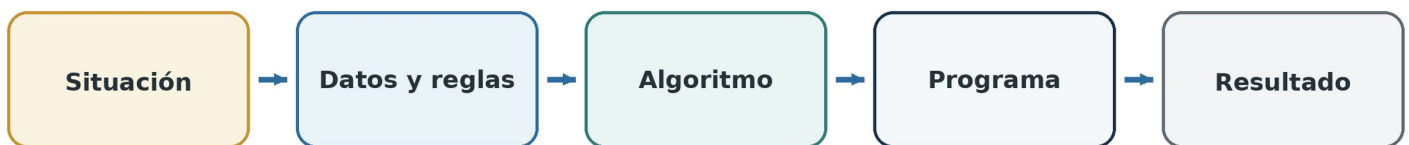


CAPÍTULO

1

Cuando una solución necesita ser ejecutable

Del problema expresado con palabras al algoritmo que puede seguirse y comprobarse



Cuando una solución necesita ser ejecutable

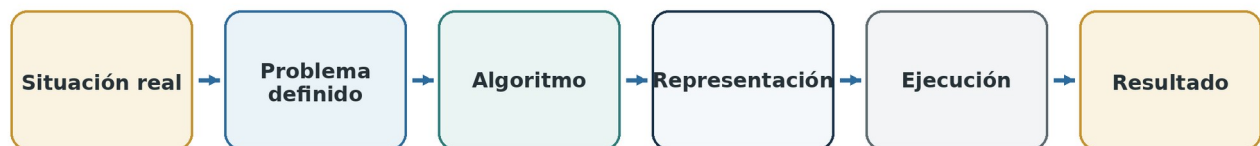
¿Qué diferencia existe entre saber resolver un problema y poder expresarlo como un programa?

Imagina una compra de 186 pesos. Entregas 200 y preguntas cuánto deben devolvarte. Una persona puede responder 14 casi de inmediato. Sin embargo, cuando intentas convertir esa respuesta en instrucciones para una computadora aparecen preguntas que antes podían pasar inadvertidas: ¿qué datos debe recibir?, ¿cuál se resta de cuál?, ¿qué ocurre si el pago es insuficiente?, ¿se aceptan centavos?, ¿cómo debe comunicarse el resultado?

La dificultad inicial de programar no consiste en aprender una larga lista de palabras reservadas. Consiste en transformar una intención humana, que puede apoyarse en contexto, intuición y supuestos implícitos, en una solución precisa que pueda seguirse paso a paso. Este capítulo se concentra en esa transformación. Todavía no escribiremos un programa completo en CLRS; primero construiremos la estructura conceptual que hará que el código del capítulo siguiente tenga sentido.

Promesa de aprendizaje: al terminar podrás identificar entradas, resultados, reglas, restricciones y subtareas de un problema sencillo; además, podrás distinguir entre problema, algoritmo, pseudocódigo y programa.

Del mundo real a un resultado comprobable



Cada transición exige decisiones: qué datos importan, qué reglas se aplican y cómo se expresan sin ambigüedad.

Figura 1.1. Del mundo real a un resultado comprobable. Elaboración propia.

Mapa del capítulo

Pregunta	Idea que construiremos
¿Qué debe resolverse?	Una especificación separa datos, resultado, reglas, restricciones y casos.
¿Cómo se resuelve?	Un algoritmo describe un procedimiento general, finito y suficientemente preciso.

Pregunta	Idea que construiremos
¿Cómo se expresa?	El pseudocódigo representa el algoritmo; el programa lo adapta a un lenguaje ejecutable.
¿Cómo se organiza?	El modelo Entrada-Proceso-Salida muestra el flujo básico de información.
¿Cómo se controla la complejidad?	La descomposición divide una solución extensa en subtarefas con propósitos claros.

Pausa y piensa: sin calcular de nuevo, explica qué información utilizaste para obtener el cambio de 14 pesos. ¿Qué parte estaba escrita en el enunciado y qué parte asumiste por experiencia?

1. Un problema no es todavía un programa

En la vida cotidiana llamamos problema a una dificultad, una pregunta o una situación no resuelta. En programación necesitamos una interpretación más precisa: un problema describe una familia de situaciones para las que se desea obtener un resultado siguiendo reglas identificables.

La palabra familia es importante. Resolver que el cambio de una compra de 186 pesos pagada con 200 es 14 demuestra que conocemos la respuesta de un caso. Programar exige construir una solución que funcione con muchos costos y pagos posibles, dentro de las condiciones que se hayan establecido.

Idea clave: un ejemplo concreto ayuda a comprender el problema, pero no sustituye una solución general.

1.1 De una situación a un problema definido

Una situación real contiene muchos detalles. No todos son relevantes para la solución computacional. En una compra podrían existir la hora, el nombre de la tienda, el color de la cartera, la forma de pago y la cantidad de artículos. Para calcular el cambio, la mayor parte de esos datos no importa. Debemos construir un modelo que conserve lo necesario y descarte, por el momento, lo que no modifica el resultado.

Este acto de seleccionar qué conservar es una forma inicial de abstracción. Abstraer no significa volver vaga una situación; significa representar sus aspectos relevantes para el propósito actual. Wing (2006) vincula el pensamiento computacional con la elección de representaciones y la descomposición de tareas complejas. En este libro utilizaremos ambas capacidades desde el primer problema.

Elemento	Pregunta de análisis	Ejemplo: calcular cambio
Datos disponibles	¿Qué información puede recibir la solución?	Costo de la compra y cantidad pagada.
Resultado esperado	¿Qué debe producirse?	Cambio que debe devolverse o faltante por cubrir.
Reglas	¿Qué relaciones determinan el resultado?	El cambio es pago menos costo cuando el pago cubre la compra.
Restricciones	¿Qué valores o condiciones se aceptan?	Cantidades no negativas; moneda con la precisión acordada.

Elemento	Pregunta de análisis	Ejemplo: calcular cambio
Casos especiales	¿Qué situaciones requieren tratamiento distinto?	Pago exacto, pago insuficiente, datos inválidos.
Criterio de éxito	¿Cómo sabremos que la solución es correcta?	El resultado coincide con la diferencia aplicable y el mensaje describe el caso.

1.2 El enunciado debe reducir la ambigüedad

Las personas completamos instrucciones incompletas con experiencia. Una computadora no comparte necesariamente ese contexto. Cuando una indicación admite varias interpretaciones, no basta con esperar que “entienda lo que quisimos decir”. La ambigüedad debe resolverse en la especificación o en el procedimiento.

De una intención ambigua a una instrucción verificable

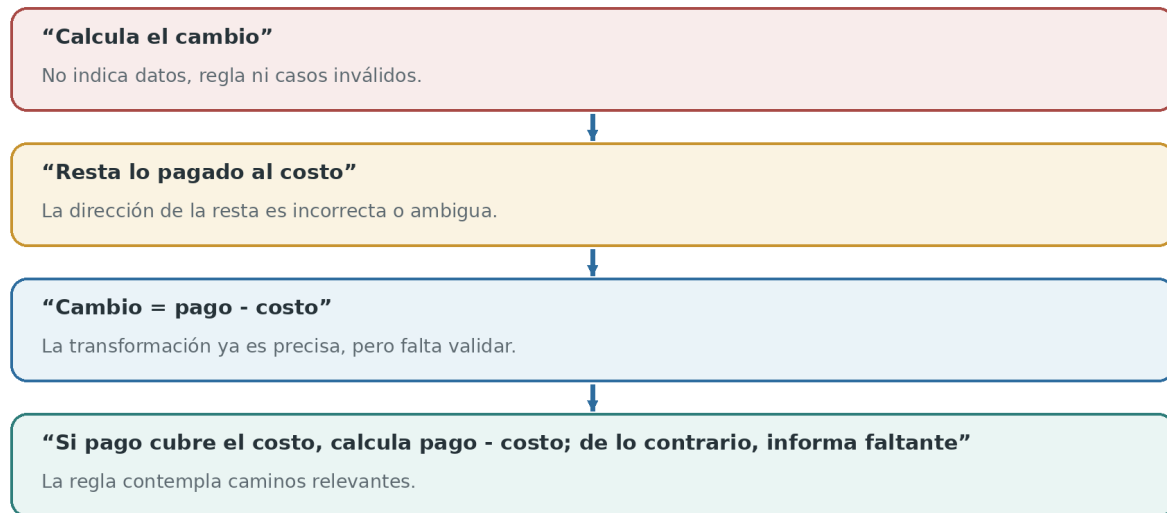


Figura 1.2. Refinamiento de una instrucción hasta volverla verificable. Elaboración propia.

La última formulación de la figura todavía no es un programa. No declara una sintaxis, un tipo de dato ni una forma exacta de mostrar el mensaje. Sin embargo, ya establece una regla que puede comprobarse con diferentes casos. Ese es el nivel de precisión que buscamos antes de concentrarnos en el lenguaje.

Advertencia: precisión no significa describir cada detalle del mundo real. Significa especificar los detalles que cambian el resultado o determinan si una entrada es aceptable.

1.3 Problemas que pueden expresarse mediante instrucciones

En este curso trabajaremos con problemas donde es posible identificar datos, reglas y una secuencia de acciones. Calcular un importe, convertir una unidad, clasificar una cantidad, buscar un valor, resumir registros o transformar texto son ejemplos adecuados.

No toda pregunta cotidiana se convierte de inmediato en un procedimiento claro. “Elige la mejor canción” depende de qué significa mejor, para quién y según qué criterios. Puede volverse un problema computacional si definimos datos y

una regla de decisión: por ejemplo, seleccionar la canción con mayor puntuación entre las que duran menos de cuatro minutos. La computadora no elimina la necesidad de definir el criterio; obliga a hacerlo visible.

Pausa y piensa: ¿qué tendrías que definir para convertir “recomienda una película interesante” en un problema que admita una solución programable?

2. De una idea de solución a un algoritmo

Una vez definido el problema podemos pensar en cómo resolverlo. La primera respuesta suele ser una idea general: “restar el costo al pago”, “sumar las calificaciones y dividir entre su cantidad” o “comparar los precios y elegir el menor”. Estas ideas son útiles, pero deben desarrollarse hasta convertirse en un procedimiento que pueda seguirse sin improvisar decisiones esenciales.

2.1 Qué entendemos por algoritmo

Un algoritmo es un procedimiento compuesto por pasos organizados para transformar unas entradas en un resultado. Su descripción debe ser lo bastante clara para que cada paso pueda realizarse y para que el procedimiento termine bajo las condiciones definidas.

Los algoritmos son centrales en computación porque los programas reales operan sobre datos mediante procedimientos. Las guías curriculares CS2023 distinguen las bases algorítmicas de los detalles particulares del lenguaje y destacan que comprender soluciones computacionales incluye conocer sus capacidades y limitaciones (ACM, IEEE-CS y AAI, 2023).

Propiedad	Qué exige	Pregunta de comprobación
Propósito definido	El procedimiento responde a un problema reconocible.	¿Qué resultado intenta producir?
Entradas identificables	Se sabe qué información necesita.	¿Con qué datos comienza?
Pasos precisos	Las acciones esenciales no dependen de adivinar la intención.	¿Dos personas interpretarían los pasos de forma compatible?
Orden coherente	Cada paso dispone de la información que necesita.	¿Algún paso usa un resultado que todavía no existe?
Finitud	El procedimiento alcanza un punto de término para los casos admitidos.	¿Existe una condición clara para detenerse?
Efectividad	Los pasos pueden realizarse con operaciones disponibles.	¿Cada acción puede ejecutarse realmente?
Generalidad limitada	Resuelve una familia de casos dentro de su especificación.	¿Funciona con otros datos válidos, no solo con el ejemplo?

2.2 Un procedimiento puede parecer correcto y no serlo

Considera estas instrucciones para calcular el promedio de tres calificaciones:

Procedimiento A
1. Suma las calificaciones. 2. Divide entre 3. 3. Comunica el resultado.

Para tres calificaciones numéricas, el procedimiento expresa una estrategia coherente. Ahora observa esta variante:

Procedimiento B

1. Divide una calificación entre 3.
2. Suma las demás.
3. Comunica el resultado.

El segundo procedimiento también tiene pasos claros y termina, pero no resuelve el mismo problema. Que una secuencia sea ordenada no garantiza que produzca el resultado correcto. La corrección depende de la relación entre el problema, las reglas y el procedimiento.

Idea clave: un algoritmo no se evalúa solo por estar bien escrito. Debe producir el resultado especificado para los casos que afirma resolver.

2.3 Seguir un algoritmo con una traza

Una traza registra qué ocurre después de cada paso. Todavía no utilizaremos variables formales, pero podemos seguir los valores intermedios. Supongamos las calificaciones 8, 9 y 7.

Paso	Acción	Información disponible después del paso
Inicio	Se reciben 8, 9 y 7.	Calificaciones: 8, 9, 7.
1	Se suman las tres calificaciones.	Suma: 24.
2	Se divide la suma entre 3.	Promedio: 8.
3	Se comunica el resultado.	Salida: 8.

La traza no demuestra por sí sola que el algoritmo sea correcto para todos los casos, pero permite detectar errores de orden, operaciones equivocadas y datos utilizados antes de estar disponibles. Más adelante, las pruebas de escritorio mostrarán también cómo cambia el estado de las variables.

2.4 El algoritmo no pertenece a un lenguaje específico

La estrategia “sumar tres cantidades y dividir el total entre tres” puede representarse en español, en un diagrama, en pseudocódigo o en diferentes lenguajes de programación. Cada representación tiene reglas propias, pero el procedimiento subyacente puede conservarse.

Esta separación tiene un valor práctico: permite concentrarse primero en la lógica del problema y después en la forma requerida por el entorno. No significa que la representación sea irrelevante. Una vez que queremos ejecutar la solución, el lenguaje debe expresar sin contradicciones las operaciones del algoritmo.

3. Problema, algoritmo, pseudocódigo y programa

Quien comienza a programar suele usar estas palabras como si fueran equivalentes. Distinguir las evita varias confusiones: escribir una solución en español no la vuelve ejecutable; ejecutar un programa sin errores no garantiza que resuelva el problema; y cambiar de lenguaje no obliga siempre a inventar un algoritmo diferente.

Cuatro niveles que no deben confundirse

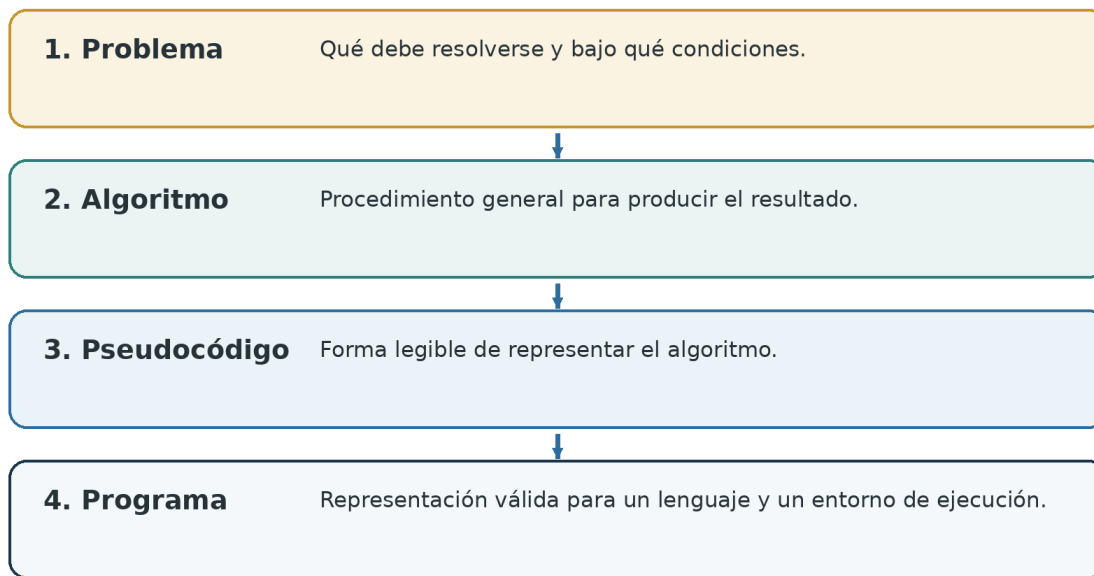


Figura 1.3. Niveles de una solución computacional. Elaboración propia.

3.1 Problema: el compromiso sobre lo que debe resolverse

El problema establece el objetivo y las condiciones. Debe permitir decidir qué casos están dentro del alcance y qué resultado se espera. Un problema mal definido puede admitir programas técnicamente válidos que produzcan respuestas incompatibles, porque nunca se acordó qué significaba una respuesta correcta.

3.2 Algoritmo: la estrategia organizada

El algoritmo explica cómo pasar de los datos al resultado. Puede describirse con distintos niveles de detalle. Al principio conviene mostrar las decisiones esenciales; después se refina hasta que cada subtarea pueda representarse de manera ejecutable.

3.3 Pseudocódigo: una representación para pensar y comunicar

El pseudocódigo utiliza estructuras parecidas a las de los lenguajes de programación, pero prioriza la legibilidad y la comunicación del procedimiento. No existe una única sintaxis universal de pseudocódigo. Diferentes libros, instituciones y herramientas adoptan convenciones distintas.

Cormen, Leiserson, Rivest y Stein (2022) presentan algoritmos en un pseudocódigo diseñado para ser legible y relativamente independiente de un lenguaje concreto. CLRS Kit Español adopta una sintaxis inspirada en ese estilo, cercana al español y orientada a la enseñanza, pero añade reglas suficientes para analizar, traducir y ejecutar archivos

`clrs`. Por eso, dentro de esta obra hablaremos de un pseudolenguaje ejecutable: conserva una apariencia pedagógica, aunque su implementación necesita una sintaxis definida.

Importante: el pseudocódigo ayuda a reducir carga sintáctica, pero no elimina la necesidad de precisión. Cuando se ejecuta, sus reglas dejan de ser opcionales.

3.4 Programa: una representación aceptada por un sistema

Un programa es una descripción expresada de acuerdo con las reglas de un lenguaje y preparada para ser procesada por un sistema de ejecución. Puede contener un algoritmo correcto o uno incorrecto. También puede fallar porque su representación viola la sintaxis o porque intenta realizar una operación no válida.

En CLRS Kit Español, la extensión analiza el archivo, construye representaciones internas, puede traducirlo a JavaScript y ejecutarlo. Además, muestra diagnósticos y genera diagramas. Estas herramientas serán útiles, pero ninguna puede reemplazar la definición del problema: el entorno puede verificar que una instrucción es válida sin saber si esa instrucción representa la intención correcta.

Nivel	Pregunta principal	Error típico
Problema	¿Qué debe resolverse?	No definir límites o resultados esperados.
Algoritmo	¿Cómo se obtiene el resultado?	Omitir un caso o aplicar una regla equivocada.
Pseudocódigo	¿Cómo se comunica el procedimiento con claridad?	Usar convenciones ambiguas o inconsistentes.
Programa	¿Cómo se expresa para que el sistema lo procese?	Violar la sintaxis, el ámbito o las operaciones disponibles.

Pausa y piensa: un programa se ejecuta sin errores y muestra 10 como promedio de 8, 9 y 7. ¿En qué nivel buscarías primero el problema: sintaxis, algoritmo o especificación? Justifica.

4. Organizar la solución con Entrada, Proceso y Salida

Cuando un problema todavía parece difuso, el modelo Entrada-Proceso-Salida ofrece una primera estructura. Obliga a identificar qué información existe al comenzar, qué transformaciones deben realizarse y qué resultado debe producirse.

Modelo Entrada - Proceso - Salida



El modelo organiza la solución, pero no sustituye la especificación de restricciones, casos inválidos y criterios de éxito

Figura 1.4. Modelo Entrada-Proceso-Salida. Elaboración propia.

4.1 Entrada: la información que la solución necesita

Una entrada es un dato disponible para el procedimiento. No tiene que provenir del teclado. Puede estar escrita en un archivo, obtenida de un sensor, calculada anteriormente, recibida desde otra función o incluida en el propio problema.

En el cálculo del cambio, el costo y el pago son entradas. La fórmula “pago menos costo” no es una entrada; es una regla del proceso. El cambio tampoco es una entrada; es el resultado que queremos producir.

4.2 Proceso: las reglas que transforman la información

El proceso reúne operaciones, decisiones y pasos intermedios. A veces se reduce a una fórmula; otras veces incluye validación, comparación, repetición, búsqueda o delegación de tareas. En este capítulo solo identificaremos esas acciones. Las estructuras concretas para decisiones y repeticiones aparecerán más adelante.

Advertencia: “procesar” no significa necesariamente hacer una operación matemática. Convertir texto a mayúsculas, comprobar un formato o elegir una categoría también son transformaciones.

4.3 Salida: la evidencia de que la tarea produjo un resultado

Una salida es información producida por la solución. Puede mostrarse en pantalla, guardarse en un archivo, enviarse a otro sistema o utilizarse como entrada de un proceso posterior. Conviene describir su significado, no solo su formato.

En el ejemplo de la compra, “14” es una cantidad; “devolver 14 pesos” expresa mejor su significado. Cuando el pago es insuficiente, la salida podría ser la cantidad faltante. Por eso la especificación debe distinguir los casos posibles.

4.4 El modelo no reemplaza las restricciones

Entrada-Proceso-Salida es una vista compacta. No muestra por sí sola qué valores son válidos, qué precisión se requiere, qué ocurre ante datos ausentes o qué criterio determina que el resultado es correcto. Esas condiciones deben acompañar al modelo.

Problema	Entrada	Proceso	Salida
Convertir grados Celsius a Fahrenheit.	Temperatura en grados Celsius.	Aplicar la relación de conversión acordada.	Temperatura equivalente en Fahrenheit.
Calcular el área de un rectángulo.	Longitud y anchura no negativas.	Multiplicar ambas medidas.	Área en unidades cuadradas.
Determinar si se alcanza una meta de ahorro.	Meta y cantidad acumulada.	Comparar ambas cantidades.	Indicación de cumplimiento y diferencia.
Contar palabras de un texto.	Texto.	Definir separadores y contabilizar unidades consideradas palabras.	Cantidad de palabras según la regla adoptada.

4.5 Caso longitudinal: registrar un gasto

El proyecto que crecerá durante el libro será un registro de gastos. En su primera aproximación no necesitamos escribir código. Necesitamos decidir qué significa registrar un gasto y qué resultado debe obtenerse.

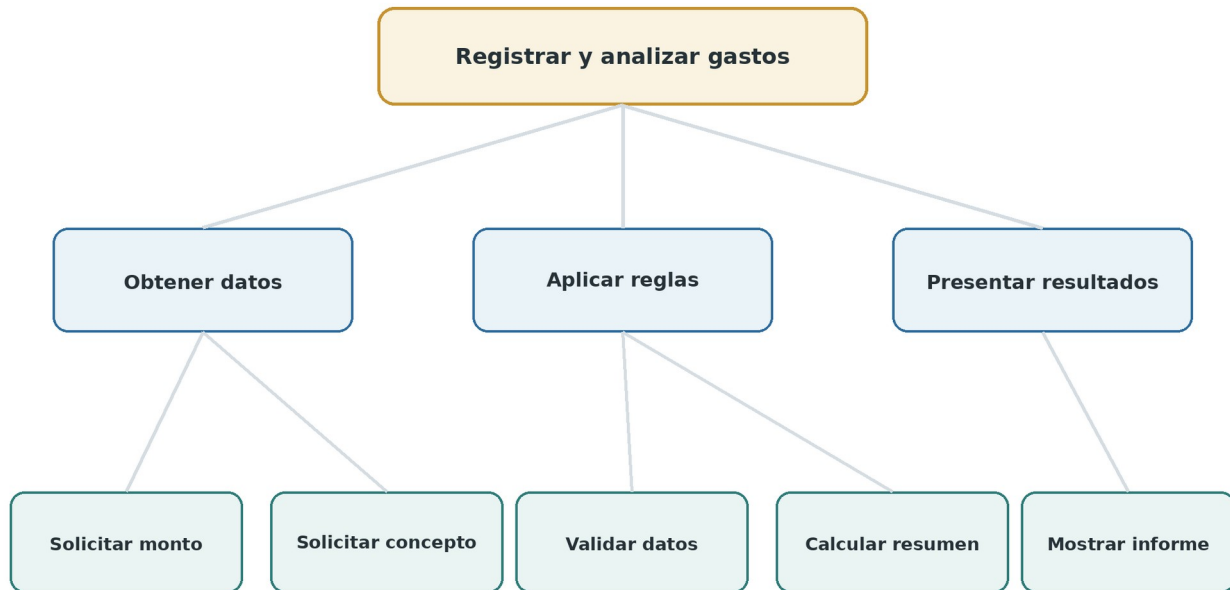
Componente	Primera definición
Entrada	Concepto del gasto y monto pagado.
Proceso	Comprobar que el monto pueda interpretarse como una cantidad válida y asociarlo con su concepto.
Salida	Confirmación que muestre el concepto y el monto registrado.
Restricciones	El concepto no debe estar vacío; el monto debe ser mayor o igual que cero; la precisión monetaria debe estar definida.
Casos especiales	Monto cero, espacios en el concepto, dato no numérico, cancelación del registro.

Esta versión todavía no calcula totales ni conserva varios registros. Es deliberadamente pequeña. Una buena primera solución no intenta abarcar todo el sistema final; establece un núcleo correcto que pueda ampliarse.

5. Descomponer antes de escribir una solución extensa

A medida que un problema incorpora más reglas, intentar resolverlo como un bloque único aumenta la carga mental. La descomposición consiste en separar el problema en subtareas más pequeñas, cada una con un propósito identificable. No elimina la dificultad: la distribuye para que podamos razonar sobre una parte a la vez.

Descomposición progresiva de un problema



La descomposición no divide el código por tamaño: divide la responsabilidad de la solución en tareas con un propósito reconocible.

Figura 1.5. Descomposición del caso longitudinal. Elaboración propia.

5.1 Refinamiento progresivo

Podemos comenzar con una acción general y reemplazarla gradualmente por acciones más concretas. Este proceso se denomina refinamiento progresivo o diseño descendente. Cada nivel responde con mayor detalle a la pregunta “¿cómo se realizará esta tarea?”.

Refinamiento del registro de un gasto

Nivel 1: Registrar un gasto.

Nivel 2: Obtener datos → validar datos → conservar registro → confirmar resultado.

Nivel 3: Solicitar concepto → retirar espacios innecesarios → comprobar que no esté vacío → solicitar monto → comprobar que sea válido → asociar concepto y monto → mostrar confirmación.

Estas líneas describen responsabilidades. Todavía no pretenden ser sintaxis CLRS.

5.2 Subtareas y contratos

Una subtarea es más útil cuando podemos describir qué necesita y qué garantiza. A esta descripción podemos llamarla contrato de manera informal. El contrato no explica todavía cada paso interno; establece la relación entre las condiciones de entrada y el resultado.

Subtarea	Necesita	Garantiza
Validar monto	Un dato capturado como monto.	Indica si puede tratarse como cantidad admitida.
Calcular cambio	Costo y pago válidos.	Produce cambio o faltante según la relación entre ambos.
Presentar registro	Concepto y monto ya aceptados.	Comunica el registro con un formato comprensible.

5.3 Precondiciones y resultados esperados

Una precondición expresa qué debe cumplirse antes de ejecutar una tarea. Un resultado esperado o poscondición describe qué debe ser cierto después si la tarea termina correctamente. Estos términos se estudiarán con mayor formalidad en otros contextos; aquí los usaremos para evitar que las subtareas dependan de supuestos invisibles.

Ejemplo: para “calcular el área de un rectángulo”, una precondición puede exigir medidas numéricas no negativas; el resultado esperado es un área igual al producto de esas medidas, expresada en unidades cuadradas.

5.4 Cómo saber cuándo dejar de dividir

No existe un número universal de niveles. Conviene detener el refinamiento cuando cada subtarea puede explicarse, probarse o convertirse en instrucciones sin ocultar decisiones importantes. Dividir más puede producir fragmentación innecesaria; dividir menos puede conservar una tarea demasiado ambigua.

Señal de que falta descomponer	Señal de fragmentación excesiva
La tarea contiene varias responsabilidades unidas por “y”.	Cada subtarea realiza una operación trivial sin aportar un propósito.
No puedes explicar cómo comprobarla de forma aislada.	Los nombres describen movimientos mínimos, no resultados.
Necesitas recordar demasiados casos al mismo tiempo.	Comprender el flujo exige saltar entre demasiadas piezas.
El mismo bloque de acciones se repite en distintos lugares.	La división no permite reutilizar ni probar nada por separado.

5.5 Descomponer por responsabilidad, no por longitud

Separar una descripción en tres grupos del mismo tamaño no garantiza una buena descomposición. Las fronteras deben seguir cambios de propósito: obtener información, validarla, calcular un resultado, almacenarlo o presentarlo. Dos pasos breves pueden pertenecer a una responsabilidad; diez pasos pueden formar otra.

Idea clave: una subtarea debe poder nombrarse por lo que consigue, no únicamente por las instrucciones que contiene.

6. Integración: especificar el cálculo del cambio

Reunamos las ideas del capítulo en una especificación más completa. El objetivo no es escribir todavía la versión CLRS, sino preparar una solución suficientemente clara para traducirla después.

6.1 Enunciado refinado

Problema: dado el costo de una compra y la cantidad entregada como pago, determinar si el pago cubre la compra. Si la cubre, informar el cambio; si no, informar la cantidad faltante.

6.2 Especificación

Aspecto	Definición
Entradas	Costo y pago, expresados como cantidades monetarias.
Salida principal	Mensaje que indique cambio, pago exacto o faltante.
Regla 1	Cuando $\text{pago} > \text{costo}$, $\text{cambio} = \text{pago} - \text{costo}$.
Regla 2	Cuando $\text{pago} = \text{costo}$, la compra queda cubierta sin cambio.
Regla 3	Cuando $\text{pago} < \text{costo}$, $\text{faltante} = \text{costo} - \text{pago}$.
Restricciones	Costo y pago no negativos; precisión de dos decimales para este ejercicio.
Datos inválidos	Cualquier entrada que no pueda interpretarse como cantidad válida o sea negativa.
Criterio de éxito	La salida describe el caso correcto y la diferencia monetaria correspondiente.

6.3 Descomposición

1. Obtener el costo.
2. Obtener el pago.
3. Comprobar que ambas entradas sean válidas.
4. Comparar el pago con el costo.
5. Calcular cambio o faltante cuando corresponda.
6. Comunicar el resultado con su significado.

6.4 Algoritmo en lenguaje estructurado

Algoritmo: determinar cambio o faltante

1. Recibir el costo y el pago.
2. Si alguno no es una cantidad válida o es negativo, informar que los datos no son aceptables y terminar.
3. Si el pago es mayor que el costo, calcular pago menos costo e informar el cambio.
4. Si el pago es igual al costo, informar que el pago es exacto.

5. Si el pago es menor que el costo, calcular costo menos pago e informar el faltante.
El capítulo 3 mostrará cómo expresar las decisiones con estructuras del lenguaje. Aquí importa que los casos estén definidos.

6.5 Casos para comprobar la especificación

Costo	Pago	Caso esperado	Resultado esperado
186.00	200.00	Pago mayor	Cambio: 14.00.
186.00	186.00	Pago exacto	Sin cambio ni faltante.
186.00	150.00	Pago insuficiente	Faltante: 36.00.
0.00	0.00	Límite admitido	Pago exacto.
-10.00	20.00	Dato inválido	Rechazar la entrada.
"cien"	200.00	Dato no numérico	Rechazar la entrada.

Los casos no prueban matemáticamente todas las entradas posibles, pero revelan si la especificación contempla caminos importantes. Observa que se incluyeron casos normales, un límite y datos inválidos. Más adelante aprenderemos a diseñar pruebas de manera sistemática.

6.6 Qué falta antes de ejecutar

La solución conceptual ya distingue entradas, reglas, casos y resultados. Sin embargo, aún falta decidir cómo representar las cantidades, cómo leerlas, cómo escribir mensajes y cómo expresar las comparaciones en la sintaxis de CLRS. Esa transición será el centro de los capítulos siguientes.

Conexión hacia adelante: en el próximo capítulo el algoritmo comenzará a recorrer instrucciones reales. Introduciremos `PRINCIPAL`, archivos `.cls`, indentación, comentarios, ejecución y lectura de diagnósticos.

7. Confusiones frecuentes

Confusión	Por qué falla	Corrección conceptual
“Si conozco una respuesta, ya tengo el algoritmo.”	Un caso no establece una regla general.	Explica cómo obtener respuestas para toda entrada admitida.
“Un algoritmo es cualquier lista de pasos.”	Los pasos pueden ser irrelevantes, incorrectos o no terminar.	Relaciona cada paso con el resultado y comprueba su orden.
“Pseudocódigo y programa son lo mismo.”	Un pseudocódigo informal puede no ser procesable.	Distingue representación legible de sintaxis ejecutable.
“Si el programa se ejecuta, resolvió el problema.”	Puede ejecutar un algoritmo equivocado.	Compara la salida con la especificación.
“Entrada significa escribir por teclado.”	Los datos pueden provenir de múltiples fuentes.	Piensa en información disponible para el proceso.
“Proceso significa fórmula.”	También puede incluir validación, elección y transformación.	Describe todas las reglas que cambian o interpretan datos.
“Descomponer es cortar el programa en partes iguales.”	El tamaño no define responsabilidad.	Divide por propósitos y resultados parciales.

8. Laboratorio conceptual: diseñar antes de programar

Este laboratorio no requiere CLRS todavía. Su objetivo es que practiques la etapa que suele omitirse cuando alguien intenta escribir código de inmediato.

Problema: calcular el costo de una estancia

Situación: un estacionamiento cobra una tarifa por hora. Se desea calcular el costo a partir del número de horas utilizadas. La tarifa es fija en esta primera versión y cualquier fracción se cobra como una hora completa.

Paso 1. Identifica información

Completa mentalmente o por escrito:

- ¿Qué dato debe recibir el procedimiento?
- ¿Qué dato debe conocerse además del tiempo?
- ¿Qué salida debe producir?
- ¿Qué significa “cualquier fracción se cobra como una hora completa”?
- ¿Qué valores deberían considerarse inválidos?

Paso 2. Propón Entrada-Proceso-Salida

Entrada	Proceso	Salida
Horas utilizadas y tarifa por hora.	Validar datos; convertir la duración a horas cobrables según la regla; multiplicar por la tarifa.	Costo total de la estancia.

Paso 3. Descompón

7. Obtener la duración utilizada.
8. Obtener o conocer la tarifa.
9. Comprobar que los datos sean válidos.
10. Determinar las horas cobrables.
11. Calcular el costo.
12. Presentar el resultado.

Paso 4. Diseña casos

Duración	Tarifa	Aspecto que comprueba	Resultado esperado
2 horas	20 por hora	Caso entero normal	40.
2.25 horas	20 por hora	Redondeo hacia arriba	60.
0 horas	20 por hora	Límite inferior	0, si el problema lo admite.
-1 hora	20 por hora	Entrada inválida	Rechazo.
2 horas	-20 por hora	Tarifa inválida	Rechazo.

Pausa y piensa: la regla de redondeo no indica qué ocurre con 2.0001 horas si la medición tiene errores. ¿Es una cuestión de algoritmo o de especificación? Explica qué tendría que definirse.

9. Reconstrucción del modelo mental

Este capítulo comenzó con una respuesta aparentemente trivial: devolver 14 pesos. Al hacer explícito el razonamiento descubrimos que programar exige separar varios niveles.

13. La situación real contiene más información de la necesaria.
14. El problema selecciona datos, objetivo, reglas, restricciones y casos relevantes.
15. El algoritmo organiza un procedimiento general para transformar entradas en resultados.
16. El pseudocódigo comunica el procedimiento con convenciones legibles.
17. El programa adopta las reglas de un lenguaje y un entorno para poder ejecutarse.
18. El modelo Entrada-Proceso-Salida ofrece una primera vista de la transformación.
19. La descomposición permite manejar soluciones más amplias mediante subtareas con propósitos claros.
20. Los casos de prueba comparan lo que se espera con lo que la solución produce.

Síntesis: programar comienza antes del código. Comienza cuando una intención se convierte en una especificación y una estrategia que admiten comprobación.

10. Ejercicios

Resuelve los ejercicios sin utilizar todavía sintaxis CLRS. Cuando un enunciado sea ambiguo, no inventes silenciosamente una regla: señala lo que debe definirse.

1. **[Reconocer]** Clasifica cada elemento del cálculo de cambio como entrada, proceso, salida, restricción o caso especial: costo, pago, pago negativo, restar, cambio, pago exacto.
2. **[Comprender]** Explica por qué “ordenar estos números: 8, 3, 5” es un caso y “ordenar cualquier lista de números admitida” describe una familia de problemas.
3. **[Analizar]** Identifica al menos tres ambigüedades en la instrucción “calcula el precio final con descuento”.
4. **[Aplicar]** Construye una tabla Entrada-Proceso-Salida para convertir kilómetros a metros. Añade restricciones y dos casos de prueba.
5. **[Aplicar]** Construye una tabla Entrada-Proceso-Salida para determinar el mayor de dos números. No escribas todavía la estructura de decisión.
6. **[Analizar]** Compara estas dos tareas: a) mostrar un saludo fijo; b) generar un saludo usando un nombre recibido. Explica cómo cambian las entradas y la generalidad.
7. **[Aplicar]** Reescribe la instrucción “saca el promedio” hasta que indique entradas, cantidad de valores y resultado esperado.
8. **[Analizar]** Diseña cinco casos para comprobar una solución que calcule el área de un rectángulo. Incluye un caso normal, un límite y datos inválidos.
9. **[Construir]** Descompón el problema “registrar una venta” en al menos cuatro subtareas. Nombra cada subtaska por su propósito.
10. **[Analizar]** Para la subtaska “validar edad”, escribe una precondition y un resultado esperado. Después indica qué información todavía falta para que la validación tenga sentido.
11. **[Comprender]** Distingue problema, algoritmo, pseudocódigo y programa en el contexto de una calculadora de propinas. Da una oración para cada nivel.

12. **[Analizar]** Una solución recibe el radio de un círculo y muestra un número. Explica qué información adicional necesitas para decidir si el resultado es correcto.
13. **[Construir]** Elabora una especificación para calcular el costo total de varios artículos cuando ya se conoce el subtotal y la tasa de impuesto. Define cómo se representa la tasa.
14. **[Comprender]** Explica por qué el procedimiento “multiplica el largo por el ancho” puede ser correcto para un rectángulo y no resolver el área de un triángulo, aunque ambos problemas soliciten un área.
15. **[Construir]** Diseña la primera especificación del caso longitudinal: registrar un gasto. Incluye concepto, monto, restricciones, casos inválidos y salida de confirmación.

Mini proyecto: especificar una calculadora de consumo

Diseña, sin programar, una solución que calcule el consumo promedio de combustible a partir de una distancia recorrida y una cantidad utilizada. Debes entregar:

- enunciado refinado del problema;
- tabla Entrada-Proceso-Salida;
- restricciones y casos inválidos;
- descomposición en subtareas;
- algoritmo en lenguaje estructurado;
- al menos seis casos con resultados esperados;
- explicación de una decisión que no pueda resolverse sin ampliar la especificación, como la unidad o la división entre cero.

Entregable	Criterio de revisión
Especificación	Permite distinguir entradas válidas, resultado esperado y situaciones fuera de alcance.
Modelo EPS	Cada dato y acción aparece en la categoría que le corresponde.
Descomposición	Las subtareas están nombradas por su propósito y no por su longitud.
Algoritmo	Los pasos están ordenados, terminan y contemplan la división entre cero.
Casos	Incluyen valores normales, límites, diferentes unidades e inválidos.

Pista graduada: si la distancia es cero o negativa, si el combustible utilizado es cero, o si las unidades no coinciden, la fórmula por sí sola no basta. Primero decide qué casos admite el problema.

11. Autoevaluación y criterio de dominio

Marca la descripción que mejor represente tu nivel actual. La meta no es responder de memoria, sino demostrarlo con un problema nuevo.

Criterio	Inicial	En desarrollo	Dominio esperado
Definir un problema	Identifico solo el resultado.	Distingo algunos datos y reglas.	Especifico entradas, resultado, reglas, restricciones, casos y criterio de éxito.
Reconocer un algoritmo	Confundo cualquier lista con un algoritmo.	Reviso orden y terminación.	Evalúo precisión, generalidad limitada y relación con la especificación.
Distinguir niveles	Uso algoritmo, pseudocódigo y programa como sinónimos.	Distingo algunos niveles.	Explico el papel y los errores propios de problema, algoritmo, pseudocódigo y programa.
Aplicar EPS	Enumero datos sin clasificarlos bien.	Separo entrada y salida.	Organizo entradas, transformaciones y salidas, y añado restricciones externas al modelo.
Descomponer	Divido por longitud.	Identifico algunas subtareas.	Divido por responsabilidad y describo qué necesita y qué garantiza cada tarea.
Diseñar casos	Pruedo un ejemplo normal.	Incluyo varios valores.	Incluyo casos normales, límites, alternativas e inválidos con resultados esperados.

Criterio para avanzar: toma un problema cotidiano nuevo y prepara una especificación, un modelo EPS, una descomposición y cinco casos de prueba sin escribir código. Si otra persona puede seguir tu razonamiento sin pedir que complete supuestos esenciales, estás listo para el siguiente capítulo.

12. Glosario breve

Término	Definición en este capítulo
Abstracción	Representación que conserva los aspectos relevantes de una situación para un propósito.
Algoritmo	Procedimiento organizado, finito y suficientemente preciso que transforma entradas en un resultado.
Caso	Conjunto concreto de datos o condiciones sobre los que se aplica una solución.
Criterio de éxito	Condición que permite decidir si la solución produjo el resultado esperado.
Descomposición	Separación de un problema en subtarefas con responsabilidades identificables.
Entrada	Información disponible para un procedimiento antes o durante su ejecución.
Especificación	Descripción del objetivo, datos, reglas, restricciones y resultados esperados de un problema.
Precondición	Condición que debe cumplirse antes de realizar una tarea.
Proceso	Conjunto de reglas y transformaciones aplicadas a la información.
Programa	Representación escrita según las reglas de un lenguaje para que un sistema pueda procesarla o ejecutarla.
Pseudocódigo	Notación legible, semejante a un lenguaje de programación, usada para representar algoritmos.
Salida	Información producida por el procedimiento.
Traza	Registro ordenado de los pasos y resultados intermedios de una ejecución o simulación.

Referencias del capítulo

ACM, IEEE Computer Society, & Association for the Advancement of Artificial Intelligence. (2023). Computer Science Curricula 2023 (Version Gamma). <https://csed.acm.org/wp-content/uploads/2023/09/Version-Gamma.pdf>

CLRS Kit Español. (2026). CLRS Kit Español, versión 1.3.1 [Documentación técnica del proyecto].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Wing, J. M. (2006). Computational thinking. Communications of the ACM, 49(3), 33-35. <https://doi.org/10.1145/1118178.1118215>

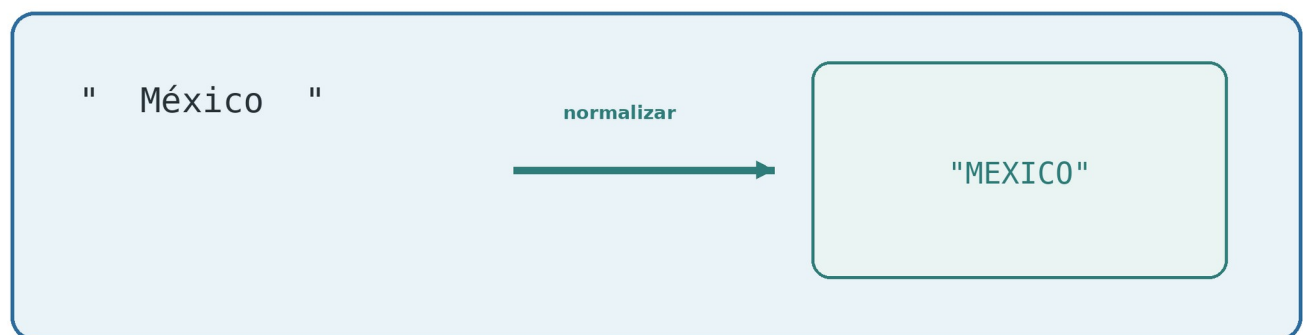
Siguiente capítulo: El primer programa y su recorrido. Convertiremos una secuencia conceptual en un archivo `.clrs``, observaremos su orden de ejecución y aprenderemos a interpretar los primeros diagnósticos.

CAPÍTULO

10

Tratar el texto como información procesable

Caracteres, normalización, búsqueda, división y análisis



Tratar el texto como información procesable

¿Cómo deja una cadena de ser un bloque indivisible y se convierte en una secuencia que puede analizarse?

Para una persona, “ México ”, “MÉXICO” y “méxico” pueden expresar el mismo nombre. Para un programa, las diferencias de mayúsculas, espacios y caracteres forman parte de la representación. Si se comparan sin preparación, pueden considerarse valores distintos.

Procesar texto exige observar dos niveles a la vez. Una cadena tiene un significado completo —un nombre, una frase o un registro—, pero también está formada por caracteres y fragmentos que pueden recorrerse, buscarse, separarse y reorganizarse.

Promesa de aprendizaje: al terminar podrás tratar cadenas como secuencias, acceder a caracteres, normalizar texto, extraer subcadenas, buscar y reemplazar fragmentos, dividir y unir cadenas, y construir algoritmos elementales de análisis textual.

Procesar texto suele requerir una cadena de transformaciones

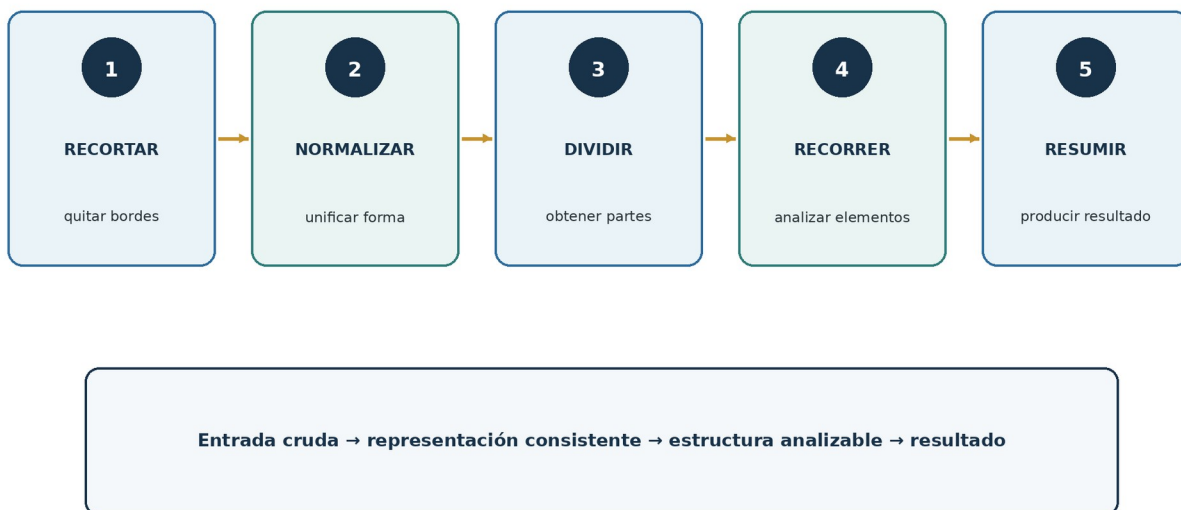


Figura 10.1. Cadena general de procesamiento de texto. Elaboración propia.

Conexión: el capítulo anterior introdujo arreglos. Ahora veremos que muchas operaciones textuales producen arreglos o necesitan recorrerlos.

10.1 Una cadena es un valor completo y una secuencia

Una cadena puede utilizarse como un valor completo: asignarse, compararse, escribirse o enviarse a una función. También puede observarse como una secuencia ordenada de caracteres. Esta segunda perspectiva permite analizar su estructura interna.

Una cadena puede observarse como una secuencia de caracteres

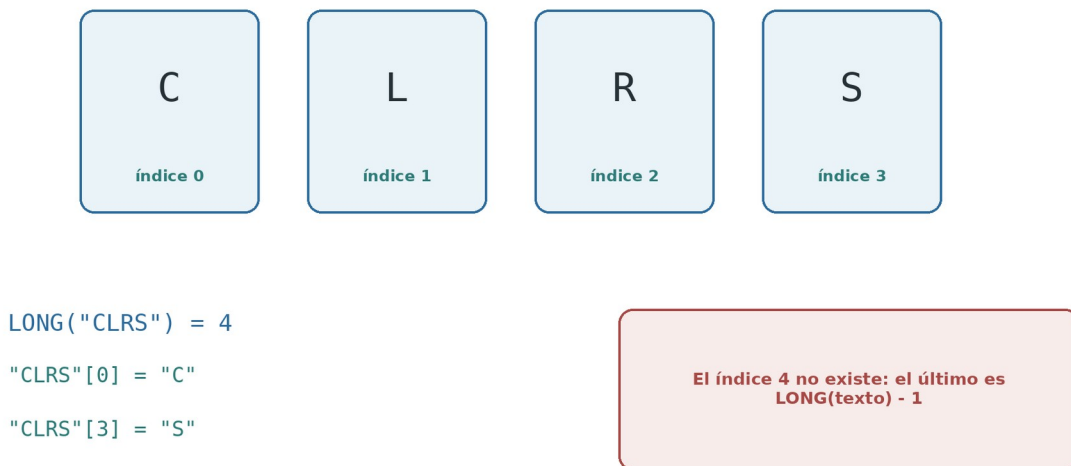


Figura 10.2. Cadena como secuencia indexada. Elaboración propia.

Acceder a caracteres

```
texto <- "CLRS"
escribir LONG(texto)
escribir texto[0]
escribir texto[LONG(texto) - 1]
```

El primer carácter ocupa el índice 0. El último ocupa $\text{LONG}(\text{texto}) - 1$. La misma relación estudiada para arreglos se reutiliza aquí.

Advertencia: una cadena vacía tiene longitud 0; por tanto, no posee un primer carácter. Antes de acceder a `texto[0]`, comprueba que exista al menos una posición.

10.2 Longitud, índices y límites

La longitud responde cuántos caracteres contiene la cadena. No responde cuál es el último índice. Si la longitud es 5, las posiciones válidas son 0, 1, 2, 3 y 4.

Expresión	Resultado	Interpretación
<code>`LONG("casa")`</code>	4	Cantidad de caracteres.
<code>`"casa"[0]`</code>	<code>`"c"`</code>	Primer carácter.
<code>`"casa"[3]`</code>	<code>`"a"`</code>	Último carácter.
<code>`"casa"[4]`</code>	Inválido	Está fuera de los límites.

Pausa y piensa: para la cadena ``"algoritmo"``, escribe la longitud, el último índice y los caracteres almacenados en las posiciones 0, 4 y 8.

10.3 Recorrer caracteres

Un recorrido visita los caracteres en orden y permite aplicar una misma pregunta o transformación a cada posición.

Mostrar cada carácter

```
texto <- "CLRS"

para i <- 0 hasta LONG(texto) - 1
  escribir texto[i]
```

La variable ``i`` representa la posición actual. El acceso ``texto[i]`` recupera el carácter asociado con esa posición.

Contar vocales

```
CONTAR_VOCALES(texto)
  cantidad <- 0
  texto <- MAYUS(texto)

  para i <- 0 hasta LONG(texto) - 1
    caracter <- texto[i]

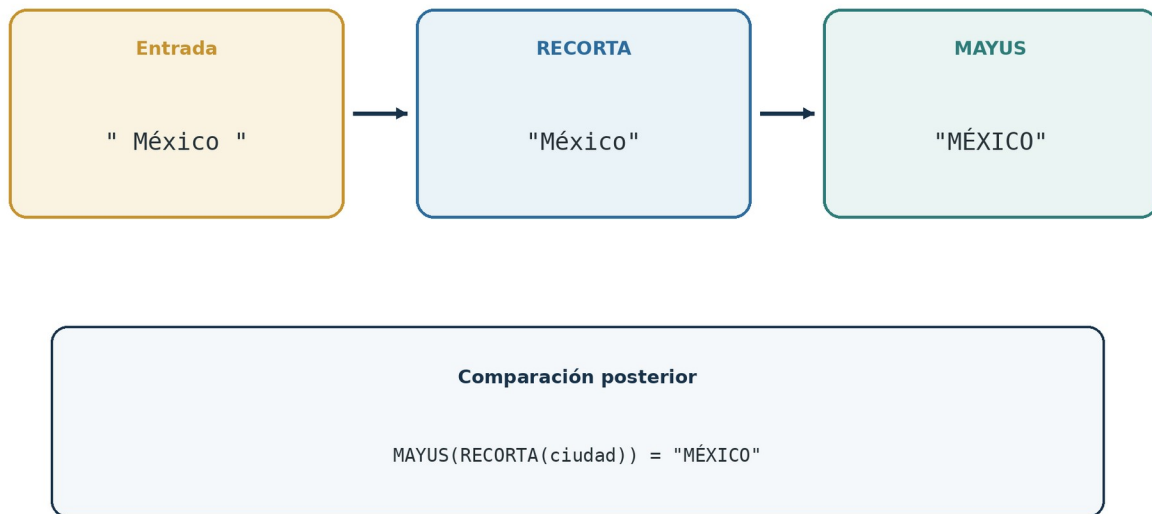
    si caracter = "A" o caracter = "E" o caracter = "I" o caracter = "O" o caracter =
"U"
      cantidad <- cantidad + 1

  retornar cantidad
```

El texto se normaliza primero a mayúsculas. Así se evita duplicar cada comparación para versiones mayúsculas y minúsculas.

10.4 Normalizar antes de comparar

Normalizar significa convertir distintas representaciones aceptables en una forma común. La normalización no corrige cualquier entrada; reduce únicamente diferencias que el problema considera irrelevantes.

Normalizar reduce diferencias que no son relevantes para el problema

La normalización elegida depende del significado: no siempre deben eliminarse acentos, símbolos o espacios internos.

Figura 10.3. Normalización de espacios y mayúsculas. Elaboración propia.

Comparación normalizada

```

ciudad <- " México "
ciudad <- RECORTA(ciudad)
ciudad <- MAYUS(ciudad)

si ciudad = "MÉXICO"
  escribir "Coincidencia"
  
```

Operación	Propósito	Precaución
`RECORTA`	Eliminar espacios al inicio y al final.	No elimina necesariamente espacios internos.
`MAYUS`	Unificar letras en mayúsculas.	Puede alterar la presentación original.
`MINUS`	Unificar letras en minúsculas.	Tiene el mismo propósito comparativo.

Importante: conserva el texto original cuando su forma sea relevante para mostrarlo después. Puedes crear una variable normalizada separada.

10.5 Comparación exacta y comparación preparada

Una comparación exacta pregunta si dos representaciones coinciden carácter por carácter. Una comparación preparada transforma primero ambos valores según las reglas del problema.

Dos estrategias

```
original <- " Ana "
escribir original = "ana"

normalizado <- MINUS(RECORTA(original))
escribir normalizado = "ana"
```

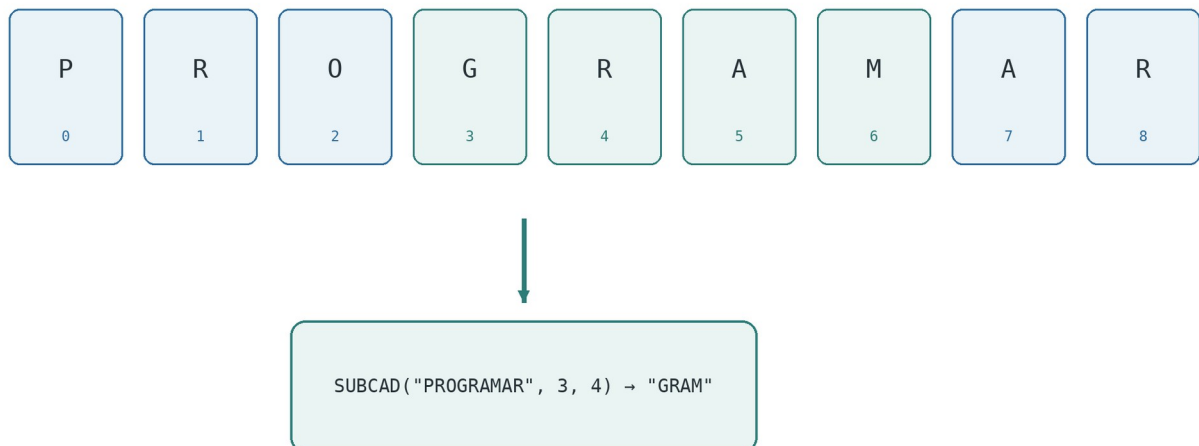
La primera comparación puede producir falso porque conserva espacios y mayúscula inicial. La segunda expresa una regla más cercana a “ignorar espacios exteriores y diferencia entre mayúsculas y minúsculas”.

Contraejemplo: normalizar números de identificación, contraseñas o códigos sensibles a mayúsculas puede destruir información válida. La normalización depende del dominio.

10.6 Extraer subcadenas

Una subcadena es una región continua de otra cadena. Puede representar un prefijo, un código interno, una fecha, una extensión o cualquier fragmento delimitado por posiciones.

Una subcadena selecciona una región continua del texto



Antes de extraer, identifica el índice inicial y la cantidad o límite que utiliza la función.

Figura 10.4. Selección de una subcadena. Elaboración propia.

Extraer una región

```
palabra <- "PROGRAMAR"
fragmento <- SUBCAD(palabra, 3, 4)
escribir fragmento
```


Antes de utilizar una función de subcadena, verifica cómo interpreta sus parámetros: posición inicial, longitud o posición final. El capítulo usa la convención documentada por la biblioteca de CLRS Kit.

Advertencia: una posición inicial negativa o un fragmento que excede la longitud disponible debe tratarse como un caso inválido o limitarse de manera explícita.

10.7 Prefijos, sufijos y formatos

Los prefijos aparecen al comienzo y los sufijos al final. Muchas validaciones sencillas pueden expresarse mediante estas regiones.

Obtener una extensión

```
archivo <- "reporte.txt"
extension <- SUBCAD(archivo, LONG(archivo) - 3, 4)

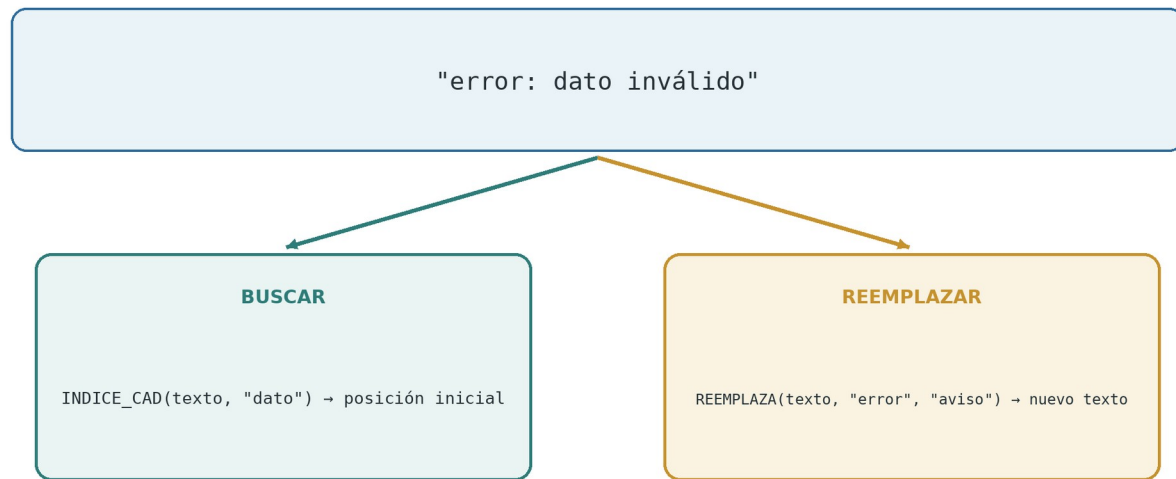
si MINUS(extension) = ".txt"
  escribir "Archivo de texto"
```

Esta solución funciona únicamente si el formato garantiza una extensión de cuatro caracteres. Para nombres generales conviene localizar primero el último punto o emplear una función específica de búsqueda.

Caso límite: ``archivo`` no contiene punto; ``datos.tar.gz`` contiene más de uno. El algoritmo debe decidir qué significado se asigna a cada caso.

10.8 Buscar texto dentro de texto

Buscar una subcadena permite saber si aparece un fragmento y, cuando la función lo permite, en qué posición comienza.

Buscar localiza; reemplazar produce una nueva representación

La cadena original puede conservarse; asigna el resultado cuando quieras actualizar el estado.

Figura 10.5. Buscar y reemplazar fragmentos. Elaboración propia.

Buscar una palabra

```

mensaje <- "error: dato inválido"
posicion <- INDICE_CAD(mensaje, "dato")

si posicion >= 0
  escribir "La palabra comienza en: " + posicion

```

Cuando no existe coincidencia, las funciones de búsqueda suelen devolver un valor especial. Confirma el valor utilizado por la biblioteca antes de construir la condición.

Idea clave: buscar responde una pregunta sobre la cadena; no modifica su contenido.

10.9 Reemplazar fragmentos

Reemplazar toma una cadena, localiza un patrón y produce una representación con otro fragmento en su lugar.

Reemplazo y asignación

```

mensaje <- "error: dato inválido"
corregido <- REEMPLAZA(mensaje, "error", "aviso")

escribir mensaje
escribir corregido

```

La cadena original puede permanecer sin cambios. Para actualizar el estado de `mensaje`, asigna el resultado a la misma variable: `mensaje <- REEMPLAZA(...)`.

Advertencia: reemplazar todas las coincidencias puede modificar fragmentos dentro de palabras más largas. Define con precisión qué patrón buscas.

10.10 Dividir una cadena en partes

Dividir transforma una cadena estructurada en un arreglo de campos. El separador indica dónde termina una parte y comienza la siguiente.

Dividir transforma texto en arreglo; unir realiza el recorrido inverso

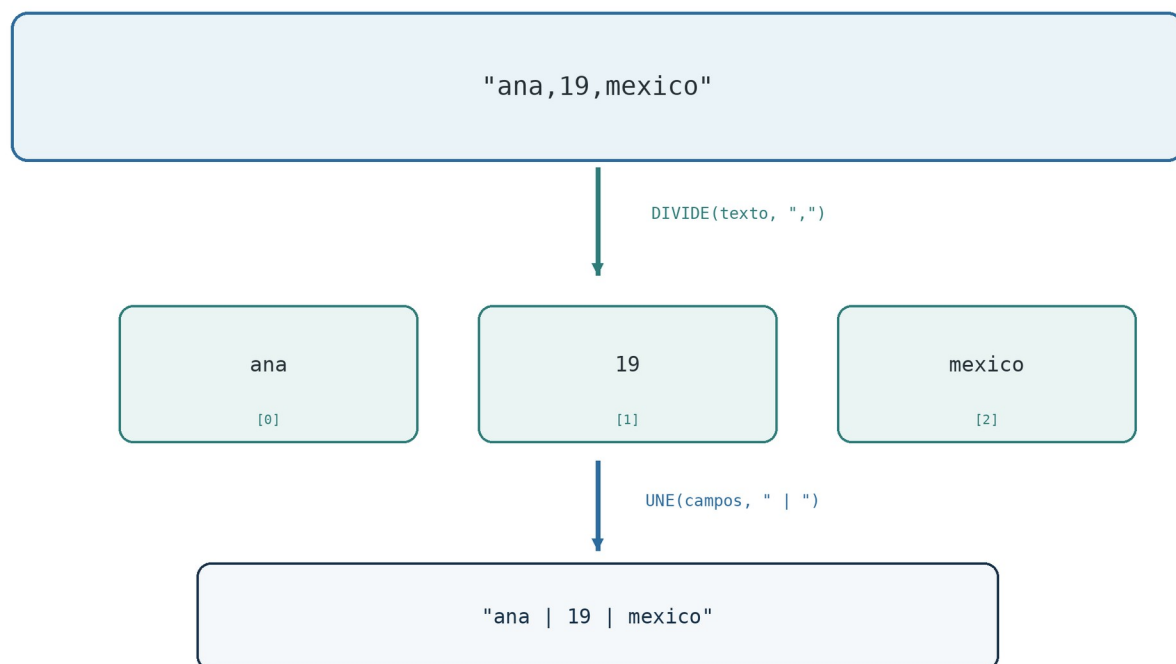


Figura 10.6. División y unión de cadenas. Elaboración propia.

Separar campos

```

línea <- "ana,19,mexico"
campos <- DIVIDE(línea, ", ")

escribir campos[0]
escribir campos[1]
escribir campos[2]

```

Después de dividir, el programa debe validar cuántos campos obtuvo. Una línea incompleta o un separador dentro de un valor puede romper la interpretación esperada.

Importante: dividir no comprende por sí mismo el significado de cada campo. Solo produce partes según un separador. El programa debe interpretarlas y convertirlas cuando sea necesario.

10.11 Unir elementos en una cadena

Unir realiza la transformación inversa: toma un arreglo de cadenas y coloca un separador entre sus elementos.

Construir una línea

```
campos[0] <- "ana"
campos[1] <- "19"
campos[2] <- "mexico"

línea <- UNE(campos, ",")
escribir línea
```

La unión es útil para presentar listas, construir registros simples o preparar texto para almacenamiento. No agrega automáticamente comillas, escapes ni reglas de formatos complejos.

10.12 Texto y conversión de tipos

Los datos obtenidos al dividir una cadena siguen siendo texto. Si un campo representa un número, debe comprobarse y convertirse antes de utilizarse en cálculos.

Interpretar un campo numérico

```
línea <- "producto,3,49.90"
campos <- DIVIDE(línea, ",")

si ES_NUM(campos[1]) y ES_NUM(campos[2])
  cantidad <- A_NUM(campos[1])
  precio <- A_NUM(campos[2])
  total <- cantidad * precio
  escribir total
sino
  escribir "Registro inválido"
```

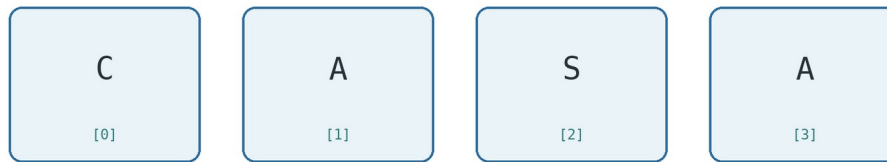
Idea clave: la estructura textual y el tipo conceptual no son lo mismo. ``3`` representa caracteres hasta que se convierte en un valor numérico.

10.13 Contar frecuencias

Un análisis de frecuencia cuenta cuántas veces aparece cada carácter o palabra. El problema combina recorrido, comparación, contadores y colecciones.

Contar frecuencias combina texto, arreglos y recorridos

```
texto = "CASA"
```



carácter	frecuencia
A	2
C	1
S	1

Cada carácter se convierte en una categoría; el conteo actualiza el estado asociado con esa categoría.

Figura 10.7. Frecuencias de caracteres. Elaboración propia.

Frecuencia de una letra

```
CONTAR_CARACTER(texto, objetivo)
  cantidad <- 0
  texto <- MAYUS(texto)
  objetivo <- MAYUS(objetivo)

  para i <- 0 hasta LONG(texto) - 1
    si texto[i] = objetivo
      cantidad <- cantidad + 1

  retornar cantidad
```

Contar todos los caracteres distintos requiere además una estructura para recordar cuáles ya fueron procesados y su frecuencia. Ese problema se retomará en ejercicios y en cursos posteriores con estructuras especializadas.

10.14 Buscar palabras y contar apariciones

Para analizar palabras, primero se normaliza y divide el texto. Después se recorre el arreglo resultante.

Contar una palabra

```

CONTAR_PALABRA(texto, objetivo)
  texto <- MINUS(RECORTA(texto))
  objetivo <- MINUS(RECORTA(objetivo))
  palabras <- DIVIDE(texto, " ")
  cantidad <- 0

  para i <- 0 hasta LONG(palabras) - 1
    si palabras[i] = objetivo
      cantidad <- cantidad + 1

  retornar cantidad

```

Esta versión es pedagógica y supone palabras separadas por un único espacio. Signos de puntuación, saltos de línea y espacios repetidos requieren una normalización más completa.

Contraejemplo: en ``"casa, casa"`, dividir solo por espacios produce ``"casa,"` y ``"casa"`. La coma impide que ambas partes sean iguales.

10.15 Eliminar duplicados de una lista textual

Eliminar duplicados exige conservar el primer ejemplar de cada valor y omitir los siguientes. La comparación debe realizarse sobre una representación normalizada si el problema considera equivalentes distintas formas.

Idea de eliminación de duplicados

```

SIN_DUPLICADOS(valores[])
  unicos[0] <- valores[0]

  para i <- 1 hasta LONG(valores) - 1
    candidato <- MINUS(RECORTA(valores[i]))
    encontrado <- FALSO

    para j <- 0 hasta LONG(unicos) - 1
      si MINUS(RECORTA(unicos[j])) = candidato
        encontrado <- VERDAD

    si no encontrado
      AGREGA(unicos, valores[i])

  retornar unicos

```

La versión supone al menos un elemento. Una implementación completa debe tratar el arreglo vacío.

La solución utiliza ciclos anidados: por cada candidato revisa los valores ya conservados. Su costo puede crecer cuadráticamente.

10.16 Invertir texto

Invertir una cadena permite practicar recorridos descendentes y construcción progresiva de texto.

Construir una cadena invertida

```

INVERTIR_TEXTO(texto)
  invertido <- ""

  para i <- LONG(texto) - 1 bajando 0
    invertido <- invertido + texto[i]

  retornar invertido

```

Para ``CLRS``, el recorrido visita S, R, L y C. Cada carácter se concatena al resultado parcial.

Pausa y piensa: completa una tabla de estado con las columnas ``i``, ``texto[i]`` e ``invertido`` para la entrada ``SOL``.

10.17 Palíndromos

Un palíndromo se lee igual de izquierda a derecha y de derecha a izquierda. La comparación debe definir primero qué diferencias se ignorarán.

Comprobación mediante inversión

```

ES_PALINDROMO(texto)
  normalizado <- MINUS(RECORTA(texto))
  retornar normalizado = INVERTIR_TEXTO(normalizado)

```

Esta versión no elimina espacios internos ni signos. Por ello reconoce ``radar``, pero no necesariamente ``Anita lava la tina``. Una versión más amplia debe filtrar caracteres antes de comparar.

Importante: la definición operacional forma parte del algoritmo. “Ignorar espacios y puntuación” debe expresarse mediante instrucciones concretas.

10.18 Ordenar texto y colecciones de palabras

Ordenar palabras puede facilitar búsquedas, presentación y detección de duplicados. La biblioteca puede ordenar arreglos, pero el resultado depende de la forma exacta de las cadenas.

Ordenar palabras normalizadas

```

texto <- "Pera manzana uva"
palabras <- DIVIDE(MINUS(texto), " ")
ORDENA(palabras)

escribir UNE(palabras, " ")

```

Normalizar antes de ordenar evita que las mayúsculas introduzcan diferencias inesperadas. Sin embargo, las reglas lingüísticas completas pueden depender del idioma y no reducirse al orden interno de caracteres.

10.19 Laboratorio CLRS: analizador elemental de texto

Construiremos un programa que reciba una frase y produzca longitud, cantidad de palabras, vocales y una versión normalizada.

Analizador de texto

```

CONTAR_VOCALES(texto)
  cantidad <- 0
  texto <- MAYUS(texto)

  para i <- 0 hasta LONG(texto) - 1
    c <- texto[i]
    si c = "A" o c = "E" o c = "I" o c = "O" o c = "U"
      cantidad <- cantidad + 1

  retornar cantidad

PRINCIPAL()
  escribir "Introduce una frase"
  leer frase

  normalizada <- RECORTA(frase)
  palabras <- DIVIDE(normalizada, " ")

  escribir "Caracteres: " + LONG(normalizada)
  escribir "Palabras: " + LONG(palabras)
  escribir "Vocales: " + CONTAR_VOCALES(normalizada)
  escribir "Mayúsculas: " + MAYUS(normalizada)

```

El conteo de palabras funciona correctamente solo bajo la suposición de un espacio entre palabras. Prueba entradas con espacios repetidos y registra el comportamiento antes de proponer una mejora.

Laboratorio: ejecuta el programa con ``"hola mundo"`, `` hola mundo ``, `"hola mundo"``` y una cadena vacía. Construye una tabla de resultados observados.

10.20 Caso longitudinal: categorías y conceptos de gastos

El registro de gastos puede almacenar conceptos y categorías como cadenas. Para generar resúmenes confiables, conviene normalizarlas antes de comparar.

Normalizar una categoría

```

NORMALIZAR_CATEGORIA(categoria)
    retornar MAYUS(RECORTA(categoria))

PRINCIPAL()
    categorias[0] <- " alimentos "
    categorias[1] <- "ALIMENTOS"
    categorias[2] <- "Transporte"

    para i <- 0 hasta LONG(categorias) - 1
        categorias[i] <- NORMALIZAR_CATEGORIA(categorias[i])

    escribir UNE(categorias, ", ")

```

Después de normalizar, las dos primeras categorías pueden tratarse como equivalentes. El siguiente paso sería contar frecuencias o acumular montos por categoría.

Decisión de diseño: guarda por separado la forma original cuando necesites presentarla exactamente como fue escrita por la persona.

10.21 Errores frecuentes al procesar texto

Error	Consecuencia	Revisión
Acceder sin comprobar longitud	Índice inválido en cadenas vacías.	Validar <code>LONG(texto) > 0</code> .
Comparar sin normalizar	Equivalencias humanas aparecen como distintas.	Definir reglas de normalización.
Normalizar de más	Se pierde información significativa.	Conservar original y justificar cambios.
Suponer un separador perfecto	Campos incompletos o desalineados.	Validar cantidad y contenido.
Usar texto numérico en cálculos	Conversión implícita o resultado inesperado.	Comprobar y convertir.
Ignorar puntuación	Conteos de palabras incorrectos.	Limpiar o tokenizar según reglas.
No asignar un reemplazo	El estado original permanece igual.	Guardar el valor producido.
Confundir índice con longitud	Acceso fuera del final.	Usar <code>LONG(texto) - 1</code> .

10.22 Crecimiento del trabajo en algoritmos de texto

Recorrer una cadena de longitud n una vez produce trabajo lineal, $O(n)$. Buscar cada carácter dentro de otra colección o comparar cada palabra con todas las anteriores puede generar ciclos anidados y aproximarse a $O(n^2)$.

Operación conceptual	Patrón	Costo intuitivo
Obtener longitud disponible por biblioteca	Consulta directa	$O(1)$ o dependiente de implementación.
Contar vocales	Un recorrido	$O(n)$.
Invertir texto	Un recorrido	$O(n)$.
Comparar cada elemento con los anteriores	Ciclos anidados	$O(n^2)$.
Dividir texto	Recorrer y crear partes	Aproximadamente $O(n)$.

Límite del análisis: estas clasificaciones expresan crecimiento general. El comportamiento concreto de las funciones de biblioteca depende de su implementación.

10.23 Reconstruye el modelo mental

El texto puede estudiarse como valor completo, secuencia de caracteres o conjunto de fragmentos. Las funciones de cadena permiten cambiar de una representación a otra: normalizar, extraer, buscar, reemplazar, dividir y unir. Los algoritmos combinan estas transformaciones con recorridos, decisiones, arreglos y funciones.

1. Una cadena tiene longitud e índices válidos desde 0 hasta longitud menos uno.
2. Recorrer permite examinar cada carácter.
3. Normalizar reduce diferencias irrelevantes definidas por el problema.
4. Las subcadenas representan regiones continuas.
5. Buscar localiza; reemplazar produce una nueva representación.
6. Dividir produce un arreglo de campos.
7. Unir produce una cadena a partir de una colección.
8. Los campos numéricos siguen siendo texto hasta comprobarse y convertirse.
9. Contar frecuencias combina recorridos, comparaciones y acumuladores.
10. Los casos límite incluyen cadenas vacías, separadores repetidos y puntuación.

Comprobación activa: explica cómo transformarías `" Ana,19,Mexico "` en tres campos limpios y cómo convertirías la edad en número.

Ejercicios

1. Para `"PSEUDOCODIGO"`, determina longitud, último índice y caracteres en 0, 5 y 11. **[Reconocer]**
2. Escribe un recorrido que muestre cada carácter junto con su índice. **[Aplicar]**
3. Construye una función que cuente espacios. **[Construir]**
4. Normaliza un nombre eliminando espacios exteriores y convirtiéndolo a mayúsculas. **[Aplicar]**
5. Explica por qué no conviene normalizar una contraseña. **[Analizar]**
6. Extrae los cuatro últimos caracteres de un nombre de archivo. **[Aplicar]**
7. Diseña casos de prueba para la extracción de extensiones. **[Probar]**

8. Busca una palabra y muestra su posición o un mensaje de ausencia. **[Construir]**
9. Reemplaza guiones por espacios y conserva el texto original. **[Aplicar]**
10. Divide ``rojo,verde,azul`` y recorre el arreglo resultante. **[Aplicar]**
11. Une un arreglo de nombres con `"; "`. **[Aplicar]**
12. Interpreta ``producto,4,25.5`` y calcula el total. **[Construir]**
13. Cuenta cuántas veces aparece una letra sin distinguir mayúsculas. **[Construir]**
14. Cuenta una palabra en una frase y documenta las limitaciones por puntuación. **[Analizar]**
15. Invierte una cadena y completa su traza. **[Aplicar]**
16. Amplía `ES\_PALINDROMO` para ignorar espacios. **[Construir]**
17. Elimina duplicados de una lista de categorías normalizadas. **[Construir]**
18. Ordena una lista de palabras después de normalizarla. **[Aplicar]**
19. Compara el costo de contar vocales y eliminar duplicados con ciclos anidados. **[Analizar]**
20. Genera un resumen de frecuencias de categorías de gasto. **[Integrar]**

Mini proyecto: analizador de palabras

Situación: construye un programa que reciba un texto y muestre una versión normalizada, cantidad de caracteres, palabras, vocales, palabra buscada, texto invertido y lista de palabras ordenadas.

Requisitos:

- Conservar el texto original.
- Crear funciones separadas para normalización, conteo de vocales e inversión.
- Dividir el texto en palabras y validar el arreglo obtenido.
- Permitir buscar una palabra sin distinguir mayúsculas.
- Ordenar las palabras normalizadas.
- Diseñar casos normales, vacíos, con espacios repetidos y puntuación.
- Generar el diagrama de flujo y explicar los recorridos principales.

Criterio de éxito: el programa debe explicar mediante sus funciones qué transformación se aplica en cada etapa y debe reconocer explícitamente sus limitaciones.

Autoevaluación

Responde sin consultar el capítulo.

Afirmación	Sí	Todavía no
Puedo relacionar longitud con índices válidos.	☐	☐
Puedo recorrer una cadena carácter por carácter.	☐	☐

Afirmación	Sí	Todavía no
Sé cuándo normalizar y cuándo conservar el original.	☐	☐
Puedo extraer una subcadena sin exceder límites.	☐	☐
Distingo buscar de reemplazar.	☐	☐
Puedo dividir una cadena y validar sus campos.	☐	☐
Puedo unir un arreglo de cadenas.	☐	☐
Comprendo que los campos numéricos siguen siendo texto.	☐	☐
Puedo construir conteos e inversiones.	☐	☐
Puedo identificar casos límite y costos básicos.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas transformar una entrada textual cruda en una estructura validada, recorrerla y producir un resultado explicable mediante funciones.

Glosario

Término	Definición de trabajo
Cadena	Secuencia ordenada de caracteres tratada también como un valor completo.
Carácter	Unidad individual localizada mediante un índice.
Normalización	Transformación hacia una representación común según reglas del problema.
Subcadena	Región continua extraída de una cadena.
Prefijo	Fragmento situado al inicio.
Sufijo	Fragmento situado al final.
Patrón	Texto que se desea localizar o reemplazar.
Separador	Fragmento que delimita partes al dividir.
Campo	Parte interpretada de una cadena estructurada.
Tokenización	Separación del texto en unidades analizables.
Frecuencia	Cantidad de apariciones de un valor.
Palíndromo	Texto que coincide con su versión invertida bajo reglas definidas.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación y código fuente de las funciones estándar para cadenas y arreglos [Repositorio de software].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Gaddis, T. (2022). Starting out with programming logic and design (6th ed.). Pearson.

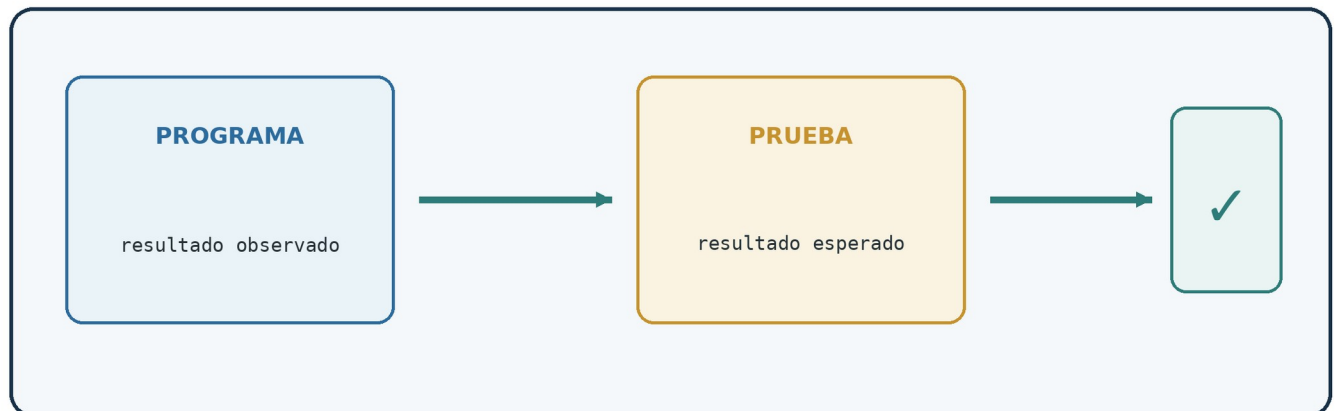
Continuidad: los arreglos y las cadenas ya permiten representar información compleja. El capítulo 11 se centrará en especificar, probar, diagnosticar y mejorar programas completos.

CAPÍTULO

11

Saber si un programa realmente funciona

Especificar, probar, diagnosticar y mejorar soluciones



Ejecutar sin errores no demuestra que el problema esté resuelto.

Saber si un programa realmente funciona

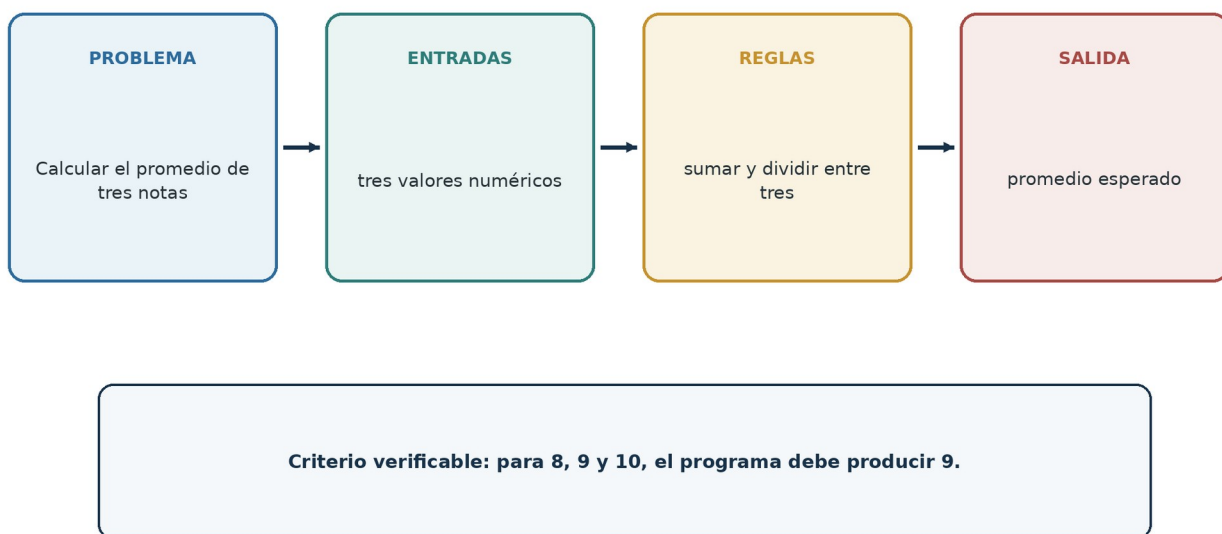
¿Cómo se pasa de “parece funcionar” a una comprobación sistemática?

Un programa puede ejecutarse sin mostrar mensajes de error y, aun así, calcular mal un promedio, aceptar datos imposibles o ignorar un caso límite. También puede fallar de manera evidente, pero dejar un mensaje que no explica por sí solo la causa. La corrección no se obtiene observando una sola ejecución favorable: se construye relacionando una especificación, casos de prueba, resultados esperados y evidencia obtenida durante la ejecución.

En este capítulo el código deja de verse únicamente como una solución que debe escribirse. Se convierte en un objeto que debe poder explicarse, someterse a pruebas, diagnosticar y mejorar. Las herramientas de CLRS Kit ayudan a localizar errores y visualizar recorridos, pero la interpretación sigue dependiendo de comprender qué debía hacer el programa.

Promesa de aprendizaje: al terminar podrás formular una especificación verificable, diseñar casos normales, límite e inválidos, distinguir errores sintácticos, semánticos, de ejecución y lógicos, seguir un proceso de depuración, usar diagnósticos y diagramas como evidencia, y revisar la calidad de un programa.

Una especificación convierte una intención en criterios comprobables



Sin un resultado esperado, una ejecución solo muestra lo que ocurrió, no si ocurrió lo correcto.

Figura 11.1. De la intención al criterio verificable. Elaboración propia.

11.1 Ejecutarse no es lo mismo que funcionar correctamente

Decir que un programa “funciona” puede significar varias cosas diferentes: que inicia, que no se detiene con un error, que produce una salida o que realmente satisface las reglas del problema. Solo la última interpretación permite hablar de corrección respecto de una especificación.

Afirmación	Lo que demuestra	Lo que todavía no demuestra
El archivo se ejecuta	La sintaxis y la estructura permiten iniciar.	Que la lógica sea correcta.
No aparece un error	No ocurrió una falla detectada en esa ejecución.	Que todos los caminos sean seguros.
Produce un número	Existe una salida observable.	Que sea el número esperado.
Coincide en un ejemplo	Ese caso concreto fue resuelto.	Que funcione para límites y datos inválidos.
Supera casos diseñados desde la especificación	Existe evidencia organizada.	Una garantía absoluta para toda entrada posible.

Idea clave: la prueba no demuestra que un programa sea perfecto; aporta evidencia de que se comporta como se espera en situaciones elegidas deliberadamente.

11.2 Especificar antes de probar

Una prueba necesita una expectativa. Para obtenerla, el problema debe describirse con suficiente precisión. Una especificación básica identifica entradas, salidas, reglas, restricciones y casos especiales.

Elemento	Pregunta	Ejemplo: promedio de tres notas
Entradas	¿Qué recibe el programa?	Tres valores numéricos.
Salida	¿Qué debe producir?	La media aritmética.
Regla	¿Cómo se obtiene?	Suma de las notas dividida entre tres.
Restricción	¿Qué valores son aceptables?	Notas entre 0 y 10.
Caso especial	¿Qué ocurre con un dato inválido?	Se rechaza y se informa la causa.

La especificación no necesita comenzar como un documento extenso. Sin embargo, debe permitir que otra persona determine si una ejecución es correcta sin adivinar la intención del autor.

Pausa y piensa: “calcular un descuento” todavía es ambiguo. ¿El porcentaje se expresa como 15 o como 0.15? ¿Puede ser mayor que 100 %? ¿El precio puede ser negativo? Convierte esas dudas en reglas explícitas.

11.3 Precondiciones y resultados esperados

Una precondición describe lo que debe cumplirse antes de ejecutar una operación. Un resultado esperado describe lo que debe observarse después cuando las precondiciones se cumplen.

Contrato de una función

```
CALCULAR_DESCUENTO(precio, porcentaje)
  retornar precio * porcentaje
```

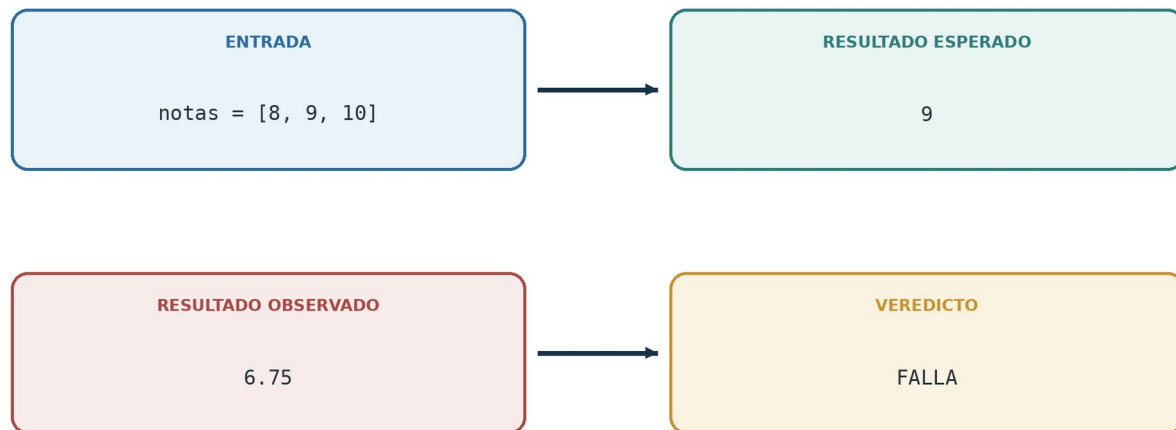
Precondiciones posibles: $\text{precio} \geq 0$ y $0 \leq \text{porcentaje} \leq 1$. Resultado esperado: monto no negativo y no mayor que el precio.

Las precondiciones ayudan a separar dos preguntas: ¿la función calcula bien con datos válidos? y ¿el programa controla correctamente los datos inválidos? Ambas son importantes, pero no son la misma prueba.

Importante: un resultado inesperado puede deberse a una lógica incorrecta o a que la prueba proporcionó datos fuera del dominio sin definir qué debía ocurrir.

11.4 Anatomía de un caso de prueba

Un caso de prueba relaciona una situación con una expectativa



La diferencia entre 9 y 6.75 orienta la investigación: el divisor usado no coincide con la especificación.

Una prueba útil registra evidencia, no solo la frase “no funciona”.

Figura 11.2. Componentes de un caso de prueba. Elaboración propia.

Un caso de prueba debe poder repetirse. Por ello conviene registrar al menos la entrada, el resultado esperado, el resultado observado y el veredicto. También puede incluir una breve razón que explique qué aspecto verifica.

ID	Propósito	Entrada	Esperado	Observado	Resultado
P-01	Caso normal	8, 9, 10	9	9	Pasa
P-02	Límite inferior	0, 0, 0	0	0	Pasa
P-03	Valor inválido	-1, 8, 9	Rechazo	-1.33	Falla

Idea clave: un caso de prueba no es solo una entrada. Es una afirmación acerca de lo que debe ocurrir con esa entrada.

11.5 Casos normales, límite e inválidos

Categoría	Propósito	Ejemplo
Normal	Representar un uso frecuente dentro del dominio.	Notas 7, 8 y 9.
Límite	Examinar extremos y cambios de comportamiento.	0, 10 y valores cercanos.
Inválido	Comprobar rechazo o manejo definido.	Cadena, nota negativa o mayor que 10.
Especial	Revisar una regla poco frecuente pero permitida.	Todas las notas iguales.
Regresión	Confirmar que una corrección anterior no se perdió.	Caso que antes reveló un defecto.

Los casos normales muestran que el recorrido común funciona. Los límites y los datos inválidos suelen descubrir defectos que permanecen ocultos en el centro del rango.

Los límites concentran errores que los valores centrales no revelan



Rango permitido: $0 \leq \text{edad} \leq 120$ **Casos sugeridos:** -1, 0, 1, 119, 120 y 121

Figura 11.3. Selección de valores alrededor de un límite. Elaboración propia.

11.6 Probar alrededor de los límites

Cuando una regla cambia en un punto, conviene probar el valor inmediatamente anterior, el valor límite y el inmediatamente posterior. Esta estrategia revela comparaciones incorrectas como usar `<` cuando se necesitaba `<=`.

Clasificación con un posible error de frontera

```
si edad > 18
    escribir "Acceso permitido"
sino
    escribir "Acceso restringido"
```

Si la regla permite acceso desde los 18 años, el operador correcto debe incluir el límite.

Edad	Esperado según la regla	Resultado del código	Conclusión
17	Restringido	Restringido	Correcto
18	Permitido	Restringido	Falla de frontera
19	Permitido	Permitido	Correcto

Pausa y piensa: para un arreglo de longitud 5, selecciona pruebas para el primer índice, el último índice válido y el primer índice inválido.

11.7 Probar caminos de decisión

Una estructura condicional crea caminos alternativos. Probar únicamente un camino deja los demás sin observar. La cobertura de ramas consiste, en este nivel, en diseñar al menos un caso que haga verdadera y otro que haga falsa cada decisión relevante.

Clasificación de saldo

```
si saldo < 0
    escribir "Deuda"
sino si saldo = 0
    escribir "Sin saldo"
sino
    escribir "Disponible"
```

Caso	Saldo	Rama esperada
R-01	-1	Deuda
R-02	0	Sin saldo
R-03	1	Disponible

No basta con contar líneas ejecutadas. Lo importante es que cada comportamiento definido por la especificación haya sido observado al menos una vez.

Advertencia: cubrir todas las ramas no garantiza que todas las expresiones, combinaciones y estados hayan sido comprobados. Es un criterio útil, no una prueba total.

11.8 Probar ciclos y colecciones

Los ciclos requieren casos que revelen su inicio, progreso y terminación. Para un recorrido de arreglo, una colección vacía, una de un elemento y otra con varios elementos producen evidencias distintas.

Situación	Qué permite observar
Arreglo vacío	Si el ciclo evita accesos inexistentes y si el resultado inicial tiene sentido.
Un elemento	Si los límites incluyen exactamente la única posición.
Varios elementos	Si la actualización se repite y acumula correctamente.
Elemento buscado al inicio	Si la búsqueda reconoce la primera posición.
Elemento buscado al final	Si el último índice se procesa.
Elemento ausente	Si la terminación y el resultado de ausencia son correctos.

Máximo con una inicialización defectuosa

```
MAXIMO(valores[])
  mayor <- 0
  para i <- 0 hasta LONG(valores) - 1
    si valores[i] > mayor
      mayor <- valores[i]
  retornar mayor
```

Con [-8, -3, -10], la función retorna 0, aunque 0 no pertenece al arreglo. El caso con todos los valores negativos revela el defecto.

Una corrección posible consiste en inicializar `mayor` con el primer elemento y recorrer desde el índice 1, después de validar que el arreglo no esté vacío.

11.9 Cuatro familias de errores

No todos los errores aparecen en el mismo momento



Un mensaje de diagnóstico puede señalar la ubicación, pero comprender la causa requiere relacionarlo con la intención del programador

Figura 11.4. Momento de aparición de distintos errores. Elaboración propia.

Clasificar el error orienta la investigación. Los errores de sintaxis y muchos errores semánticos pueden detectarse antes de ejecutar. Los errores de ejecución aparecen durante un recorrido concreto. Los errores lógicos requieren comparar el comportamiento con la especificación.

Tipo	Ejemplo conceptual	Pregunta útil
Sintaxis	Falta un paréntesis o la indentación no forma el bloque esperado.	¿Qué estructura esperaba el analizador?
Semántico	Se invoca una función inexistente o se usa una referencia no válida.	¿Qué nombre o relación no puede resolverse?
Ejecución	Una función de biblioteca o <code>`LANZAR_ERROR`</code> detiene el programa.	¿Qué estado y entrada condujeron a la falla?
Lógico	El promedio divide entre cuatro cuando existen tres notas.	¿En qué difiere el observado del esperado?

11.10 Leer diagnósticos sin obedecerlos mecánicamente

CLRS Kit presenta diagnósticos estáticos en línea, mensajes en español y entradas en el panel Problemas. Cuando la ubicación está disponible, el mensaje señala línea y columna. Esta información reduce la región que debe inspeccionarse, pero no reemplaza la interpretación.

Ejemplo de diagnóstico

× Error de sintaxis

Se esperaba «)», pero se encontró «escribir».
Línea 2, columna 5

Estado: No ejecutado

1. Identifica la familia del error.
2. Lee qué se esperaba y qué se encontró.
3. Revisa la ubicación indicada y la instrucción anterior.
4. Relaciona el mensaje con la estructura completa, no solo con el carácter marcado.
5. Corrige una causa probable y vuelve a ejecutar.

Advertencia: el lugar señalado puede ser donde el analizador descubre la inconsistencia, no necesariamente donde comenzó. Un paréntesis faltante en una línea anterior puede hacer que el error aparezca después.

Nota técnica: los detalles internos se ocultan en la salida normal. El modo `CLRS\_DEBUG=1` está destinado al desarrollo de la extensión, no a la resolución ordinaria de ejercicios.

11.11 Diagnósticos estáticos y estado del editor

Los diagnósticos estáticos se actualizan mientras se edita. Esto permite detectar referencias, problemas estructurales o errores léxicos antes de intentar ejecutar. Sin embargo, un editor sin advertencias no implica que el resultado sea correcto.

Señal	Interpretación adecuada
Subrayado y entrada en Problemas	Existe un problema detectado por el análisis disponible.
Sin diagnósticos	No se detectó un problema estático conocido.
Error al ejecutar	El recorrido concreto produjo una falla.
Salida inesperada sin error	Probable defecto lógico o especificación incompleta.

La ausencia de diagnósticos debe dar paso a pruebas, no a una confianza automática. El analizador no intenta inferir todos los tipos ni todos los valores que existirán durante la ejecución.

Idea clave: las herramientas responden preguntas diferentes: el diagnóstico pregunta si detecta una inconsistencia; la prueba pregunta si el comportamiento satisface una expectativa.

11.12 Depurar mediante hipótesis

Depurar es un ciclo de investigación, no una secuencia de cambios al azar

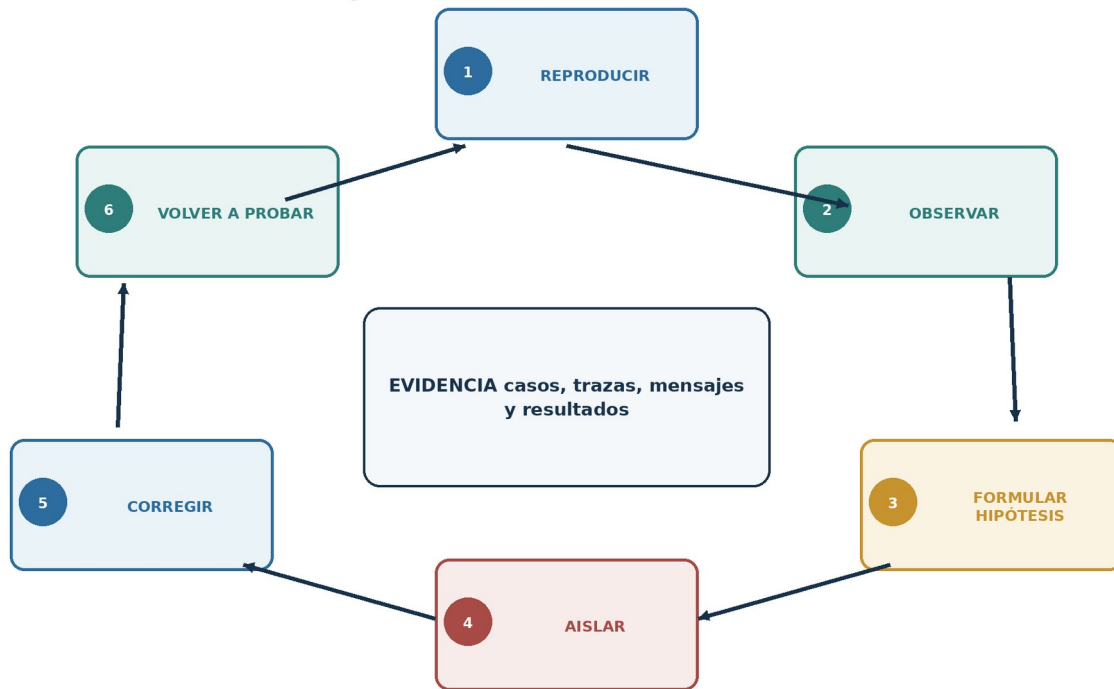


Figura 11.5. Ciclo de depuración basado en evidencia. Elaboración propia.

Cambiar varias líneas al azar puede ocultar la causa original e introducir defectos nuevos. Una depuración disciplinada comienza con un caso reproducible y una diferencia concreta entre lo esperado y lo observado.

Paso	Acción
Reproducir	Conservar una entrada que provoque siempre el problema.
Observar	Registrar salida, estado, mensaje y punto del recorrido.
Hipótesis	Proponer una causa que explique la evidencia.
Aislar	Reducir el programa, la entrada o la función implicada.
Corregir	Aplicar el cambio mínimo coherente con la causa.
Volver a probar	Ejecutar el caso original y los casos relacionados.

11.13 Usar trazas para localizar el primer estado incorrecto

Una salida final equivocada suele originarse varias instrucciones antes. La traza permite comparar estados esperados y observados hasta localizar la primera divergencia.

Promedio ponderado con una regla equivocada

```
total <- 0
total <- total + nota1 * 0.10
total <- total + nota2 * 0.25
total <- total + nota3 * 0.40
total <- total + nota4 * 0.15
escribir total
```

Los pesos suman 0.90. La tercera ponderación debía ser 0.50.

Paso	Valor esperado de total	Valor observado	¿Coinciden?
Inicial	0	0	Sí
Nota 1	0.8	0.8	Sí
Nota 2	3.05	3.05	Sí
Nota 3	7.55	6.65	No
Nota 4	8.75	7.85	No

La primera diferencia aparece al procesar la tercera nota. Esta observación reduce la investigación a una regla concreta en lugar de revisar todo el programa.

Método práctico: detente en la primera divergencia. Los errores posteriores pueden ser consecuencias del mismo defecto.

11.14 Aislar funciones antes de revisar el programa completo

Las funciones permiten probar responsabilidades de manera independiente. Si el total de una compra es incorrecto, puede comprobarse por separado el subtotal, el descuento, el impuesto y la combinación final.

Pruebas directas

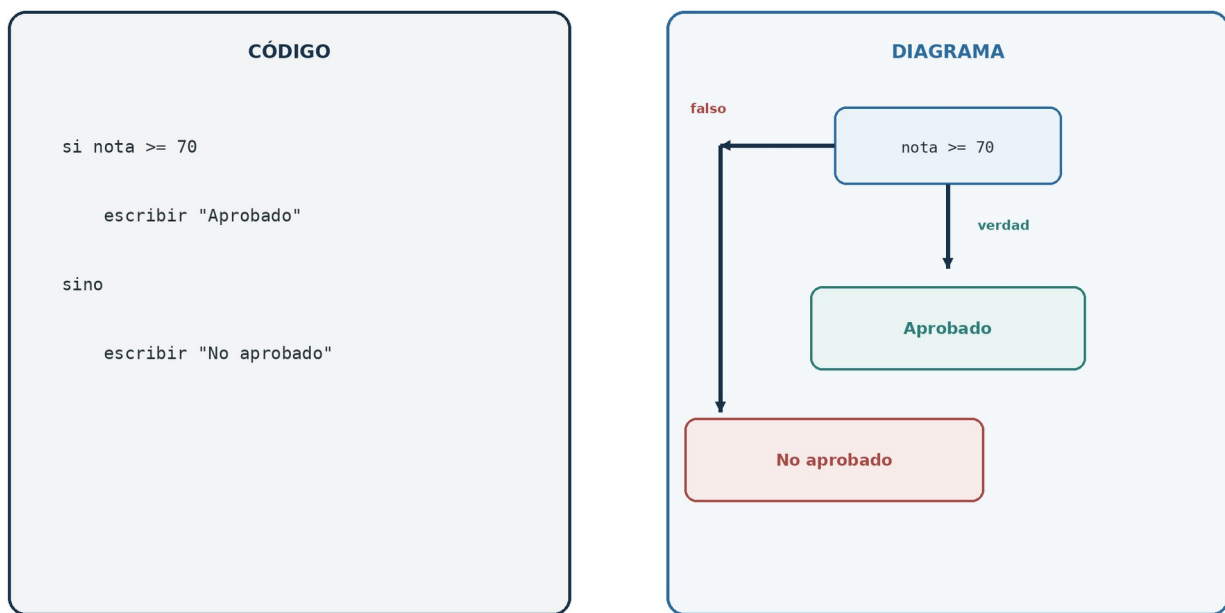
```
PRINCIPAL()
  escribir CALCULAR_SUBTOTAL(100, 3)           // esperado: 300
  escribir CALCULAR_DESCUENTO(300, 0.10)       // esperado: 30
  escribir CALCULAR_IMPUESTO(270, 0.16)        // esperado: 43.2
  escribir CALCULAR_TOTAL(300, 30, 43.2)       // esperado: 313.2
```

Cuando las funciones intermedias producen lo esperado, la causa puede encontrarse en la coordinación, en el orden de los argumentos o en el uso de los retornos. Cuando una función aislada falla, el área de búsqueda se reduce.

Idea clave: la descomposición funcional no solo organiza el código; también crea unidades que pueden especificarse y probarse por separado.

11.15 El diagrama de flujo como instrumento de revisión

Código y diagrama muestran el mismo recorrido desde perspectivas distintas



El diagrama ayuda a revisar caminos; el código conserva las expresiones exactas y los detalles de implementación.

Figura 11.6. Correspondencia entre código y diagrama de flujo. Elaboración propia.

El comando para mostrar el diagrama de flujo genera una representación visual del programa y de sus funciones. La visualización ayuda a revisar decisiones, ciclos y llamadas que pueden resultar difíciles de seguir en un bloque extenso.

- Comprueba que cada condición tenga los caminos previstos.
- Localiza ciclos cuyo regreso no conduce a una actualización visible.
- Revisa si una función queda aislada porque nunca se invoca.
- Compara el orden general del diagrama con la especificación.
- Exporta el SVG cuando necesites documentar la revisión.

Límite de la figura: el diagrama muestra la estructura del recorrido, pero no demuestra que una fórmula, un límite o una condición sean correctos. Debe leerse junto con casos de prueba y trazas.

11.16 Revisar caminos imposibles o faltantes

Una cadena `sino si` se evalúa en orden. Una condición demasiado amplia puede absorber casos que deberían llegar a una rama posterior.

Orden defectuoso de condiciones

```
si calificacion >= 60
  escribir "Aprobado"
sino si calificacion >= 90
  escribir "Excelente"
sino
  escribir "No aprobado"
```

La rama "Excelente" nunca se alcanza: todo valor mayor o igual que 90 ya satisface la primera condición.

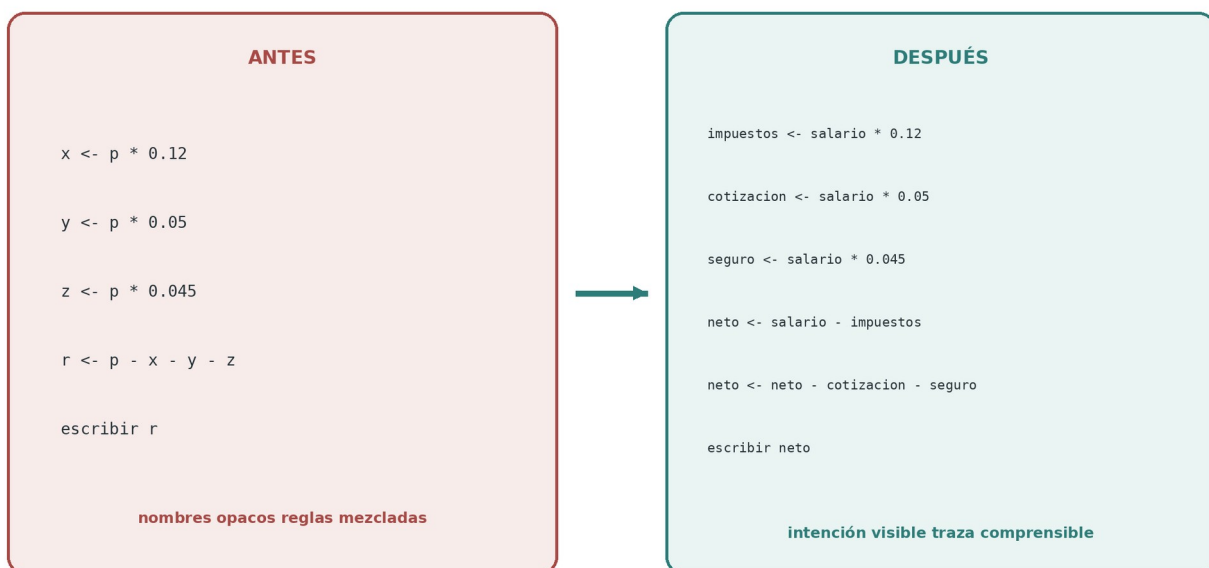
El diagrama permite observar el orden, pero una tabla de casos muestra la consecuencia: para 95, el resultado esperado "Excelente" se convierte en "Aprobado". La corrección consiste en ordenar primero la condición más específica.

Pausa y piensa: diseña tres calificaciones que recorran todas las ramas después de corregir el orden.

11.17 Calidad: hacer visible la intención

La calidad del código no sustituye las pruebas. Sin embargo, un programa legible permite construir mejores hipótesis, seguir estados y reconocer reglas. La claridad reduce el costo humano de comprobar y modificar.

Mejorar calidad reduce la dificultad de comprobar y modificar



La calidad no garantiza corrección, pero hace más probable descubrir y corregir errores.

Figura 11.7. Refactorización orientada a la comprensión. Elaboración propia.

Práctica	Beneficio para la comprobación
Nombres significativos	Permiten relacionar variables y funciones con la especificación.
Funciones pequeñas y cohesivas	Facilitan pruebas aisladas y localización de causas.
Evitar duplicación	Impide que una regla tenga versiones inconsistentes.
Condiciones simples	Hace visibles los caminos y los límites.
Comentarios útiles	Explican decisiones o restricciones no evidentes.
Eliminar código innecesario	Reduce estados y caminos que deben analizarse.

11.18 Comentarios que ayudan y comentarios que estorban

Un comentario útil explica una decisión, una precondition, una limitación o una razón que no puede expresarse claramente con nombres y estructura. Un comentario que repite la instrucción añade ruido y puede quedar desactualizado.

Comentario	Evaluación
<code>`// suma uno a i` junto a `i <- i + 1`</code>	Repite lo visible; aporta poco.
<code>`// -1 termina la captura y no se registra como gasto`</code>	Explica el significado del centinela.
<code>`// usar 0.50 porque la tercera nota vale 50 %`</code>	Documenta una regla del problema.
<code>`// corregido`</code>	No explica qué cambió ni por qué.

Criterio: prefiere que el código diga qué hace y reserva el comentario para explicar por qué se tomó una decisión.

11.19 Simplificar condiciones sin cambiar su significado

Las condiciones duplicadas o contradictorias dificultan la cobertura de ramas. Antes de simplificar, utiliza una tabla de casos para asegurar que el comportamiento permanezca igual.

Condición redundante
<pre> si edad >= 18 y edad > 15 escribir "Mayor de edad" Si edad >= 18, entonces edad > 15 ya se cumple. La segunda comparación no añade un caso nuevo.</pre>

Versión más clara

```

si edad >= 18
    escribir "Mayor de edad"

```

Después de refactorizar, deben volver a ejecutarse los casos de frontera 17, 18 y 19. Una mejora estructural también puede introducir errores y necesita verificación.

11.20 Volver a probar después de una corrección

Corregir el caso que reveló un defecto es necesario, pero insuficiente. El cambio puede afectar comportamientos que antes eran correctos. Las pruebas de regresión repiten casos anteriores para comprobar que la solución no retrocedió.

Después de corregir...	Vuelve a probar...
Un límite de edad	Valores por debajo, en y por encima del límite.
La inicialización del máximo	Arreglos positivos, negativos, de un elemento y vacíos.
El orden de una cadena `sino si`	Un caso para cada rama.
Una fórmula de descuento	Porcentaje cero, normal y máximo.
Una función compartida	Todos los programas o funciones que la invocan.

Idea clave: cada defecto descubierto debe convertirse en un caso que pueda repetirse. Así, el error deja de ser solo un incidente y se transforma en conocimiento comprobable.

11.21 Laboratorio CLRS: comprobar el registro de gastos

El registro de gastos ya contiene validación, ciclos, funciones, arreglos y texto. Ahora se convertirá en un objeto de prueba.

Regla	Caso	Resultado esperado
Un gasto debe ser positivo	-20	Rechazo con mensaje.
Un gasto no debe superar el saldo	saldo 100, gasto 120	Rechazo; saldo permanece 100.
El centinela no se registra	-1	Termina sin modificar arreglos.
Un gasto válido reduce el saldo	saldo 100, gasto 40	Saldo 60 y movimiento agregado.
El total coincide con los gastos	20, 30, 10	Total 60.
La categoría se normaliza	" alimentos "	Categoría comparable con "ALIMENTOS".

Función sometida a prueba

```
ES_GASTO_VALIDO(gasto, saldo)
    retornar ES_NUM(gasto) y gasto > 0 y gasto <= saldo
```

Prueba primero la función de validación con valores concretos. Después prueba la coordinación: confirma que el programa no modifique el saldo ni agregue elementos cuando la validación sea falsa.

Laboratorio: construye una tabla con entrada, saldo inicial, longitud inicial del arreglo, resultado esperado, saldo observado y longitud observada. Ejecuta al menos seis casos.

11.22 Investigación guiada de un defecto lógico

Programa con una falla

```
PROMEDIO(notas[])
    total <- 0
    para i <- 0 hasta LONG(notas) - 1
        total <- total + notas[i]
    retornar total / 4

PRINCIPAL()
    notas[0] <- 8
    notas[1] <- 9
    notas[2] <- 10
    escribir PROMEDIO(notas)
```

6. Especifica el resultado esperado.
7. Ejecuta y registra el resultado observado.
8. Traza `total` durante el ciclo.
9. Localiza la primera operación que no coincide con la regla.
10. Formula la corrección mínima.
11. Agrega casos con uno, tres y cinco elementos.
12. Refactoriza para dividir entre `LONG(notas)`.

Conclusión esperada: el ciclo acumula correctamente; el defecto se encuentra en una suposición fija sobre la longitud. La colección debe determinar el divisor.

11.23 Lista de revisión antes de considerar terminado un programa

Área	Preguntas
Especificación	¿Entradas, salidas, reglas y restricciones están claras?
Casos	¿Existen pruebas normales, límite, inválidas y especiales?

Área	Preguntas
Decisiones	¿Cada rama relevante se ejecutó?
Ciclos	¿Se probaron cero, una y varias iteraciones cuando aplica?
Funciones	¿Se probaron de forma aislada y con sus precondiciones?
Diagnósticos	¿El panel Problemas está limpio y los mensajes fueron comprendidos?
Resultados	¿Se compararon esperado y observado?
Regresión	¿Se repitieron los casos anteriores después de cambios?
Calidad	¿Los nombres, funciones y condiciones expresan intención?
Documentación	¿El diagrama y los comentarios explican decisiones reales?

11.24 Reconstruye el modelo mental

Comprobar un programa comienza antes de ejecutarlo. La especificación define lo esperado; los casos de prueba seleccionan situaciones; la ejecución produce evidencia; el diagnóstico y la traza ayudan a localizar causas; la corrección se confirma mediante nuevas pruebas y regresión; y la calidad del código reduce la dificultad de todo el proceso.

13. Ejecutarse sin errores no demuestra corrección.
14. Una prueba necesita una expectativa derivada de la especificación.
15. Los casos normales, límite e inválidos revelan comportamientos distintos.
16. Las decisiones requieren casos para sus ramas.
17. Los ciclos requieren pruebas de inicio, progreso y terminación.
18. Los errores sintácticos, semánticos, de ejecución y lógicos aparecen de formas diferentes.
19. Un diagnóstico orienta, pero no explica automáticamente la causa.
20. La depuración debe apoyarse en evidencia e hipótesis.
21. Las trazas localizan la primera divergencia del estado.
22. Los diagramas ayudan a revisar recorridos, no fórmulas.
23. La calidad facilita comprobar, modificar y explicar.
24. Después de corregir, deben repetirse casos de regresión.

Comprobación activa: explica cómo investigarías un programa que no muestra errores, pero produce 6.75 cuando esperabas 9.

Ejercicios

Realiza primero los ejercicios de especificación y predicción sin ejecutar.

1. Especifica entradas, salida, reglas y restricciones de un conversor Celsius-Fahrenheit. [\[Especificar\]](#)
2. Diseña tres casos normales y cuatro casos límite para una nota permitida entre 0 y 10. [\[Diseñar\]](#)
3. Completa una tabla de prueba para `CALCULAR_AREA_CIRCULO(radio)`. [\[Aplicar\]](#)

4. Explica por qué una entrada aislada no constituye un caso de prueba completo. [Analizar]
5. Diseña casos que cubran las ramas de una clasificación negativa, cero y positiva. [Diseñar]
6. Selecciona pruebas para un arreglo vacío, de un elemento y de cinco elementos. [Diseñar]
7. Identifica un error de frontera causado por usar `<` en lugar de `<=`. [Analizar]
8. Clasifica cuatro ejemplos como sintácticos, semánticos, de ejecución o lógicos. [Reconocer]
9. Corrige un programa con un paréntesis faltante y explica por qué el diagnóstico aparece en la línea siguiente. [Diagnosticar]
10. Construye una traza para una suma acumulada que omita el último elemento. [Trazar]
11. Prueba `MAXIMO` con todos los valores negativos y explica la falla de inicialización. [Analizar]
12. Aísla y prueba las funciones de una calculadora de compra. [Probar]
13. Diseña una prueba de regresión para un defecto de descuento ya corregido. [Diseñar]
14. Genera un diagrama de flujo e identifica una rama que nunca se alcanza. [Representar]
15. Refactoriza variables `x`, `y`, `z` y `r` para hacer visible una fórmula de salario. [Mejorar]
16. Elimina una condición redundante y demuestra con casos que el comportamiento no cambia. [Mejorar]
17. Escribe un comentario útil para un centinela y otro que deba eliminarse. [Evaluar]
18. Construye una lista de revisión para una búsqueda lineal. [Integrar]
19. Investiga un promedio que divide siempre entre cuatro, aunque el arreglo tenga otra longitud. [Depurar]
20. Documenta un defecto real encontrado en tus programas: entrada, esperado, observado, causa y corrección. [Integrar]

Mini proyecto: expediente de comprobación

Situación: elige uno de los programas desarrollados en capítulos anteriores y construye un expediente que permita a otra persona comprobarlo sin preguntarte qué debería ocurrir.

El expediente debe incluir:

- Descripción precisa del problema.
- Entradas, salidas, reglas, restricciones y casos especiales.
- Tabla con al menos diez casos de prueba.
- Casos normales, límite, inválidos y de cobertura de ramas.
- Resultados observados y veredictos.
- Una traza de un caso significativo.
- Captura o transcripción de un diagnóstico corregido.
- Diagrama de flujo exportado.
- Una refactorización de calidad y sus pruebas de regresión.
- Conclusión sobre limitaciones que permanecen.

Criterio de éxito: otra persona debe poder repetir las pruebas, reconocer qué comportamiento se esperaba y comprender la evidencia utilizada para aceptar o rechazar el programa.

Autoevaluación

Responde sin consultar el capítulo. Después utiliza un programa real para comprobar tus respuestas.

Afirmación	Sí	Todavía no
Puedo distinguir ejecución de corrección.	☐	☐
Puedo escribir una especificación verificable.	☐	☐
Diseño casos normales, límite e inválidos.	☐	☐
Puedo cubrir ramas y casos de ciclos.	☐	☐
Distingo cuatro familias de errores.	☐	☐
Puedo interpretar un diagnóstico sin corregir al azar.	☐	☐
Uso trazas para localizar la primera divergencia.	☐	☐
Puedo probar funciones de forma aislada.	☐	☐
Uso el diagrama como instrumento de revisión.	☐	☐
Vuelvo a probar después de corregir.	☐	☐
Puedo mejorar nombres, condiciones y funciones.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas especificar un programa, diseñar pruebas reproducibles, localizar un defecto mediante evidencia y demostrar que la corrección no dañó casos anteriores.

Glosario

Término	Definición de trabajo
Especificación	Descripción verificable de entradas, salidas, reglas y restricciones.
Precondición	Condición que debe cumplirse antes de usar una operación.
Resultado esperado	Comportamiento que debe observarse para una entrada dada.
Caso de prueba	Entrada, expectativa, resultado observado y veredicto reproducibles.
Caso límite	Prueba situada en un extremo o cambio de comportamiento.
Cobertura de ramas	Evidencia de que los caminos alternativos fueron ejecutados.
Defecto	Causa en el programa que produce un comportamiento incorrecto.
Falla	Manifestación observable de un defecto durante una ejecución.

Término	Definición de trabajo
Diagnóstico	Mensaje producido por una herramienta para señalar un problema detectado.
Depuración	Proceso de reproducir, localizar y corregir la causa de una falla.
Hipótesis	Explicación provisional que debe contrastarse con evidencia.
Regresión	Pérdida de un comportamiento que antes era correcto.
Refactorización	Mejora estructural que pretende conservar el comportamiento.
Traza	Registro ordenado de estados y pasos de ejecución.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación técnica de diagnósticos, ejecución y diagramas de flujo [Repositorio de software].

McConnell, S. (2004). Code complete: A practical handbook of software construction (2nd ed.). Microsoft Press.

Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). John Wiley & Sons.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

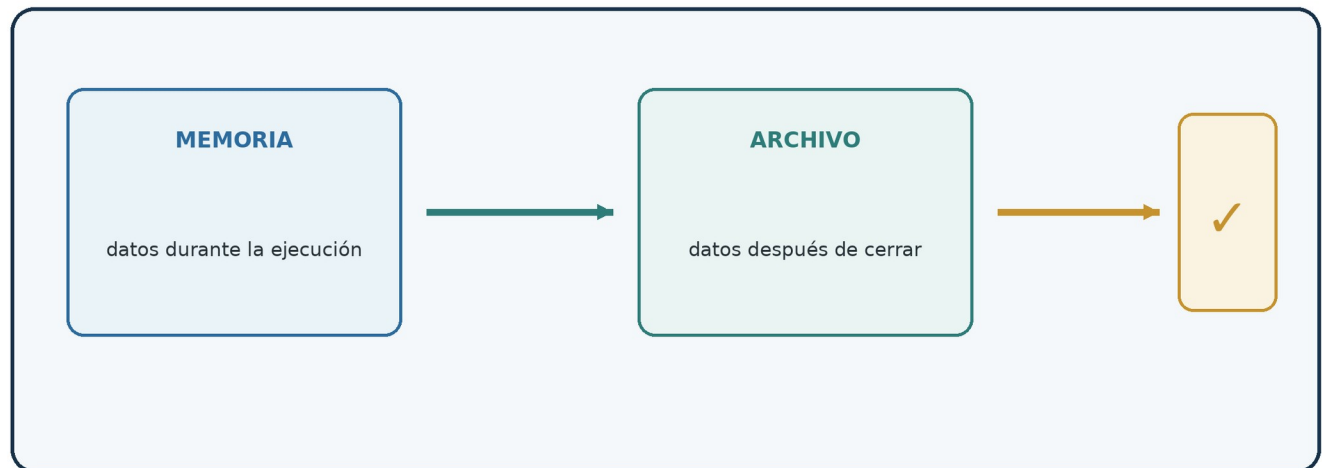
Continuidad: el programa ya puede especificarse, probarse y mejorarse. El capítulo 12 integrará la biblioteca estándar, la persistencia básica y un proyecto completo que reúna todo el recorrido del libro.

CAPÍTULO

12

Integrar datos, archivos y decisiones de diseño

Biblioteca estándar, persistencia y proyecto integrador



El programa completo une estructuras, biblioteca, pruebas y decisiones justificadas.

Integrar datos, archivos y decisiones de diseño

¿Cómo se convierte un conjunto de técnicas en un programa completo que conserva información?

Hasta ahora, cada ejecución comenzó con un estado nuevo. Las variables y los arreglos permitieron organizar información mientras el programa estaba activo, pero al cerrarlo los datos desaparecieron. Un registro de gastos que olvida todos los movimientos deja de ser útil, aunque sus cálculos sean correctos.

El cierre de la obra integra tres capacidades. Primero, seleccionar funciones de la biblioteca estándar según el problema. Segundo, conservar información mediante archivos de texto. Tercero, justificar cómo se relacionan entrada, validación, funciones, colecciones, pruebas, diagramas y costo dentro de un proyecto completo.

Promesa de aprendizaje: al terminar podrás elegir funciones de biblioteca por su contrato, crear, escribir, leer, medir y eliminar archivos, diseñar un formato textual básico, separar procesamiento y almacenamiento, reconstruir datos persistentes y completar un proyecto integrador documentado y probado.

La biblioteca estándar agrupa capacidades que ya tienen nombre y contrato



Elegir una función exige conocer qué recibe, qué devuelve, qué modifica y cuánto trabajo realiza.

Figura 12.1. Familias principales de la biblioteca estándar de CLRS. Elaboración propia.

Alcance: la persistencia se estudiará con archivos de texto y formatos sencillos. Bases de datos, serialización estructurada, concurrencia y seguridad de almacenamiento pertenecen a cursos posteriores.

12.1 La biblioteca como conjunto de decisiones ya implementadas

Una biblioteca estándar reúne operaciones comunes para que cada programa no tenga que reconstruirlas. Usar `RAIZ`, `DIV`, `ORDENA` o `LEER\_ARCHIVO` delega detalles de implementación, pero no elimina la necesidad de comprender el problema.

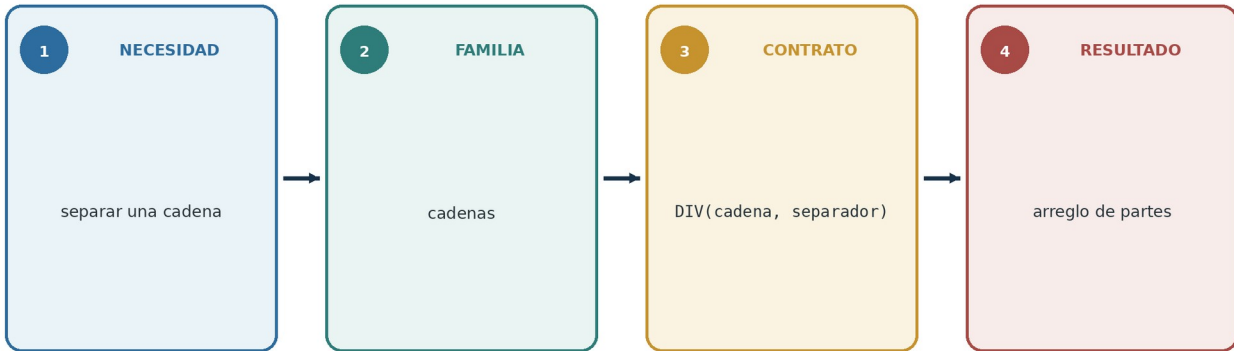
La pregunta correcta no es “¿qué función puedo usar?”, sino “¿qué transformación necesito y qué contrato ofrece la función?”.

Necesidad	Función posible	Resultado
Obtener la raíz cuadrada	`RAIZ(x)`	Número.
Normalizar una categoría	`MAYUS(RECORTA(texto))`	Cadena.
Separar campos	`DIV(texto, separador)`	Arreglo.
Comprobar un tipo	`ES_NUM(valor)`	Lógico.
Agregar un elemento	`AGREGA(arreglo, valor)`	Arreglo modificado.
Leer un archivo	`LEER_ARCHIVO(ruta)`	Cadena con su contenido.

Idea clave: la biblioteca reduce código repetitivo; la responsabilidad de elegir, validar y combinar sus operaciones sigue perteneciendo a quien diseña el programa.

12.2 Elegir por contrato, no por semejanza del nombre

Seleccionar una función comienza por la necesidad, no por memorizar nombres



Después se revisan casos límite: cadena vacía, separador ausente, campos incompletos y tipos incompatibles.

Figura 12.2. Proceso para seleccionar una función de biblioteca. Elaboración propia.

El contrato de una función incluye al menos sus entradas, su resultado observable y sus condiciones de uso. Dos funciones pueden parecer cercanas y resolver problemas distintos.

Función	Pregunta que responde	Detalle relevante
`ES_NUM(valor)`	¿El valor ya es numérico?	No convierte.
`ES_CAD_NUM(cadena)`	¿La cadena representa un número?	Examina texto.
`A_NUM(valor)`	¿Cuál es su conversión numérica?	Puede producir un valor no útil si la entrada no es válida.
`INDICE(arreglo, valor)`	¿En qué posición aparece?	Retorna una posición, no un lógico.
`CONT(arreglo, valor)`	¿Está contenido?	Retorna `VERDAD` o `FALSO`.

Pausa y piensa: para validar una entrada textual antes de sumarla, ¿usarías únicamente `A\_NUM` o combinarías una comprobación con la conversión? Justifica el orden.

12.3 Las familias de funciones trabajan juntas

Un problema real rara vez se resuelve con una sola llamada. La preparación de una categoría puede combinar funciones de cadenas; el análisis de cantidades puede unir división, conversión y operaciones de arreglos.

Preparar una categoría

```
NORMALIZAR_CATEGORIA(categoria)
  categoria <- RECORTA(categoria)
  categoria <- MAYUS(categoria)
  retornar categoria
```

Separar y convertir un registro

```
campos <- DIV("COMIDA|150.50", "|")
categoria <- campos[0]
monto <- A_NUM(campos[1])
```

La composición hace visible una cadena de transformaciones. Cada paso puede comprobarse por separado y el resultado intermedio conserva un nombre significativo.

Advertencia: encadenar muchas llamadas en una sola expresión puede ocultar qué transformación falló. Usa variables intermedias cuando aporten observabilidad.

12.4 El costo no desaparece dentro de la biblioteca

Una llamada breve puede representar trabajo proporcional al tamaño de los datos

EXISTE_ARCHIVO(ruta)	Comprobar metadatos	$O(1)$
LEER_ARCHIVO(ruta)	Leer n caracteres	$O(n)$
ESCRIBIR_ARCHIVO(ruta, texto)	Agregar m caracteres	$O(m)$
DIV(texto, separador)	Recorrer la cadena	$O(n)$
ORDENA(arreglo)	Ordenar n elementos	$O(n \log n)$

La biblioteca oculta detalles de implementación, no elimina el costo de procesar datos.

Figura 12.3. Trabajo oculto detrás de llamadas breves. Elaboración propia.

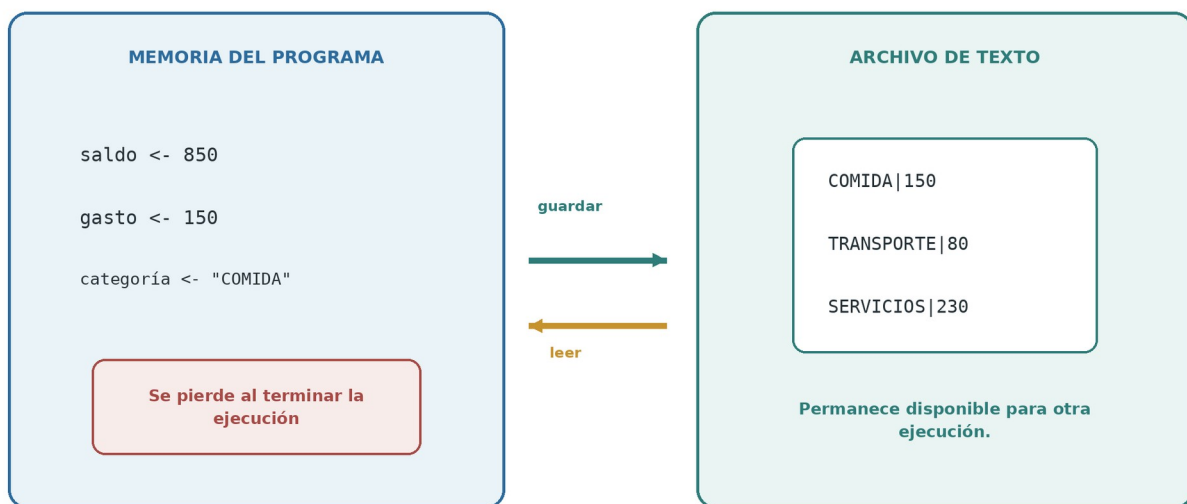
La notación de una llamada puede ocupar una sola línea mientras la operación recorre una cadena, copia un arreglo o reordena elementos. El visualizador de costo permite mantener esta relación entre sintaxis breve y trabajo realizado.

Patrón	Ejemplo	Interpretación
Constante	<code>`EXISTE_ARCHIVO(ruta)`</code>	No depende directamente de la longitud del contenido.
Lineal	<code>`LEER_ARCHIVO(ruta)`</code>	Procesa una cantidad proporcional al tamaño leído.
Lineal	<code>`DIV(texto, " ")`</code>	Examina la cadena para localizar separadores.
$n \log n$	<code>`ORDENA(valores)`</code>	Ordena una colección.

Importante: en esta obra el costo se utiliza para explicar crecimiento, no para reemplazar mediciones ni demostrar formalmente la implementación.

12.5 De memoria temporal a información persistente

Persistir significa conservar información más allá de una ejecución



El archivo no reemplaza las variables: permite reconstruir datos cuando el programa vuelve a comenzar.

Figura 12.4. Relación entre memoria de ejecución y archivo persistente. Elaboración propia.

La memoria del programa mantiene el estado necesario para la ejecución actual. Un archivo conserva una representación de ciertos datos para que otra ejecución pueda recuperarlos.

Persistir no significa guardar todas las variables. Significa decidir qué información merece sobrevivir, con qué formato se escribirá y cómo podrá reconstruirse después.

Pregunta	Decisión de diseño
¿Qué debe conservarse?	Los movimientos, no mensajes temporales ni contadores de interfaz.
¿Cuándo se guarda?	Después de aceptar y validar cada registro.
¿Cómo se representa?	Una línea por movimiento.
¿Cómo se recupera?	Leer, dividir líneas, dividir campos y convertir cantidades.
¿Qué sucede si falta el archivo?	Crearlo o iniciar un historial vacío.

Idea clave: un archivo contiene una representación de los datos, no los conceptos del programa. Recuperar exige interpretar esa representación.

12.6 Rutas y nombres de archivo

Las funciones de archivo reciben una ruta. Para los primeros programas conviene utilizar un nombre relativo sencillo, como `"gastos.txt"`, que se resolverá desde el directorio de ejecución.

Ruta básica

```
ruta <- "gastos.txt"
escribir "Archivo de trabajo: " + ruta
```

Las rutas absolutas dependen del sistema operativo y del equipo. Una ruta relativa facilita trasladar el proyecto, aunque exige comprender desde qué carpeta se ejecuta.

Advertencia: no construyas rutas mezclando separadores específicos del sistema sin necesidad. En un proyecto introductorio, conserva los archivos dentro de la carpeta de trabajo.

12.7 Comprobar y crear sin destruir información

`EXISTE_ARCHIVO(ruta)` permite decidir si es necesario preparar un archivo. `CREAR_ARCHIVO(ruta)` crea un archivo vacío; si se utiliza sobre uno existente, su contenido puede reemplazarse por vacío. Por eso no debe ejecutarse indiscriminadamente en cada inicio.

Preparación segura

```
PREPARAR_ARCHIVO(ruta)
  si no EXISTE_ARCHIVO(ruta)
    CREAR_ARCHIVO(ruta)
```

Práctica peligrosa

```
PRINCIPAL( )
  ruta <- "gastos.txt"
  CREAR_ARCHIVO(ruta)
  escribir "Archivo preparado"
```

Si el archivo ya contenía movimientos, esta decisión puede borrar el historial.

Idea clave: antes de crear, sobrescribir o eliminar, el programa debe decidir qué debe ocurrir con la información existente.

12.8 Escribir agrega contenido

`ESCRIBIR\_ARCHIVO(ruta, contenido)` agrega el contenido al final. Esta semántica es apropiada para un historial en el que cada operación produce una nueva línea.

Agregar un movimiento

```
ruta <- "gastos.txt"
registro <- "COMIDA|150.50\n"
ESCRIBIR_ARCHIVO(ruta, registro)
```

El salto de línea separa registros consecutivos. Sin él, dos escrituras podrían quedar unidas y perder la frontera entre movimientos.

Advertencia: agregar no corrige registros previos. Para modificar una línea específica sería necesario leer, transformar todo el contenido y volver a escribir con una estrategia diferente.

12.9 Leer y distinguir ausencia de contenido

`LEER\_ARCHIVO(ruta)` devuelve el contenido completo como cadena. Cuando el archivo no existe, la implementación de CLRS devuelve una cadena vacía. Esto evita una falla inmediata, pero obliga a diferenciar entre “archivo ausente” y “archivo existente sin contenido” cuando esa diferencia sea importante.

Lectura defensiva

```
contenido <- LEER_ARCHIVO("gastos.txt")

si ES_VAC(RECORTA(contenido))
  escribir "No hay movimientos registrados"
sino
  escribir contenido
```

Para conocer la existencia de manera explícita se utiliza `EXISTE\_ARCHIVO`. Para conocer el tamaño se utiliza `PESO\_ARCHIVO`, que retorna cero si el archivo no existe.

Situación	`EXISTE_ARCHIVO`	`LEER_ARCHIVO`	`PESO_ARCHIVO`
No existe	`FALSO`	''''	0
Existe y está vacío	`VERDAD`	''''	0
Existe con datos	`VERDAD`	Contenido	Número mayor que 0

Importante: una misma observación, como contenido vacío, puede corresponder a estados distintos. Elige la función que permita responder la pregunta real.

12.10 Diseñar el ciclo de vida del archivo

El ciclo de vida de un archivo debe diseñarse de forma explícita

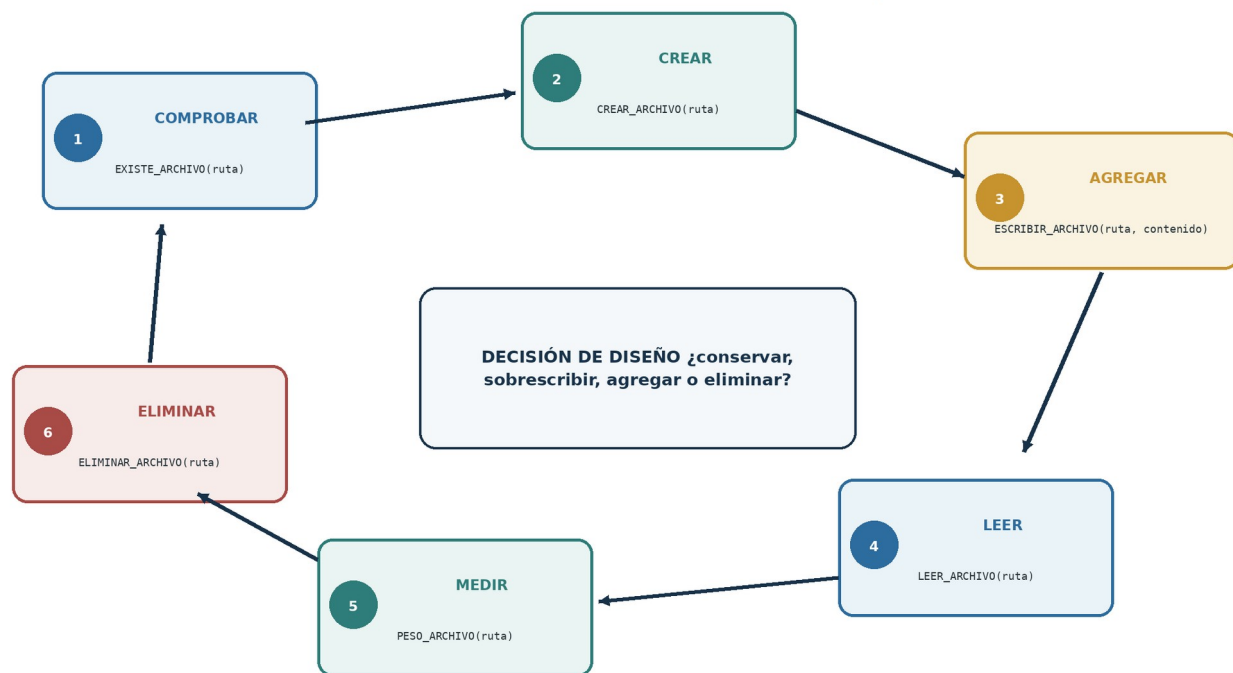


Figura 12.5. Operaciones del ciclo de vida de un archivo. Elaboración propia.

No todos los programas necesitan todas las operaciones. Un historial puede comprobar, crear, agregar y leer. Una herramienta de mantenimiento podría además medir o eliminar.

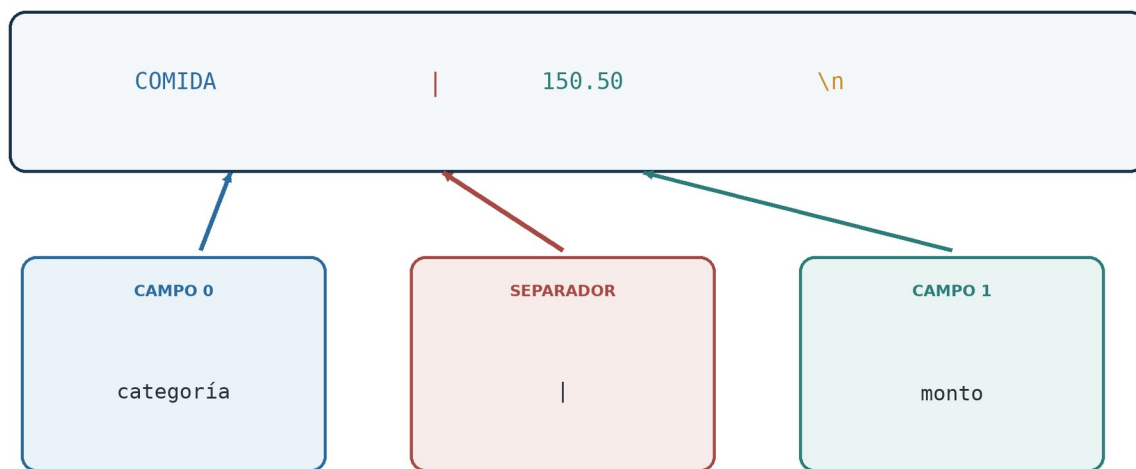
Operación	Cuándo utilizarla	Riesgo
Comprobar	Antes de depender de un archivo.	Confundir ausencia con vacío.
Crear	Al inicializar un recurso nuevo.	Vaciar uno existente.
Agregar	Al registrar nueva información.	Duplicar datos.
Leer	Al reconstruir el estado.	Interpretar un formato dañado.

Operación	Cuándo utilizarla	Riesgo
Medir	Al informar tamaño o detectar vacío.	No confundir bytes con cantidad de registros.
Eliminar	Al reiniciar deliberadamente.	Pérdida irreversible del historial.

Pausa y piensa: ¿un botón “reiniciar historial” debería llamar directamente a `ELIMINAR\_ARCHIVO` o pedir una confirmación y volver a crear el archivo? Explica el flujo.

12.11 Un formato de registros separado por campos

Un formato de texto define cómo escribir y reconstruir cada registro



Limitación: si la categoría contiene “|” o un salto de línea, el formato necesita reglas de escape adicionales.

Figura 12.6. Formato básico de una línea de gasto. Elaboración propia.

Un formato elemental puede asignar una línea a cada registro y utilizar ` ` como separador entre categoría y monto.

Formar una línea

```
FORMATEAR_GASTO(categoria, monto)
  categoria <- MAYUS(RECORTA(categoria))
  retornar categoria + "|" + A_CAD(monto) + "\n"
```

El formato es legible y puede procesarse con `DIV`. También tiene límites: no define cómo escapar el separador, los saltos de línea ni campos faltantes. Estas limitaciones deben declararse.

Simplificación pedagógica: el formato sirve para comprender persistencia y reconstrucción. No pretende sustituir CSV, JSON, bases de datos ni formatos con reglas completas de escape.

12.12 Reconstruir registros desde texto

La lectura produce una sola cadena. Para recuperar registros se realiza una transformación por niveles:

1. Recortar saltos sobrantes.
2. Dividir el contenido por líneas.
3. Recorrer cada línea.
4. Dividir cada línea por el separador de campos.
5. Comprobar la cantidad y el tipo de los campos.
6. Convertir el monto a número.

Calcular el total guardado

```
CALCULAR_TOTAL_HISTORIAL(contenido)
  contenido <- RECORTA(contenido)

  si ES_VAC(contenido)
    retornar 0

  lineas <- DIV(contenido, "\n")
  total <- 0

  para i <- 0 hasta LONG(lineas) - 1
    campos <- DIV(lineas[i], "|")

    si LONG(campos) >= 2 y ES_CAD_NUM(campos[1])
      total <- total + A_NUM(campos[1])

  retornar total
```

La función ignora líneas que no contienen al menos dos campos numéricamente interpretables. Esa es una política de tolerancia. Otra solución podría lanzar un error y detenerse. La elección depende del propósito del programa.

Decisión de diseño: tolerar datos dañados permite continuar, pero puede ocultar pérdida de información. Rechazarlos hace visible el problema, pero impide obtener un resultado parcial.

12.13 Separar procesamiento y almacenamiento

Una función de cálculo no debería depender necesariamente de una ruta. Recibir el contenido como cadena permite probarla sin crear archivos.

Separación de responsabilidades

```

LEER_HISTORIAL(ruta)
  retornar LEER_ARCHIVO(ruta)

CALCULAR_TOTAL_HISTORIAL(contenido)
  // procesa la cadena recibida
  retornar total

PRINCIPAL()
  contenido <- LEER_HISTORIAL("gastos.txt")
  total <- CALCULAR_TOTAL_HISTORIAL(contenido)
  escribir "Total: " + total

```

Esta organización permite comprobar `CALCULAR\_TOTAL\_HISTORIAL` con cadenas construidas directamente en pruebas. La interacción con el sistema de archivos queda concentrada.

Idea clave: la persistencia es una responsabilidad; interpretar y calcular sobre los datos es otra. Separarlas mejora las pruebas y permite cambiar el almacenamiento con menor impacto.

12.14 Manejar errores y datos incompatibles

Las operaciones de archivo pueden fallar por permisos, rutas inválidas u otras condiciones externas. CLRS Kit presenta los errores de ejecución con mensajes en español. El programa también debe detectar inconsistencias propias del formato.

Validar una línea

```

ES_LINEA_GASTO_VALIDA(línea)
  campos <- DIV(línea, "|")

  si LONG(campos) != 2
    retornar FALSO

  retornar no ES_VAC(RECORTA(campos[0])) y ES_CAD_NUM(campos[1])

```

`LANZAR\_ERROR` puede utilizarse cuando continuar produciría resultados engañosos.

Rechazar un formato dañado

```

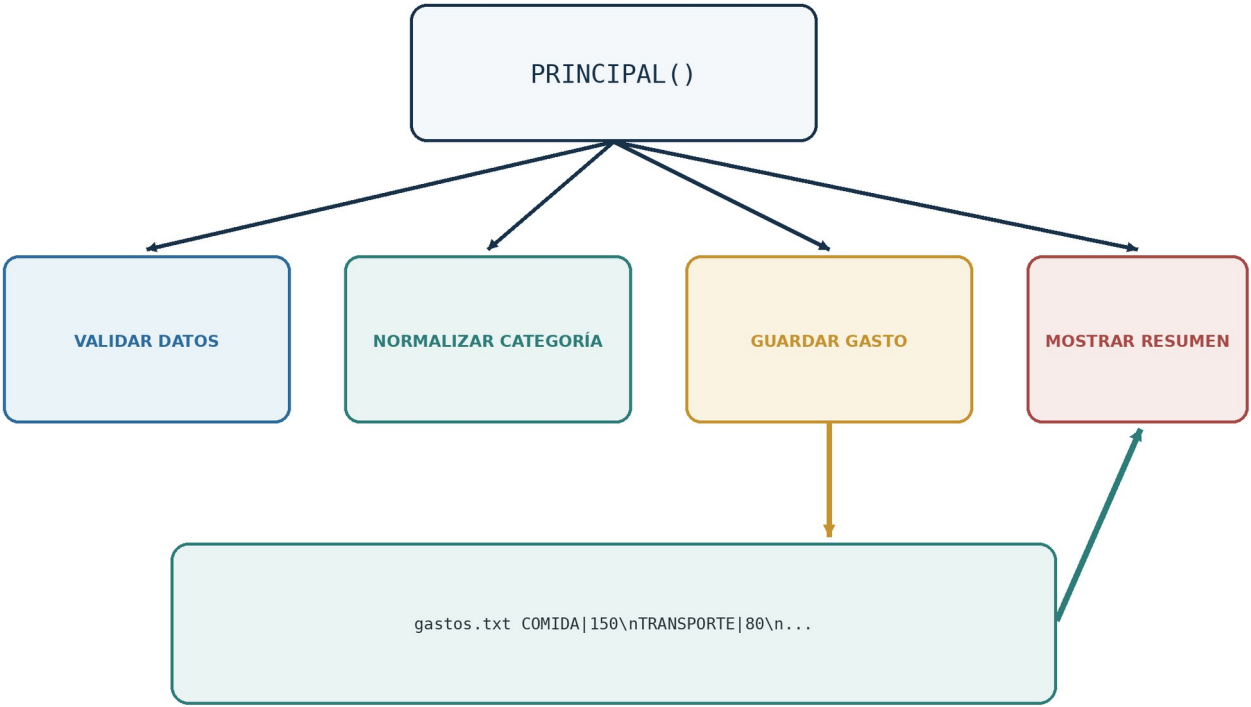
si no ES_LINEA_GASTO_VALIDA(línea)
  LANZAR_ERROR("El historial contiene un registro inválido")

```

Advertencia: no captures o ignores un error solo para que el programa “siga funcionando”. Decide qué información puede recuperarse con seguridad.

12.15 Proyecto integrador: gestor persistente de gastos

El proyecto integrador conecta entrada, reglas, memoria, archivo y presentación



Cada flecha representa una transferencia de datos o una decisión que debe poder explicarse y probarse.

Figura 12.7. Arquitectura del proyecto integrador. Elaboración propia.

El proyecto final amplía el caso longitudinal. Permitirá registrar movimientos, conservarlos en un archivo y recuperar un resumen en ejecuciones posteriores.

Componente	Responsabilidad
`ES_CATEGORIA_VALIDA`	Comprobar que la categoría sea texto no vacío.
`ES_MONTO_VALIDO`	Aceptar números positivos o cadenas numéricas positivas.
`A_MONTO`	Convertir la entrada validada a número.
`NORMALIZAR_CATEGORIA`	Recortar y convertir a mayúsculas.
`PREPARAR_ARCHIVO`	Crear solo cuando no existe.
`GUARDAR_GASTO`	Formar una línea y agregarla.
`CALCULAR_TOTAL_HISTORIAL`	Reconstruir montos y acumularlos.
`MOSTRAR_HISTORIAL`	Presentar contenido y total.
`PRINCIPAL`	Coordinar el menú y las llamadas.

12.16 Implementación guiada del proyecto

Gestor persistente de gastos

```

ES_CATEGORIA_VALIDA(valor)
    retornar ES_CAD(valor) y no ES_VAC(RECORTA(valor))

ES_MONTO_VALIDO(valor)
    si ES_NUM(valor)
        retornar valor > 0

    si ES_CAD(valor)
        retornar ES_CAD_NUM(valor) y A_NUM(valor) > 0

    retornar FALSO

A_MONTO(valor)
    si ES_NUM(valor)
        retornar valor

    retornar A_NUM(valor)

NORMALIZAR_CATEGORIA(categoria)
    retornar MAYUS(RECORTA(categoria))

PREPARAR_ARCHIVO(ruta)
    si no EXISTE_ARCHIVO(ruta)
        CREAR_ARCHIVO(ruta)

GUARDAR_GASTO(ruta, categoria, monto)
    categoria <- NORMALIZAR_CATEGORIA(categoria)
    registro <- categoria + "|" + A_CAD(monto) + "\n"
    ESCRIBIR_ARCHIVO(ruta, registro)

CALCULAR_TOTAL_HISTORIAL(contenido)
    contenido <- RECORTA(contenido)

    si ES_VAC(contenido)
        retornar 0

    lineas <- DIV(contenido, "\n")
    total <- 0

    para i <- 0 hasta LONG(lineas) - 1
        campos <- DIV(lineas[i], "|")

        si LONG(campos) >= 2 y ES_CAD_NUM(campos[1])
            total <- total + A_NUM(campos[1])

    retornar total

MOSTRAR_HISTORIAL(ruta)
    contenido <- LEER_ARCHIVO(ruta)

    si ES_VAC(RECORTA(contenido))
        escribir "No hay movimientos registrados"
        retornar 0

    escribir "--- HISTORIAL ---"
    escribir contenido
    retornar CALCULAR_TOTAL_HISTORIAL(contenido)

PRINCIPAL()
    ruta <- "gastos.txt"

```

```

PREPARAR_ARCHIVO(ruta)
opcion <- 0

mientras opcion != 3
  escribir "1. Registrar gasto"
  escribir "2. Mostrar historial"
  escribir "3. Salir"
  leer opcion

  si opcion = 1
    categoria <- ""
    monto <- ""

    escribir "Categoría:"
    leer categoria
    escribir "Monto:"
    leer monto

    si ES_CATEGORIA_VALIDA(categoria) y ES_MONTO_VALIDO(monto)
      monto <- A_MONTO(monto)
      GUARDAR_GASTO(ruta, categoria, monto)
      escribir "Gasto guardado"
    sino
      escribir "Datos inválidos"
  sino si opcion = 2
    total <- MOSTRAR_HISTORIAL(ruta)
    escribir "Total acumulado: " + total
  sino si opcion != 3
    escribir "Opción inválida"

```

Observación: el programa conserva los registros en texto y vuelve a calcular el total al leerlos. No mantiene todavía identificadores, fechas, edición ni eliminación selectiva. Esas ausencias son limitaciones declaradas, no errores ocultos.

12.17 Seguir una sesión completa

Paso	Acción	Estado o archivo
1	El programa inicia.	Comprueba `gastos.txt`.
2	El archivo no existe.	Lo crea vacío.
3	Se registra `Comida`, 150.	Normaliza a `COMIDA`.
4	Guarda el movimiento.	Archivo: `COMIDA 150\n`.
5	Se registra `Transporte`, 80.	Agrega otra línea.
6	Se solicita historial.	Lee todo el contenido.
7	Divide líneas y campos.	Convierte 150 y 80.
8	Acumula los montos.	Total = 230.

La traza permite comprobar que la persistencia no consiste únicamente en escribir. El recorrido completo incluye validación, representación, almacenamiento, recuperación, interpretación y cálculo.

Pausa y piensa: ¿qué resultado obtendría la función si el archivo contiene una línea vacía al final? Explica el efecto de `RECORTA` antes de `DIV`.

12.18 Casos de prueba del proyecto

Caso	Estado inicial	Acción	Resultado esperado
Archivo ausente	No existe	Iniciar	Se crea vacío.
Historial vacío	Existe y mide 0	Mostrar	Mensaje sin movimientos; total 0.
Gasto válido	Vacío	Registrar COMIDA, 150	Se agrega una línea.
Monto textual válido	Vacío	Registrar "80.5"	Se convierte y guarda.
Monto negativo	Cualquier estado	Registrar -10	Se rechaza.
Categoría vacía	Cualquier estado	Registrar espacios	Se rechaza.
Dos registros	Dos líneas válidas	Mostrar	Total correcto.
Línea dañada	Campo numérico inválido	Calcular	Se ignora según la política actual.
Reinicio	Archivo con datos	Volver a ejecutar	Los datos permanecen.

Prueba de regresión esencial: después de cualquier cambio, confirma que iniciar el programa no vacíe un archivo existente.

12.19 Revisar decisiones y limitaciones

Un proyecto integrador debe explicar no solo lo que hace, sino por qué se diseñó de esa manera y qué no resuelve.

Decisión	Razón	Limitación
Una línea por gasto	Facilita agregar y leer.	No permite saltos en campos.
Separador ` `	Es visible y fácil de dividir.	Requiere escape si aparece en texto.
Agregar al final	Conserva historial.	Editar exige reescritura.
Total recalculado	Evita guardar datos derivados.	Lee todo el archivo.
Funciones separadas	Facilita pruebas.	Aumenta cantidad de componentes.
Ignorar líneas dañadas	Permite resultado parcial.	Puede ocultar pérdida.

Criterio profesional inicial: una simplificación es aceptable cuando está declarada, sus consecuencias se comprenden y no se presenta como una solución universal.

12.20 Entregables del proyecto final

El proyecto puede presentarse como evidencia acumulada de todo el aprendizaje del libro.

1. Archivo `.clrs`` ejecutable.
2. Descripción del problema y del usuario previsto.
3. Especificación de entradas, salidas, reglas y restricciones.
4. Mapa de funciones y responsabilidades.
5. Descripción del formato persistente.
6. Tabla de casos de prueba y resultados observados.
7. Diagrama de flujo exportado.
8. Explicación del costo de los recorridos principales.
9. Lista de errores encontrados y correcciones.
10. Limitaciones y posibles extensiones.

Criterio	Evidencia de dominio
Corrección	Los casos normales y límite producen resultados esperados.
Persistencia	Los datos sobreviven a una nueva ejecución.
Validación	Las entradas inválidas no contaminan el archivo.
Organización	Las funciones tienen responsabilidades reconocibles.
Representación	El formato puede escribirse y reconstruirse.
Pruebas	La evidencia permite repetir la comprobación.
Costo	Se distinguen recorridos lineales y operaciones constantes.
Comunicación	Las decisiones y limitaciones están justificadas.

12.21 Reconstruye el recorrido completo del libro

El programa final puede comprenderse como una cadena de decisiones acumuladas:

1. Un problema se convierte en entradas, reglas y resultados.
2. Los valores forman un estado que cambia mediante instrucciones.
3. Las condiciones eligen caminos.
4. Los ciclos repiten procesos con progreso y terminación.
5. Las funciones dividen responsabilidades y transfieren datos.
6. Los arreglos y cadenas permiten procesar colecciones.
7. Las pruebas comparan resultados observados y esperados.
8. La biblioteca aporta operaciones con contratos conocidos.
9. Los archivos conservan representaciones entre ejecuciones.
10. El proyecto integra código, pruebas, diagramas, costo y explicación.

Idea final: programar no consiste en acumular sintaxis. Consiste en representar un problema, organizar transformaciones, observar evidencia y justificar decisiones hasta obtener una solución comprensible.

Ejercicios

Realiza los ejercicios sobre copias de archivos de práctica. No uses información importante.

1. Clasifica diez funciones de la biblioteca por familia y tipo de resultado. **[Reconocer]**
2. Compara ``ES_NUM``, ``ES_CAD_NUM`` y ``A_NUM`` con tres entradas distintas. **[Analizar]**
3. Selecciona funciones para normalizar una categoría con espacios y minúsculas. **[Aplicar]**
4. Predice el costo de leer, dividir y recorrer un archivo de n caracteres. **[Analizar]**
5. Crea un archivo solo cuando no exista. **[Construir]**
6. Demuestra qué ocurre si ``CREAR_ARCHIVO`` se ejecuta sobre un historial existente. **[Experimentar]**
7. Escribe tres líneas y comprueba el peso del archivo. **[Aplicar]**
8. Diseña un formato ``nombre|calificación`` y declara sus limitaciones. **[Diseñar]**
9. Reconstruye un arreglo de líneas mediante ``DIV``. **[Aplicar]**
10. Valida que cada línea tenga exactamente dos campos. **[Construir]**
11. Calcula el promedio de calificaciones guardadas en texto. **[Integrar]**
12. Decide si una línea dañada debe ignorarse o lanzar un error. Justifica. **[Evaluar]**
13. Separa una función que lee el archivo de otra que procesa su contenido. **[Mejorar]**
14. Prueba una función de procesamiento sin utilizar archivos reales. **[Probar]**
15. Añade una opción para mostrar la cantidad de registros válidos. **[Construir]**
16. Añade una confirmación antes de eliminar el archivo. **[Construir]**
17. Diseña una prueba que demuestre persistencia entre ejecuciones. **[Probar]**
18. Genera el diagrama del proyecto e identifica las llamadas de archivo. **[Representar]**
19. Explica por qué el peso del archivo no equivale a la cantidad de registros. **[Analizar]**
20. Propón una extensión del formato para incluir fecha y describe el impacto. **[Diseñar]**

Variaciones del proyecto integrador

Proyecto	Registro persistente	Procesamiento
Sistema de calificaciones	<code>`alumno nota`</code>	Promedio, máximo y aprobación.
Inventario	<code>`producto cantidad precio`</code>	Existencias y valor total.
Analizador de texto	Texto completo	Frecuencias y palabras comunes.
Registro de hábitos	<code>`fecha hábito estado`</code>	Cumplimiento por período.
Estadísticas personales	Una cantidad por línea	Media, mediana y varianza.

Restricción: elige un proyecto cuya complejidad permita explicar cada decisión. Agregar características sin poder probarlas o justificarlas reduce la calidad del resultado.

Autoevaluación

Responde sin consultar el capítulo. Después demuestra cada respuesta mediante un programa pequeño.

Afirmación	Sí	Todavía no
Selecciono funciones de biblioteca por su contrato.	☐	☐
Distingo comprobar, convertir y procesar.	☐	☐
Comprendo qué operaciones tienen costo lineal.	☐	☐
Puedo crear un archivo sin destruir uno existente.	☐	☐
Puedo agregar y leer contenido.	☐	☐
Distingo ausencia de archivo y archivo vacío.	☐	☐
Puedo diseñar un formato textual básico.	☐	☐
Puedo reconstruir registros desde una cadena.	☐	☐
Separo almacenamiento y procesamiento.	☐	☐
Puedo probar persistencia entre ejecuciones.	☐	☐
Justifico decisiones y limitaciones del proyecto.	☐	☐

Criterio de dominio: has completado la ruta cuando puedes construir un programa persistente, explicar su formato, probarlo desde archivos vacíos y con datos, y justificar la relación entre biblioteca, funciones, costo y calidad.

Glosario

Término	Definición de trabajo
Biblioteca estándar	Conjunto de funciones disponibles para operaciones comunes.
Contrato	Entradas, resultado, efectos y condiciones de uso de una función.
Persistencia	Conservación de información más allá de una ejecución.
Archivo de texto	Secuencia persistente de caracteres.

Término	Definición de trabajo
Ruta	Ubicación utilizada para identificar un archivo.
Formato	Reglas para representar datos mediante texto.
Registro	Grupo de campos que representa una entidad o movimiento.
Separador	Texto utilizado para distinguir campos o registros.
Serialización básica	Conversión de datos del programa a una representación almacenable.
Reconstrucción	Interpretación de la representación para recuperar valores.
Contenido derivado	Dato calculable a partir de otros, como un total.
Tolerancia	Política que permite continuar ante ciertos datos defectuosos.
Sobrescritura	Reemplazo del contenido anterior.
Agregación al final	Incorporación de contenido después de lo existente.
Costo oculto	Trabajo realizado internamente por una llamada breve.
Proyecto integrador	Solución que combina y justifica los aprendizajes principales.

Referencias del capítulo

CLRS Kit Español. (2026). Catálogo semántico de la biblioteca estándar y documentación de funciones de archivos [Repositorio de software].

CLRS Kit Español. (2026). Implementación de la biblioteca estándar: lectura, escritura, creación, comprobación, medición y eliminación de archivos [Código fuente].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2nd ed.). The MIT Press.

Cierre de la obra: el pseudocódigo dejó de ser una descripción informal y se convirtió en un medio para ejecutar, observar, probar, visualizar y conservar soluciones. El siguiente paso no es memorizar otro lenguaje: es transferir estos modelos mentales a nuevos lenguajes y problemas.

CAPÍTULO

2

El primer programa y su recorrido

Del archivo .clrs a una ejecución que puede observarse, seguirse y corregirse



El primer programa y su recorrido

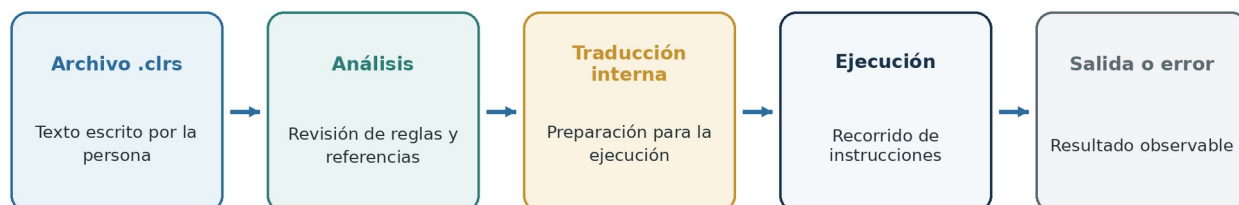
¿Qué ocurre desde que escribimos una instrucción hasta que aparece un resultado?

Una receta puede contener los pasos correctos y aun así fracasar si alguien los ejecuta en otro orden. Agregar el agua después de hornear, cobrar antes de calcular el total o mostrar una conversión antes de obtenerla no son cambios de estilo: alteran el resultado. En un programa, el orden también forma parte de la solución.

En el capítulo anterior convertimos situaciones en problemas definidos, identificamos entradas, procesos y salidas, y descompusimos soluciones en subtarear. Ahora cruzaremos el límite entre una estrategia escrita y una ejecución observable. Crearemos un archivo `.clrs`, reconoceremos su punto de entrada, seguiremos instrucciones una por una y aprenderemos a usar los diagnósticos como evidencia para corregir el programa.

Promesa de aprendizaje: al terminar podrás crear y ejecutar un programa pequeño en CLRS, explicar el papel de `PRINCIPAL`, la extensión `.clrs`, los comentarios y la indentación, elaborar una traza de su estado e interpretar los componentes básicos de un diagnóstico.

De un archivo a un resultado observable



Modelo simplificado: permite comprender el recorrido sin estudiar todavía el parser, el AST o el transpilador.

Figura 2.1. Recorrido simplificado desde el archivo `.clrs` hasta una salida o un error. Elaboración propia.

Alcance del capítulo: usaremos asignaciones y `escribir` para observar una ejecución, pero las variables, los tipos, las expresiones y la entrada con `leer` se estudiarán formalmente en capítulos posteriores. Aquí nos interesa el recorrido.

2.1 Ejecutar no es lo mismo que leer

Cuando una persona observa varias instrucciones puede anticipar su efecto sin detenerse en cada detalle. La computadora no realiza esa lectura global. Un entorno de ejecución procesa una representación válida y produce una secuencia concreta de acciones. Cada instrucción se alcanza en un momento determinado y opera con la información disponible en ese momento.

Para comenzar construiremos un modelo deliberadamente sencillo: el programa tiene un punto de entrada, recorre las instrucciones de su cuerpo de arriba hacia abajo y termina cuando ya no quedan instrucciones pendientes. Más adelante aparecerán decisiones, ciclos y funciones que desviarán o interrumpirán ese recorrido lineal.

Idea clave: el código es una descripción estática; la ejecución es el proceso dinámico que ocurre cuando sus instrucciones se recorren.

Elemento	Mientras observamos el archivo	Durante la ejecución
Instrucción	Texto que describe una acción.	Acción que se realiza cuando el recorrido llega a ella.
Orden	Posición de las líneas y estructura de bloques.	Secuencia efectiva en que se alcanzan las instrucciones.
Estado	No se ve por completo en el texto.	Conjunto de valores disponibles en ese momento.
Salida	Instrucciones que podrían comunicar información.	Texto o valores que realmente aparecen.
Error	Posible defecto todavía no observado.	Interrupción o diagnóstico producido por una condición concreta.

Pausa y piensa: si un programa contiene tres instrucciones correctas, ¿puede producir un resultado incorrecto únicamente por su orden? Construye un ejemplo cotidiano antes de continuar.

2.2 El archivo que reconoce CLRS Kit

CLRS Kit Español recomienda utilizar la extensión de archivo `.clrs`. Esa terminación permite que Visual Studio Code asocie el documento con las herramientas del lenguaje: resaltado, indentación, diagnósticos, comandos de ejecución, diagramas y otras funciones de la extensión. El contenido sigue siendo texto, pero el entorno ya sabe qué reglas debe aplicar.

Un nombre como `primer-programa.clrs` tiene dos partes: `primer-programa` identifica el archivo para la persona y `.clrs` identifica el lenguaje para el entorno. Cambiar únicamente la terminación no repara un contenido inválido, pero guardar el código con otra extensión puede impedir que algunas ayudas se activen.

Nombre	Interpretación	Recomendación
primer-programa.clrs	Archivo asociado con CLRS.	Adecuado.
primer-programa.txt	Archivo de texto genérico.	Puede perder soporte específico.
programa.clrs.txt	La última extensión es <code>.txt</code> .	Renombrar y comprobar extensiones visibles.
capitulo2.clrs	Nombre descriptivo dentro de una carpeta de trabajo.	Adecuado.

Advertencia: el archivo que se ejecuta suele ser el documento activo. Antes de interpretar un resultado extraño, comprueba qué pestaña está seleccionada y si guardaste los cambios más recientes.

2.3 `PRINCIPAL`: dónde comienza el recorrido

Un archivo puede contener varias funciones. Para que el entorno sepa cuál coordina la ejecución, CLRS permite usar `PRINCIPAL` como identificador del punto de entrada. En este libro lo escribiremos en mayúsculas para conservar una convención estable y reconocible.

Primer programa ejecutable

```
// Mi primer programa
PRINCIPAL()
    escribir "Hola desde CLRS"
    escribir "El programa terminó"
Las líneas desplazadas cuatro espacios forman el cuerpo de `PRINCIPAL`.
```

La primera línea comienza con `//`; es un comentario y no produce ninguna acción. La segunda identifica el punto de entrada. Las dos instrucciones indentadas forman su cuerpo. Al ejecutar, se muestra primero el saludo y después el mensaje final.

Anatomía de un primer programa CLRS

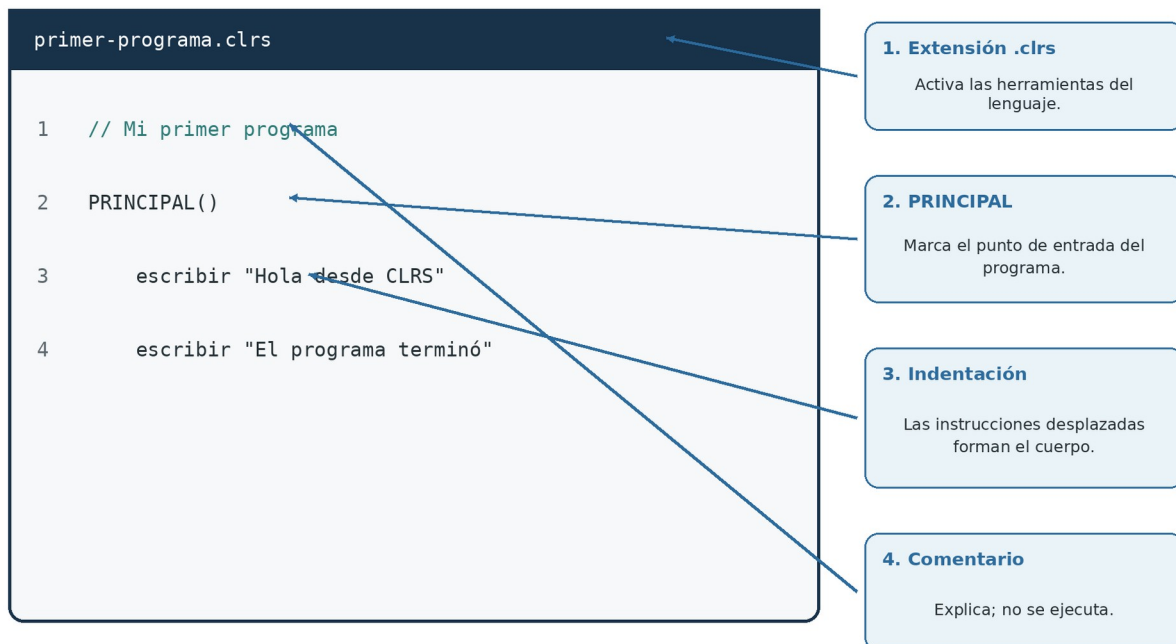


Figura 2.2. Componentes visibles de un primer programa CLRS. Elaboración propia.

Importante: no es necesario escribir `PRINCIPAL()` de nuevo para llamarlo. Cuando existe como punto de entrada, el entorno inicia allí la ejecución.

2.4 Comentarios, líneas y bloques

Los comentarios permiten explicar el propósito de una decisión, dejar una observación o separar una intención del código que la implementa. CLRS admite comentarios de una sola línea mediante `//`. Todo lo que aparece después de esos dos caracteres, en la misma línea, se interpreta como comentario.

Comentarios de una línea

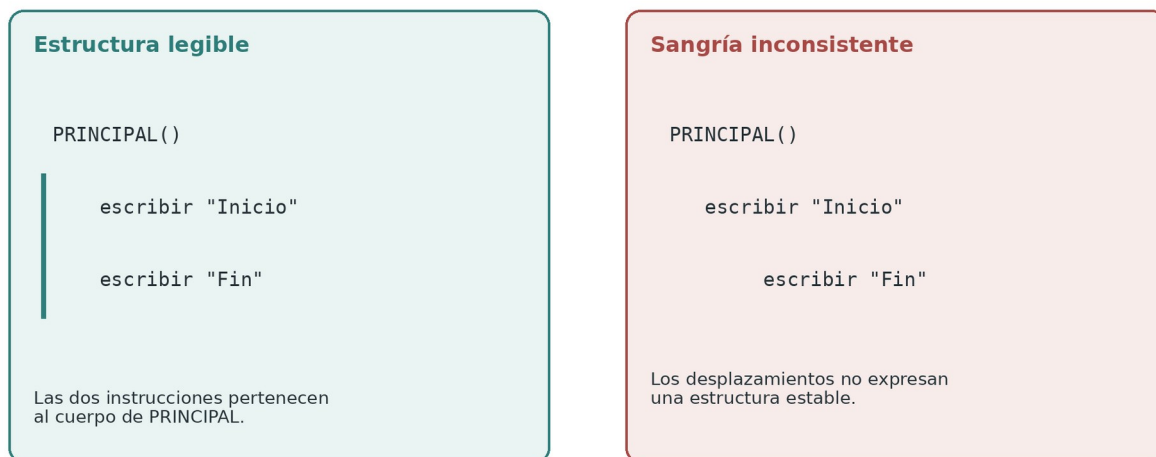
```
// Este comentario ocupa una línea completa
PRINCIPAL()
    escribir "Inicio" // Esta nota acompaña una instrucción
```

Un comentario útil explica por qué existe una decisión o qué supone el ejemplo. Repetir literalmente la instrucción agrega ruido: ``escribir "Inicio" // escribe Inicio`` no ayuda a comprender nada que el código no muestre ya.

Advertencia: un comentario no corrige el programa. Escribir “aquí se valida el dato” no realiza ninguna validación si no existe una instrucción que la implemente.

La indentación desplaza una línea hacia la derecha para mostrar que pertenece a un bloque. En CLRS no es un adorno opcional: participa en la estructura que reconoce el lenguaje. Dentro de ``PRINCIPAL``, todas las instrucciones de su cuerpo deben conservar una sangría coherente.

La indentación hace visible la pertenencia a un bloque



En CLRS, una tabulación avanza a intervalos de cuatro columnas; la consistencia importa más que decorar el código.

Figura 2.3. La indentación expresa qué instrucciones pertenecen al cuerpo de ``PRINCIPAL``. Elaboración propia.

Las tabulaciones avanzan a intervalos de cuatro columnas y pueden alinearse con espacios. Para aprender con menos errores, conviene configurar el editor para insertar siempre el mismo tipo de sangría y permitir que la extensión complete los bloques cuando corresponda.

Pausa y piensa: sin ejecutar, marca qué líneas pertenecen a ``PRINCIPAL`` en un ejemplo propio. Después borra la sangría de una línea y explica qué relación visual se pierde.

2.5 De guardar a ejecutar en Visual Studio Code

Visual Studio Code reúne archivos, extensiones y comandos en una misma interfaz. La extensión de CLRS incorpora acciones específicas para ejecutar el archivo activo, generar JavaScript, mostrar el costo algorítmico y construir diagramas. En este capítulo utilizaremos únicamente la ejecución directa.

1. Crea o abre una carpeta destinada a los ejercicios del libro.
2. Crea un archivo llamado ``primer-programa.clrs``.
3. Escribe el programa anterior respetando mayúsculas, comillas, paréntesis e indentación.
4. Guarda el archivo.
5. Abre la Paleta de comandos con ``Ctrl+Shift+P`` en Windows y Linux, o mediante el menú correspondiente.
6. Busca y selecciona ``Ejecutar código CLRS``.
7. Observa la salida producida por el entorno y compárala con tu predicción.

La Paleta de comandos de Visual Studio Code concentra acciones disponibles en el contexto actual. También puede existir un botón aportado por la extensión; la ruta mediante la paleta resulta útil porque hace visible el nombre exacto del comando.

Antes de ejecutar	Pregunta de comprobación
Archivo activo	¿La pestaña seleccionada es la que quiero ejecutar?
Extensión	¿El nombre termina realmente en <code>`clrs`</code> ?
Guardado	¿Los últimos cambios ya se conservaron?
Estructura	¿ <code>`PRINCIPAL()`</code> tiene un cuerpo indentado?
Predicción	¿Qué salida espero y en qué orden?

Hábito recomendado: predice la salida antes de ejecutar. La ejecución deja de ser una prueba al azar y se convierte en una comparación entre lo esperado y lo observado.

2.6 El estado: la información disponible ahora

Para seguir programas que realizan cálculos necesitamos observar algo que no aparece como una ventana independiente: su estado. En este capítulo llamaremos estado al conjunto de valores disponibles en un momento de la ejecución. Cada asignación puede incorporar o modificar una parte de ese estado.

Notación provisional: la forma ``nombre <- valor`` indica que el nombre queda asociado con el valor situado a la derecha. En el próximo capítulo estudiaremos con precisión variables, valores, literales y tipos.

Conversión con un valor conocido

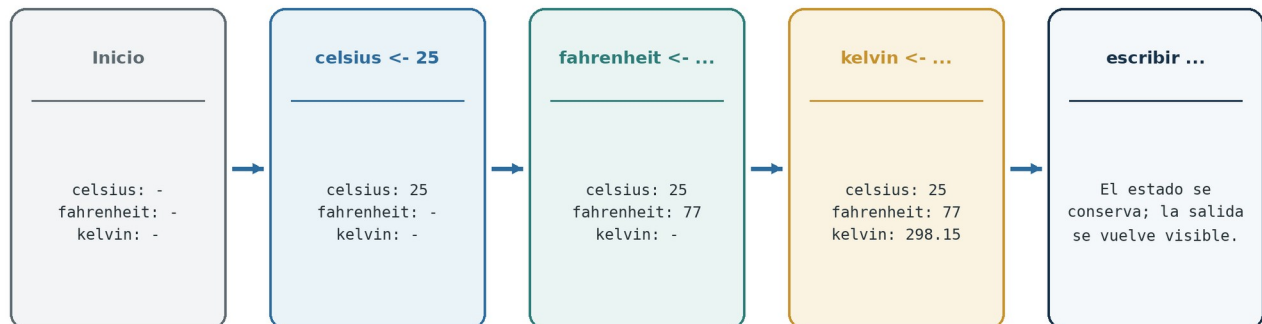
```
PRINCIPAL()
  celsius <- 25
  fahrenheit <- ((9 / 5) * celsius) + 32
  kelvin <- celsius + 273.15
```

```
  escribir "Temperatura en F: " + fahrenheit
  escribir "Temperatura en K: " + kelvin
```

Este ejemplo adapta uno de los programas proporcionados para concentrarse en la secuencia. La lectura y validación se incorporarán más adelante.

El programa fija una temperatura de 25 grados Celsius, calcula dos conversiones y después presenta los resultados. Si las expresiones se evalúan como esperamos, ``fahrenheit`` queda asociado con 77 y ``kelvin`` con 298.15.

Una ejecución como secuencia de cambios de estado



Estado: conjunto de valores disponibles en un momento de la ejecución.

Traza: registro ordenado de instrucciones, valores y salidas observadas.

El guion (-) indica que todavía no se ha establecido un valor en la traza didáctica.

Figura 2.4. Evolución del estado durante la conversión de temperatura. Elaboración propia.

La figura no representa cajas físicas dentro de la computadora. Es un modelo para razonar: después de cada instrucción anotamos qué información debería estar disponible. Esa anotación es una traza.

Paso	Instrucción alcanzada	Estado relevante después del paso	Salida
1	<code>`celsius <- 25`</code>	celsius = 25	-
2	<code>`fahrenheit <- ((9 / 5) * celsius) + 32`</code>	celsius = 25; fahrenheit = 77	-
3	<code>`kelvin <- celsius + 273.15`</code>	celsius = 25; fahrenheit = 77; kelvin = 298.15	-
4	<code>`escribir "Temperatura en F: " + fahrenheit`</code>	Sin cambio relevante	Temperatura en F: 77
5	<code>`escribir "Temperatura en K: " + kelvin`</code>	Sin cambio relevante	Temperatura en K: 298.15

Pausa y piensa: cambia mentalmente ``celsius`` a 0, -40 y 100. Predice ambas salidas antes de ejecutar cada caso. Si tu predicción difiere, revisa la expresión paso a paso.

2.7 Por qué el orden cambia el resultado

Dos programas pueden contener las mismas operaciones y no ser equivalentes. Cada instrucción utiliza el estado disponible cuando se alcanza. Si modificamos ese estado antes de una operación, la operación trabaja con valores diferentes.

Programa A: aumentar y después duplicar

```
PRINCIPAL()
  cantidad <- 10
  cantidad <- cantidad + 5
  cantidad <- cantidad * 2
  escribir cantidad
```

Programa B: duplicar y después aumentar

```
PRINCIPAL()
  cantidad <- 10
  cantidad <- cantidad * 2
  cantidad <- cantidad + 5
  escribir cantidad
```

El Programa A produce 30: primero obtiene 15 y después lo duplica. El Programa B produce 25: primero obtiene 20 y después agrega 5. Las instrucciones de actualización son las mismas, pero su composición no es conmutativa.

Momento	Programa A	Programa B
Inicial	cantidad = 10	cantidad = 10
Después de la segunda asignación	cantidad = 15	cantidad = 20
Después de la tercera asignación	cantidad = 30	cantidad = 25
Salida	30	25

Idea clave: una traza explica no solo qué valor aparece al final, sino cómo cada instrucción produjo el estado utilizado por la siguiente.

Contraejemplo: sumar 5 y después sumar 3 produce lo mismo que invertir esas dos sumas. Esto no significa que el orden nunca importe; significa que algunas operaciones particulares admiten reordenamiento y otras no.

2.8 Salida visible y estado interno

No todo lo que ocurre durante la ejecución aparece en la consola. Las asignaciones de la conversión modifican el estado, pero solo `escribir` vuelve visible una representación de los valores. Por eso una salida correcta puede depender de varios pasos que el usuario nunca observa directamente.

Mismo cálculo, distinta observabilidad

```
PRINCIPAL()  
  subtotal <- 800  
  impuestos <- subtotal * 0.12  
  total <- subtotal + impuestos
```

Este programa puede terminar sin mostrar nada. El cálculo existe en su estado, pero no se solicitó una salida. Agregar `escribir total` hace visible el resultado; no crea el cálculo, únicamente lo comunica.

Cálculo con salida

```
PRINCIPAL()  
  subtotal <- 800  
  impuestos <- subtotal * 0.12  
  total <- subtotal + impuestos  
  escribir "Total: " + total
```

Advertencia: que la consola esté vacía no demuestra por sí solo que el programa no se ejecutó. Puede significar que ninguna instrucción produjo salida o que el recorrido no llegó a ella.

Los ejemplos de salario y promedio que proporcionaste separan cálculo y presentación mediante funciones. Esa organización se estudiará más adelante. Por ahora conservamos los pasos en `PRINCIPAL` para que el recorrido sea visible sin introducir llamadas, parámetros ni retornos al mismo tiempo.

2.9 Cómo construir una prueba de escritorio

Una prueba de escritorio consiste en simular manualmente una ejecución utilizando valores concretos. No reemplaza la ejecución real, pero obliga a explicar qué ocurre en cada paso y ayuda a localizar la primera diferencia entre lo esperado y lo observado.

8. Numera las instrucciones ejecutables del fragmento.
9. Anota el estado inicial conocido.
10. Recorre una instrucción a la vez.
11. Evalúa primero la parte derecha de una asignación con los valores actuales.
12. Actualiza únicamente el nombre situado a la izquierda.
13. Registra cualquier salida en una columna separada.
14. Detente en la primera instrucción inválida o en el final del programa.

Importante: no completes la traza con el valor que “debería” producir el programa según el enunciado. Registra el valor que realmente resulta de las instrucciones escritas.

Programa para trazar

```
PRINCIPAL( )
  precio <- 120
  descuento <- precio * 0.10
  precioFinal <- precio - descuento
  escribir precioFinal
```

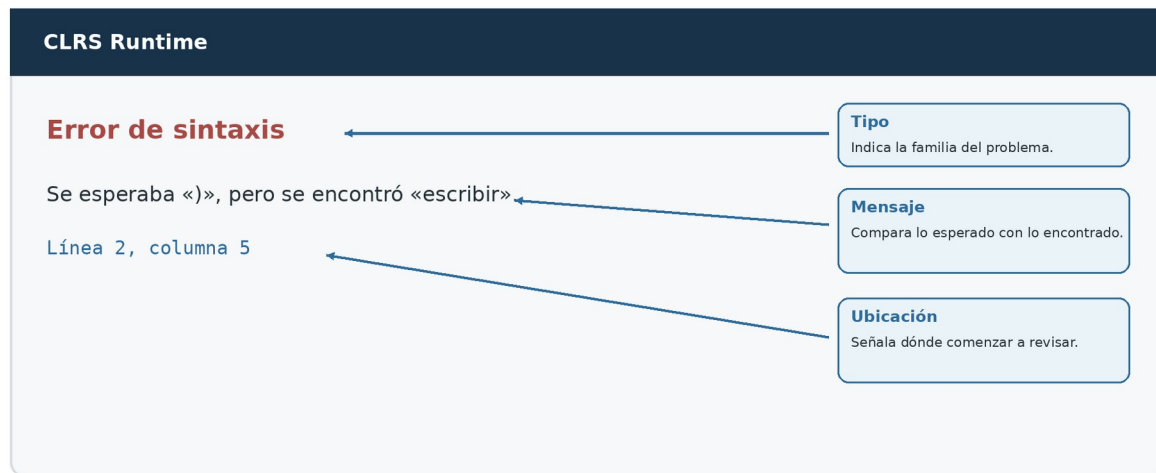
Paso	precio	descuento	precioFinal	Salida
Inicio	-	-	-	-
1	120	-	-	-
2	120	12	-	-
3	120	12	108	-
4	120	12	108	108

Pausa y piensa: ¿qué cambiaría si la tercera asignación fuera `precioFinal <- precio + descuento`? La sintaxis seguiría siendo válida, pero el algoritmo ya no representaría un descuento.

2.10 Los errores como información

Cuando el entorno no puede analizar o ejecutar el código, produce un diagnóstico. El objetivo no es castigar a quien programa, sino hacer observable una discrepancia entre las reglas esperadas y el texto encontrado, o entre la ejecución esperada y lo que realmente ocurrió.

Cómo leer un diagnóstico sin adivinar



La ubicación es un punto de partida. El origen real puede estar en el símbolo anterior, por ejemplo un paréntesis que nunca se cerró.

Figura 2.5. Partes básicas de un diagnóstico de CLRS. Elaboración propia a partir del formato documentado por el proyecto.

Categoría	Qué significa en este nivel	Ejemplo inicial
Sintaxis	El texto no cumple una regla de la estructura del lenguaje.	Falta un paréntesis o una comilla.
Semántica	La estructura puede reconocerse, pero una referencia o uso no satisface las reglas del lenguaje.	Usar un nombre o llamada de manera no permitida.
Ejecución	El programa comenzó a ejecutarse y falló durante el recorrido.	Una operación recibe un valor que no puede manejar.
Lógica	El programa se ejecuta, pero no resuelve correctamente el problema.	Suma un descuento en lugar de restarlo.

Los tres primeros tipos pueden ser informados por las herramientas de CLRS según la fase donde aparezcan. El error lógico suele requerir comparar la salida con una especificación, una predicción o casos de prueba. Ningún mensaje puede saber por sí solo cuál era tu intención.

Ejemplo con un paréntesis sin cerrar

```
PRINCIPAL(
  escribir "Hola"
```

El mensaje puede señalar la línea siguiente porque allí se vuelve evidente que faltaba ``).

15. Lee el tipo de error sin intentar corregir todavía.
16. Ubica la línea y la columna indicadas.
17. Revisa el símbolo señalado y el inmediatamente anterior.
18. Compara con una forma válida conocida.
19. Realiza un solo cambio que tenga una causa explicable.
20. Guarda y vuelve a ejecutar.

Advertencia: cambiar muchos símbolos al azar puede hacer desaparecer un mensaje y crear otro problema. Una corrección útil parte de una hipótesis: “falta cerrar el paréntesis iniciado en la línea 1”.

2.11 Primer catálogo de problemas frecuentes

Síntoma	Posible causa	Comprobación útil
No aparece el comando de CLRS.	La extensión no está instalada, habilitada o el archivo no se reconoce como CLRS.	Revisar la vista Extensiones y la terminación <code>\.cls`</code> .
Se ejecuta una versión anterior.	Los cambios no se guardaron o se ejecutó otra pestaña.	Guardar y comprobar el archivo activo.
La salida aparece en orden inesperado.	Las instrucciones están en otro orden o existe una estructura que desvía el flujo.	Realizar una traza desde <code>`PRINCIPAL`</code> .
No aparece salida.	No existe <code>`escribir`</code> o el recorrido no alcanza esa instrucción.	Identificar las instrucciones de salida.
El editor marca un bloque.	La indentación es inconsistente.	Alinear el cuerpo a cuatro columnas.
Se esperaba un símbolo.	Falta cerrar una comilla, paréntesis o expresión.	Revisar el símbolo anterior a la ubicación.
El resultado es numéricamente incorrecto.	El orden o la expresión no representa el proceso esperado.	Comparar la traza con la especificación.

Idea clave: antes de atribuir el problema al entorno, identifica qué evidencia observas: comando ausente, diagnóstico, salida vacía, valor incorrecto o archivo equivocado. Cada síntoma orienta una revisión distinta.

2.12 Laboratorio CLRS: temperatura en dos escalas

Propósito: crear, ejecutar, trazar y modificar una versión reducida del programa de conversión de temperatura. La lectura y la validación del dato se omiten para no introducir todavía ciclos ni comprobación de tipos.

Crea el archivo `conversion-temperatura.clrs` y copia el siguiente programa:

Versión inicial del laboratorio

```
// Convierte una temperatura conocida
PRINCIPAL()
  celsius <- 25
  fahrenheit <- ((9 / 5) * celsius) + 32
  kelvin <- celsius + 273.15

  escribir "Celsius: " + celsius
  escribir "Fahrenheit: " + fahrenheit
  escribir "Kelvin: " + kelvin
```

Antes de ejecutar, completa esta tabla:

Caso	Celsius	Fahrenheit esperado	Kelvin esperado
A	25		
B	0		
C	-40		
D	100		

21. Ejecuta el caso A y compara línea por línea con tu predicción.
22. Sustituye únicamente el valor de `celsius` para los casos B, C y D.
23. Guarda antes de cada ejecución.
24. Introduce deliberadamente el error `PRINCIPAL(` y registra el diagnóstico.
25. Restaura el paréntesis y elimina una comilla de la última salida. Compara el nuevo mensaje.
26. Corrige el archivo y verifica que las tres salidas vuelvan a aparecer.

Pregunta de reflexión: ¿qué información obtuviste de la ejecución que no era visible al leer el archivo? ¿Qué información obtuviste de la traza que la salida final no mostraba?

2.13 Primera versión ejecutable del caso longitudinal

El caso del libro es un registro y análisis de gastos. Todavía no solicitaremos datos al usuario ni almacenaremos varios movimientos. Construiremos una versión mínima con un gasto conocido para practicar estado, orden y salida.

Registro de un gasto conocido

```
// Primera versión del registro de gastos
PRINCIPAL()
  concepto <- "Transporte"
  monto <- 85
  saldoInicial <- 500
  saldoRestante <- saldoInicial - monto

  escribir "Gasto registrado: " + concepto
  escribir "Monto: " + monto
  escribir "Saldo restante: " + saldoRestante
```

Paso	Cambio de estado	Salida acumulada
1	concepto = "Transporte"	-
2	monto = 85	-
3	saldoInicial = 500	-
4	saldoRestante = 415	-
5	Sin cambio	Gasto registrado: Transporte
6	Sin cambio	Monto: 85
7	Sin cambio	Saldo restante: 415

Este programa ya es ejecutable, pero sigue siendo poco general: los datos están escritos dentro del archivo. Esa limitación es deliberada. Primero aprendemos a observar una ejecución estable; después permitiremos que el estado dependa de entradas proporcionadas durante la ejecución.

Conexión hacia adelante: el capítulo siguiente explicará qué son exactamente ``concepto``, ``monto``, ``saldoInicial`` y ``saldoRestante``; por qué algunos valores llevan comillas; y qué significa que una variable cambie.

2.15 Reconstruye el recorrido

Sin volver a leer las secciones anteriores, intenta explicar el recorrido completo de un programa CLRS utilizando estas palabras: archivo, extensión, `PRINCIPAL`, bloque, instrucción, estado, salida, diagnóstico y traza.

27. El archivo `.clrs` contiene la representación escrita.
28. La extensión permite que el entorno active las herramientas del lenguaje.
29. `PRINCIPAL` establece el punto de entrada.
30. La indentación delimita las instrucciones que pertenecen al cuerpo.
31. La ejecución alcanza instrucciones en un orden concreto.
32. Las asignaciones construyen o modifican el estado.
33. `escribir` vuelve visible parte de la información.
34. Una traza registra los estados y salidas paso a paso.
35. Los diagnósticos aportan tipo, mensaje y ubicación para iniciar la corrección.

Comprobación activa: cubre la lista anterior e intenta reconstruirla con tus propias palabras. Después explica qué evidencia utilizarías para demostrar cada paso.

Ejercicios

Realiza primero una predicción o traza. Ejecuta únicamente después de haber escrito qué esperas observar.

1. **[Reconocer]** Escribe el programa mínimo que muestre dos mensajes en orden. Explica por qué invertir las líneas cambia la salida.
2. **[Comprender]** Clasifica como texto estático o evento de ejecución: archivo `.clrs``, instrucción, estado actual, salida mostrada, comentario y diagnóstico producido.
3. **[Comprender]** Explica la función de ``PRINCIPAL`` sin utilizar la frase “porque así lo pide la sintaxis”.
4. **[Aplicar]** Corrige un archivo llamado ``ejemplo.clrs.txt`` y explica por qué ocultar extensiones en el sistema puede causar confusión.
5. **[Analizar]** Escribe tres comentarios: uno útil, uno redundante y uno engañoso. Explica la diferencia.
6. **[Aplicar]** Construye una traza del Programa A de la sección 2.7 y verifica el resultado 30.
7. **[Construir]** Modifica el Programa B para que produzca 30 sin copiar exactamente el Programa A. Justifica el nuevo orden.
8. **[Aplicar]** Un programa calcula ``area <- base * altura`` y no muestra nada. Explica si existe cálculo, salida o ambos. Después agrega una salida.
9. **[Aplicar]** Diseña una tabla de traza para un programa que aplica 15% de descuento a un precio de 240.
10. **[Analizar]** Identifica el tipo probable de problema: falta una comilla; se suma un descuento; la extensión no aparece; el programa divide durante la ejecución por un valor inválido.
11. **[Construir]** Escribe un diagnóstico ficticio con tipo, mensaje, línea y columna para un paréntesis faltante. Después explica qué revisarías primero.
12. **[Construir]** Crea un programa con tres asignaciones cuya salida cambie al invertir dos de ellas. Incluye ambas trazas.
13. **[Construir]** Adapta el programa del círculo proporcionado para utilizar un radio fijo y mostrar radio, área y perímetro, sin ``leer``, ``mientras`` ni validación.
14. **[Construir]** Adapta el programa de salario neto a una versión lineal dentro de ``PRINCIPAL`` con salario bruto fijo. Traza impuestos, cotización, seguro y salario neto.
15. **[Analizar]** Explica por qué la versión lineal del ejercicio anterior es útil para aprender el recorrido, pero menos organizada que la versión con ``OBTSALNETO``.
16. **[Experimentar]** Crea un archivo con un error sintáctico deliberado, registra el mensaje exacto, formula una hipótesis, corrígelo y describe la evidencia de que la corrección funcionó.
17. **[Construir]** Construye la segunda versión del caso longitudinal cambiando concepto, monto y saldo inicial. Predice y verifica el saldo restante.

Mini proyecto: recibo de una compra conocida

Situación: una tienda desea mostrar un recibo elemental a partir de datos escritos inicialmente dentro del programa: producto, precio unitario, cantidad y dinero recibido.

El programa debe calcular subtotal, cambio y mostrar cada dato en un orden comprensible. Para conservar el alcance del capítulo, utiliza valores conocidos y supone que el pago es suficiente.

Entregable	Criterio
Archivo `recibo.clrs`	Se reconoce como archivo CLRS y contiene `PRINCIPAL`.
Programa	Incluye comentarios útiles, sangría coherente, asignaciones y salidas.
Predicción	Indica la salida completa antes de ejecutar.
Traza	Registra producto, precio, cantidad, subtotal, pago y cambio.
Experimento	Cambia únicamente la cantidad y explica qué estados y salidas se modifican.
Diagnóstico	Introduce y corrige un error de paréntesis o comillas.

Estructura inicial sugerida

```
PRINCIPAL( )
  producto <- "Cuaderno"
  precio <- 48
  cantidad <- 3
  pago <- 200

  subtotal <- precio * cantidad
  cambio <- pago - subtotal

  // Agrega aquí las salidas del recibo
```

Criterio de éxito: otra persona debe poder leer tu traza y reconstruir por qué cada valor estaba disponible cuando se produjo la salida.

Autoevaluación

Selecciona la descripción que mejor represente lo que puedes demostrar con un archivo nuevo, no solo lo que recuerdas del capítulo.

Criterio	Inicial	En desarrollo	Dominio esperado
Crear y ejecutar	Necesito copiar todos los pasos.	Creo el archivo, pero confundo guardado, pestaña o comando.	Creo, guardo y ejecuto el archivo correcto de forma consciente.
Explicar `PRINCIPAL`	Lo identifico sin explicar su función.	Sé que inicia el programa, pero no relaciono cuerpo e indentación.	Explico punto de entrada, bloque y orden inicial.
Trazar estado	Solo anoto la salida final.	Registro algunos valores, pero salto actualizaciones.	Sigo una instrucción a la vez y separo estado de salida.
Interpretar diagnósticos	Cambio símbolos al azar.	Uso línea y mensaje, pero no formulo una causa.	Identifico tipo, ubicación e hipótesis antes de corregir.
Prededir	Ejecuto primero y explico después.	Predigo ejemplos simples.	Predigo, ejecuto y explico cualquier diferencia.

Criterio para avanzar: crea un programa lineal nuevo con al menos cuatro cambios de estado y tres salidas. Predice, traza, ejecuta y corrige un error deliberado. Si puedes explicar todo el recorrido sin depender únicamente de la consola, estás listo para estudiar valores y variables.

Glosario

Término	Definición en este capítulo
Archivo <code>.clrs`</code>	Documento de texto asociado con las herramientas de CLRS Kit Español.
Bloque	Conjunto de instrucciones que pertenecen a una estructura y se expresa mediante indentación.
Comentario	Texto explicativo iniciado con <code>`//`</code> que no se ejecuta.
Diagnóstico	Información producida por una herramienta para describir un error o problema detectado.
Ejecución	Proceso dinámico de recorrer instrucciones y producir efectos.
Estado	Conjunto de valores disponibles en un momento de la ejecución.
Instrucción	Unidad del programa que describe una acción reconocible por el lenguaje.
Punto de entrada	Lugar desde el que comienza el recorrido principal del programa.
Salida	Información que el programa hace observable mediante una instrucción de comunicación.
Traza	Registro ordenado de instrucciones, estados y salidas de una ejecución concreta.

Referencias del capítulo

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2nd ed.). The MIT Press.

CLRS Kit Español. (2026). CLRS Kit Español, versión 1.3.1 [Documentación técnica del proyecto].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Microsoft. (n.d.). Extension Marketplace. Visual Studio Code Documentation. Recuperado el 1 de agosto de 2026.

Microsoft. (n.d.). User interface. Visual Studio Code Documentation. Recuperado el 1 de agosto de 2026.

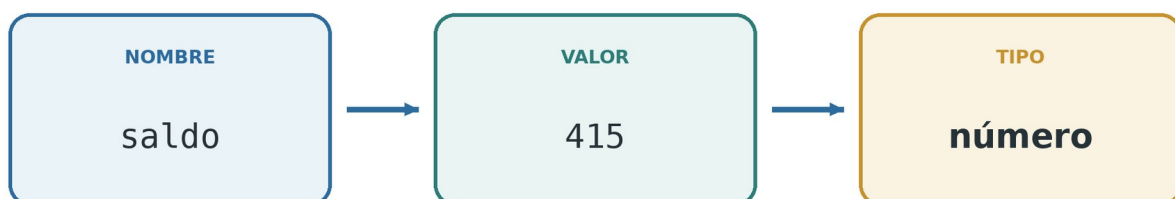
Siguiente capítulo: Los valores que forman el estado de un programa. Estudiaremos qué son los literales, identificadores y variables; cómo una asignación crea o actualiza información; y por qué número, cadena y lógico no son intercambiables en todos los contextos.

CAPÍTULO

3

Los valores que forman el estado de un programa

Nombres, datos y tipos para representar información durante la ejecución



Los valores que forman el estado de un programa

¿Cómo conserva un programa la información que cambia mientras se ejecuta?

Un recibo puede mostrar “85”, “Transporte” y “Pagado”. Para una persona, los tres elementos tienen significados distintos aunque aparezcan impresos con la misma tinta. Para un programa también importa distinguir si algo representa una cantidad, un texto o una afirmación lógica. Esa diferencia determina qué operaciones tienen sentido y qué resultados puede producir.

En el capítulo anterior seguimos el recorrido de un programa y utilizamos la palabra estado para describir la información disponible en un momento de la ejecución. Ahora abriremos ese estado: estudiaremos qué es un valor, cómo se escribe mediante un literal, cómo recibe un nombre y qué significa que CLRS permita cambiar tanto el valor como su tipo durante la ejecución.

Promesa de aprendizaje: al terminar podrás distinguir dato, valor, literal, identificador y variable; reconocer números, cadenas y valores lógicos; explicar la asignación y la mutabilidad; y utilizar comprobaciones y conversiones de tipo sin confundir flexibilidad con ausencia de reglas.

Un nombre permite volver a encontrar un valor



La variable no es el valor: es el nombre mediante el cual el programa accede al valor asociado en ese momento.

En CLRS el nombre puede asociarse después con otro valor, incluso de otro tipo.

Figura 3.1. Relación entre identificador, valor actual y tipo del valor. Elaboración propia.

Alcance del capítulo: utilizaremos asignaciones y salidas pequeñas para observar valores, pero dejaremos las expresiones aritméticas, relacionales, lógicas y la entrada con `leer` para el capítulo siguiente.

3.1 El estado no es una sola cosa

Cuando ejecutamos el registro de gastos del capítulo anterior, el estado contenía asociaciones como ``concepto = "Transporte"`, `monto = 85` y `saldoRestante = 415``. Cada asociación reúne tres ideas diferentes: un nombre elegido por quien programa, un valor disponible en ese momento y una clase de valor que determina cómo puede interpretarse.

Pregunta	Ejemplo	Respuesta
¿Cómo lo encontramos?	<code>`monto`</code>	Mediante un identificador.
¿Qué información conserva ahora?	<code>`85`</code>	Un valor.
¿Cómo se escribió directamente?	<code>`85`</code> en el código	Mediante un literal.
¿Qué clase de valor es?	número	Mediante su tipo.

Idea clave: una variable es un nombre asociado con un valor. El valor puede cambiar durante la ejecución; el nombre permite volver a localizar la información.

La palabra variable suele explicarse como una “caja”. Esa analogía ayuda a imaginar que algo se conserva, pero es incompleta. Una variable no es necesariamente una caja física ni contiene el valor de la misma forma en todos los lenguajes. Para este libro será más preciso pensar en una asociación: el nombre ``monto`` permite acceder al valor que tiene asignado en el estado actual.

Pausa y piensa: en ``saldoRestante <- 415``, identifica por separado el nombre, el símbolo de asignación y el valor. ¿Cuál de esos elementos podría permanecer igual aunque el saldo cambiara?

3.2 Dato, valor y representación

En conversación cotidiana usamos “dato” y “valor” casi como sinónimos. En programación conviene distinguirlos. Un dato es una unidad de información considerada por el problema; un valor es la forma concreta que esa información adopta dentro de una ejecución. La representación es la manera en que ese valor se escribe, almacena o muestra.

Situación	Dato del problema	Valor en una ejecución	Representación en CLRS
Registrar un gasto	monto del gasto	85	<code>`85`</code>
Nombrar el gasto	concepto	Transporte	<code>`"Transporte"`</code>
Indicar si es necesario	condición lógica	verdadero	<code>`VERDAD`</code>
Esperar texto todavía vacío	descripción	cadena sin caracteres	<code>`""`</code>

La representación importa porque el lenguaje debe reconocer dónde comienza y termina cada valor. Las comillas de una cadena no forman parte del texto conceptual que queremos registrar, pero sí forman parte de la escritura del literal en el código. Sin ellas, el lenguaje intentaría interpretar ``Transporte`` como un nombre, no como texto.

Importante: el valor y su representación no son idénticos. El número veinticinco puede representarse como ``25``; el texto formado por los caracteres 2 y 5 se representa como ```"25"```.

3.3 Literales: valores escritos directamente

Un literal es una forma de escribir un valor directamente en el código. En ``monto <- 85``, el fragmento ``85`` es un literal numérico. En ``concepto <- "Transporte"```, el fragmento entre comillas es un literal de cadena. En ``esNecesario <- VERDAD``, la palabra reservada representa un valor lógico.

Tres familias de valores literales en CLRS



Las comillas no son decoración: forman parte de la representación escrita y cambian la clase de valor.

Por eso `25` y `"25"` se parecen visualmente, pero no representan lo mismo para el programa.

Figura 3.2. Familias de literales utilizadas en CLRS. Elaboración propia.

Literal	Valor interpretado	Tipo
<code>`25`</code>	cantidad veinticinco	Número
<code>`-40`</code>	cantidad negativa cuarenta	Número
<code>`273.15`</code>	cantidad con parte decimal	Número
<code>``"25"``</code>	texto formado por los caracteres 2 y 5	Cadena
<code>``""``</code>	cadena vacía	Cadena
<code>`VERDAD`</code>	afirmación lógica verdadera	Lógico
<code>`FALSO`</code>	afirmación lógica falsa	Lógico

Contraejemplo: ```"VERDAD"``` no es el valor lógico ``VERDAD``; es una cadena que contiene siete caracteres. Del mismo modo, ```"85"``` no es el número ``85``.

3.4 Números: representar cantidades

Los valores numéricos permiten representar cantidades que pueden participar en cálculos: temperaturas, salarios, porcentajes, radios, calificaciones y montos. CLRS trata los valores numéricos de forma dinámica; no exige declarar por separado si una cantidad es entera o real antes de asignarla.

Valores numéricos

```
PRINCIPAL()
  edad <- 23
  temperatura <- -40
  porcentaje <- 12.5
  saldo <- 0

  escribir edad
  escribir temperatura
  escribir porcentaje
  escribir saldo
```

Aunque 23 no tenga parte decimal escrita y 12.5 sí, ambos son valores numéricos para las operaciones habituales de CLRS. En este nivel nos interesa más la intención del dato que una clasificación interna detallada entre enteros y reales.

Advertencia: una cifra no se vuelve numérica solo por parecer número. Si aparece entre comillas, su tipo es cadena. Esa diferencia será decisiva al construir cálculos en el capítulo siguiente.

Pausa y piensa: clasifica `0`, `-0.5`, `"0"`, `"-0.5"` y `FALSO`. Después explica qué pista visual utilizaste.

3.5 Cadenas: conservar texto sin interpretarlo como nombre

Una cadena es una secuencia de caracteres utilizada como texto. Puede contener letras, números, espacios, signos y caracteres acentuados. En CLRS se escribe entre comillas dobles. Las comillas permiten que el lenguaje distinga el contenido textual de los identificadores y palabras reservadas.

Cadenas con contenidos distintos

```
PRINCIPAL()
  concepto <- "Transporte"
  folio <- "A-102"
  codigoPostal <- "06000"
  mensaje <- "El gasto fue registrado"
  vacia <- ""

  escribir concepto
  escribir folio
  escribir codigoPostal
  escribir mensaje
```

El código postal se conserva como cadena porque sus caracteres identifican una zona; no representa una cantidad con la que pretendamos sumar o multiplicar. Además, una cadena permite conservar ceros iniciales que podrían perderse si el dato se interpretara como número.

Contenido	¿Número o cadena?	Razón
Edad de una persona: 23	Número	Representa una cantidad que puede compararse o calcularse.
Código de empleado: "0023"	Cadena	Identifica; los ceros forman parte del código.
Monto: 23.50	Número	Representa una cantidad monetaria.
Folio: "23.50"	Cadena	El punto forma parte del identificador textual.

Idea clave: el tipo adecuado depende del significado del dato, no solo de su apariencia.

3.6 Valores lógicos: representar afirmaciones

Un valor lógico expresa una de dos posibilidades: `VERDAD` o `FALSO`. Se utiliza para representar condiciones como “el gasto fue pagado”, “el dato es válido” o “la cuenta está activa”. En este capítulo los asignaremos directamente; más adelante aprenderemos a producirlos mediante comparaciones y operadores lógicos.

Estado lógico de un registro

```
PRINCIPAL ( )
  fuePagado <- VERDAD
  requiereFactura <- FALSO

  escribir fuePagado
  escribir requiereFactura
```

El valor lógico no es una palabra decorativa. Permite que el programa trate una afirmación como información. En capítulos posteriores, una decisión podrá consultar ese valor y elegir un camino distinto.

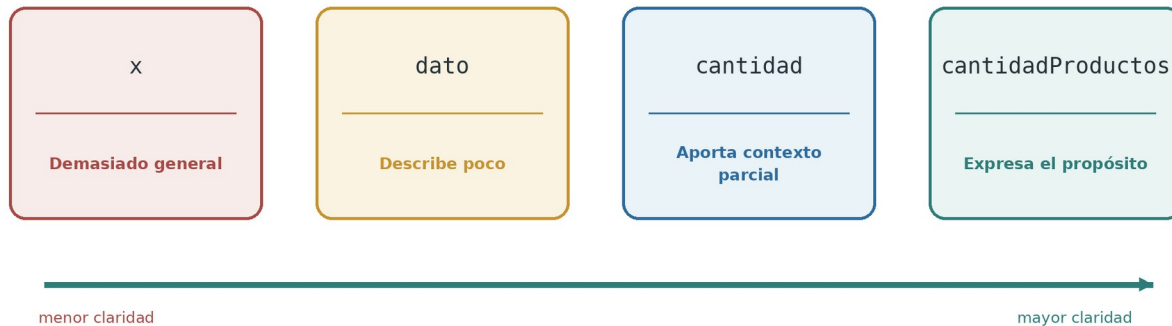
Nombre sugerido	Valor posible	Pregunta que representa
`fuePagado`	VERDAD o FALSO	¿El pago ya se realizó?
`esValido`	VERDAD o FALSO	¿El dato cumple las reglas?
`tieneDescuento`	VERDAD o FALSO	¿Se debe aplicar descuento?
`continuar`	VERDAD o FALSO	¿El proceso debe seguir?

Advertencia: utilizar `FALSO` como valor inicial de una variable que después debe contener un número es posible en CLRS, pero mezcla dos significados: “todavía no hay número” y “la afirmación es falsa”. Más adelante veremos alternativas de validación y diseño.

3.7 Identificadores: nombrar para comprender

Un identificador es el nombre con el que el programa se refiere a una variable, una función u otro elemento. En este capítulo nos concentraremos en variables. Un buen identificador no solo cumple las reglas del lenguaje; también comunica qué representa el valor.

Un identificador debe revelar la intención



El nombre más largo no siempre es mejor; debe ser tan específico como el contexto lo requiere.

Figura 3.3. Diferentes niveles de claridad en los identificadores. Elaboración propia.

Nombre	Evaluación	Mejora posible
`x`	Solo resulta claro en un contexto matemático muy breve.	`radio`, `temperatura` o `monto`.
`dato`	Indica que existe información, pero no cuál.	`salarioBruto` o `notaFinal`.
`num1`	Puede servir en un ejercicio genérico, pero pierde significado en programas reales.	`dividendo`, `precioUnitario`.
`saldoRestante`	Describe el propósito y el momento del valor.	Conservar si el contexto lo justifica.

CLRS es sensible a mayúsculas y minúsculas. Por tanto, `monto`, `Monto` y `MONTO` pueden interpretarse como identificadores distintos. Para variables utilizaremos nombres descriptivos en estilo camelCase: la primera palabra comienza en minúscula y las siguientes inician con mayúscula, como `salarioBruto` o `cantidadProductos`.

- El nombre debe describir el papel del dato, no su valor actual.
- Evita abreviaturas que solo tú puedas recordar.
- No incluyas el tipo en el nombre cuando no aporte comprensión.
- Conserva una convención estable dentro del programa.
- Usa nombres más breves cuando el contexto sea pequeño y matemático; usa nombres más específicos cuando el programa crezca.

Pausa y piensa: propón identificadores para: el radio de un círculo, la temperatura en Celsius, el salario después de deducciones y la respuesta a “¿es mayor de edad?”.

3.8 La primera asignación crea la variable

CLRS no utiliza una instrucción separada para declarar variables. La primera asignación con ``<-`` introduce el nombre y le proporciona un valor inicial. Por eso una variable no puede crearse sin un valor: su aparición útil en el estado comienza con una asociación concreta.

Primera asignación y actualizaciones

```
PRINCIPAL ( )
  saldo <- 500
  saldo <- 415
  saldo <- 300
  escribir saldo
```

La primera asignación crea la variable; las siguientes reutilizan el mismo nombre y cambian el valor actual.

Momento	Instrucción	Estado de `saldo`
Antes de la primera asignación	-	No existe una asociación disponible en esta traza.
Después de `saldo <- 500`	Primera asignación	500
Después de `saldo <- 415`	Actualización	415
Después de `saldo <- 300`	Actualización	300

Idea clave: el símbolo ``<-`` se lee “recibe” o “queda asociado con”. La información fluye desde la expresión de la derecha hacia el nombre de la izquierda.

Advertencia: la asignación no debe leerse como una igualdad matemática permanente. ``saldo <- 415`` describe un cambio de estado; una instrucción posterior puede asignar otro valor.

3.9 Mutabilidad: el estado puede cambiar

Una variable es mutable cuando su valor puede actualizarse después de su creación. En CLRS todas las variables son mutables; el lenguaje no dispone de constantes. Esta característica permite representar procesos: un saldo disminuye, una cantidad aumenta o una respuesta se reemplaza cuando llega nueva información.

El mismo nombre a través del tiempo

```

PRINCIPAL()
  estadoPedido <- "Recibido"
  escribir estadoPedido

  estadoPedido <- "Preparando"
  escribir estadoPedido

  estadoPedido <- "Enviado"
  escribir estadoPedido

```

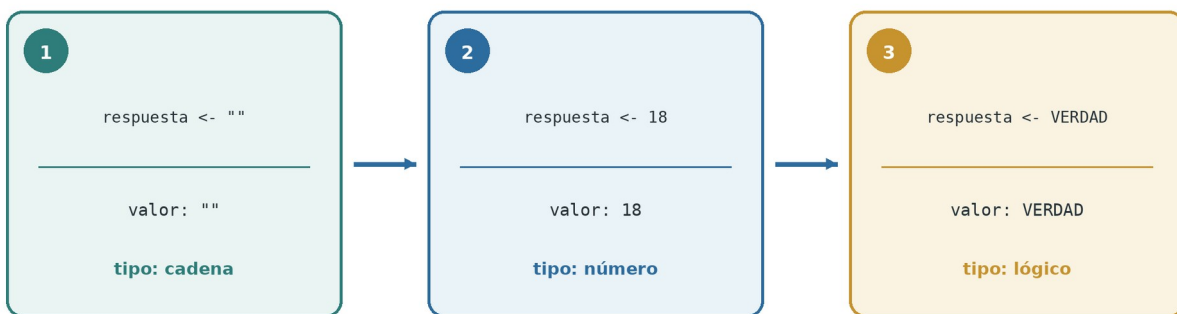
El nombre `estadoPedido` mantiene su propósito, mientras el valor refleja momentos distintos del proceso. La mutabilidad tiene sentido cuando los valores describen una realidad cambiante. Sin embargo, actualizar sin una razón clara dificulta seguir el programa.

Criterio de diseño: si el significado del dato cambió por completo, suele convenir otro identificador. Si solo cambió el estado de la misma entidad, reutilizar el nombre puede ser apropiado.

3.10 Tipado dinámico: el valor actual determina el tipo

CLRS es dinámicamente tipado: no se fija un tipo permanente al nombre cuando se crea la variable. El tipo se conoce a partir del valor asociado durante la ejecución. Por ello un mismo identificador puede recibir primero una cadena, después un número y finalmente un valor lógico.

Una variable mutable puede cambiar de valor y de tipo



El nombre `respuesta` permanece; lo que cambia es el valor asociado y, por tanto, el tipo del valor actual.

Esta flexibilidad es útil, pero también puede ocultar errores si el cambio no tiene una intención clara.

Figura 3.4. Cambio de valor y de tipo bajo el mismo identificador. Elaboración propia.

Cambio de tipo permitido

```

PRINCIPAL()
  respuesta <- ""
  escribir respuesta

  respuesta <- 18
  escribir respuesta

  respuesta <- VERDAD
  escribir respuesta

```

El programa puede ejecutar estas asignaciones porque no existe una declaración que restrinja `respuesta` a un único tipo. Esa flexibilidad reduce sintaxis inicial, pero exige disciplina: cada uso debe ser coherente con el valor que realmente existe en ese momento.

Precisión conceptual: es habitual decir que “la variable cambia de tipo”. De forma más exacta, el nombre se asocia con valores de tipos diferentes a lo largo de la ejecución.

Advertencia: que el lenguaje permita un cambio no significa que el diseño sea claro. Si `respuesta` unas veces significa texto, otras edad y otras validación, el nombre deja de comunicar una intención estable.

3.11 Tipado débil: algunas conversiones pueden ocurrir implícitamente

Además de ser dinámico, CLRS es débilmente tipado. Esto significa que, en ciertos contextos, puede realizar conversiones implícitas para combinar valores de clases diferentes. La comodidad resulta útil al presentar información, pero puede producir resultados inesperados cuando quien programa supone que dos valores serán tratados de una forma distinta.

Mismo aspecto, tipos diferentes

```

PRINCIPAL()
  cantidad <- 25
  textoCantidad <- "25"

  escribir cantidad
  escribir textoCantidad

```

Las dos salidas pueden verse iguales, pero el estado conserva valores de tipos distintos.

La consola muestra representaciones textuales. Por eso observar `25` en pantalla no demuestra por sí solo si el valor original era número o cadena. Para razonar con precisión necesitamos conocer cómo fue creado, qué operaciones se aplicaron y, cuando sea necesario, comprobar su tipo.

Riesgo	Suposición incorrecta	Práctica más segura
Confiar en la apariencia	“Se ve como 25, así que es número”.	Comprobar el tipo o controlar el origen del dato.

Riesgo	Suposición incorrecta	Práctica más segura
Cambiar de tipo sin intención	“El nombre puede contener cualquier cosa”.	Mantener un significado estable para cada variable.
Convertir todo automáticamente	“Si falla, convierto”.	Validar primero si la conversión representa el problema.
Usar valores centinela ambiguos	`FALSO` significa “aún no hay número”.	Separar disponibilidad, validez y valor cuando el programa crezca.

Idea clave: el tipado débil no elimina los tipos. Los valores siguen teniendo tipos; el lenguaje solo es más permisivo al relacionarlos en ciertos contextos.

3.12 Comprobar qué tipo de valor tenemos

La biblioteca estándar de CLRS incluye funciones que responden preguntas sobre el tipo del valor. Estas funciones devuelven `VERDAD` o `FALSO` y permiten hacer explícita una distinción que la apariencia de la salida no muestra.

Función	Pregunta que responde	Ejemplo
`ES_NUM(valor)`	¿El valor es numérico?	`ES_NUM(25)` produce VERDAD.
`ES_CAD(valor)`	¿El valor es una cadena?	`ES_CAD("25")` produce VERDAD.
`ES_LOG(valor)`	¿El valor es lógico?	`ES_LOG(FALSO)` produce VERDAD.

Observar tipos mediante comprobaciones

```
PRINCIPAL()
  monto <- 85
  concepto <- "Transporte"
  esNecesario <- VERDAD

  escribir ES_NUM(monto)
  escribir ES_CAD(concepto)
  escribir ES_LOG(esNecesario)
```

Estas comprobaciones no cambian el valor. Solo producen información lógica acerca de él. En el capítulo de decisiones utilizaremos ese resultado para elegir qué debe hacer el programa.

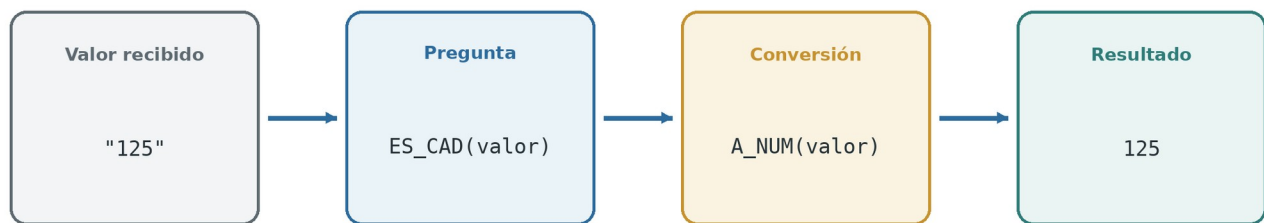
Pausa y piensa: predice el resultado de `ES\_NUM("85")`, `ES\_CAD(85)`, `ES\_LOG("FALSO")` y `ES\_CAD("")`. Justifica cada respuesta a partir de la representación.

3.13 Convertir cuando cambia la representación necesaria

A veces el dato llega con una representación y el programa necesita otra. CLRS ofrece funciones para convertir explícitamente valores: `A\_CAD` produce una cadena, `A\_NUM` produce un número y `A\_LOG` produce un valor lógico. Una conversión explícita hace visible la intención y evita depender únicamente de conversiones implícitas.

Función	Resultado buscado	Uso típico
`A_CAD(valor)`	Cadena	Preparar un valor para combinarlo o almacenarlo como texto.
`A_NUM(valor)`	Número	Transformar texto numérico antes de calcular.
`A_LOG(valor)`	Lógico	Interpretar un valor según las reglas lógicas de la biblioteca.

Comprobar antes de convertir



Comprobar el tipo responde “¿qué clase de valor tengo?”. Convertir responde “¿qué representación necesito para la siguiente operación?”.

La conversión no debe utilizarse para ocultar datos inválidos ni para sustituir una validación del problema.

Figura 3.5. Comprobación y conversión como pasos distintos. Elaboración propia.

Conversión explícita

```

PRINCIPAL()
  textoMonto <- "125"
  monto <- A_NUM(textoMonto)

  escribir ES_CAD(textoMonto)
  escribir ES_NUM(monto)
  
```

La conversión crea un nuevo valor; no demuestra por sí misma que el dato sea válido para el problema. Por ejemplo, un número puede ser técnicamente convertible y aun así resultar inaceptable como radio si es negativo, o como calificación si está fuera del rango permitido.

Advertencia: comprobar el tipo y validar el significado son tareas diferentes. `ES\_NUM(radio)` puede producir VERDAD aunque el radio sea -5; la regla “el radio debe ser mayor que cero” pertenece al problema.

3.14 Elegir valores iniciales sin ocultar el significado

En varios programas introductorios se asigna `""` o `FALSO` antes de leer un dato. Esa técnica crea la variable y proporciona un estado que una validación posterior puede reconocer. En CLRS es válida, pero conviene entender qué comunica cada elección.

Patrón observado en programas de entrada

```
radio <- ""
```

```
mientras no ES_NUM(radio)
  leer radio
```

El ciclo se estudiará más adelante. Aquí solo observamos que radio comienza como cadena vacía y después puede recibir un número.

Inicialización	Qué representa literalmente	Posible intención
`radio <- ""`	Una cadena sin caracteres.	Todavía no se ha recibido el radio.
`radio <- FALSO`	Una afirmación lógica falsa.	Todavía no se ha recibido un número.
`radio <- 0`	La cantidad cero.	Valor inicial numérico; puede confundirse con una entrada real.

No existe un valor inicial universalmente correcto. La elección depende de qué estados necesita distinguir el programa. En ejemplos pequeños, una cadena vacía puede funcionar como señal temporal. En programas mayores suele ser más claro separar la información: una variable conserva el valor y otra indica si ya fue capturado o validado.

Separar valor y estado de validación

```
PRINCIPAL()
  radio <- 0
  radioFueCapturado <- FALSO
```

```
  escribir radio
  escribir radioFueCapturado
```

Este fragmento no realiza todavía la lectura; muestra dos responsabilidades diferentes.

Criterio de diseño: un valor inicial debe hacer que el estado sea comprensible. Si obliga a recordar una interpretación secreta, como “FALSO significa que falta un número”, conviene buscar una representación más explícita.

3.15 Confusiones frecuentes entre nombres, valores y tipos

Confusión	Ejemplo	Consecuencia
Omitir comillas en texto	<code>`concepto <- Transporte`</code>	El lenguaje busca un identificador llamado Transporte.
Agregar comillas a un número	<code>`monto <- "85"`</code>	El dato queda como cadena, no como cantidad numérica.
Cambiar el significado del nombre	<code>`monto <- VERDAD`</code>	El identificador deja de expresar una intención estable.
Usar mayúsculas distintas	<code>`saldo`</code> y <code>`Saldo`</code>	Se crean o consultan nombres diferentes.
Suponer que la salida revela el tipo	Ambos muestran 85	Se pierde la diferencia entre número y cadena.
Convertir sin validar el dominio	<code>`A_NUM(texto)`</code>	Puede obtenerse un número que incumple las reglas del problema.

Advertencia: un programa puede ejecutarse y producir salida aun cuando los tipos elegidos no representen bien el problema. La ausencia de error sintáctico no garantiza un modelo correcto.

Para diagnosticar una confusión de tipos, reconstruye el estado: identifica cómo se creó la variable, qué valor recibió en cada paso, qué tipo tenía ese valor y qué operación se intentó realizar. Esta revisión suele ser más útil que observar únicamente la última línea.

3.16 Laboratorio CLRS: observar valores y tipos

Propósito: construir un programa pequeño que permita comparar números, cadenas y valores lógicos mediante su salida y las funciones de comprobación.

Crea el archivo ``valores-y-tipos.clrs`` con el siguiente contenido:

Programa inicial del laboratorio

```
// Observa valores y tipos
PRINCIPAL()
  numero <- 25
  textoNumero <- "25"
  mensaje <- "25 grados"
  disponible <- VERDAD

  escribir numero
  escribir textoNumero
  escribir mensaje
  escribir disponible

  escribir ES_NUM(numero)
  escribir ES_NUM(textoNumero)
  escribir ES_CAD(textoNumero)
  escribir ES_LOG(disponible)
```

Antes de ejecutar, completa la tabla de predicción:

Nombre	Valor escrito	Tipo esperado	Resultado de comprobación
numero	25		ES_NUM(numero) =
textoNumero	"25"		ES_NUM(textoNumero) =
textoNumero	"25"		ES_CAD(textoNumero) =
disponible	VERDAD		ES_LOG(disponible) =

1. Ejecuta el archivo y compara la salida con tu predicción.
2. Cambia `numero <- 25` por `numero <- "25"` y vuelve a predecir antes de ejecutar.
3. Restaura el número y asigna `mensaje <- 25`. Explica qué cambió en el significado de la variable.
4. Agrega `numeroComoTexto <- A_CAD(numero)` y comprueba su tipo.
5. Agrega `textoComoNumero <- A_NUM(textoNumero)` y comprueba su tipo.
6. Cambia el identificador `disponible` por uno que describa mejor una situación concreta, por ejemplo `productoDisponible`.

Pregunta de reflexión: ¿por qué dos valores que producen la misma salida visual pueden exigir tratamientos diferentes en un cálculo?

3.17 El estado tipado del caso longitudinal

Retomemos el registro de gastos. En el capítulo anterior observamos una primera versión con concepto, monto, saldo inicial y saldo restante. Ahora añadiremos un dato lógico y haremos explícito qué representa cada valor.

Registro de gasto con distintos tipos

```
// Estado básico de un gasto
PRINCIPAL()
  concepto <- "Transporte"
  monto <- 85
  esNecesario <- VERDAD
  saldoInicial <- 500
  saldoRestante <- 415

  escribir concepto
  escribir monto
  escribir esNecesario
  escribir saldoRestante
```

El estado de un gasto combina valores de distintos tipos

Nombre	Valor actual	Tipo
concepto	"Transporte"	cadena
monto	85	número
esNecesario	VERDAD	lógico
saldoRestante	415	número

El estado no es una sola cifra: es el conjunto de asociaciones que el programa puede utilizar en ese momento.

Nombrar bien permite comprender el propósito de cada valor sin depender de comentarios redundantes.

Figura 3.6. Componentes del estado de un gasto. Elaboración propia.

Variable	Significado estable	Tipo del valor actual
`concepto`	Nombre o descripción del gasto.	Cadena
`monto`	Cantidad desembolsada.	Número
`esNecesario`	Clasificación lógica del gasto.	Lógico
`saldoInicial`	Cantidad disponible antes del gasto.	Número
`saldoRestante`	Cantidad disponible después del gasto.	Número

Este estado todavía contiene un valor calculado manualmente: `saldoRestante <- 415`. En el próximo capítulo construiremos expresiones para que el programa produzca ese valor a partir de `saldoInicial` y `monto`, en lugar de escribirlo directamente.

Conexión hacia adelante: comprender el tipo de cada valor permitirá decidir qué operaciones son válidas. Restar tiene sentido para dos cantidades; combinar texto y cantidades sirve para presentar mensajes; comparar valores producirá resultados lógicos.

3.18 Reconstruye el modelo mental

Un programa conserva información mediante asociaciones que evolucionan. El identificador permite localizar una asociación; la variable es el nombre que participa en el estado; el valor es la información concreta; el literal es una forma de escribirla directamente; y el tipo describe la clase de valor y las operaciones que tienen sentido.

7. Un dato pertenece al problema que queremos representar.
8. Un valor es una instancia concreta de ese dato durante una ejecución.
9. Un literal escribe directamente un valor en el código.
10. Un identificador proporciona un nombre para volver a acceder al valor.
11. La primera asignación crea la variable y le da un valor inicial.
12. Las asignaciones posteriores modifican el estado.
13. CLRS permite que el mismo nombre reciba valores de tipos distintos.
14. Los tipos principales estudiados son número, cadena y lógico.
15. Las funciones `ES\_NUM`, `ES\_CAD` y `ES\_LOG` comprueban tipos.
16. Las funciones `A\_NUM`, `A\_CAD` y `A\_LOG` realizan conversiones explícitas.

Comprobación activa: cubre la lista y explica con un ejemplo propio cada concepto. Si utilizas la misma palabra para identificador, variable, literal y valor, vuelve a construir el ejemplo separando sus funciones.

Ejercicios

Realiza las predicciones antes de ejecutar. En los ejercicios de diseño, justifica el tipo y el identificador elegidos.

1. Identifica el literal, el valor, el tipo y el identificador en `temperatura <- -40``. **[Reconocer]**
2. Clasifica como número, cadena o lógico: `0``, `""`, `FALSO``, `"FALSO"`, `-12.5``, `""``. **[Reconocer]**
3. Explica por qué `25`` y `"25"` pueden producir una salida parecida y, sin embargo, no son intercambiables. **[Comprender]**
4. Propón nombres claros para: precio antes de impuestos, nombre del cliente, condición de pago y cantidad de intentos. **[Aplicar]**
5. Mejora los identificadores `x``, `dato2``, `res`` y `flag`` dentro de un programa que calcula el área de un círculo. **[Aplicar]**
6. Construye una tabla de estado para tres asignaciones sucesivas a `estadoPedido``. **[Aplicar]**
7. Escribe un programa donde una variable cambie de cadena a número. Explica por qué el lenguaje lo permite y por qué podría ser confuso. **[Analizar]**
8. Predice `ES_NUM("85")``, `ES_CAD("85")``, `ES_LOG(FALSO)`` y `ES_CAD(FALSO)``. Verifica después. **[Aplicar]**
9. Convierte la cadena `"125"` a número y comprueba ambos tipos antes y después. **[Aplicar]**
10. Decide si cada dato debe ser número o cadena: código postal, matrícula, peso, folio, temperatura y número telefónico. Justifica. **[Analizar]**

11. Explica qué se pierde cuando todos los valores se almacenan como cadenas “porque después se pueden convertir”.
[Analizar]
12. Corrige ``concepto <- Transporte`` y explica qué interpretación intentaba realizar el lenguaje. [Aplicar]
13. Compara ``radio <- ""``, ``radio <- FALSO`` y ``radio <- 0`` como estados iniciales. Indica una ventaja y un riesgo de cada opción. [Analizar]
14. Diseña el estado inicial de un libro prestado: título, número de páginas, disponible y código de inventario. Elige tipos y valores. [Construir]
15. Construye un programa que represente una cuenta de usuario mediante nombre, edad, correo y condición de activación. Comprueba los cuatro tipos. [Construir]
16. Introduce deliberadamente comillas alrededor de un valor numérico y documenta qué comprobaciones cambian. [Experimentar]
17. Adapta el ejemplo de temperatura proporcionado para que ``C``, ``F`` y ``K`` comiencen con nombres descriptivos y valores fijos. Clasifica sus tipos. [Construir]
18. Reescribe la inicialización del programa de salario neto separando el salario numérico de una variable lógica que indique si ya fue capturado. [Construir]

Mini proyecto: ficha de un producto

Situación: una tienda necesita representar el estado inicial de un producto antes de realizar cálculos o decisiones.

Crea un archivo ``ficha-producto.clrs`` que represente, mediante variables, la siguiente información: nombre del producto, código, precio, cantidad disponible, condición de disponibilidad y una descripción que pueda estar vacía.

Requisito	Criterio de éxito
Identificadores	Cada nombre expresa la intención del dato.
Tipos	Los códigos se conservan como texto y las cantidades como números.
Lógicos	La disponibilidad utiliza <code>`VERDAD`</code> o <code>`FALSO`</code> sin comillas.
Salida	El programa muestra todos los valores en un orden comprensible.
Comprobación	Usa <code>`ES_NUM`</code> , <code>`ES_CAD`</code> y <code>`ES_LOG`</code> al menos una vez.
Conversión	Crea una versión textual del precio mediante <code>`A_CAD`</code> .
Traza	Registra el estado después de cada primera asignación.

Incluye una explicación breve de por qué cada valor tiene el tipo elegido. Después cambia deliberadamente el precio de número a cadena, ejecuta las comprobaciones y documenta qué evidencia demuestra el cambio.

Autoevaluación

Responde sin consultar el capítulo. Después vuelve a las secciones que no puedas explicar con un ejemplo.

Afirmación	Sí	Todavía no
Puedo distinguir dato, valor, literal, identificador y variable.	☐	☐
Puedo reconocer números, cadenas y valores lógicos por su representación.	☐	☐
Puedo explicar qué hace la primera asignación.	☐	☐
Puedo construir una traza de actualizaciones de una variable.	☐	☐
Puedo explicar tipado dinámico sin decir que “no existen tipos”.	☐	☐
Puedo explicar el riesgo del tipado débil.	☐	☐
Puedo comprobar y convertir tipos mediante la biblioteca de CLRS.	☐	☐
Puedo elegir identificadores y tipos según el significado del dato.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas observar una asignación y explicar qué nombre cambia, qué valor queda asociado, cuál es su tipo y por qué esa representación es adecuada para el problema.

Glosario

Término	Definición de trabajo
Dato	Unidad de información relevante para el problema.
Valor	Información concreta que existe durante una ejecución.
Representación	Forma en que un valor se escribe, almacena o muestra.
Literal	Notación que escribe directamente un valor en el código.
Identificador	Nombre utilizado para referirse a un elemento del programa.
Variable	Identificador asociado con un valor que puede cambiar.
Asignación	Instrucción que asocia un nombre con el valor de la derecha.
Mutabilidad	Capacidad de cambiar el valor asociado con una variable.
Tipo	Clasificación de un valor que orienta su interpretación y operaciones.
Tipado dinámico	Modelo en el que el tipo se determina durante la ejecución a partir del valor.
Tipado débil	Modelo permisivo respecto a conversiones y combinaciones entre tipos.
Cadena vacía	Cadena que no contiene caracteres, representada por <code>""</code> .
Conversión	Transformación explícita o implícita de una representación de valor a otra.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación del proyecto, versión 1.3.1 [Documento técnico proporcionado para la obra].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Scott, M. L. (2015). Programming language pragmatics (4th ed.). Morgan Kaufmann.

Continuidad: en el capítulo 4 utilizaremos estos valores para construir expresiones aritméticas, comparaciones, condiciones lógicas y mensajes de entrada y salida. El estado dejará de ser una colección de valores escritos y comenzará a transformarse mediante operaciones.

CAPÍTULO

4

Expresar cálculos comparaciones y comunicación

Combinar valores, producir resultados y seguir la transformación del estado



Expresar cálculos, comparaciones y comunicación

¿Cómo se combinan valores para producir nueva información y comunicarse con el usuario?

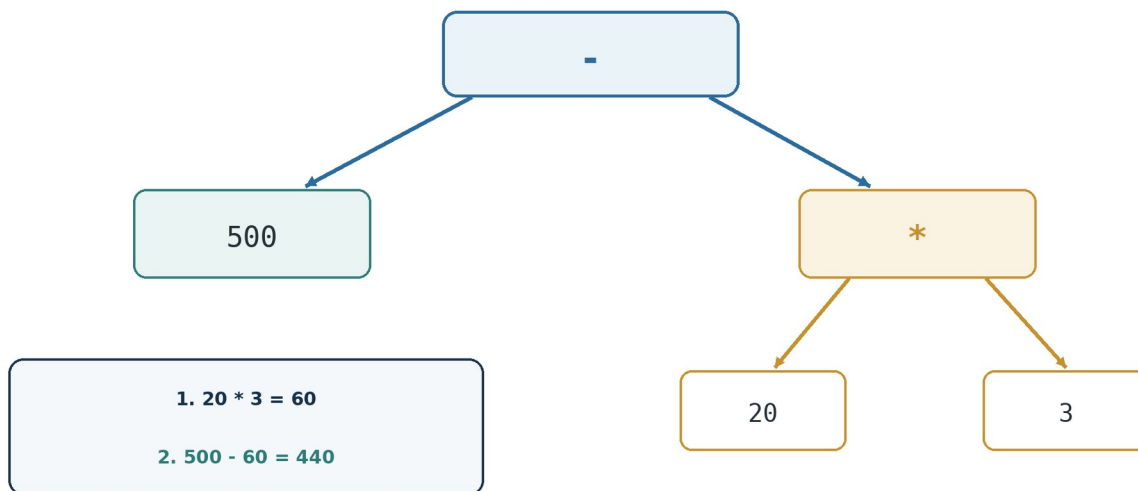
Una fórmula puede ser correcta en papel y producir un resultado inesperado en un programa. No siempre falla la matemática: a veces el problema está en el orden con que se evalúan las operaciones, en el tipo de los valores o en la manera de construir la salida.

En el capítulo anterior aprendimos a representar información mediante valores, variables y tipos. Ahora utilizaremos esos valores como partes de expresiones. Una expresión puede calcular una cantidad, comparar dos datos, combinar afirmaciones lógicas o construir un mensaje. Al evaluarla, el programa produce un nuevo valor que puede asignarse, escribirse o utilizarse en otra expresión.

Promesa de aprendizaje: al terminar podrás construir expresiones aritméticas, relacionales y lógicas; controlar su agrupación mediante paréntesis; leer y escribir datos; diferenciar concatenación de valores separados; y completar trazas que expliquen cómo una expresión transforma el estado.

La agrupación determina qué operación ocurre primero

Expresión: $500 - 20 * 3$



Sin paréntesis, la multiplicación se evalúa antes que la resta.

Figura 4.1. Evaluación de una expresión según su agrupación. Elaboración propia.

Alcance del capítulo: calcularemos condiciones lógicas, pero todavía no las utilizaremos para elegir caminos. Las estructuras `sí`, `síno sí` y `síno` se introducirán en el capítulo 5.

4.1 De conservar valores a producir resultados

Una asignación como `saldoInicial <- 500` incorpora un valor al estado. Una asignación como `saldoRestante <- saldoInicial - monto` hace algo más: solicita que el programa consulte valores existentes, evalúe una operación y asocie el resultado con otro nombre.

Una expresión transforma valores

```
PRINCIPAL()
  saldoInicial <- 500
  monto <- 85
  saldoRestante <- saldoInicial - monto

  escribir saldoRestante
```

Elemento	Fragmento	Función
Valores de entrada	<code>saldoInicial</code> , <code>monto</code>	Proporcionan la información utilizada.
Operador	<code>-</code>	Describe la transformación solicitada.
Expresión	<code>saldoInicial - monto</code>	Combina operandos y operador.
Resultado	<code>415</code>	Es el valor producido al evaluar.
Asignación	<code>saldoRestante <- ...</code>	Actualiza el estado con el resultado.

Idea clave: una expresión es una construcción que, al evaluarse, produce un valor. Ese valor puede ser numérico, de cadena o lógico.

Pausa y piensa: en `area <- PI() * radio^2`, identifica los valores u operandos, los operadores, la llamada que produce π y el nombre que recibirá el resultado.

4.2 Operandos, operadores y evaluación

Los valores o expresiones sobre los que actúa un operador se llaman operandos. En `precio * cantidad`, `precio` y `cantidad` son operandos; `*` es el operador. Evaluar significa determinar el valor que resulta de aplicar las reglas de la expresión al estado actual.

Expresión	Operandos	Operador principal	Valor producido
<code>8 + 3</code>	8 y 3	Suma	11
<code>saldo - gasto</code>	Valores actuales de ambas variables	Resta	Número
<code>monto > limite</code>	monto y limite	Mayor que	Lógico
<code>"Total: " + total</code>	Cadena y valor de total	Suma/concatenación	Cadena

Expresión	Operandos	Operador principal	Valor producido
`no esValido`	Valor de esValido	Negación	Lógico

Una expresión puede contener otras expresiones. En `(precio \* cantidad) - descuento`, la multiplicación forma una subexpresión cuyo resultado se utiliza después en la resta. La evaluación avanza desde las partes que deben resolverse primero hacia la operación principal.

Importante: el mismo símbolo puede tener comportamientos diferentes según los tipos. `+` suma números, pero también puede participar en la construcción de texto. Por eso comprender los tipos sigue siendo necesario.

4.3 Construir cálculos aritméticos

Las expresiones aritméticas representan cálculos. CLRS permite utilizar suma, resta, multiplicación, división y potencia. La escritura debe reflejar con precisión la fórmula que se desea evaluar.

Operación	Operador CLRS	Ejemplo	Resultado
Suma	`+`	`12 + 5`	17
Resta	`-`	`12 - 5`	7
Multiplicación	`*`	`12 * 5`	60
División	`/`	`12 / 5`	2.4
Potencia	`^`	`5^2`	25

Operaciones con dos números

```
PRINCIPAL( )
  a <- 20
  b <- 4

  suma <- a + b
  resta <- a - b
  producto <- a * b
  cociente <- a / b
  cuadrado <- a^2

  escribir suma, resta, producto, cociente, cuadrado
```

Advertencia: la división entre cero no representa una operación válida para el problema. La validación y el manejo de ese caso se estudiarán al introducir decisiones.

En una fórmula, utiliza nombres que preserven el significado. `denominador <- 2 \* a` comunica más que `d <- 2 \* a`. Separar una fórmula en resultados intermedios también puede mejorar la lectura, siempre que cada nombre tenga un propósito claro.

4.4 Cuando el orden escrito no es el orden evaluado

Las expresiones se escriben de izquierda a derecha, pero no todas las operaciones tienen la misma prioridad. En ``500 - 20 * 3``, la multiplicación se resuelve antes que la resta. El resultado es 440, no 1440 ni 60.

Orden práctico de evaluación de expresiones

1	Paréntesis	()	Agrupar de forma explícita
2	Potencia	^	Eleva un valor
3	Signo unario	+ -	Aplica signo a una expresión
4	Multiplicación y división	* /	Operaciones multiplicativas
5	Suma y resta	+ -	Operaciones aditivas
6	Comparaciones	< <= > >= = !=	Producen valores lógicos
7	Negación lógica	no	Invierte una comparación o afirmación
8	Conjunción	y	Exige que ambas condiciones sean verdaderas
9	Disyunción	o	Acepta que al menos una sea verdadera

Los paréntesis son la herramienta más segura para expresar la intención cuando una fórmula es compleja.

Figura 4.2. Orden práctico para leer expresiones de CLRS. Elaboración propia.

La precedencia indica qué operadores se evalúan antes cuando no existen paréntesis. Para las operaciones aritméticas del mismo nivel, la evaluación habitual avanza de izquierda a derecha. En expresiones con varias potencias o combinaciones complejas, utiliza paréntesis para evitar depender de una interpretación difícil de recordar.

Expresión	Agrupación interpretada	Resultado
<code>`10 + 2 * 3`</code>	<code>`10 + (2 * 3)`</code>	16
<code>`(10 + 2) * 3`</code>	<code>`(10 + 2) * 3`</code>	36
<code>`20 / 5 * 2`</code>	<code>`(20 / 5) * 2`</code>	8
<code>`20 / (5 * 2)`</code>	<code>`20 / (5 * 2)`</code>	2
<code>`2 + 3^2`</code>	<code>`2 + (3^2)`</code>	11

Regla práctica: los paréntesis no solo cambian resultados; también documentan la intención. Aunque la precedencia ya produzca el cálculo deseado, agrupa cuando la fórmula pueda leerse de más de una manera.

Pausa y piensa: predice ``100 - 20 / 2 + 5``. Después reescribe la expresión con paréntesis que hagan explícito el orden que utilizaste.

4.5 Traducir fórmulas a expresiones

Traducir una fórmula no consiste en copiar símbolos sin pensar. Primero identifica las cantidades, después las operaciones y finalmente la agrupación. Dos ejemplos útiles son la conversión de temperatura y el cálculo de un círculo.

Conversión de Celsius

```
PRINCIPAL()
  celsius <- 25
  fahrenheit <- ((9 / 5) * celsius) + 32
  kelvin <- celsius + 273.15

  escribir "Temperatura en F: " + fahrenheit
  escribir "Temperatura en K: " + kelvin
```

La fórmula de Fahrenheit combina una multiplicación y una suma. Los paréntesis alrededor de $9 / 5$ no son indispensables para el resultado, pero ayudan a reconocer el factor de conversión. Los paréntesis exteriores hacen visible que primero se calcula el producto y después se suma 32.

Área y perímetro de un círculo

```
PRINCIPAL()
  radio <- 5
  area <- PI() * (radio^2)
  perimetro <- 2 * PI() * radio

  escribir "Área: " + area
  escribir "Perímetro: " + perimetro
```

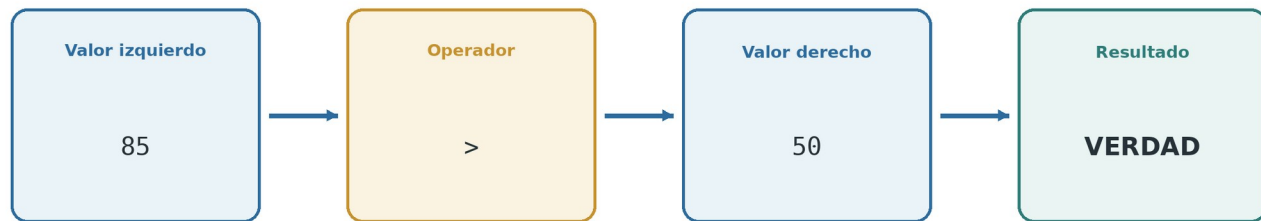
Paso de traducción	Temperatura	Círculo
Identificar entradas	celsius	radio
Identificar resultados	fahrenheit y kelvin	area y perimetro
Reconocer constantes	9/5, 32, 273.15	π y 2
Agrupar	factor por temperatura, después suma	potencia del radio y productos
Nombrar	nombres con unidad o significado	nombres geométricos

Advertencia: una fórmula bien traducida puede seguir siendo inválida para ciertas entradas. Un radio negativo no tiene sentido en este problema; esa regla pertenece a la validación, no a la aritmética.

4.6 Comparar también es calcular

Una comparación es una expresión que produce `VERDAD` o `FALSO`. No modifica por sí sola el recorrido del programa. Primero calcula una afirmación; en el capítulo siguiente aprenderemos a utilizar ese resultado para elegir una acción.

Comparar valores produce un nuevo valor lógico



La comparación no “elige” todavía una acción. Solo calcula una afirmación que después podrá utilizar una decisión.

Por eso una expresión relacional puede asignarse, escribirse o combinarse con otras expresiones lógicas.

Figura 4.3. Una comparación produce un valor lógico. Elaboración propia.

Relación	Operador	Ejemplo	Resultado
Igualdad	<code>`==`</code>	<code>`10 = 10`</code>	VERDAD
Desigualdad	<code>`!=`</code>	<code>`10 != 8`</code>	VERDAD
Mayor que	<code>`>`</code>	<code>`85 > 50`</code>	VERDAD
Menor que	<code>`<`</code>	<code>`85 < 50`</code>	FALSO
Mayor o igual	<code>`>=`</code>	<code>`10 >= 10`</code>	VERDAD
Menor o igual	<code>`<=`</code>	<code>`9 <= 10`</code>	VERDAD

Guardar resultados de comparaciones

```

PRINCIPAL()
  monto <- 85
  limite <- 50

  superaLimite <- monto > limite
  coincide <- monto = limite

  escribir superaLimite
  escribir coincide
  
```

Importante: en CLRS ``=`` compara igualdad y ``<=`` asigna. Confundirlos cambia por completo la intención: uno produce un valor lógico; el otro modifica el estado.

4.7 Combinar afirmaciones lógicas

Los operadores `y`, `o` y `no` permiten construir afirmaciones a partir de otras. Sus operandos deben interpretarse como valores lógicos. Estas expresiones son especialmente útiles para representar reglas que dependen de varias condiciones.

Operador	Interpretación	Ejemplo	Cuándo produce VERDAD
`y`	Conjunción	`esNumero y esPositivo`	Cuando ambas partes son verdaderas.
`o`	Disyunción	`esEfectivo o esTarjeta`	Cuando al menos una parte es verdadera.
`no`	Negación	`no estaVacio`	Cuando la expresión original es falsa.

Expresiones lógicas sin decisiones

```

PRINCIPAL()
  monto <- 85
  esNumero <- ES_NUM(monto)
  esPositivo <- monto > 0
  esMontoAceptable <- esNumero y esPositivo

  escribir esNumero
  escribir esPositivo
  escribir esMontoAceptable

```

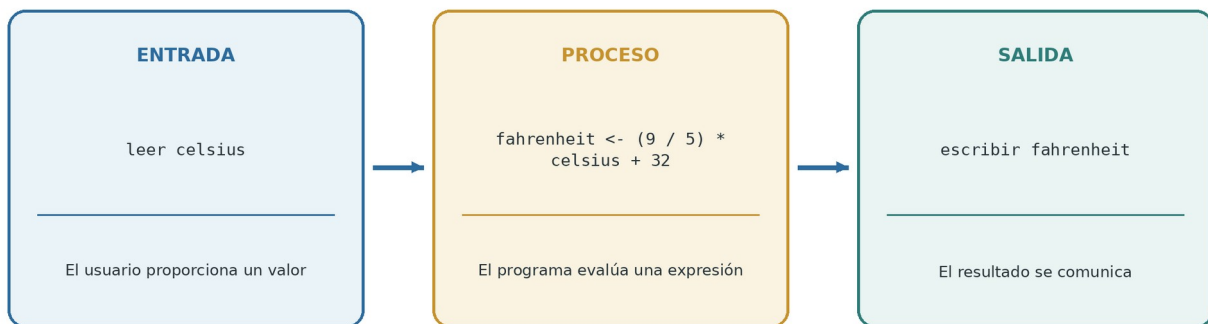
La expresión `esNumero y esPositivo` no repite las operaciones de comprobación y comparación: utiliza los valores lógicos ya almacenados. Esta separación facilita seguir el estado y explicar qué representa cada regla.

Pausa y piensa: si `esNumero` vale VERDAD y `esPositivo` vale FALSO, predice `esNumero y esPositivo`, `esNumero o esPositivo` y `no esPositivo`.

4.8 Leer: incorporar información durante la ejecución

Hasta ahora la mayoría de los valores estaban escritos directamente en el programa. La instrucción ``leer`` permite que un valor llegue desde la entrada de consola. Al ejecutarse, modifica el estado porque asigna el dato recibido a su destino.

Entrada, proceso y salida durante una ejecución



``leer`` modifica el estado; la expresión transforma valores; ``escribir`` hace observable una parte del resultado.

La pantalla no muestra automáticamente todo el estado: solo aquello que el programa decide comunicar.

Figura 4.4. Entrada, proceso y salida en un programa. Elaboración propia.

Leer una temperatura

```

PRINCIPAL()
  celsius <- 0

  escribir "Ingresa la temperatura en Celsius"
  leer celsius

  fahrenheit <- ((9 / 5) * celsius) + 32
  escribir "Temperatura en F: " + fahrenheit
  
```

En CLRS, ``leer`` asigna un valor, pero no declara por sí sola una variable. Por eso el destino debe haber sido creado mediante una asignación en el ámbito correspondiente. La inicialización ``celsius <- 0`` hace explícita esa asociación antes de la lectura.

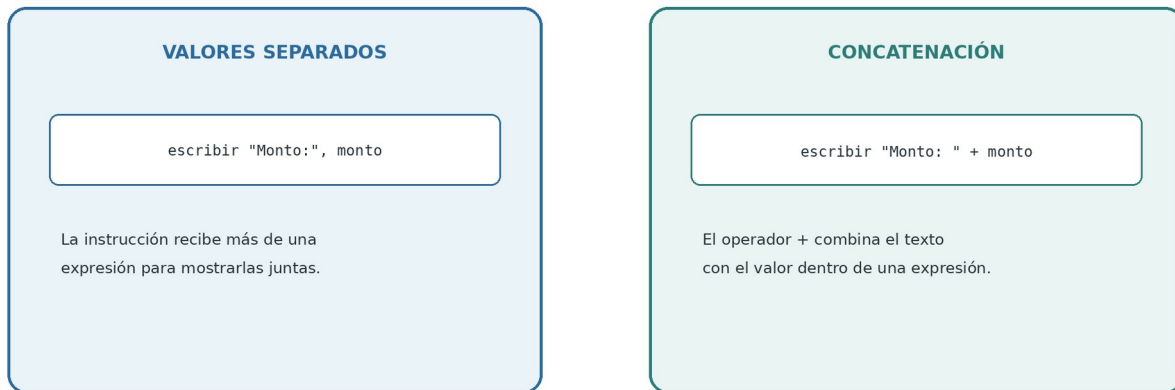
El tipo del valor leído se determina automáticamente. La lectura de ``25`` puede producir un número; la lectura de texto produce una cadena. El programa puede comprobar después el tipo mediante funciones como ``ES_NUM`` o ``ES_CAD``. La validación repetitiva de entradas se estudiará junto con los ciclos.

Advertencia: leer un dato no garantiza que sea adecuado para el problema. La entrada puede tener el tipo esperado y aun así violar una regla, como un radio negativo o una calificación mayor que el máximo permitido.

4.9 Escribir: hacer observable una parte del estado

La instrucción `escribir` muestra valores en la consola. Puede recibir un valor, varias expresiones separadas por comas, una expresión concatenada o un arreglo completo. En este capítulo nos interesa distinguir dos formas habituales de presentar mensajes.

Separar valores y construir una cadena no son lo mismo



En expresiones mixtas, los paréntesis ayudan a dejar claro qué cálculo debe completarse antes de formar el mensaje.

Ejemplo: `escribir "Total: " + (precio * cantidad)`

Figura 4.5. Valores separados y concatenación. Elaboración propia.

Formas de escribir

```
PRINCIPAL()
  concepto <- "Transporte"
  monto <- 85

  escribir monto
  escribir concepto, monto
  escribir "Monto: " + monto
  escribir "Gasto: " + concepto + " - " + monto
```

Con comas, `escribir` recibe varias expresiones y las presenta en una misma instrucción. Con `+`, se construye una expresión que combina valores. Esta diferencia importa cuando también hay cálculos: `"Total: " + (precio * cantidad)` hace explícito que la multiplicación debe terminar antes de formar el mensaje.

Advertencia: no confundas mostrar dos valores con sumarlos. `escribir a, b` presenta ambos; `escribir a + b` evalúa primero una sola expresión. Si ambos son números, el segundo caso muestra su suma.

4.10 Seguir el estado mediante una traza

Una traza registra el estado después de cada instrucción relevante. Es especialmente útil cuando una expresión depende de resultados anteriores. La salida final puede ser correcta y aun así ocultar pasos innecesarios o una interpretación accidental.

Una traza registra cómo cada expresión cambia el estado

Paso	Instrucción	celsius	fahrenheit	kelvin	Salida
0	Estado inicial	0	-	-	-
1	leer celsius	25	-	-	-
2	fahrenheit <- (9 / 5) * celsius + 32	25	77	-	-
3	kelvin <- celsius + 273.15	25	77	298.15	-
4	escribir fahrenheit	25	77	298.15	77
5	escribir kelvin	25	77	298.15	298.15

El guion indica que la variable todavía no tiene un valor relevante para la traza.

La salida aparece solo cuando se ejecuta `escribir`; los cálculos anteriores modifican el estado interno.

Figura 4.6. Traza de una conversión de temperatura. Elaboración propia.

Para construir una traza:

1. Anota las variables que forman parte del estado relevante.
2. Registra sus valores iniciales.
3. Lee una instrucción completa y evalúa primero su lado derecho.
4. Actualiza únicamente las asociaciones afectadas por esa instrucción.
5. Registra la salida solo cuando se ejecuta `escribir`.
6. Repite hasta terminar el recorrido.

Instrucción	Qué observar
Asignación con literal	Aparece o cambia una asociación.
Asignación con expresión	Se consultan operandos, se evalúa y se actualiza un nombre.
`leer destino`	El destino recibe el valor ingresado.
`escribir expresión`	La expresión se evalúa y su resultado se hace visible.
Comparación	Se produce VERDAD o FALSO, aunque no exista decisión.

Idea clave: la traza explica el proceso, no solo el resultado. Dos programas que muestran lo mismo pueden recorrer estados distintos.

4.11 Tipos dentro de las expresiones

Las operaciones adquieren sentido a partir de sus operandos. Multiplicar cantidades numéricas es coherente; intentar multiplicar una descripción textual no representa un cálculo válido. El operador `+` exige especial atención porque puede sumar números o combinar texto.

Expresión	Interpretación prevista	Riesgo
`10 + 5`	Suma numérica: 15	Ninguno si ambos son números.
`"10" + "5"`	Concatenación: "105"	Puede parecer una suma si se ignoran las comillas.
`"Total: " + 15`	Construcción de un mensaje	Interviene conversión implícita.
`A_NUM("10") + 5`	Conversión explícita y suma: 15	La cadena debe ser convertible.
`A_CAD(10) + "5"`	Conversión explícita y concatenación	El resultado es texto.

Hacer explícita la intención

```
PRINCIPAL()
  textoCantidad <- "10"
  cantidad <- A_NUM(textoCantidad)
  total <- cantidad + 5

  mensaje <- "Total: " + A_CAD(total)
  escribir mensaje
```

CLRS puede realizar conversiones implícitas en ciertas combinaciones, pero hacer explícita la conversión mejora el razonamiento cuando el tipo forma parte central de la intención. No es necesario convertir cada valor mecánicamente; conviene hacerlo cuando evita ambigüedad o documenta una frontera entre texto y cálculo.

Advertencia: la conversión resuelve una representación, no una regla del dominio. `A\_NUM("-5")` puede producir un número, pero no convierte un radio negativo en una entrada aceptable.

4.12 Errores frecuentes al construir expresiones

Error	Ejemplo	Consecuencia o corrección
Confundir asignación e igualdad	`monto = 85` cuando se quiere guardar	Usa ` <code><-</code> ` para asignar y ` <code>=</code> ` para comparar.
Omitir agrupación	`a + b / c`	Comprueba si se desea `a + (b / c)` o `(a + b) / c`.
Mezclar texto y cálculo sin intención	`"Total: " + precio * cantidad`	Escribe `"Total: " + (precio * cantidad)`.
Dividir entre cero	`a / 0`	El problema requiere validación antes de dividir.
Leer en un nombre no creado	`leer radio` sin asignación previa	Inicializa el destino en el ámbito correspondiente.

Error	Ejemplo	Consecuencia o corrección
Suponer que escribir cambia el estado	`escribir saldo`	La instrucción comunica; no actualiza saldo.
Observar solo la salida	El número final parece correcto	Construye una traza para revisar el proceso.

Diagnóstico práctico: cuando una expresión produce un resultado extraño, revisa en este orden: valores actuales, tipos, agrupación, precedencia, operador utilizado y asignación del resultado.

4.13 Laboratorio CLRS: conversor de temperatura

Propósito: leer un valor, evaluar dos fórmulas, construir mensajes y comprobar la transformación del estado mediante una traza.

Crea el archivo `conversion-temperatura.clrs`:

Programa inicial del laboratorio

```
// Convierte Celsius a Fahrenheit y Kelvin
PRINCIPAL()
  celsius <- 0

  escribir "Ingresa la temperatura en Celsius"
  leer celsius

  fahrenheit <- ((9 / 5) * celsius) + 32
  kelvin <- celsius + 273.15

  escribir "Temperatura en F: " + fahrenheit
  escribir "Temperatura en K: " + kelvin
```

Antes de ejecutar, calcula manualmente los resultados para 0 °C, 25 °C y -40 °C. Después completa una traza para uno de esos casos.

- Ejecuta el programa con 25 y compara la salida con tu cálculo.
- Cambia la fórmula de Fahrenheit por $(9 / (5 * celsius)) + 32$. Predice el efecto antes de ejecutar.
- Restaura la fórmula correcta y elimina sus paréntesis exteriores. Explica por qué el resultado se conserva.
- Cambia la salida por `escribir "F:", fahrenheit` y compara la forma de presentación.
- Agrega `esNumero <- ES\_NUM(celsius)` y escribe el resultado.
- Construye una tabla de estado desde el valor inicial hasta las dos salidas.

Pregunta de reflexión: ¿qué parte del programa pertenece a la entrada, cuál al proceso y cuál a la salida? ¿En qué paso aparece por primera vez cada valor calculado?

4.14 El caso longitudinal: calcular y comunicar un gasto

En el capítulo anterior el saldo restante estaba escrito como un literal. Ahora el programa lo calculará. También producirá comparaciones que describen el gasto, aunque todavía no elegirá acciones con ellas.

Estado transformado mediante expresiones

```
// Registro elemental de un gasto
PRINCIPAL()
  concepto <- "Transporte"
  saldoInicial <- 500
  monto <- 85
  limiteAviso <- 100

  saldoRestante <- saldoInicial - monto
  superaLimite <- monto > limiteAviso
  conservaSaldo <- saldoRestante >= 0

  escribir "Concepto: " + concepto
  escribir "Monto: " + monto
  escribir "Saldo restante: " + saldoRestante
  escribir "Supera el límite: " + superaLimite
  escribir "Conserva saldo: " + conservaSaldo
```

Nombre	Expresión que lo produce	Tipo del resultado
`saldoRestante`	`saldoInicial - monto`	Número
`superaLimite`	`monto > limiteAviso`	Lógico
`conservaSaldo`	`saldoRestante >= 0`	Lógico
Mensaje de salida	Cadena + valor	Cadena presentada

El programa ya puede describir situaciones relevantes, pero aún no cambia su conducta. Aunque `superaLimite` sea VERDAD, todas las instrucciones de salida se ejecutan. En el capítulo siguiente utilizaremos ese valor para decidir si debe mostrarse una advertencia.

Conexión hacia adelante: una decisión es una forma de utilizar el resultado lógico de una expresión para controlar qué instrucciones se ejecutan.

4.15 Reconstruye el modelo mental

Los valores del estado se convierten en información nueva mediante expresiones. Los operadores describen relaciones o transformaciones; la precedencia y los paréntesis determinan la agrupación; la evaluación produce un valor; la asignación incorpora ese valor al estado; `leer` introduce información externa; y `escribir` hace observable una parte del resultado.

- Una expresión produce un valor cuando se evalúa.
- Los operandos pueden ser literales, variables, llamadas o subexpresiones.

15. Las operaciones aritméticas producen números.
16. Las comparaciones producen valores lógicos.
17. Los operadores lógicos combinan o niegan afirmaciones.
18. La precedencia decide la agrupación cuando faltan paréntesis.
19. Los paréntesis expresan y documentan la intención.
20. ``leer`` modifica el estado con un valor de entrada.
21. ``escribir`` comunica resultados sin modificar las variables.
22. Una traza permite explicar cómo cada paso transforma el estado.

Comprobación activa: explica con un mismo ejemplo la diferencia entre expresión, resultado, asignación y salida. Después construye una comparación y señala por qué su resultado es lógico.

Ejercicios

Predice antes de ejecutar. Cuando una respuesta dependa de la agrupación, escribe explícitamente los paréntesis utilizados.

1. Identifica operandos y operador principal en ``saldoInicial - monto``. **[Reconocer]**
2. Calcula ``10 + 2 * 3``, ``(10 + 2) * 3`` y ``10 + (2 * 3)``. **[Aplicar]**
3. Reescribe ``a + b / c`` para representar dos interpretaciones distintas. **[Aplicar]**
4. Traduce a CLRS: el promedio de tres números, el precio con 16 % de impuesto y el área de un rectángulo. **[Construir]**
5. Construye una expresión para el salario neto con deducciones de 12 %, 5 % y 4.5 %. **[Construir]**
6. Explica la diferencia entre ``<-`` y ``=`` mediante dos líneas válidas. **[Comprender]**
7. Predice ``8 > 3``, ``8 = 3``, ``8 != 3``, ``8 <= 8`` y ``no (8 < 3)``. **[Aplicar]**
8. Con ``esNumero = VERDAD`` y ``esPositivo = FALSO``, evalúa las combinaciones con ``y``, ``o`` y ``no``. **[Aplicar]**
9. Compara ``escribir a, b`` con ``escribir a + b`` para `a = 10` y `b = 5`. **[Analizar]**
10. Construye una salida clara para precio, cantidad y total. Usa paréntesis alrededor del cálculo. **[Aplicar]**
11. Completa una traza para ``a <- 4``, ``b <- a + 3``, ``a <- b * 2``, ``escribir a``. **[Aplicar]**
12. Explica por qué `10 + 5` no representa la misma operación que ``10 + 5``. **[Comprender]**
13. Convierte explícitamente `125` y úsalo en una suma. Después construye un mensaje con el resultado. **[Aplicar]**
14. Diseña un programa que lea base y altura y calcule el área de un triángulo. **[Construir]**
15. Diseña un programa que lea dos números y muestre suma, resta, multiplicación y división. No valides todavía la división entre cero. **[Construir]**
16. Encuentra y corrige la ambigüedad en ``escribir "Promedio: " + n1 + n2 + n3 / 3``. **[Analizar]**
17. Representa mediante valores lógicos: una edad está dentro de 18 a 65 y un monto es número y mayor que cero. **[Construir]**
18. Adapta el cálculo del círculo para leer el radio y mostrar área y perímetro en mensajes separados. **[Construir]**
19. Crea una tabla de estado para la conversión de 100 °C a Fahrenheit y Kelvin. **[Analizar]**
20. Introduce deliberadamente un error de paréntesis en una fórmula, documenta la salida y explica la causa. **[Experimentar]**

Mini proyecto: calculadora de compra

Situación: una compra contiene precio unitario, cantidad y descuento. El programa debe leer los datos, calcular subtotal, descuento aplicado y total, y comunicar los resultados.

Crea `calculadora-compra.clrs` con los siguientes requisitos:

Requisito	Criterio de éxito
Entrada	Lee precio unitario, cantidad y porcentaje de descuento.
Cálculo	Produce subtotal, importe del descuento y total.
Agrupación	Usa paréntesis que hagan explícitas las fórmulas.
Comparaciones	Calcula si el total supera 1 000 y si el descuento es mayor que cero.
Salida	Presenta etiquetas y resultados en un orden comprensible.
Traza	Documenta el estado con un caso de prueba.
Pruebas	Usa al menos tres casos, incluido descuento cero.

No utilices todavía decisiones ni ciclos. El objetivo es que todas las expresiones se evalúen y que sus resultados queden disponibles para el capítulo siguiente.

Autoevaluación

Responde sin consultar el capítulo. Después vuelve a las secciones que no puedas explicar con un ejemplo propio.

Afirmación	Sí	Todavía no
Puedo distinguir operandos, operadores, expresión y resultado.	☐	☐
Puedo traducir una fórmula sencilla a CLRS.	☐	☐
Puedo explicar y controlar la precedencia con paréntesis.	☐	☐
Puedo construir comparaciones que produzcan valores lógicos.	☐	☐
Puedo combinar condiciones mediante y, o y no.	☐	☐
Puedo explicar cómo leer modifica el estado.	☐	☐
Puedo diferenciar concatenación de valores separados por comas.	☐	☐
Puedo construir una traza de entrada, cálculo y salida.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas recibir valores, construir una fórmula con agrupación explícita, explicar el tipo del resultado, incorporarlo al estado y mostrarlo mediante una salida comprensible.

Glosario

Término	Definición de trabajo
Expresión	Construcción que produce un valor al evaluarse.
Operando	Valor o subexpresión sobre la que actúa un operador.
Operador	Símbolo o palabra que solicita una operación.
Evaluación	Proceso de determinar el valor de una expresión.
Subexpresión	Expresión contenida dentro de otra expresión.
Precedencia	Prioridad relativa que determina qué operador se evalúa primero.
Agrupación	Organización de partes de una expresión, explícita mediante paréntesis.
Expresión aritmética	Expresión que representa un cálculo numérico.
Expresión relacional	Comparación que produce VERDAD o FALSO.
Expresión lógica	Combinación o negación de valores lógicos.
Entrada	Información que llega al programa durante la ejecución.
Salida	Información que el programa comunica al exterior.
Concatenación	Construcción de una cadena a partir de varios valores o fragmentos.
Traza	Registro ordenado de estados y salidas durante la ejecución.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación del proyecto, versión 1.3.1 [Documento técnico proporcionado para la obra].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Scott, M. L. (2015). Programming language pragmatics (4th ed.). Morgan Kaufmann.

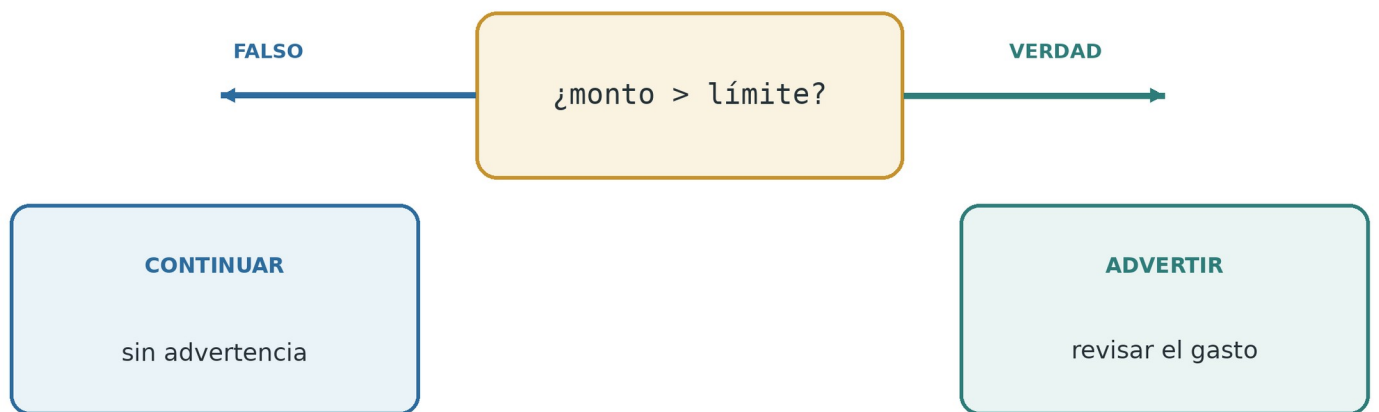
Continuidad: en el capítulo 5 utilizaremos los valores lógicos producidos por comparaciones y expresiones para elegir caminos de ejecución. El programa dejará de ejecutar siempre la misma secuencia y comenzará a responder de forma distinta según sus datos.

CAPÍTULO

5

Hacer que el programa elija

Condiciones, caminos alternativos, validación y pruebas de ramas



Hacer que el programa elija

¿Cómo puede un mismo programa responder de manera distinta ante datos diferentes?

Un programa de descuentos puede calcular correctamente un total y aun así estar mal diseñado si aplica la misma respuesta a todas las compras. La dificultad no está en obtener un número, sino en decidir qué debe ocurrir cuando ese número cae en una categoría, supera un límite o incumple una regla.

Hasta ahora, los programas siguieron una secuencia: cada instrucción válida se ejecutó después de la anterior. En este capítulo utilizaremos los valores lógicos del capítulo 4 para controlar el recorrido. Una condición ya no será solo un resultado que puede escribirse; se convertirá en una puerta que permite o impide ejecutar un bloque.

Promesa de aprendizaje: al terminar podrás construir decisiones simples y alternativas; ordenar cadenas `sino si`; elegir entre condiciones compuestas y decisiones anidadas; validar tipo y rango; y diseñar pruebas que recorran cada rama y sus límites.

Una condición transforma una secuencia en caminos posibles

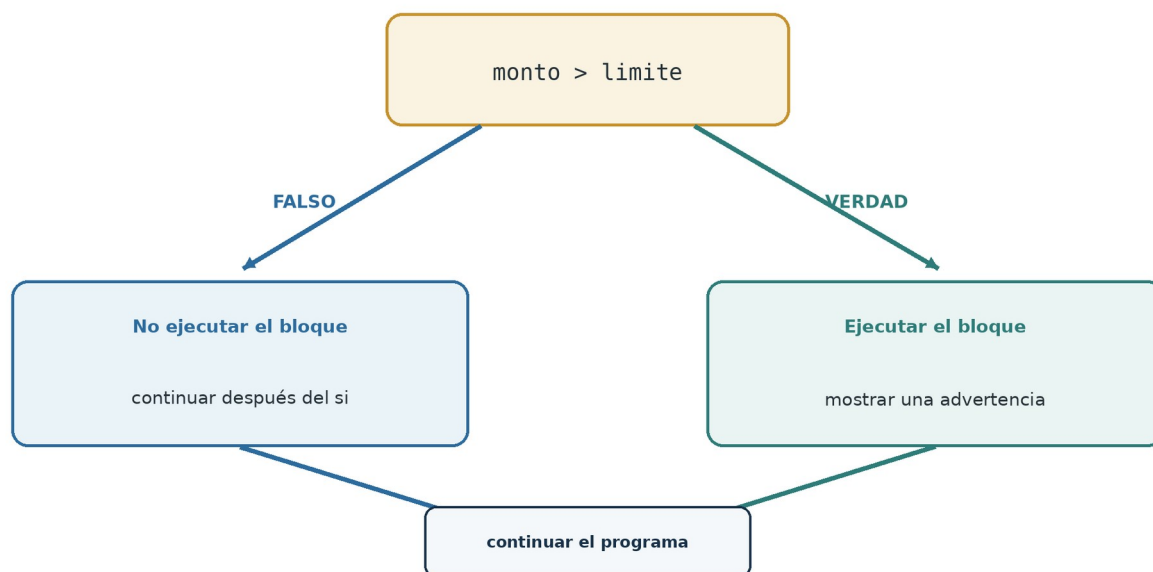


Figura 5.1. Una condición divide el recorrido en caminos posibles. Elaboración propia.

Alcance del capítulo: las decisiones se ejecutarán una vez por cada encuentro durante el recorrido. La repetición de decisiones mediante ciclos se estudiará en el capítulo 6.

5.1 De calcular una condición a controlar el recorrido

La expresión ``monto > limite`` produce ``VERDAD`` o ``FALSO``. Por sí sola, esa expresión no muestra mensajes ni cambia el saldo. La estructura ``si`` utiliza el resultado para decidir si debe ejecutar un bloque indentado.

Primera decisión

```
PRINCIPAL()
  monto <- 125
  limite <- 100

  si monto > limite
    escribir "El monto supera el límite"

  escribir "Fin de la revisión"
```

Como 125 es mayor que 100, la condición produce ``VERDAD`` y se ejecuta la instrucción indentada. Después, el recorrido continúa con ``escribir "Fin de la revisión"``. Si ``monto`` fuera 80, el bloque se omitiría, pero la instrucción posterior seguiría ejecutándose.

Parte	Fragmento	Responsabilidad
Palabra de control	<code>`si`</code>	Inicia una decisión.
Condición	<code>`monto > limite`</code>	Produce un valor lógico.
Bloque	Línea indentada	Se ejecuta solo cuando la condición es VERDAD.
Continuación	Línea sin esa indentación	Se ejecuta después de resolver la decisión.

Idea clave: una condición describe una pregunta lógica; una decisión utiliza la respuesta para controlar instrucciones.

Pausa y piensa: predice la salida si ``monto`` vale 100. La comparación usa ``>``, no ``>=``; el valor límite pertenece a la rama falsa.

5.2 La estructura ``si`` y el bloque indentado

CLRS delimita los bloques mediante indentación. No se escribe ``entonces``, una llave de apertura ni una instrucción de cierre. La posición de la línea comunica si pertenece al bloque de la decisión.

Dos instrucciones en el mismo bloque

```

PRINCIPAL( )
  saldo <- 300
  monto <- 350

  si monto > saldo
    escribir "Saldo insuficiente"
    escribir "La operación no debería continuar"

  escribir "Revisión terminada"

```

Las dos primeras salidas están al mismo nivel de indentación y pertenecen al bloque. La última vuelve al nivel de la asignación y queda fuera de la decisión.

Error	Consecuencia	Corrección
No indentar después de `si`	No existe un bloque válido.	Indentar al menos una instrucción.
Indentar una línea de más	La línea queda condicionada sin intención.	Volver al nivel exterior.
Mezclar bloques visualmente	Dificulta reconocer el recorrido.	Usar indentación consistente.
Escribir una condición no significativa	Puede depender de conversiones implícitas.	Construir una expresión lógica explícita.

Advertencia: la indentación no es decoración. Cambiarla puede modificar qué instrucciones se ejecutan y también puede producir un error de sintaxis.

5.3 Elegir entre dos caminos con `si-sino`

Una decisión simple responde “¿ejecuto este bloque?”. Cuando el problema exige una respuesta para ambos resultados, se utiliza `sino`. Exactamente uno de los dos bloques se ejecutará.

Clasificar una operación

```

PRINCIPAL( )
  saldo <- 500
  monto <- 85

  si monto <= saldo
    escribir "Operación permitida"
  sino
    escribir "Saldo insuficiente"

```

Resultado de `monto <= saldo`	Bloque ejecutado	Bloque omitido
VERDAD	`si`	`sino`
FALSO	`sino`	`si`

No es necesario escribir dos condiciones complementarias. La segunda rama ya representa todos los casos en los que la condición del `si` es falsa.

Redundancia que conviene evitar

```

si monto <= saldo
    escribir "Permitida"
sino si monto > saldo
    escribir "Rechazada"

```

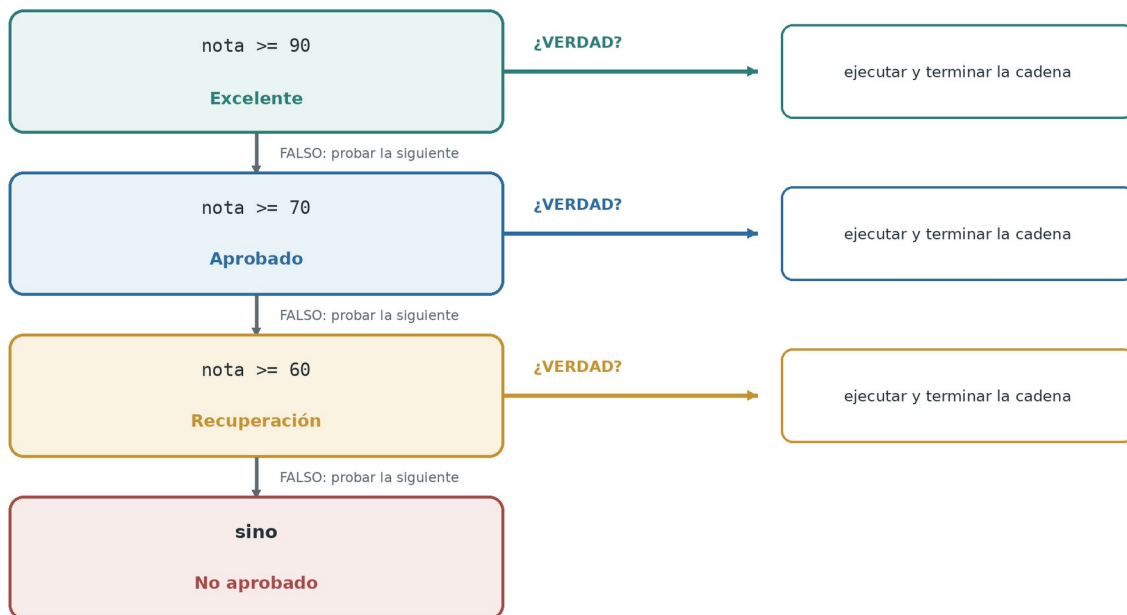
La segunda comparación repite la negación exacta de la primera. `sino` expresa la intención con menos posibilidades de contradicción.

Importante: `sino` no tiene condición propia. Su significado depende del fracaso de todas las condiciones anteriores de la misma cadena.

5.4 Clasificar varios casos con `sino si`

Cuando existen más de dos categorías, una cadena `si - sino si - sino` evalúa condiciones de arriba hacia abajo. En cuanto encuentra la primera condición verdadera, ejecuta ese bloque y omite el resto de la cadena.

Una cadena selecciona la primera condición verdadera



El orden forma parte del algoritmo: una condición demasiado general puede impedir que se evalúen casos posteriores.

Figura 5.2. Selección de la primera condición verdadera. Elaboración propia.

Clasificación de una nota

```

PRINCIPAL()
  nota <- 84

  si nota >= 90
    escribir "Excelente"
  sino si nota >= 70
    escribir "Aprobado"
  sino si nota >= 60
    escribir "Recuperación"
  sino
    escribir "No aprobado"

```

Para `nota = 84`, la primera comparación es falsa y la segunda es verdadera. Se escribe “Aprobado”. Aunque `84 >= 60` también sería verdadero, esa condición ya no se evalúa porque la cadena terminó en la primera coincidencia.

Regla de orden: coloca primero los casos más específicos o los límites superiores. Si comenzaras con `nota >= 60`, una nota de 95 quedaría clasificada como “Recuperación”.

5.5 Diseñar rangos sin huecos ni superposiciones

Una clasificación por rangos debe responder tres preguntas: ¿qué valores pertenecen a cada categoría?, ¿qué ocurre exactamente en los límites? y ¿queda algún valor sin clasificar?

Los límites deben pertenecer a una sola categoría



Pruebas de frontera recomendadas: 59, 60, 69, 70, 89, 90 y 100.

Cada valor límite confirma qué comparación incluye el extremo y cuál lo excluye.

Figura 5.3. Fronteras de una clasificación de notas. Elaboración propia.

Valor	Condición que lo acepta primero	Categoría
59	Ninguna condición anterior	No aprobado
60	`nota >= 60`	Recuperación
69	`nota >= 60`	Recuperación
70	`nota >= 70`	Aprobado

Valor	Condición que lo acepta primero	Categoría
89	`nota >= 70`	Aprobado
90	`nota >= 90`	Excelente

Al ordenar de mayor a menor, cada condición puede expresar solo el límite inferior. Las condiciones anteriores ya descartaron los valores superiores. Esta forma reduce expresiones largas como `nota >= 70 y nota < 90`, aunque ambas pueden ser correctas según el contexto.

Pausa y piensa: diseña categorías para edades 0-11, 12-17, 18-64 y 65 o más. Escribe los casos frontera que utilizarías para comprobarlas.

5.6 Condiciones compuestas

Los operadores `y`, `o` y `no` permiten expresar reglas con varias afirmaciones. La decisión sigue teniendo una sola condición, pero esa condición puede combinar resultados lógicos.

Intención	Condición	Lectura
Exigir dos requisitos	`esCliente y total >= 1000`	Ambas afirmaciones deben ser VERDAD.
Aceptar alternativas	`pago = "efectivo" o pago = "tarjeta"`	Al menos una afirmación debe ser VERDAD.
Excluir una situación	`no estaBloqueado`	Invierte el valor lógico.
Expresar un rango	`edad >= 18 y edad <= 65`	El valor debe cumplir ambos límites.

Descuento con dos requisitos

```
PRINCIPAL()
  esCliente <- VERDAD
  total <- 1250

  si esCliente y total >= 1000
    escribir "Aplica descuento preferente"
  sino
    escribir "No aplica descuento preferente"
```

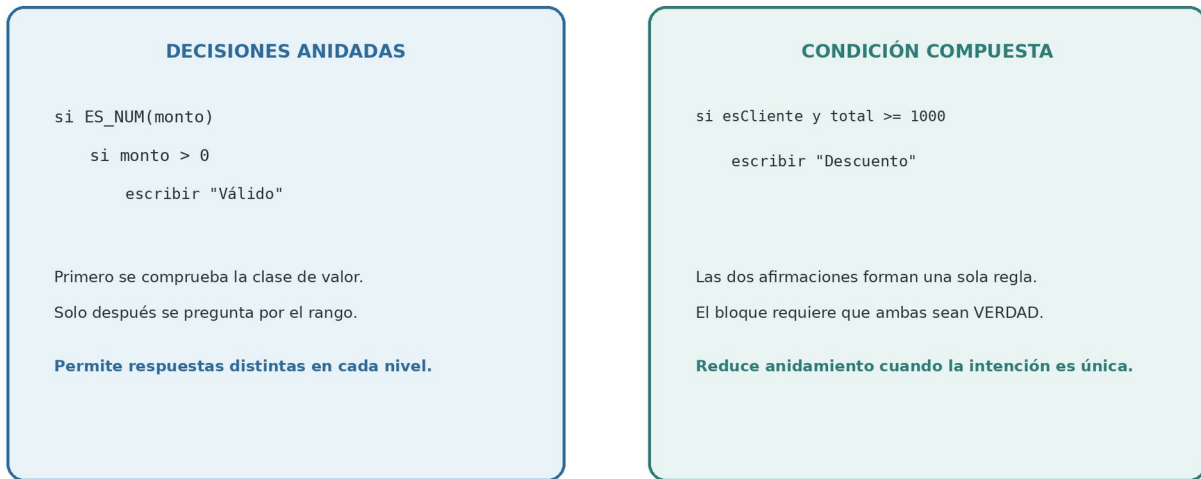
Los paréntesis son útiles cuando se combinan varios operadores: `si (esCliente o tieneCupon) y total >= 500`. La agrupación comunica que primero se acepta una de las dos credenciales y después se exige el total mínimo.

Advertencia: una condición más corta no siempre es más clara. Agrupa y nombra resultados intermedios cuando la regla de negocio contenga varias ideas.

5.7 Decisiones anidadas

Una decisión está anidada cuando aparece dentro del bloque de otra. El anidamiento es apropiado cuando una pregunta solo tiene sentido después de que otra haya sido resuelta.

Anidar y combinar condiciones responden preguntas distintas



No elimines anidamiento a costa de perder información sobre por qué una validación falló.

Figura 5.4. Decisiones anidadas y condiciones compuestas. Elaboración propia.

Validar en dos niveles
<pre> PRINCIPAL() monto <- "" leer monto si ES_NUM(monto) si monto > 0 escribir "Monto válido" sino escribir "El monto debe ser mayor que cero" sino escribir "Se esperaba un número" </pre>

El primer nivel determina si es razonable preguntar por el rango. El segundo distingue un número aceptable de uno no positivo. Cada fallo puede producir un mensaje específico.

Conviene anidar cuando...	Conviene combinar cuando...
La segunda pregunta depende conceptualmente de la primera.	Las afirmaciones forman una sola regla.
Se necesitan mensajes distintos para cada fallo.	Solo importa si el conjunto completo se cumple.
Cada nivel controla tareas diferentes.	El anidamiento oculta una intención sencilla.

Criterio de diseño: reduce anidamiento innecesario, pero no sacrifiques la capacidad de explicar por qué un dato fue rechazado.

5.8 Validar datos antes de utilizarlos

Validar significa comprobar que un valor cumple las condiciones necesarias para participar en el problema. Una entrada puede ser numérica y aun así ser inválida: un radio negativo tiene el tipo esperado, pero viola el dominio del cálculo.

Validar significa impedir que un dato inválido avance

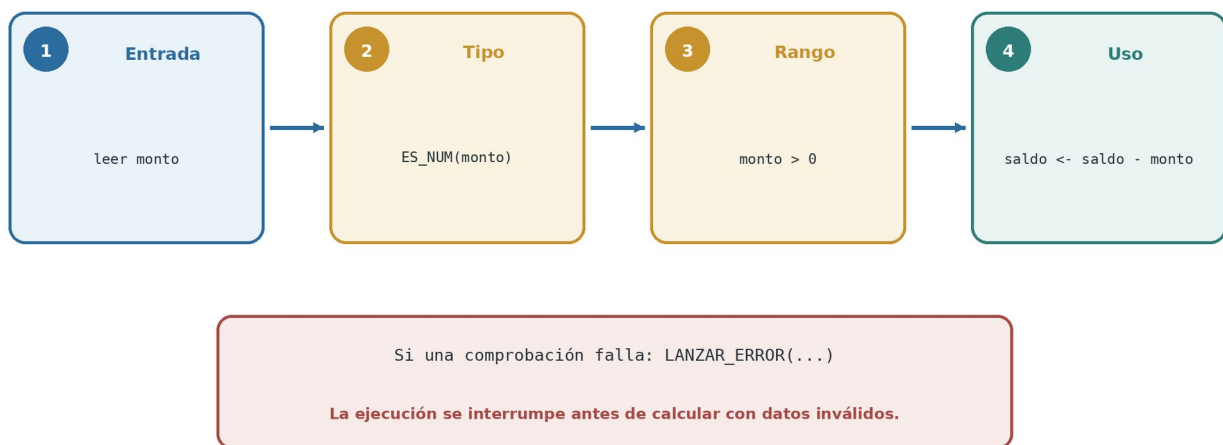


Figura 5.5. Validación por tipo y rango antes del cálculo. Elaboración propia.

Validación de un radio

```

PRINCIPAL()
  radio <- ""
  escribir "Introduce el radio"
  leer radio

  si no ES_NUM(radio)
    LANZAR_ERROR("El radio debe ser numérico")

  si radio <= 0
    LANZAR_ERROR("El radio debe ser mayor que cero")

  area <- PI() * radio^2
  escribir "Área: " + area
  
```

Las dos decisiones se mantienen separadas porque detectan causas diferentes. Si una falla, `LANZAR\_ERROR` interrumpe la ejecución y evita que el cálculo use un dato inválido.

Clase de validación	Pregunta	Ejemplo
Presencia	¿Existe un valor utilizable?	Cadena no vacía.
Tipo	¿Es de la clase esperada?	`ES_NUM(radio)`.
Rango	¿Está dentro de límites válidos?	`radio > 0`.
Relación	¿Es coherente con otros datos?	`inicio <= fin`.
Regla del problema	¿Cumple una restricción específica?	No superar el saldo disponible.

Importante: validar no consiste en convertir cualquier entrada hasta que “funcione”. La conversión cambia una representación; la validación decide si el dato es aceptable para el problema.

5.9 Errores de lógica frecuentes en decisiones

Error	Ejemplo	Por qué falla
Usar el operador equivocado	`edad > 18` cuando 18 debe aceptarse	Excluye el límite.
Ordenar mal una cadena	`nota >= 60` antes de `nota >= 90`	La condición general captura casos superiores.
Crear un hueco	Una categoría termina en 69 y la siguiente inicia en 71	El valor 70 no tiene respuesta.
Superponer categorías independientes	Dos `si` separados para clases excluyentes	Puede ejecutar más de una clasificación.
Confundir asignación e igualdad	Usar `<-` donde se desea comparar	Actualiza en vez de preguntar.
Indentar mal	Una salida queda dentro del bloque equivocado	El recorrido real difiere de la intención.
Usar `sino` como si tuviera condición	Esperar que pruebe una regla nueva	Solo representa el fracaso previo.

Dos `si` independientes no forman una cadena

```
si nota >= 60
  escribir "Aprobó"
```

```
si nota >= 90
  escribir "Excelente"
```

Para nota = 95 se ejecutan ambos bloques. Esto puede ser correcto si se desean dos mensajes, pero es incorrecto si las categorías deben ser mutuamente excluyentes.

Idea clave: elige entre decisiones independientes y una cadena según si pueden ejecutarse varios resultados o debe seleccionarse solo uno.

5.10 Probar ramas y fronteras

Ejecutar una decisión con un único dato no demuestra que esté bien diseñada. Cada resultado posible debe ser recorrido por al menos un caso de prueba. Cuando existe un límite, conviene probar un valor anterior, el valor exacto y uno posterior.

Probar decisiones exige recorrer cada rama y sus fronteras

Caso	Entrada	Camino esperado	Resultado
Normal bajo	monto = 80	si: FALSO	Sin advertencia
Frontera	monto = 100	monto > 100: FALSO	Sin advertencia
Frontera + 1	monto = 101	si: VERDAD	Mostrar advertencia
Tipo inválido	monto = "abc"	validación: VERDAD	Lanzar error
Rango inválido	monto = -5	segunda validación	Lanzar error

Cobertura de ramas: al menos un caso debe ejecutar cada resultado posible de la decisión.

Pruebas de frontera: usa valores justo antes, exactamente en y justo después de cada límite.

Figura 5.6. Casos para cubrir ramas y fronteras. Elaboración propia.

Estrategia	Pregunta que responde
Caso normal verdadero	¿El bloque se ejecuta cuando debe?
Caso normal falso	¿El bloque se omite o se elige la alternativa?
Valor de frontera	¿El operador incluye o excluye el límite?
Dato inválido	¿La validación detiene el programa?
Combinación lógica	¿Se probaron combinaciones relevantes de VERDAD y FALSO?
Cada categoría	¿Todas las ramas de una cadena pueden alcanzarse?

La cobertura de ramas no garantiza por sí sola que el programa sea correcto, pero evita una revisión superficial en la que solo se observa el camino más común.

Pausa y piensa: para `si total >= 1000`, propone tres casos de frontera. Después añade uno que pruebe un tipo inválido y otro que pruebe una regla relacionada, como saldo suficiente.

5.11 Utilizar el diagrama de flujo para revisar caminos

El generador de diagramas de CLRS Kit representa una decisión como una bifurcación. Esta vista ayuda a comprobar si existe una salida para cada caso, si una cadena tiene demasiados niveles y si una instrucción quedó accidentalmente dentro de un bloque.

- Compara cada condición del código con una bifurcación del diagrama.
- Sigue por separado el camino verdadero y el falso.
- Comprueba dónde vuelven a reunirse los caminos.
- Revisa que cada `sino si` aparezca en el orden previsto.
- Confirma que las instrucciones posteriores estén fuera de la decisión cuando deban ejecutarse siempre.

Laboratorio breve: crea un programa con `si-sino`, genera su diagrama y cambia la indentación de una salida. Corrige cualquier error sintáctico y observa después cómo cambia el camino representado.

5.12 Laboratorio CLRS: clasificar y validar un gasto

Este laboratorio amplía el caso longitudinal. El programa recibe un gasto, comprueba que sea numérico y positivo, determina si puede cubrirse con el saldo y clasifica su magnitud.

Registro de gasto con decisiones

```
PRINCIPAL()
  saldo <- 500
  monto <- ""

  escribir "Introduce el monto del gasto"
  leer monto

  si no ES_NUM(monto)
    LANZAR_ERROR("El monto debe ser numérico")

  si monto <= 0
    LANZAR_ERROR("El monto debe ser mayor que cero")

  si monto > saldo
    escribir "El gasto supera el saldo disponible"
  sino si monto >= 200
    escribir "Gasto alto"
  sino si monto >= 100
    escribir "Gasto medio"
  sino
    escribir "Gasto bajo"

  saldoRestante <- saldo - monto
  escribir "Saldo proyectado: " + saldoRestante
```

Antes de ejecutar, predice la salida para 80, 100, 200, 500, 501, 0 y una cadena no numérica. Después registra el camino real. Observa que un monto mayor que el saldo recibe una advertencia antes de llegar a las categorías de magnitud.

Entrada	Primera condición decisiva	Salida principal esperada
80	Todas las condiciones anteriores son falsas	Gasto bajo
100	`monto >= 100`	Gasto medio
200	`monto >= 200`	Gasto alto
500	`monto >= 200`	Gasto alto
501	`monto > saldo`	Supera el saldo
0	`monto <= 0`	Error
`"abc"`	`no ES_NUM(monto)`	Error

Observación crítica: el programa todavía calcula un saldo proyectado negativo cuando el gasto supera el saldo, porque la advertencia no interrumpe la ejecución. Decide si la regla correcta debería advertir o rechazar; después modifica el diseño con `LANZAR\_ERROR` si la operación no debe continuar.

5.13 Mejorar la claridad de una decisión

Una decisión correcta puede seguir siendo difícil de entender. La claridad mejora cuando las condiciones expresan una idea, las ramas contienen responsabilidades relacionadas y los nombres reemplazan cálculos repetidos.

Nombrar una condición compleja

```
esClientePreferente <- esCliente y total >= 1000

si esClientePreferente
  escribir "Aplica descuento preferente"
```

El nombre no cambia el resultado; hace visible qué significa la combinación. Esta técnica es útil cuando una condición se reutiliza o cuando su propósito no puede comprenderse con una lectura rápida.

Señal de dificultad	Mejora posible
Condición demasiado larga	Separar resultados lógicos con nombres significativos.
Muchos niveles de indentación	Validar y terminar temprano, o extraer una responsabilidad en una función más adelante.
Misma expresión repetida	Calcularla una vez y conservar el resultado.
Mensajes vagos	Explicar qué regla falló y qué se esperaba.
Cadena extensa y cambiante	Revisar si el problema necesita otra representación en cursos posteriores.

Importante: no conviertas cada comparación en una variable. Introduce un nombre cuando aporte significado, evite repetición o facilite probar una regla.

5.14 Caso longitudinal: el programa comienza a responder

En capítulos anteriores, el registro de gastos calculaba valores y mostraba resultados sin alterar su conducta. Ahora puede rechazar entradas inválidas, advertir sobre una insuficiencia y clasificar el gasto. El estado sigue siendo el mismo tipo de conjunto de asociaciones; lo nuevo es que no todas las instrucciones se ejecutan en todos los recorridos.

1. La entrada actualiza el valor de `monto`.
2. Las validaciones deciden si la ejecución puede continuar.
3. La comparación con `saldo` distingue una operación cubierta de una insuficiente.
4. La cadena clasifica el monto en una sola categoría.
5. La salida depende del camino recorrido.

Conexión hacia adelante: para registrar varios gastos será necesario volver a ejecutar decisiones con nuevos datos. El capítulo 6 incorporará ciclos y mostrará cómo garantizar progreso y terminación.

5.15 Reconstruye el modelo mental

Una decisión conecta una expresión lógica con el flujo de ejecución. `si` ejecuta un bloque solo cuando la condición es verdadera; `sino` representa el caso restante; `sino si` permite probar categorías en orden; el anidamiento crea preguntas dependientes; y la validación evita que datos inaceptables alcancen cálculos posteriores.

6. Una condición produce VERDAD o FALSO.
7. Una decisión utiliza ese resultado para seleccionar instrucciones.
8. La indentación delimita los bloques de CLRS.
9. `si-sino` selecciona exactamente una de dos ramas.
10. Una cadena ejecuta la primera condición verdadera.
11. El orden de las condiciones forma parte del algoritmo.
12. Los límites deben probarse explícitamente.
13. Las decisiones anidadas expresan preguntas dependientes.
14. Las condiciones compuestas reúnen afirmaciones en una sola regla.
15. Validar exige comprobar tipo, rango y reglas del problema.
16. Las pruebas deben cubrir ramas y fronteras.

Comprobación activa: explica con un ejemplo la diferencia entre dos `si` independientes y una cadena `si-sino si-sino`. Después identifica una situación en la que convenga anidar.

Ejercicios

Predice cada recorrido antes de ejecutar. En los ejercicios de clasificación, incluye casos de frontera.

1. Indica qué líneas se ejecutan cuando `monto = 100` en `si monto > 100`. **[Reconocer]**
2. Escribe una decisión que muestre “Positivo” solo cuando un número sea mayor que cero. **[Aplicar]**
3. Transforma la decisión anterior en `si-sino` para distinguir positivo de no positivo. **[Aplicar]**
4. Clasifica una temperatura como bajo cero, templada o alta. Define y documenta tus límites. **[Construir]**
5. Corrige una cadena que prueba `nota >= 60` antes de `nota >= 90`. **[Analizar]**
6. Diseña los casos frontera para categorías de edad 0-11, 12-17, 18-64 y 65 o más. **[Analizar]**
7. Escribe una condición para aceptar una edad entre 18 y 65, ambos incluidos. **[Aplicar]**
8. Construye una condición que acepte pago en efectivo o tarjeta, pero rechace cualquier otra cadena. **[Aplicar]**
9. Explica cuándo dos `si` independientes pueden ejecutar dos bloques para la misma entrada. **[Comprender]**
10. Convierte dos decisiones complementarias en una estructura `si-sino`. **[Mejorar]**
11. Valida que un radio sea numérico y mayor que cero antes de calcular el área. **[Construir]**
12. Valida tres coeficientes numéricos y rechaza `a = 0` antes de aplicar la fórmula cuadrática. **[Construir]**
13. Para un discriminante negativo, utiliza una decisión que lance un error antes de calcular la raíz. **[Aplicar]**
14. Diseña una clasificación de salario en tres rangos sin huecos ni superposiciones. **[Construir]**
15. Compara una condición compuesta con una decisión anidada para validar tipo y rango. **[Analizar]**
16. Encuentra el error de indentación en un programa donde una salida final se ejecuta solo en la rama falsa. **[Detectar]**

17. Diseña una tabla con casos que cubran todas las ramas de un descuento de 0 %, 5 % y 10 %. **[Probar]**
18. Genera un diagrama de flujo para una clasificación de notas y compáralo con el código. **[Representar]**
19. Modifica el laboratorio para rechazar gastos superiores al saldo en lugar de solo advertir. **[Mejorar]**
20. Crea deliberadamente una superposición entre rangos, demuestra el fallo con una prueba y corrígelo. **[Experimentar]**

Mini proyecto: cotizador con reglas

Situación: un servicio calcula un precio base y aplica reglas según tipo de cliente, monto de compra y método de pago. El programa debe validar entradas y seleccionar un único descuento.

Requisito	Criterio de éxito
Entradas	Lee total, tipo de cliente y método de pago.
Validación de tipo	Comprueba que el total sea numérico.
Validación de rango	Rechaza totales menores o iguales que cero.
Descuento	Selecciona 10 %, 5 % o 0 % mediante una cadena ordenada.
Regla compuesta	Combina al menos dos afirmaciones con `y` u `o`.
Salida	Muestra regla aplicada, descuento y total final.
Pruebas	Incluye cada rama, fronteras e input inválido.
Diagrama	Genera y revisa el diagrama de flujo.

Documenta por qué el orden de las condiciones impide que una categoría general capture un caso más específico.

Autoevaluación

Responde sin consultar el capítulo. Después vuelve a las secciones que no puedas justificar con una traza o un caso de prueba.

Afirmación	Sí	Todavía no
Puedo distinguir una condición de una decisión.	☐	☐
Puedo delimitar correctamente un bloque mediante indentación.	☐	☐
Puedo construir decisiones `si`, `si-sino` y cadenas `sino si`.	☐	☐
Puedo ordenar condiciones de una clasificación.	☐	☐
Puedo detectar huecos y superposiciones en rangos.	☐	☐
Puedo decidir entre anidamiento y condición compuesta.	☐	☐
Puedo validar tipo y rango antes de calcular.	☐	☐
Puedo diseñar casos que cubran ramas y fronteras.	☐	☐
Puedo revisar el recorrido mediante un diagrama de flujo.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas especificar categorías sin ambigüedad, escribir una cadena que seleccione una sola respuesta y demostrar su comportamiento mediante casos normales, límites e inválidos.

Glosario

Término	Definición de trabajo
Condición	Expresión cuyo resultado se interpreta como VERDAD o FALSO.
Decisión	Estructura que selecciona instrucciones según una condición.
Rama	Camino posible dentro de una decisión.
Bloque	Conjunto de instrucciones agrupadas mediante indentación.
`si`	Estructura que ejecuta un bloque cuando la condición es verdadera.
`sino`	Rama que representa el fracaso de las condiciones anteriores.
`sino si`	Rama con una condición adicional dentro de una cadena.
Cadena de decisiones	Secuencia ordenada que selecciona la primera condición verdadera.
Decisión anidada	Decisión situada dentro del bloque de otra.
Condición compuesta	Condición formada por varias expresiones lógicas.
Validación	Comprobación de que un dato cumple requisitos antes de utilizarse.
Frontera	Valor exacto que separa dos categorías o comportamientos.
Cobertura de ramas	Conjunto de pruebas que ejecuta cada resultado posible de una decisión.
Mutuamente excluyente	Propiedad de categorías que no deben seleccionarse simultáneamente.

Referencias del capítulo

CLRS Kit Español. (2026). Código fuente y documentación del analizador sintáctico [Repositorio de software].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Scott, M. L. (2015). Programming language pragmatics (4th ed.). Morgan Kaufmann.

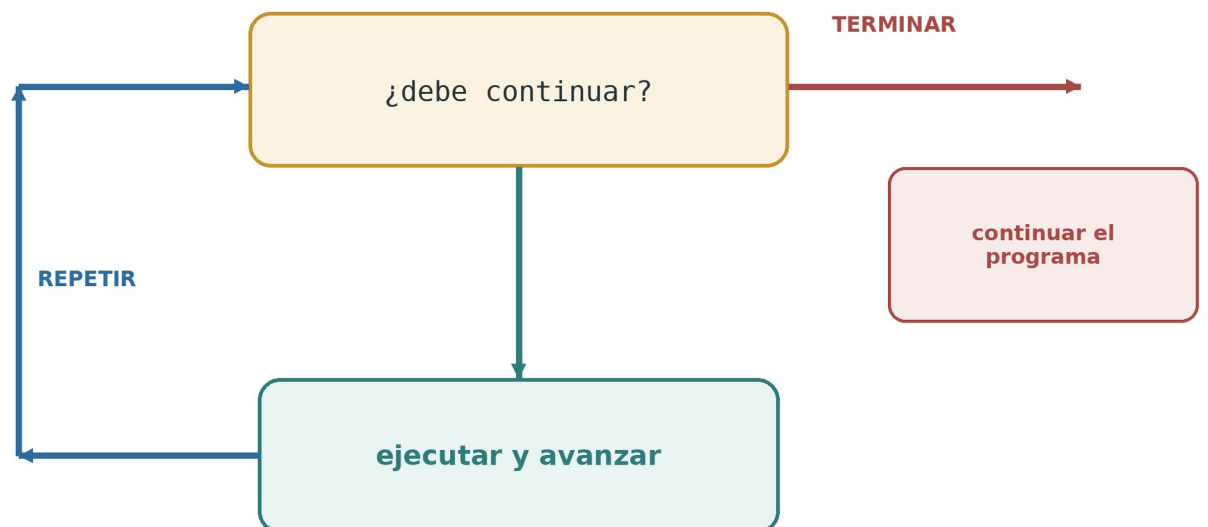
Continuidad: en el capítulo 6 las decisiones aparecerán dentro de ciclos. El nuevo desafío será repetir un proceso mientras el estado cambie, sin perder el control del progreso ni de la terminación.

CAPÍTULO

6

Repetir sin perder el control

Ciclos, progreso, terminación y crecimiento del trabajo



Repetir sin perder el control

¿Cómo se repite un proceso garantizando que avance y termine?

Imagina un programa que debe mostrar cien números, registrar una cantidad desconocida de gastos o seguir solicitando un dato hasta que sea válido. Copiar la misma instrucción muchas veces puede producir un resultado, pero obliga a decidir manualmente cuántas copias necesita cada problema. Además, cualquier cambio debe repetirse en todas ellas.

Los ciclos permiten describir una repetición una sola vez. Esa ventaja introduce una responsabilidad nueva: el programa debe saber cuándo continuar, cómo cambiar su estado y en qué momento detenerse. Un ciclo sin progreso puede ejecutar el mismo bloque indefinidamente; uno con límites incorrectos puede omitir datos o procesarlos una vez de más.

Promesa de aprendizaje: al terminar podrás construir ciclos `para` y `mientras`; reconocer contadores, acumuladores, banderas y centinelas; seguir iteraciones mediante trazas; detectar ciclos infinitos y errores de frontera; y relacionar recorridos simples con $O(1)$, $O(n)$, $O(n^2)$ y $O(\log n)$.

Todo ciclo necesita estado, condición, trabajo y progreso

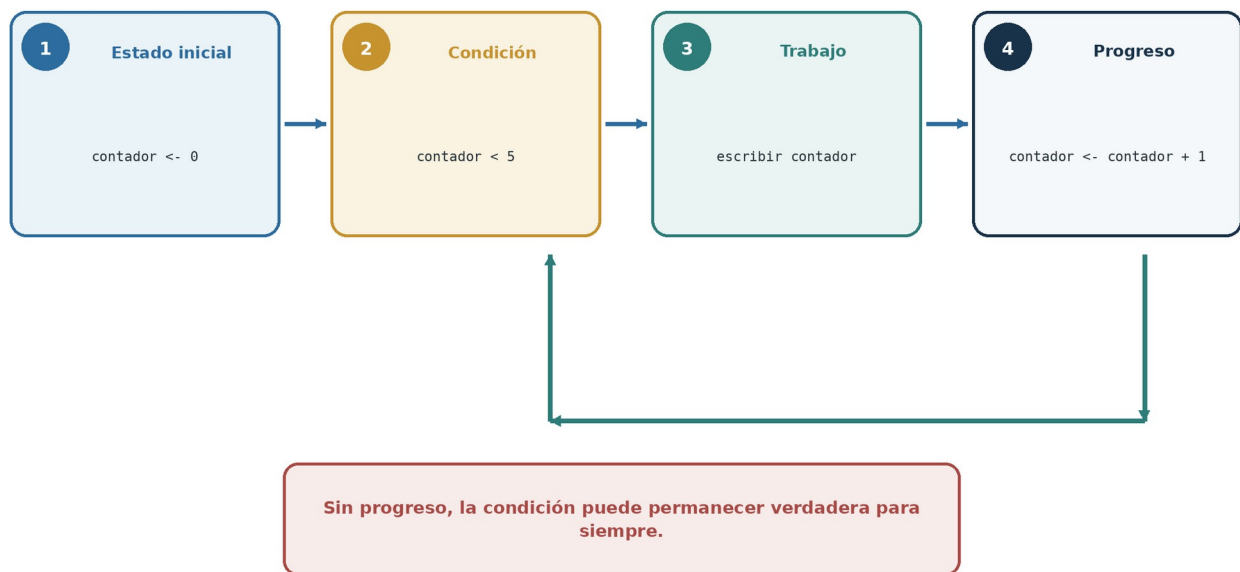


Figura 6.1. Componentes mínimos de una repetición controlada. Elaboración propia.

Alcance del capítulo: la complejidad se utilizará para desarrollar intuición sobre el crecimiento del trabajo. No se demostrarán recurrencias ni se estudiarán técnicas formales avanzadas.

6.1 El problema de repetir instrucciones

Una secuencia fija funciona cuando la cantidad de operaciones también es fija. Para mostrar los números del 1 al 5 se podrían escribir cinco instrucciones:

Repetición escrita manualmente

```
escribir 1
escribir 2
escribir 3
escribir 4
escribir 5
```

La solución produce la salida, pero no expresa la idea general: comenzar en un valor, mostrarlo, avanzar y detenerse al alcanzar un límite. Si el límite cambia de 5 a 500, el diseño deja de ser razonable.

Un ciclo concentra esa regla en una estructura. El bloque contiene el trabajo repetido y el control del ciclo determina cuántas veces se ejecuta.

Idea clave: un ciclo no significa “hacer algo muchas veces” de manera vaga. Significa repetir un bloque bajo una regla de control explícita.

Pausa y piensa: identifica qué parte del ejemplo manual representa el estado, cuál representa el avance y cuál funciona como límite.

6.2 Las cuatro preguntas de todo ciclo

Antes de escribir la sintaxis, conviene responder cuatro preguntas. Si alguna queda sin respuesta, el ciclo todavía no está diseñado.

Pregunta	Elemento	Ejemplo
¿Desde qué estado comienza?	Inicialización	<code>`i <- 0`</code>
¿Cuándo debe continuar?	Condición o límite	<code>`i < 5`</code>
¿Qué trabajo se repite?	Bloque	<code>`escribir i`</code>
¿Cómo se acerca al final?	Progreso	<code>`i <- i + 1`</code>

En un ciclo ``para``, la estructura expresa automáticamente parte del control. En un ciclo ``mientras``, quien programa debe construir y mantener explícitamente el estado que modifica la condición.

Advertencia: una condición verdadera no demuestra que el ciclo esté bien. También debe existir un camino por el que esa condición pueda volverse falsa.

6.3 Recorrer una cantidad previsible con `para`

El ciclo `para` es adecuado cuando el recorrido puede describirse mediante un valor inicial y un límite final. La variante `hasta` avanza de forma ascendente.

Recorrido ascendente

```
PRINCIPAL()
  para i <- 1 hasta 5
    escribir i
```

La variable `i` toma sucesivamente los valores 1, 2, 3, 4 y 5. El límite se incluye. Después de ejecutar el bloque con 5, el ciclo termina y el programa continúa con la siguiente instrucción exterior.

Parte	Fragmento	Función
Variable de iteración	`i`	Conserva la posición actual del recorrido.
Inicio	`1`	Primer valor utilizado.
Dirección	`hasta`	Indica avance ascendente.
Límite	`5`	Último valor incluido.
Bloque	`escribir i`	Trabajo ejecutado en cada iteración.

Pausa y piensa: ¿cuántas veces se ejecuta el bloque de `para i <- 0 hasta 5`? Enumera los valores antes de responder.

6.4 Recorrer en sentido descendente y contar iteraciones

La variante `bajando` reduce automáticamente la variable de iteración hasta alcanzar el límite indicado.

Cuenta regresiva

```
PRINCIPAL()
  para i <- 5 bajando 1
    escribir i

  escribir "Despegue"
```

Los valores escritos son 5, 4, 3, 2 y 1. Tanto el inicio como el límite forman parte del recorrido. Esta inclusión explica muchos errores de una posición.

Recorrido	Valores	Iteraciones
`0 hasta 4`	0, 1, 2, 3, 4	5
`1 hasta 5`	1, 2, 3, 4, 5	5

Recorrido	Valores	Iteraciones
`5 bajando 1`	5, 4, 3, 2, 1	5
`3 bajando 0`	3, 2, 1, 0	4

Para límites enteros y recorrido unitario, la cantidad de iteraciones puede razonarse como la distancia entre los extremos más una unidad:

$$\text{iteraciones} = |\text{límite} - \text{inicio}| + 1$$

Importante: la fórmula ayuda a contar un recorrido simple, pero no sustituye la enumeración cuando las expresiones son más complejas o el límite cambia.

6.5 Elegir entre `para` y `mientras`

`para` y `mientras` expresan dos formas de controlar la repetición

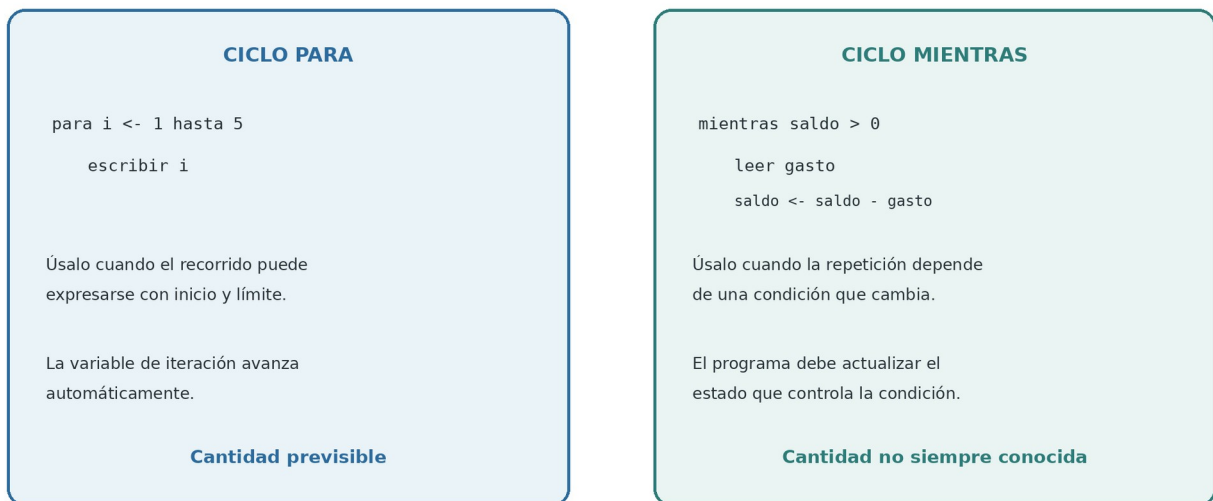


Figura 6.2. Dos formas de controlar una repetición. Elaboración propia.

La diferencia no es que uno pueda hacer cosas que el otro jamás podría expresar. La diferencia pedagógica y de diseño está en qué información comunica con claridad.

Situación	Elección habitual	Razón
Mostrar del 1 al 20	`para`	Inicio y límite conocidos.
Recorrer posiciones de una colección	`para`	El rango depende de la longitud.
Solicitar hasta recibir un número	`mientras`	No se conoce cuántos intentos habrá.
Repetir mientras exista saldo	`mientras`	La condición depende del estado.

Situación	Elección habitual	Razón
Reducir un valor a la mitad hasta 1	`mientras`	El avance es multiplicativo, no unitario.

Criterio práctico: usa `para` cuando la idea central sea recorrer un intervalo. Usa `mientras` cuando la idea central sea conservar una condición.

6.6 Repetir mientras una condición sea verdadera

El ciclo `mientras` comprueba una condición antes de cada repetición. Si la condición es falsa desde el comienzo, el bloque no se ejecuta ninguna vez.

Ciclo controlado manualmente

```
PRINCIPAL()
  i <- 0

  mientras i < 4
    escribir i
    i <- i + 1

  escribir "Fin"
```

Una traza permite comprobar el progreso antes de ejecutar

Iteración	i al comprobar	i < 4	Salida	i después del avance
1	0	VERDAD	0	1
2	1	VERDAD	1	2
3	2	VERDAD	2	3
4	3	VERDAD	3	4
salida	4	FALSO	-	4

La condición se comprueba antes de cada repetición. Cuando es FALSO, el bloque ya no se ejecuta.

Los valores 0, 1, 2 y 3 se escriben; 4 solo aparece en la comprobación final.

Figura 6.3. Traza del ciclo mientras. Elaboración propia.

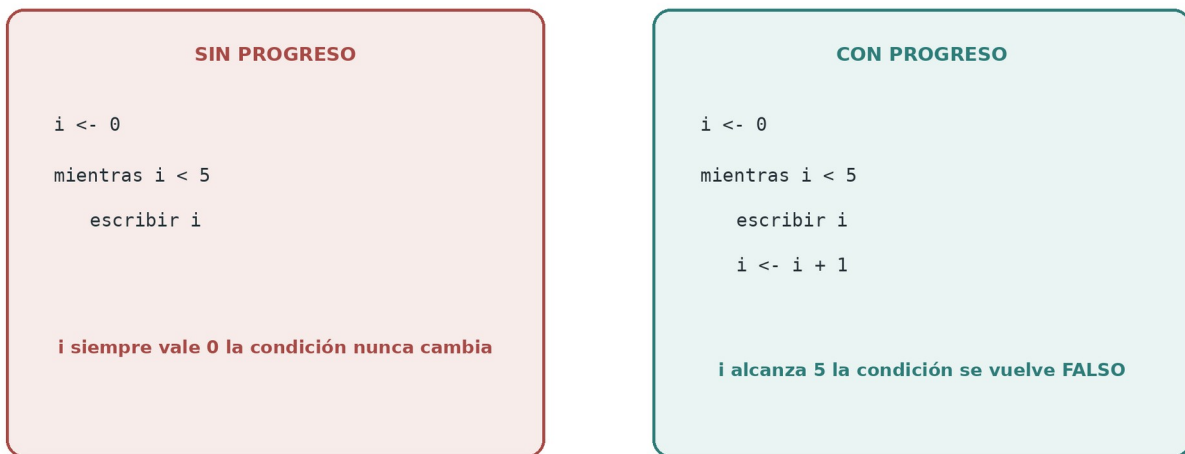
La comprobación final con `i = 4` es necesaria aunque no produzca salida. Es el momento en que el programa descubre que ya no debe repetir el bloque.

Idea clave: la condición se comprueba antes de entrar y antes de volver a entrar. Por eso un ciclo `mientras` puede ejecutarse cero veces.

6.7 Progreso y terminación

La terminación puede analizarse sin ejecutar miles de veces. Basta con identificar qué variable influye en la condición, cómo cambia y si ese cambio la acerca a un estado falso.

Un ciclo termina solo si el estado puede volver falsa su condición



Inicio correcto + condición alcanzable + avance coherente = argumento básico de terminación.

Figura 6.4. La actualización distingue un ciclo finito de uno infinito. Elaboración propia.

Pregunta	Ejemplo correcto	Señal de riesgo
¿Qué controla la condición?	`i` en `i < 5`	La condición depende de un valor que nunca cambia.
¿En qué dirección debe avanzar?	Aumentar `i`	Se actualiza en sentido contrario.
¿Puede alcanzar o cruzar el límite?	0, 1, 2, 3, 4, 5	El paso salta entre estados que conservan la condición.
¿Hay ramas sin progreso?	Todas actualizan `i`	Una decisión evita la actualización en ciertos casos.

Advertencia: un ciclo puede ser infinito aunque contenga una asignación. Lo importante es que la asignación modifique el estado que sostiene la condición en la dirección adecuada.

6.8 Patrones fundamentales de iteración

Muchos problemas de repetición comparten una forma. Reconocerla reduce la carga de diseñar desde cero y permite revisar si falta alguna inicialización o actualización.

Patrones que convierten una repetición en una tarea reconocible



Figura 6.5. Patrones recurrentes de iteración. Elaboración propia.

6.8.1 Contador

Un contador registra cuántas veces ocurre un evento. Comienza normalmente en cero y aumenta cuando se cumple la regla que se desea contar.

Contar valores positivos

```
positivos <- 0
para i <- 1 hasta 5
  leer valor

  si valor > 0
    positivos <- positivos + 1

escribir "Cantidad positiva: " + positivos
```

6.8.2 Acumulador

Un acumulador combina valores sucesivos. Para sumar, su identidad inicial es cero; para multiplicar, suele ser uno.

Acumular cinco gastos

```
total <- 0

para i <- 1 hasta 5
  leer gasto
  total <- total + gasto

escribir "Total: " + total
```

Pausa y piensa: ¿qué resultado produciría el acumulador si comenzara en 100? ¿Sería un error o podría representar una cantidad previa? La respuesta depende de la especificación.

6.8.3 Bandera

Una bandera conserva un estado lógico: si algo ocurrió, si se encontró un valor o si el programa debe seguir intentando.

Conservar si apareció un valor grande

```
hayGrande <- FALSO

para i <- 1 hasta 4
  leer valor

  si valor > 1000
    hayGrande <- VERDAD

escribir hayGrande
```

6.8.4 Centinela

Un centinela es un valor reservado para indicar que la entrada terminó. Debe elegirse de modo que no pueda confundirse con un dato normal válido.

Finalizar con la palabra fin

```
entrada <- ""

mientras entrada != "fin"
  escribir "Escribe un dato o fin"
  leer entrada

  si entrada != "fin"
    escribir "Recibido: " + entrada
```

Advertencia: si `fin` también fuera un dato legítimo, no serviría como centinela. La regla de terminación debe distinguirse de los valores que se desean procesar.

6.9 Repetir una entrada hasta que sea válida

La validación estudiada en el capítulo 5 puede combinarse con `mientras` para permitir nuevos intentos en lugar de interrumpir inmediatamente el programa. Este patrón aparece en varios de los programas de referencia del proyecto.

Solicitar un radio numérico

```
PRINCIPAL()
  radio <- ""

  mientras no ES_NUM(radio)
    escribir "Introduce el radio"
    leer radio

    si no ES_NUM(radio)
      escribir "Solo se aceptan números"

  escribir "Radio válido: " + radio
```

El estado inicial se elige para que la condición sea verdadera y el ciclo solicite al menos una entrada. Cada lectura reemplaza `radio`; cuando el valor es numérico, la siguiente comprobación produce `FALSO` y el ciclo termina.

Importante: esta validación solo comprueba el tipo. Un radio numérico puede seguir siendo inválido si es cero o negativo. Se pueden combinar reglas en la condición o añadir una comprobación posterior.

Validar tipo y rango

```
radio <- ""

mientras no ES_NUM(radio) o radio <= 0
  escribir "Introduce un radio mayor que cero"
  leer radio
```

En un lenguaje dinámico, evaluar `radio <= 0` cuando `radio` aún es una cadena puede depender de conversiones implícitas. Una alternativa más segura es separar la validación de tipo y la de rango en dos momentos o usar una bandera lógica. La claridad importa más que reducir líneas.

6.10 Buscar, obtener máximos y detenerse temprano

Una búsqueda combina repetición, comparación y una condición de salida. Aunque los arreglos se estudiarán en el capítulo 9, el patrón puede comprenderse con entradas sucesivas.

Buscar el número 7 en cinco intentos

```

encontrado <- FALSO
intento <- 1

mientras intento <= 5 y no encontrado
  leer valor

  si valor = 7
    encontrado <- VERDAD

  intento <- intento + 1

escribir encontrado

```

La condición combina dos motivos para continuar: aún quedan intentos y el valor todavía no se encontró. Si cualquiera de cumplirse, el ciclo termina.

Conservar el mayor de cuatro valores

```

leer mayor

para i <- 2 hasta 4
  leer valor

  si valor > mayor
    mayor <- valor

escribir "Mayor: " + mayor

```

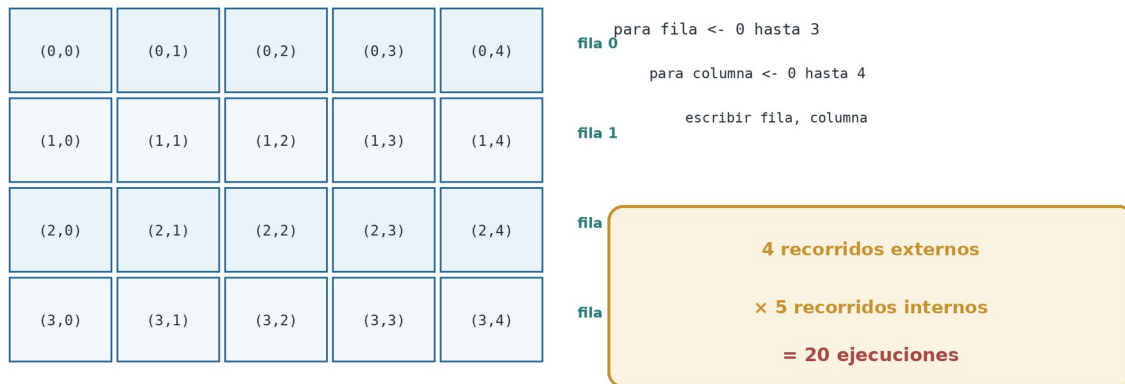
El primer valor inicializa el máximo. Empezar siempre en 0 fallaría si todos los datos fueran negativos.

Idea clave: máximo y mínimo deben inicializarse con un dato real o con una garantía explícita del dominio, no con un valor arbitrario que pueda alterar el resultado.

6.11 Ciclos anidados

Un ciclo puede aparecer dentro del bloque de otro. El ciclo interior completa todo su recorrido por cada valor del ciclo exterior. Esta estructura es útil para tablas, combinaciones de posiciones y datos organizados en filas y columnas.

Un ciclo anidado completa un recorrido interno por cada recorrido externo



La indentación revela qué ciclo contiene al otro y qué instrucciones se repiten en cada nivel.

Figura 6.6. Recorrido de una cuadrícula mediante ciclos anidados. Elaboración propia.

Tabla de multiplicar

```

PRINCIPAL()
  para fila <- 1 hasta 3
    para columna <- 1 hasta 4
      producto <- fila * columna
      escribir fila + " x " + columna + " = " + producto
    
```

El bloque interior se ejecuta 4 veces por cada uno de los 3 valores de `fila`: en total, 12 productos. La multiplicación de cantidades solo funciona cuando cada recorrido mantiene una cantidad estable e independiente.

Advertencia: la cantidad de ciclos escritos no determina por sí sola el costo. Importa cuántas iteraciones realiza cada uno y si sus límites dependen entre sí.

6.12 Errores frecuentes de repetición

Error	Ejemplo o causa	Consecuencia
Olvidar el avance	`i` nunca cambia	Ciclo infinito.

Error	Ejemplo o causa	Consecuencia
Avanzar en dirección incorrecta	Aumentar con condición `i > 0`	Se aleja de la salida.
Error de una posición	Usar `hasta LONG(A)` más adelante	Accede una posición extra.
Inicialización incorrecta	Contador empieza en 1 sin razón	Resultado desplazado.
Actualizar dos veces	La estructura y el bloque cambian el índice	Se omiten valores.
Condición invertida	`mientras ES_NUM(valor)` al validar	Repite los válidos o no entra con inválidos.
Reiniciar dentro	`total <- 0` dentro del ciclo	El acumulador pierde lo anterior.
Progreso condicional	Solo actualiza en una rama	Algunos datos bloquean el ciclo.

Acumulador reiniciado incorrectamente

```
para i <- 1 hasta 5
  total <- 0
  leer valor
  total <- total + valor
```

Al final solo se conserva el último valor. La inicialización debe ocurrir antes del ciclo.

Método de revisión: marca la inicialización, encierra la condición, subraya el avance y simula el primer, segundo y último recorrido. Este procedimiento detecta gran parte de los errores sin depender de la intuición.

6.13 Del número de iteraciones al crecimiento del trabajo

Cuando el tamaño de la entrada aumenta, también puede aumentar la cantidad de veces que se ejecuta un bloque. La notación Big O describe el patrón dominante de ese crecimiento. En este capítulo se utiliza como lenguaje de comparación, no como medición exacta de tiempo.

El costo describe cómo crece el trabajo cuando aumenta la entrada

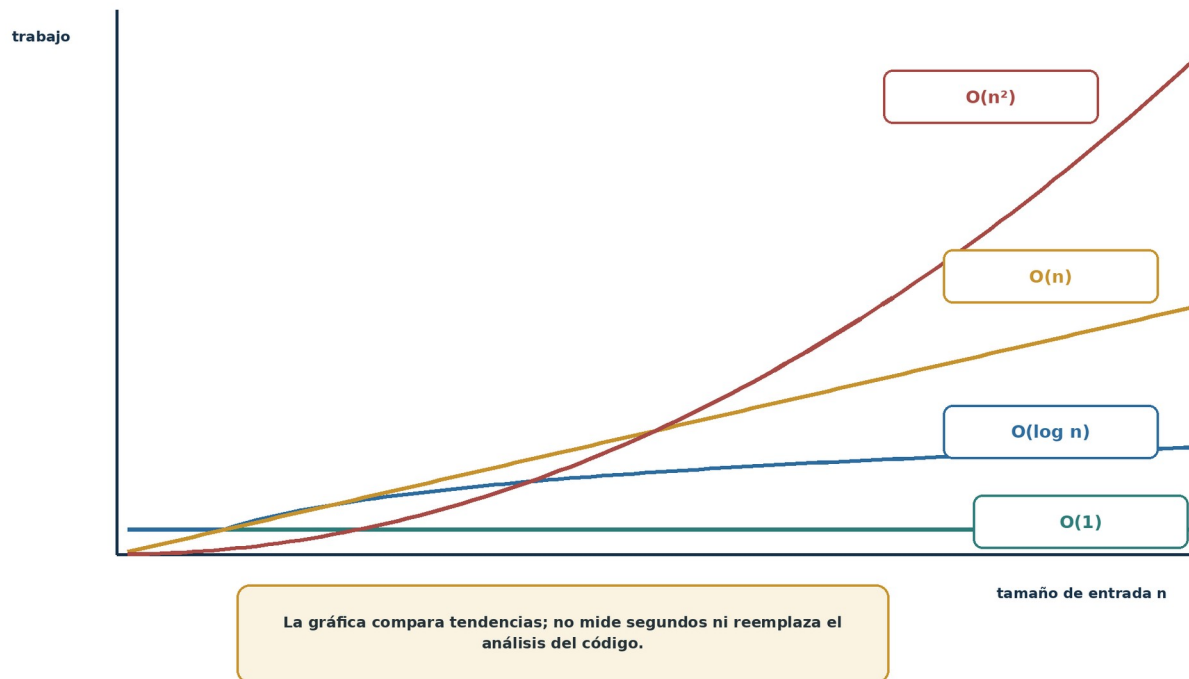


Figura 6.7. Tendencias elementales de crecimiento. Elaboración propia.

Patrón	Ejemplo conceptual	Interpretación
$O(1)$	Asignar y escribir una cantidad fija	El trabajo no crece con n .
$O(n)$	Un recorrido de 0 hasta $n - 1$	El trabajo crece aproximadamente en proporción a n .
$O(n^2)$	Dos recorridos independientes de tamaño n	Duplicar n aproxima cuatro veces el trabajo.
$O(\log n)$	Dividir el problema entre 2 en cada paso	El número de pasos crece lentamente.

Reducción multiplicativa

```
i <- n
```

```
mientras i > 1
  escribir i
  i <- PISO(i / 2)
```

Para $n = 32$, los valores aproximados son 32, 16, 8, 4 y 2. Cada paso reduce a la mitad el estado relevante.

El ciclo anterior no realiza n pasos. La pregunta es cuántas veces puede dividirse n entre 2 antes de alcanzar 1. Ese número corresponde a un crecimiento logarítmico.

Importante: Big O elimina constantes y detalles menores para comparar tendencias. No afirma que todos los programas $O(n)$ tarden lo mismo ni que un $O(\log n)$ siempre sea más rápido para cualquier entrada concreta.

6.14 Interpretar el visualizador de costo de CLRS Kit

CLRS Kit puede mostrar expresiones de costo y una clasificación asintótica junto al código. La herramienta es más útil cuando confirma una predicción hecha antes de activarla.

1. Identifica primero el tamaño relevante, como ``n`` o ``LONG(A)``.
2. Predice cuántas veces se ejecuta el bloque.
3. Determina si los ciclos anidados multiplican recorridos.
4. Observa si el estado avanza por suma constante o reducción multiplicativa.
5. Activa la visualización y compara el resultado con tu razonamiento.
6. Si aparece ``O(?)``, revisa qué relación no puede demostrarse con seguridad.

Advertencia: la clasificación automática no sustituye el análisis manual. Un resultado conservador o desconocido puede ser más correcto que una etiqueta aparentemente precisa basada en una suposición.

6.15 Revisar un ciclo mediante el diagrama de flujo

En el diagrama de flujo, la condición aparece como una bifurcación. La rama verdadera conduce al bloque y regresa a la condición; la falsa abandona la repetición. Este regreso visual ayuda a detectar instrucciones que quedaron fuera del ciclo o avances situados en una rama incompleta.

- Sigue la rama verdadera y confirma que regresa a la condición.
- Localiza la instrucción que modifica el estado de control.
- Comprueba que la rama falsa conduzca a la continuación del programa.
- En ciclos anidados, identifica qué regreso pertenece a cada condición.
- Compara la cantidad de bloques del diagrama con la indentación del código.

Laboratorio breve: crea un ``mientras`` que escriba del 0 al 3, genera su diagrama, mueve accidentalmente el avance fuera del bloque y observa cómo el error sintáctico o el recorrido resultante revela el problema.

6.16 Laboratorio CLRS: registrar varios gastos

El caso longitudinal ya puede recibir más de un gasto. El programa acumulará el total, contará registros y permitirá finalizar mediante un centinela numérico.

Registro repetido de gastos

```

PRINCIPAL()
    saldo <- 1000
    totalGastado <- 0
    cantidad <- 0
    monto <- 0

    escribir "Introduce un gasto mayor que cero"
    escribir "Escribe -1 para terminar"
    leer monto

    mientras monto != -1
        si no ES_NUM(monto)
            escribir "El monto debe ser numérico"
        sino si monto <= 0
            escribir "El monto debe ser mayor que cero"
        sino
            totalGastado <- totalGastado + monto
            cantidad <- cantidad + 1
            saldo <- saldo - monto

            escribir "Saldo actual: " + saldo

        escribir "Introduce otro gasto o -1"
        leer monto

    escribir "Gastos registrados: " + cantidad
    escribir "Total gastado: " + totalGastado
    escribir "Saldo final: " + saldo

```

El centinela se lee antes de entrar y al final de cada recorrido. Esa simetría garantiza que la condición utilice el dato más reciente. Los acumuladores solo se actualizan en la rama válida.

Entrada secuencial	Cantidad final	Total	Observación
100, 50, -1	2	150	Dos gastos válidos.
-1	0	0	El ciclo no entra.
0, 25, -1	1	25	El cero se rechaza y se continúa.
`"abc"`, 10, -1	1	10	La cadena se rechaza.

Observación crítica: el programa permite que el saldo se vuelva negativo. Decide si el problema solo debe informar o impedir gastos superiores al saldo; después añade la regla correspondiente.

6.17 Diseñar ciclos legibles

Un ciclo correcto también debe comunicar su propósito. La claridad mejora cuando el nombre del estado revela su función y el bloque no mezcla demasiadas responsabilidades.

Señal de dificultad	Mejora
Variable llamada `x` controla el ciclo	Usar `intento`, `indice`, `restantes` o un nombre contextual.

Señal de dificultad	Mejora
Condición larga y repetida	Conservar resultados lógicos con nombres significativos.
Bloque extenso	Separar responsabilidades mediante funciones en los capítulos siguientes.
Varios avances dispersos	Centralizar o justificar claramente cada actualización.
Centinela ambiguo	Elegir un valor imposible dentro del dominio normal.
Ciclos anidados profundos	Revisar el diseño y representar el recorrido con una figura o función.

Idea clave: el ciclo debe permitir responder con rapidez: qué se repite, qué cambia y por qué terminará.

6.18 Reconstruye el modelo mental

La repetición controlada surge de una relación entre estado y flujo. El estado inicial permite comenzar; la condición decide si se ejecuta el bloque; el trabajo modifica o utiliza datos; y el progreso acerca la ejecución a una salida. `para` expresa recorridos delimitados, mientras que `mientras` conserva una condición cuyo cambio debe gestionarse explícitamente.

- Un ciclo repite un bloque bajo una regla de control.
- Cada iteración es una ejecución completa del bloque.
- `para` incluye el valor inicial y el límite final.
- `hasta` recorre en forma ascendente y `bajando` en forma descendente.
- `mientras` comprueba la condición antes de cada entrada.
- El progreso debe modificar el estado que sostiene la condición.
- Contadores, acumuladores, banderas y centinelas resuelven tareas recurrentes.
- Los ciclos anidados multiplican recorridos cuando sus tamaños son independientes.
- Las trazas revelan errores de una posición, reinicios y falta de progreso.
- Big O describe el crecimiento dominante del trabajo.
- El visualizador de costo confirma razonamientos, pero no los reemplaza.

Comprobación activa: explica por qué `mientras i < 10` no basta para demostrar que un ciclo termina. Después describe una actualización correcta y una incorrecta para esa condición.

Ejercicios

Predice la cantidad de iteraciones y la salida antes de ejecutar cada solución.

- Enumera los valores de `para i <- 0 hasta 6` y calcula la cantidad de iteraciones. **[Reconocer]**
- Enumera los valores de `para i <- 8 bajando 3`. **[Reconocer]**
- Escribe un ciclo `para` que muestre los números del 10 al 20. **[Aplicar]**
- Escribe una cuenta regresiva de 5 a 0 y muestra “Fin” después del ciclo. **[Aplicar]**

5. Convierte un recorrido `para` de 0 a 4 en un ciclo `mientras` equivalente. [Transformar]
6. Completa la inicialización y el avance faltantes en `mientras i <= 10`. [Aplicar]
7. Detecta por qué `i <- 5; mientras i > 0; i <- i + 1` no termina. [Analizar]
8. Construye una traza para un ciclo que escriba 2, 4, 6 y 8. [Representar]
9. Solicita cinco números y cuenta cuántos son negativos. [Construir]
10. Solicita cuatro valores y calcula su suma mediante un acumulador. [Construir]
11. Calcula el promedio de cuatro valores usando total y contador. [Construir]
12. Solicita entradas hasta recibir la palabra `fin` y cuenta las entradas válidas. [Construir]
13. Valida repetidamente una temperatura hasta que sea numérica. [Aplicar]
14. Obtén el mayor de cinco números, incluidos posibles negativos. [Construir]
15. Diseña una búsqueda con máximo seis intentos y terminación temprana. [Construir]
16. Explica por qué inicializar un máximo en 0 puede fallar. [Analizar]
17. Crea una tabla de 3 filas y 6 columnas mediante ciclos anidados. Calcula las ejecuciones internas. [Construir]
18. Clasifica como $O(1)$, $O(n)$, $O(n^2)$ u $O(\log n)$ cuatro fragmentos propios. [Analizar]
19. Ejecuta un ciclo multiplicativo con $n = 64$ y registra cada valor. [Experimentar]
20. Genera el diagrama de un ciclo `mientras` y señala condición, cuerpo, regreso y salida. [Representar]
21. Corrige un acumulador que se reinicia dentro del ciclo. [Mejorar]
22. Diseña casos de prueba para un ciclo de validación: válido al primer intento, inválido y luego válido, y centinela inmediato. [Probar]

Mini proyecto: registro repetido de calificaciones

Situación: un programa debe recibir calificaciones hasta que se escriba -1, rechazar valores fuera del intervalo 0-100 y producir un resumen.

Requisito	Criterio de éxito
Centinela	-1 termina sin contarse como calificación.
Validación	Rechaza tipos no numéricos y valores fuera de 0-100.
Contador	Registra la cantidad aceptada.
Acumulador	Calcula la suma de calificaciones.
Máximo y mínimo	Conserva los extremos aceptados.
Promedio	Evita dividir entre cero si no hubo datos.
Pruebas	Incluye centinela inmediato, límites y datos inválidos.
Costo	Explica por qué el recorrido principal es $O(n)$.

Documenta con una traza al menos cuatro entradas consecutivas y explica qué cambia en cada iteración.

Autoevaluación

Responde sin consultar el capítulo. Después vuelve a las secciones que no puedas justificar mediante una traza.

Afirmación	Sí	Todavía no
Puedo identificar estado inicial, condición, bloque y progreso.	☐	☐
Puedo construir recorridos ascendentes y descendentes con `para`.	☐	☐
Puedo contar iteraciones con límites inclusivos.	☐	☐
Puedo decidir entre `para` y `mientras`.	☐	☐
Puedo demostrar por qué un ciclo termina.	☐	☐
Puedo detectar un ciclo infinito o un error de una posición.	☐	☐
Puedo usar contador, acumulador, bandera y centinela.	☐	☐
Puedo seguir ciclos anidados mediante una traza.	☐	☐
Puedo relacionar recorridos con $O(1)$, $O(n)$, $O(n^2)$ y $O(\log n)$.	☐	☐
Puedo interpretar el visualizador de costo sin depender de él.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas diseñar un ciclo, justificar su terminación, predecir su cantidad de iteraciones y explicar cómo cambia su estado en una prueba de escritorio.

Glosario

Término	Definición de trabajo
Ciclo	Estructura que repite un bloque bajo una regla de control.
Iteración	Una ejecución completa del bloque de un ciclo.
Estado inicial	Valores existentes antes de la primera comprobación.
Condición de continuación	Expresión que determina si el ciclo vuelve a ejecutarse.
Progreso	Cambio de estado que acerca el ciclo a su terminación.
Terminación	Momento en que la condición deja de permitir repeticiones.
`para`	Ciclo orientado a un recorrido entre un inicio y un límite.
`hasta`	Dirección ascendente del ciclo `para`.
`bajando`	Dirección descendente del ciclo `para`.
`mientras`	Ciclo que repite un bloque mientras una condición sea verdadera.
Contador	Variable que registra la cantidad de ocurrencias.
Acumulador	Variable que combina valores sucesivos.
Bandera	Variable lógica que conserva si ocurrió una condición.
Centinela	Valor especial que marca el final de una secuencia de entradas.
Ciclo anidado	Ciclo incluido dentro del bloque de otro ciclo.
Error de una posición	Desfase que procesa un elemento de más o de menos.
Ciclo infinito	Ciclo cuya condición nunca llega a ser falsa.
Big O	Notación que describe el crecimiento asintótico dominante del trabajo.

Referencias del capítulo

CLRS Kit Español. (2026). Código fuente y documentación de las estructuras `para`, `mientras` y del análisis de costo [Repositorio de software].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Gaddis, T. (2022). Starting out with programming logic and design (6th ed.). Pearson.

Scott, M. L. (2015). Programming language pragmatics (4th ed.). Morgan Kaufmann.

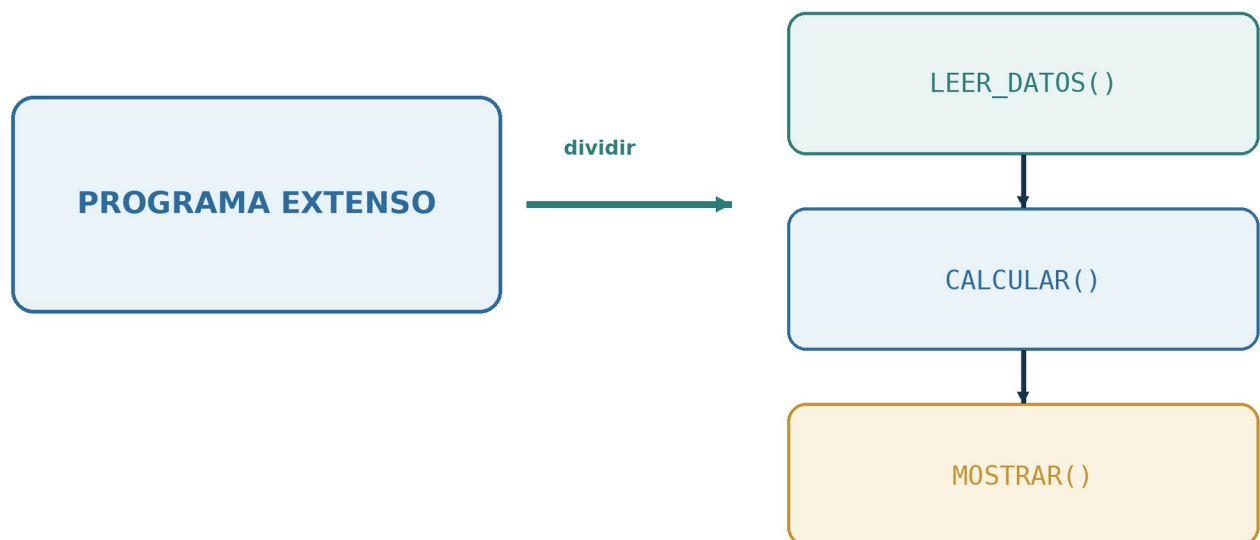
Continuidad: los ciclos permiten repetir responsabilidades, pero los programas extensos todavía concentran demasiadas instrucciones en un solo recorrido. El capítulo 7 mostrará cómo convertir subtarefas en funciones con parámetros y valores de retorno.

CAPÍTULO

7

Convertir subtareas en funciones

Responsabilidades, parámetros, argumentos y valores de retorno



Convertir subtareas en funciones

¿Cómo se evita que un programa creciente se convierta en una secuencia difícil de entender?

Un programa pequeño puede leerse de principio a fin. Cuando aparecen validaciones, cálculos, ciclos, decisiones y mensajes, esa lectura deja de ser sencilla. Una modificación en el cálculo del salario puede obligar a buscar entre instrucciones de entrada; una fórmula repetida puede corregirse en un lugar y permanecer equivocada en otro.

Las funciones permiten separar una solución en tareas con nombre. Cada función recibe la información que necesita, realiza una responsabilidad y, cuando corresponde, devuelve un resultado. El programa deja de ser una sola lista de instrucciones y comienza a expresar una colaboración entre partes.

Promesa de aprendizaje: al terminar podrás identificar subtareas, definir e invocar funciones, relacionar parámetros con argumentos, devolver valores mediante `retornar`, distinguir cálculo y presentación, diseñar interfaces sencillas y coordinar un programa desde `PRINCIPAL()`.

Una función reúne una responsabilidad bajo un nombre

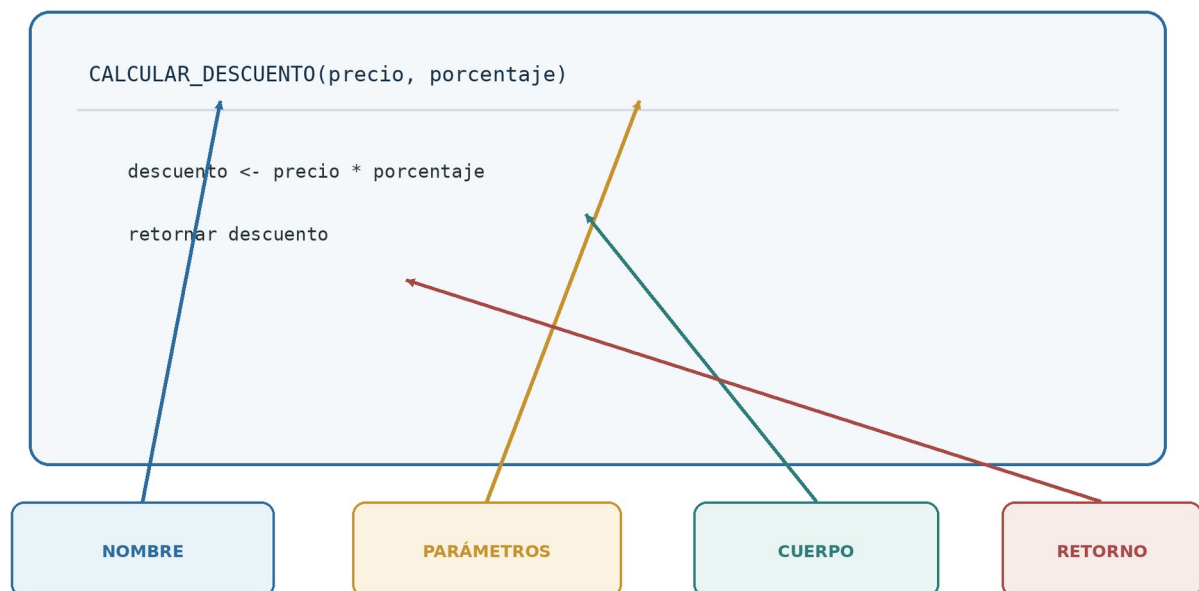


Figura 7.1. Anatomía conceptual de una función. Elaboración propia.

Alcance del capítulo: se introducirá el uso de funciones, pero el ámbito detallado de variables y la recursión se desarrollarán en el capítulo siguiente.

7.1 Cuando una sola secuencia deja de ser suficiente

Considera un programa que calcula el salario neto, valida el salario bruto, obtiene tres deducciones y muestra un resumen. Todo puede escribirse dentro de `PRINCIPAL()`, pero varias ideas distintas quedan mezcladas:

- solicitar y validar el salario bruto;
- calcular impuestos;
- calcular cotización social;
- calcular seguro médico;
- obtener el salario neto;
- presentar el desglose.

Estas ideas forman una secuencia válida, pero no una organización clara. Si las tasas cambian, quien mantiene el programa debe localizar cada fórmula. Si otro programa necesita el mismo cálculo, deberá copiarlo.

Idea clave: una función no se crea porque el código sea largo, sino porque existe una responsabilidad que merece un nombre, puede comprenderse de forma separada o puede reutilizarse.

Programa monolítico

```
PRINCIPAL()
  salarioBruto <- 10000
  impuestos <- salarioBruto * 0.12
  cotizacion <- salarioBruto * 0.05
  seguro <- salarioBruto * 0.045
  salarioNeto <- salarioBruto - impuestos - cotizacion - seguro

  escribir "Salario neto: " + salarioNeto
```

El programa es correcto para el ejemplo, pero el cálculo queda unido a la presentación. El primer paso de la descomposición consiste en preguntar: ¿qué resultado podría calcularse sin necesidad de escribirlo todavía?

7.2 Descomponer por responsabilidades

Descomponer no significa cortar el programa en fragmentos del mismo tamaño. Significa reconocer responsabilidades: tareas que tienen un propósito coherente y un resultado identificable.

Responsabilidad	Entrada necesaria	Resultado o efecto
Obtener salario neto	Salario bruto y tasas	Cantidad neta.
Calcular descuento	Precio y porcentaje	Monto descontado.
Validar una calificación	Valor recibido	Valor lógico.
Mostrar un resumen	Datos ya calculados	Texto visible en consola.
Solicitar un dato	Mensaje y reglas	Valor aceptado.

Una responsabilidad bien delimitada permite describir una función con una oración breve: “calcula el salario neto a partir del salario bruto” o “muestra el resumen de una operación”. Si la descripción necesita varias tareas unidas por muchos “y”, probablemente la función todavía mezcla propósitos.

Pausa y piensa: en el programa de salario, separa con una marca las instrucciones de cálculo y las de presentación. ¿Qué bloque podría reutilizarse en otro programa sin cambiar su lógica?

7.3 Definir una función

En CLRS una función se define escribiendo un identificador seguido de paréntesis y un bloque indentado. Los parámetros, cuando existen, aparecen dentro de los paréntesis.

Primera función

```
SALUDAR()  
  escribir "Hola"
```

La línea `SALUDAR()` introduce la definición. Las instrucciones indentadas forman su cuerpo. La definición describe una capacidad disponible, pero no solicita todavía su ejecución.

Definir una función no es lo mismo que invocarla



Figura 7.2. Diferencia entre definición e invocación. Elaboración propia.

Importante: la indentación distingue el cuerpo de la función de las instrucciones exteriores. Una llamada no debe recibir accidentalmente un bloque indentado, porque podría interpretarse como una nueva definición.

7.4 Invocar una función y seguir el recorrido

Invocar una función significa solicitar que se ejecute su cuerpo. Una llamada utiliza el nombre de la función y paréntesis.

Definición y llamada

```
SALUDAR()
    escribir "Hola"

PRINCIPAL()
    escribir "Antes"
    SALUDAR()
    escribir "Después"
```

El recorrido comienza en `PRINCIPAL()`. Después de escribir “Antes”, la llamada transfiere temporalmente el control a `SALUDAR()`. Cuando el cuerpo termina, el recorrido regresa a la instrucción posterior a la llamada y escribe “Después”.

Paso	Lugar	Acción
1	`PRINCIPAL`	Escribe “Antes”.
2	`PRINCIPAL`	Encuentra la llamada `SALUDAR()`.
3	`SALUDAR`	Ejecuta el cuerpo y escribe “Hola”.
4	`PRINCIPAL`	Regresa y escribe “Después”.

Modelo mental: una llamada no copia visualmente el cuerpo dentro de `PRINCIPAL`; cambia el punto de ejecución, realiza la tarea y regresa.

7.5 Funciones sin parámetros

Una función sin parámetros es adecuada cuando la tarea no necesita información variable suministrada por quien la llama. Puede mostrar un encabezado, presentar instrucciones o ejecutar una acción fija.

Encabezado reutilizable

```
MOSTRAR_ENCABEZADO()
    escribir "-----"
    escribir "      REGISTRO DE GASTOS"
    escribir "-----"

PRINCIPAL()
    MOSTRAR_ENCABEZADO()
    escribir "Bienvenido"
```

La ventaja no es ahorrar tres líneas una sola vez. La ventaja aparece cuando el mismo encabezado se necesita en varias partes o cuando se desea que `PRINCIPAL()` comunique la intención general.

Advertencia: no conviertas cada línea en una función. Una función debe aportar una unidad de significado, no solo desplazar código a otro lugar.

7.6 Parámetros: información que una función necesita

Una función se vuelve más general cuando recibe información. Los parámetros son nombres declarados en la definición y utilizados dentro del cuerpo.

Función con un parámetro

```
SALUDAR(nombre)
  escribir "Hola " + nombre

PRINCIPAL( )
  SALUDAR("Ana")
  SALUDAR("Luis")
```

El parámetro `nombre` no representa siempre el mismo valor. En cada llamada recibe el argumento correspondiente: primero `"Ana"` y después `"Luis"`. El cuerpo permanece igual; cambia la información con la que trabaja.

Concepto	Aparece en	Ejemplo
Parámetro	Definición	`nombre` en `SALUDAR(nombre)`.
Argumento	Llamada	`"Ana"` en `SALUDAR("Ana")`.
Correspondencia	Durante la llamada	`nombre` recibe `"Ana"`.

Idea clave: el parámetro pertenece a la descripción general de la tarea; el argumento pertenece a un uso concreto.

7.7 Varios parámetros y correspondencia por posición

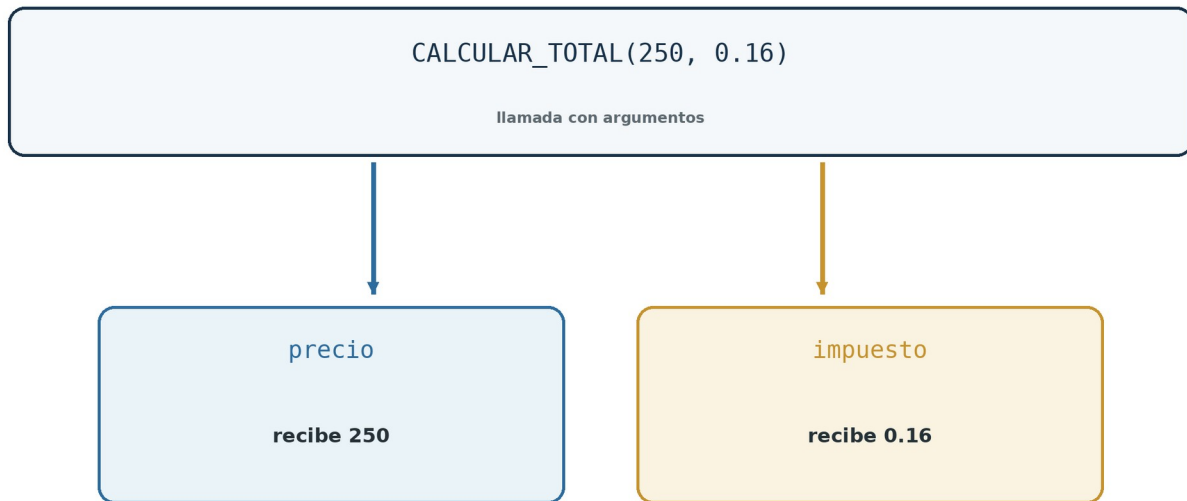
Las funciones pueden recibir varios parámetros separados por comas. En una llamada, los argumentos se relacionan con ellos según su posición.

Calcular un impuesto

```
CALCULAR_IMPUESTO(precio, tasa)
  impuesto <- precio * tasa
  retornar impuesto

PRINCIPAL( )
  impuesto <- CALCULAR_IMPUESTO(250, 0.16)
  escribir impuesto
```

Los argumentos alimentan a los parámetros por posición



Cambiar el orden de los argumentos puede cambiar el significado aunque ambos sean números.

Figura 7.3. Correspondencia posicional entre argumentos y parámetros. Elaboración propia.

La cantidad y el orden forman parte de la interfaz. `CALCULAR_IMPUESTO(250, 0.16)` expresa precio y tasa. Invertirlos produce un cálculo distinto aunque ambos argumentos sean numéricos.

Advertencia: nombres claros reducen errores dentro de la función, pero no corrigen una llamada con argumentos en orden equivocado. Revisa la interfaz antes de invocar.

7.8 Los argumentos pueden ser valores, variables o expresiones

Una llamada no está limitada a literales. Cada argumento puede ser una expresión evaluable.

Formas de proporcionar argumentos

```
precio <- 500
tasa <- 0.10

a <- CALCULAR_IMPUESTO(500, 0.10)
b <- CALCULAR_IMPUESTO(precio, tasa)
c <- CALCULAR_IMPUESTO(precio + 100, tasa / 2)
```

Antes de entrar en la función, el programa evalúa los argumentos. En la tercera llamada, ``precio + 100`` produce 600 y ``tasa / 2`` produce 0.05. Esos resultados se asocian después con los parámetros.

Pausa y piensa: sin ejecutar, calcula los valores de `a`, `b` y `c`. Después explica qué recibe cada parámetro en la tercera llamada.

7.9 Devolver un resultado con `retornar`

Una función de cálculo debe comunicar su resultado al punto que la llamó. La instrucción `retornar` produce ese valor y finaliza la ejecución de la función.

Función que devuelve un valor

```
SUMAR(a, b)
  resultado <- a + b
  retornar resultado

PRINCIPAL()
  total <- SUMAR(25, 30)
  escribir "Total: " + total
```

La llamada `SUMAR(25, 30)` se comporta como una expresión cuyo valor es 55. Por eso puede aparecer a la derecha de una asignación, dentro de otra expresión o directamente en `escribir`.

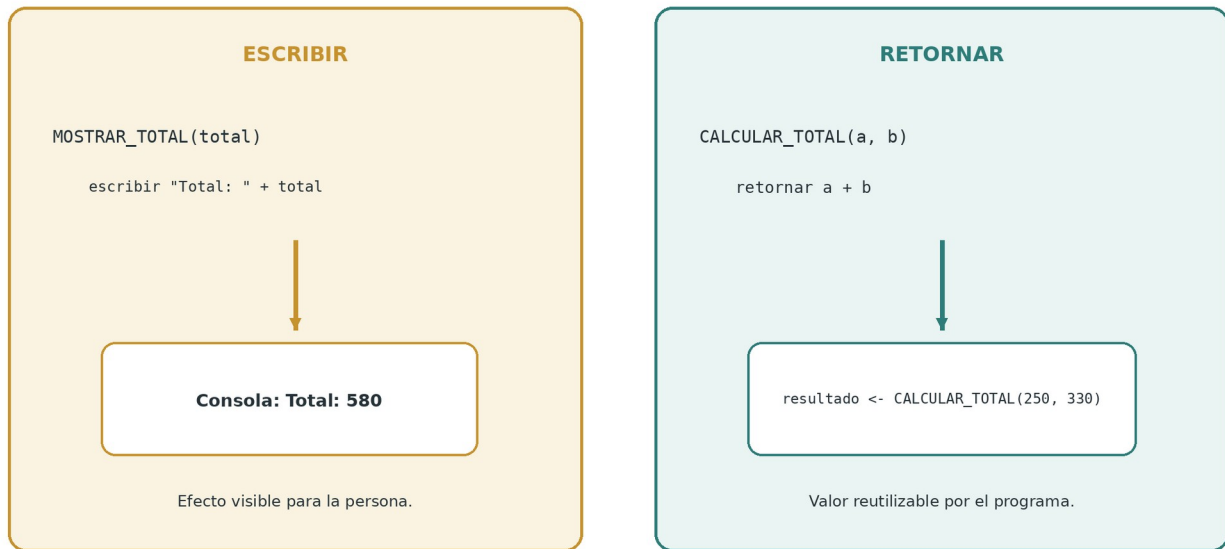
Reutilizar el valor retornado

```
subtotal <- SUMAR(25, 30)
doble <- subtotal * 2
escribir SUMAR(10, 5)
escribir "Con impuesto: " + (SUMAR(25, 30) * 1.16)
```

Importante: el valor retornado no aparece automáticamente en la consola. El programa decide si lo almacena, combina, compara o muestra.

7.10 `escribir` no reemplaza a `retornar`

Mostrar un valor y devolver un valor resuelven necesidades distintas



Una función puede mostrar, devolver o hacer ambas cosas, pero mezclar responsabilidades sin necesidad dificulta su reutilización.

Figura 7.4. Mostrar y devolver son operaciones diferentes. Elaboración propia.

Una función que solo escribe puede ser útil para presentar información. Sin embargo, el texto visible no queda disponible como valor calculable para otras instrucciones.

Versión difícil de reutilizar

```
SUMAR_Y_MOSTRAR(a, b)
  escribir a + b
```

```
PRINCIPAL()
  resultado <- SUMAR_Y_MOSTRAR(10, 20)
```

La función muestra 30, pero no retorna el valor 30. La asignación no obtiene el resultado esperado.

Versión reutilizable

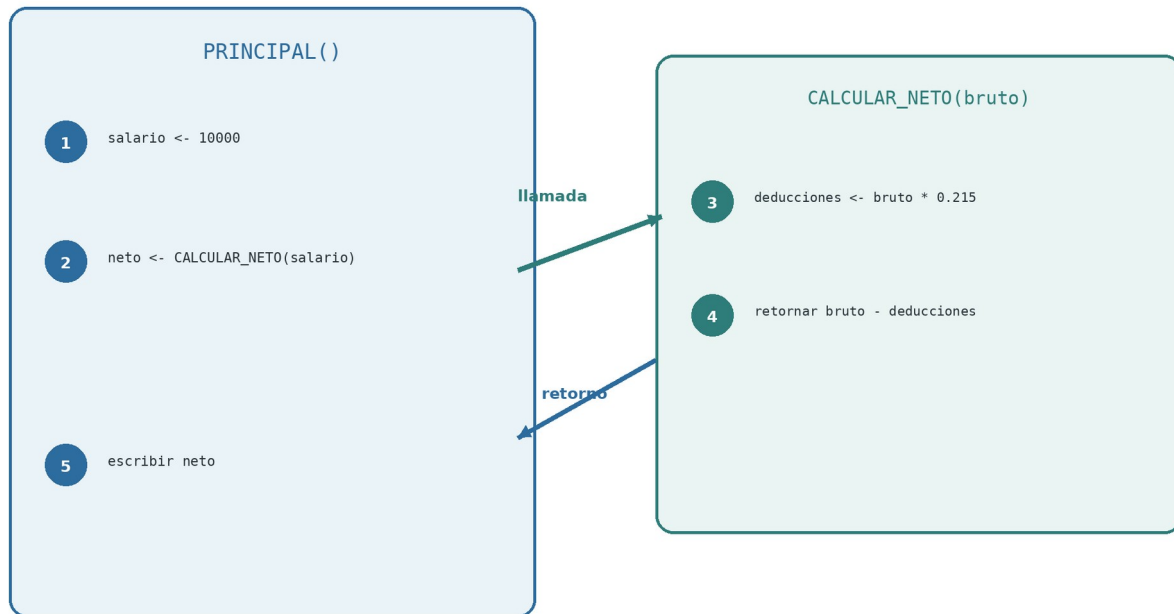
```
SUMAR(a, b)
  retornar a + b
```

```
PRINCIPAL()
  resultado <- SUMAR(10, 20)
  escribir resultado
```

Criterio práctico: si otra parte del programa necesita operar con el resultado, la función debe retornarlo. Si la responsabilidad es comunicarse con la persona, puede escribirlo.

7.11 Seguir una llamada que devuelve un valor

Una llamada interrumpe temporalmente el recorrido y después regresa



La función llamada termina; el programa continúa en la instrucción posterior a la llamada.

Figura 7.5. Flujo de una llamada con retorno. Elaboración propia.

El valor retornado ocupa temporalmente el lugar de la llamada. Si `CALCULAR_NETO(10000)` retorna 7850, la asignación se comporta como si se hubiera escrito `neto <- 7850`.

Momento	Estado relevante
Antes de la llamada	<code>`salario = 10000`</code> ; <code>`neto`</code> aún no tiene el resultado.
Dentro de la función	El parámetro <code>`bruto`</code> recibe 10000.
Al retornar	La función produce 7850.
Después de la llamada	<code>`neto`</code> queda asociado con 7850.

Pausa y piensa: ¿qué cambiaría en el recorrido si la función escribiera el salario neto pero no lo retornara?

7.12 Retornar una expresión o una variable intermedia

Las dos formas siguientes pueden ser correctas. La elección depende de cuánto ayude una variable intermedia a comprender el cálculo.

Retorno directo

```
CALCULAR_AREA_CIRCULO(radio)
  retornar PI() * radio^2
```

Retorno mediante un nombre

```
CALCULAR_AREA_CIRCULO(radio)
  area <- PI() * radio^2
  retornar area
```

La primera versión es compacta. La segunda hace visible el significado del resultado y ofrece un punto natural para revisar o ampliar el cálculo. Una variable intermedia es útil cuando nombra una idea; no es obligatoria para toda expresión.

Idea clave: la claridad no depende de escribir más o menos líneas, sino de que cada nombre y cada paso aporten información.

7.13 Separar cálculo, entrada y presentación

Una función de cálculo suele ser más fácil de probar y reutilizar cuando recibe sus datos como parámetros y retorna un resultado, sin solicitar entradas ni producir mensajes.

Responsabilidades separadas

```
CONVERTIR_A_FAHRENHEIT(celsius)
  retornar ((9 / 5) * celsius) + 32

MOSTRAR_TEMPERATURA(celsius, fahrenheit)
  escribir "Celsius: " + celsius
  escribir "Fahrenheit: " + fahrenheit

PRINCIPAL()
  celsius <- 20
  fahrenheit <- CONVERTIR_A_FAHRENHEIT(celsius)
  MOSTRAR_TEMPERATURA(celsius, fahrenheit)
```

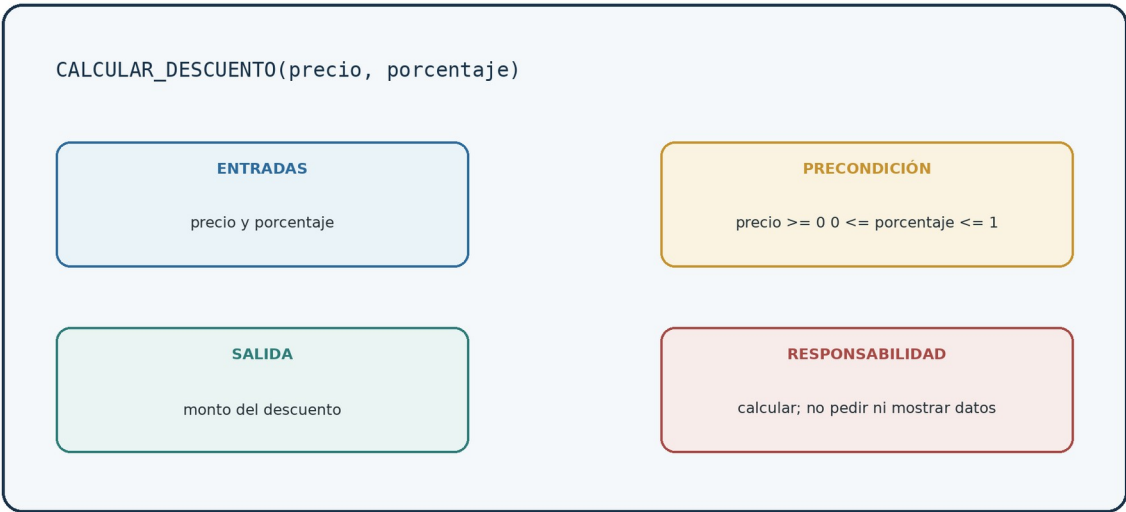
Ahora `CONVERTIR\_A\_FAHRENHEIT` puede probarse con distintos valores sin depender de la consola. `MOSTRAR\_TEMPERATURA` se concentra en la presentación. `PRINCIPAL()` coordina ambas.

Advertencia: separar responsabilidades no significa prohibir funciones de entrada o salida. Significa evitar que una función mezcle tareas sin una razón clara.

7.14 Diseñar la interfaz de una función

La interfaz es la parte visible para quien utiliza la función: su nombre, parámetros y valor de retorno esperado. Debe comunicar qué hace y qué necesita.

La interfaz de una función actúa como un contrato de uso



Una buena interfaz permite usar la función sin conocer cada instrucción de su cuerpo.

Figura 7.6. Contrato conceptual de una función. Elaboración propia.

Pregunta de diseño	Ejemplo
¿Qué responsabilidad tiene?	Calcular el monto de un descuento.
¿Qué información necesita?	Precio y porcentaje.
¿Qué condiciones deben cumplir los datos?	Precio no negativo; porcentaje entre 0 y 1.
¿Qué devuelve?	Cantidad descontada.
¿Qué no debería hacer?	Solicitar datos o decidir cómo mostrarlos.

Las precondiciones describen supuestos que deben cumplirse para usar correctamente la función. En capítulos posteriores se podrá decidir dónde validarlas; por ahora, deben al menos reconocerse y documentarse.

Idea clave: una función bien diseñada puede comprenderse primero desde su interfaz y después, solo si es necesario, desde su implementación.

7.15 Nombres que expresan acciones

El nombre de una función debe permitir anticipar su responsabilidad. En CLRS se adopta como convención frecuente el uso de mayúsculas y guion bajo para separar palabras.

Nombre	Evaluación	Mejora posible
`HACER()`	No revela la tarea.	`CALCULAR_SALARIO_NETO()`

Nombre	Evaluación	Mejora posible
<code>`DATOS()`</code>	No expresa una acción.	<code>`MOSTRAR_DATOS()`</code> o <code>`LEER_DATOS()`</code>
<code>`CALCULAR()`</code>	Demasiado general.	<code>`CALCULAR_PROMEDIO()`</code>
<code>`OBTENERSALNETO()`</code>	Abreviado y menos legible.	<code>`OBTENER_SALARIO_NETO()`</code>
<code>`VALIDAR_MONTO()`</code>	Acción y objeto claros.	Adecuado si realmente valida.

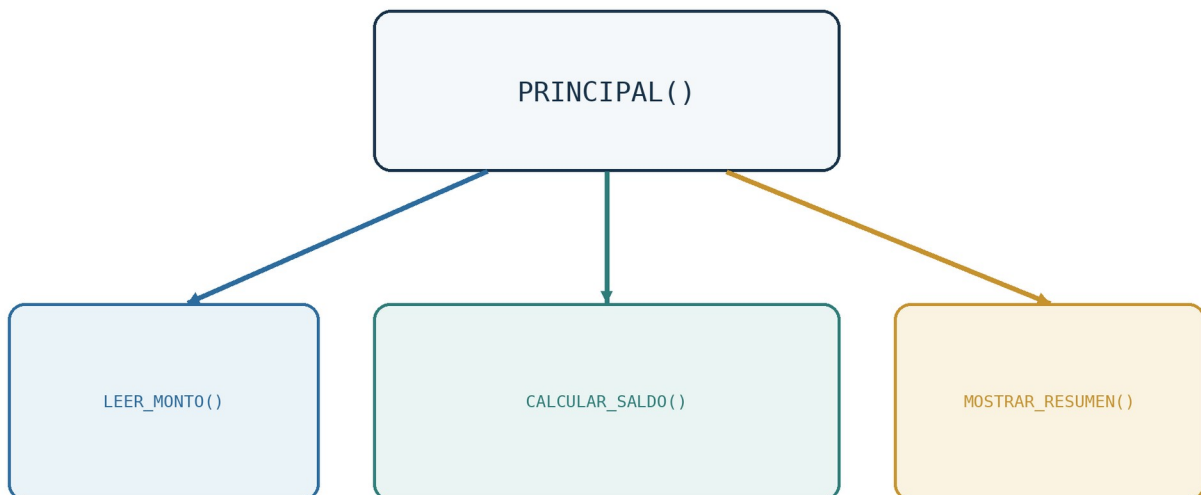
No todos los nombres deben ser largos. Deben contener la información necesaria para distinguir la función dentro del contexto del programa.

Advertencia: un nombre preciso no corrige una función que realiza varias responsabilidades. La implementación debe cumplir la promesa del nombre.

7.16 ``PRINCIPAL()`` como coordinador

Cuando un programa se divide en funciones, ``PRINCIPAL()`` puede mostrar el recorrido general: obtener datos, realizar cálculos y presentar resultados. Los detalles quedan delegados a funciones especializadas.

PRINCIPAL coordina; las funciones especializadas resuelven subtarear



La coordinación expresa el orden general sin ocultarlo entre detalles de cálculo, lectura o presentación.

Cada función debe tener una razón clara para cambiar y un nombre que permita anticipar su trabajo.

Figura 7.7. Organización de un programa alrededor de `PRINCIPAL`. Elaboración propia.

Coordinación de alto nivel

```

PRINCIPAL()
  salarioBruto <- LEER_SALARIO()
  salarioNeto <- CALCULAR_SALARIO_NETO(salarioBruto)
  MOSTRAR_RESUMEN(salarioBruto, salarioNeto)

```

Aunque todavía no se muestre el cuerpo de las funciones, el recorrido puede comprenderse. Esta lectura de alto nivel es una señal de buena organización.

Importante: `PRINCIPAL()` no tiene que contener toda la lógica. Su responsabilidad principal es establecer el orden general y coordinar resultados.

7.17 Refactorización guiada: salario neto

Refactorizar significa mejorar la estructura del programa sin cambiar el resultado que debe producir. Partiremos del cálculo monolítico y extraeremos responsabilidades.

Versión organizada

```

CALCULAR_IMPUESTOS(salarioBruto)
  retornar salarioBruto * 0.12

CALCULAR_COTIZACION(salarioBruto)
  retornar salarioBruto * 0.05

CALCULAR_SEGURO(salarioBruto)
  retornar salarioBruto * 0.045

CALCULAR_SALARIO_NETO(salarioBruto)
  impuestos <- CALCULAR_IMPUESTOS(salarioBruto)
  cotizacion <- CALCULAR_COTIZACION(salarioBruto)
  seguro <- CALCULAR_SEGURO(salarioBruto)

  retornar salarioBruto - impuestos - cotizacion - seguro

MOSTRAR_SALARIO(salarioBruto, salarioNeto)
  escribir "Salario bruto: " + salarioBruto
  escribir "Salario neto: " + salarioNeto

PRINCIPAL()
  salarioBruto <- 10000
  salarioNeto <- CALCULAR_SALARIO_NETO(salarioBruto)
  MOSTRAR_SALARIO(salarioBruto, salarioNeto)

```

Esta división hace explícitas las reglas. Sin embargo, no existe una única descomposición correcta. También podría utilizarse una sola función que calcule todas las deducciones si el lenguaje de datos disponible permitiera devolverlas como conjunto. Por ahora, la solución usa valores individuales.

Decisión de diseño: crear demasiadas funciones diminutas puede fragmentar el cálculo. Conserva una función separada cuando su regla tenga significado propio, pueda cambiar de forma independiente o se reutilice.

7.18 Una función puede invocar otras funciones

La descomposición forma niveles. Una función general puede coordinar funciones más específicas, como ``CALCULAR_SALARIO_NETO`` en el ejemplo anterior.

Nivel	Responsabilidad
<code>`PRINCIPAL`</code>	Coordina el programa completo.
<code>`CALCULAR_SALARIO_NETO`</code>	Coordina los cálculos del salario.
<code>`CALCULAR_IMPUESTOS`</code>	Aplica una regla específica.
<code>`CALCULAR_COTIZACION`</code>	Aplica otra regla específica.
<code>`CALCULAR_SEGURO`</code>	Aplica otra regla específica.

Esta jerarquía permite leer la solución con distintos niveles de detalle. Primero se entiende el propósito general; después se inspeccionan las funciones que requieren atención.

Modelo mental: descomponer no elimina la complejidad del problema. La distribuye en unidades que pueden comprenderse y comprobarse por separado.

7.19 Errores frecuentes al trabajar con funciones

Error	Consecuencia	Revisión
Definir pero no invocar	La tarea nunca se ejecuta.	Localizar las llamadas.
Invocar un nombre inexistente	Referencia inválida.	Comprobar escritura y definición.
Cantidad incorrecta de argumentos	No hay correspondencia completa.	Comparar llamada e interfaz.
Orden incorrecto	Los parámetros reciben significados equivocados.	Revisar posición.
Usar <code>`escribir`</code> en lugar de <code>`retornar`</code>	El cálculo no queda disponible.	Preguntar quién necesita el valor.
Olvidar usar el retorno	El resultado se calcula pero se descarta.	Asignar, combinar o mostrar.
Función con demasiadas tareas	Difícil de probar y modificar.	Separar responsabilidades.
Nombre impreciso	La intención permanece oculta.	Usar acción y objeto.
Duplicar cálculo dentro y fuera	Surgen versiones inconsistentes.	Concentrar la regla.

Resultado descartado

```
CALCULAR_AREA_CIRCULO(5)
escribir "Cálculo terminado"
```

La función puede retornar el área, pero la llamada no conserva ni muestra ese valor.

Método de revisión: para cada llamada, señala qué argumentos entrega, qué función recibe el control, qué valor retorna y qué hace la instrucción llamadora con ese valor.

7.20 Representar funciones en el diagrama de flujo

CLRS Kit representa las funciones como subgrafos independientes. Esta separación visual ayuda a reconocer el flujo principal y las responsabilidades delegadas.

- Localiza el subgrafo de ``PRINCIPAL()`` y sigue el orden de sus llamadas.
- Identifica qué funciones producen un valor mediante ``retornar``.
- Distingue una llamada de la ejecución interna de la función.
- Comprueba que cada subgrafo tenga una responsabilidad reconocible.
- Si un subgrafo es demasiado extenso, revisa si mezcla varias tareas.

Laboratorio breve: crea ``SUMAR(a, b)`` y ``MOSTRAR_RESULTADO(valor)``, invócalas desde ``PRINCIPAL()`` y genera el diagrama. Explica qué parte representa la coordinación y cuáles representan subtareas.

7.21 Laboratorio CLRS: organizar el registro de gastos

El registro de gastos del capítulo anterior concentraba lectura, validación, cálculo y presentación en un solo bloque. Ahora se separarán algunas responsabilidades sin introducir todavía arreglos.

Registro de gastos organizado mediante funciones

```

ES_MONTO_VALIDO(monto)
    retornar ES_NUM(monto) y monto > 0

CALCULAR_SALDO(saldoActual, gasto)
    retornar saldoActual - gasto

MOSTRAR_MOVIMIENTO(gasto, saldo)
    escribir "Gasto registrado: " + gasto
    escribir "Saldo actual: " + saldo

PRINCIPAL()
    saldo <- 1000
    gasto <- 0

    escribir "Introduce un gasto o -1 para terminar"
    leer gasto

    mientras gasto != -1
        si ES_MONTO_VALIDO(gasto)
            saldo <- CALCULAR_SALDO(saldo, gasto)
            MOSTRAR_MOVIMIENTO(gasto, saldo)
        sino
            escribir "Monto inválido"

    escribir "Introduce otro gasto o -1"
    leer gasto

    escribir "Saldo final: " + saldo

```

`ES\_MONTO\_VALIDO` responde una pregunta lógica. `CALCULAR\_SALDO` produce un valor. `MOSTRAR\_MOVIMIENTO` genera salida. `PRINCIPAL()` conserva la secuencia general del registro.

Función	Entrada	Salida o efecto	Responsabilidad
`ES_MONTO_VALIDO`	Monto	Lógico	Comprobar una regla.
`CALCULAR_SALDO`	Saldo y gasto	Nuevo saldo	Realizar un cálculo.
`MOSTRAR_MOVIMIENTO`	Gasto y saldo	Texto en consola	Presentar información.

Observación crítica: la validación no comprueba todavía que el gasto sea menor o igual al saldo. Añade esa regla modificando la interfaz de validación o incorporando una función específica. Justifica tu decisión.

7.22 Probar funciones de forma aislada

Una función de cálculo puede comprobarse con llamadas pequeñas antes de integrarla al programa completo.

Pruebas manuales de una función

```

PRINCIPAL()
    escribir CALCULAR_SALDO(1000, 100)
    escribir CALCULAR_SALDO(500, 0)
    escribir CALCULAR_SALDO(200, 250)

```


Llamada	Resultado esperado	Pregunta
<code>`CALCULAR_SALDO(1000, 100)`</code>	900	Caso normal.
<code>`CALCULAR_SALDO(500, 0)`</code>	500	¿El cero debe ser permitido por esta función?
<code>`CALCULAR_SALDO(200, 250)`</code>	-50	¿El cálculo o la validación debe impedirlo?

La función puede calcular correctamente incluso cuando los datos violan una regla del negocio. Por eso conviene distinguir la corrección matemática de la validación de uso.

Idea clave: una interfaz clara permite diseñar casos de prueba directamente a partir de sus entradas, precondiciones y salida esperada.

7.23 Criterios para decidir si una función está bien delimitada

Criterio	Pregunta de revisión
Una responsabilidad	¿Puede describirse con una acción principal?
Nombre informativo	¿Anticipa el resultado o efecto?
Entradas necesarias	¿Recibe solo la información que utiliza?
Salida definida	¿Queda claro qué retorna o qué efecto produce?
Pocas razones de cambio	¿Una modificación ajena obligaría a editarla?
Tamaño comprensible	¿Puede seguirse sin perder su propósito?
Reutilización razonable	¿Puede emplearse en más de un contexto o probarse aislada?

Estos criterios no son reglas mecánicas. Una función puede ser específica y utilizarse una sola vez, pero seguir siendo valiosa si revela una idea importante y reduce la carga de comprender ``PRINCIPAL()``.

Advertencia: la reutilización no debe convertirse en una obsesión. Primero busca una estructura comprensible; después identifica qué partes realmente comparten una responsabilidad.

7.24 Reconstruye el modelo mental

Una función transforma una subtaska en una unidad con nombre. La definición establece la interfaz y el cuerpo; la llamada suministra argumentos; los parámetros reciben esos valores; y ``retornar`` entrega un resultado al punto de llamada. ``PRINCIPAL()`` coordina las funciones para expresar el recorrido completo.

1. Descomponer significa separar responsabilidades, no dividir por cantidad de líneas.
2. Definir una función crea una capacidad; invocarla solicita su ejecución.

3. Los parámetros pertenecen a la definición y los argumentos a la llamada.
4. La correspondencia entre argumentos y parámetros es posicional.
5. Una llamada puede utilizar literales, variables o expresiones como argumentos.
6. `retornar` produce un valor reutilizable y finaliza la función.
7. `escribir` produce salida visible, pero no sustituye un retorno.
8. Las funciones de cálculo son más reutilizables cuando no mezclan entrada y presentación.
9. La interfaz incluye nombre, parámetros, precondiciones y resultado esperado.
10. `PRINCIPAL()` debe comunicar el orden general del programa.
11. Una función puede coordinar otras funciones.
12. La descomposición permite probar y modificar partes con menor interferencia.

Comprobación activa: explica con tus palabras qué ocurre desde que `PRINCIPAL()` evalúa `total <- SUMAR(10, 5)` hasta que `total` recibe 15.

Ejercicios

Realiza primero los ejercicios de predicción y análisis sin ejecutar el código.

1. Distingue la definición y la invocación en un programa que contenga `SALUDAR(nombre)` y `SALUDAR("Marta")`. **[Reconocer]**
2. Identifica los parámetros y argumentos de `CALCULAR\_TOTAL(250, 0.16)`. **[Reconocer]**
3. Escribe una función `MOSTRAR\_SEPARADOR()` sin parámetros. **[Aplicar]**
4. Escribe `MOSTRAR\_MENSAJE(mensaje)` y realiza tres llamadas diferentes. **[Aplicar]**
5. Crea una función `MULTIPLICAR(a, b)` que retorne el producto. **[Construir]**
6. Utiliza el retorno de `MULTIPLICAR` dentro de una expresión que sume 10. **[Aplicar]**
7. Explica por qué una función que solo ejecuta `escribir a + b` no sirve para asignar el resultado a una variable. **[Analizar]**
8. Convierte una función que escribe el área de un círculo en una función que la retorna. **[Mejorar]**
9. Diseña la interfaz de `CALCULAR\_PROMEDIO\_TRES`: nombre, parámetros, precondiciones y retorno. **[Diseñar]**
10. Refactoriza un conversor de Celsius para separar cálculo y presentación. **[Construir]**
11. Crea `ES\_TEMPERATURA\_VALIDA(valor)` y define qué regla comprobará. **[Construir]**
12. Predice el resultado de `SUMAR(2 + 3, 4 \* 2)`. Explica qué argumentos llegan a la función. **[Analizar]**
13. Detecta el error en una llamada que invierte `DIVIDIR(dividendo, divisor)`. **[Analizar]**
14. Propón nombres más claros para `HACER`, `DATOS`, `OP` y `CALC`. **[Mejorar]**
15. Divide un programa de salario bruto en funciones de cálculo y presentación. **[Construir]**
16. Decide si tres cálculos de deducción deben ser funciones separadas o una sola función. Justifica. **[Evaluar]**
17. Construye un mapa de llamadas para un programa con `PRINCIPAL`, `CALCULAR\_TOTAL` y `MOSTRAR\_TOTAL`. **[Representar]**
18. Escribe tres casos de prueba para `CALCULAR\_DESCUENTO(precio, porcentaje)`. **[Probar]**
19. Identifica una precondición que `CALCULAR\_DESCUENTO` no debería ignorar. **[Analizar]**
20. Corrige una función que recibe parámetros que nunca utiliza. **[Mejorar]**
21. Refactoriza un programa repetitivo para que una fórmula aparezca en un solo lugar. **[Construir]**

22. Genera el diagrama de flujo de un programa con tres funciones y explica sus subgrafos. [\[Representar\]](#)

Mini proyecto: calculadora de compra organizada

Situación: un programa recibe precio, cantidad, porcentaje de descuento e impuesto; calcula subtotal, descuento, impuesto y total; y presenta un comprobante.

Función sugerida	Responsabilidad
<code>`CALCULAR_SUBTOTAL(precio, cantidad)`</code>	Obtener precio por cantidad.
<code>`CALCULAR_DESCUENTO(subtotal, porcentaje)`</code>	Obtener el monto descontado.
<code>`CALCULAR_IMPUESTO(base, tasa)`</code>	Obtener el impuesto.
<code>`CALCULAR_TOTAL(subtotal, descuento, impuesto)`</code>	Combinar los resultados.
<code>`MOSTRAR_COMPROBANTE(...)`</code>	Presentar los valores calculados.

Requisitos:

- Cada función debe tener una responsabilidad clara.
- Las funciones de cálculo deben retornar valores.
- ``PRINCIPAL()`` debe coordinar el orden general.
- Incluye casos de prueba con descuento cero, cantidad uno y valores límite.
- Genera el diagrama de flujo y señala las llamadas.
- Explica qué función cambiaría si se modifica la tasa de impuesto.

Criterio de éxito: el recorrido general debe comprenderse leyendo ``PRINCIPAL()`` sin necesidad de inspeccionar inmediatamente todos los cuerpos.

Autoevaluación

Responde sin consultar el capítulo. Después revisa mediante un ejemplo ejecutable.

Afirmación	Sí	Todavía no
Puedo identificar responsabilidades dentro de un programa extenso.	☐	☐
Distingo definición e invocación.	☐	☐
Puedo escribir funciones con y sin parámetros.	☐	☐
Distingo parámetros y argumentos.	☐	☐
Comprendo la correspondencia posicional.	☐	☐
Puedo usar una llamada como expresión.	☐	☐
Distingo <code>`retornar`</code> de <code>`escribir`</code> .	☐	☐

Afirmación	Sí	Todavía no
Puedo diseñar una interfaz con entradas y salida.	☐	☐
Puedo separar cálculo, entrada y presentación.	☐	☐
Puedo organizar el recorrido desde <code>`PRINCIPAL()`</code> .	☐	☐
Puedo probar una función de forma aislada.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas tomar un programa monolítico, extraer al menos dos responsabilidades, explicar el recorrido de sus llamadas y utilizar correctamente sus valores de retorno.

Glosario

Término	Definición de trabajo
Función	Subprograma con nombre que reúne una responsabilidad.
Definición	Descripción de la interfaz y el cuerpo de una función.
Invocación	Solicitud de ejecutar una función.
Llamada	Expresión o instrucción utilizada para invocar una función.
Parámetro	Nombre que recibe información dentro de la definición.
Argumento	Valor o expresión proporcionado en una llamada.
Correspondencia posicional	Asociación de cada argumento con el parámetro de la misma posición.
Cuerpo	Bloque indentado de instrucciones de una función.
Retorno	Valor entregado por una función al punto de llamada.
`retornar`	Instrucción que produce el valor de una función y finaliza su ejecución.
Interfaz	Nombre, parámetros y resultado observable de una función.
Precondición	Supuesto que debe cumplir una entrada para utilizar correctamente la función.
Responsabilidad	Tarea coherente que justifica la existencia de una función.
Descomposición funcional	Organización de una solución mediante funciones relacionadas.
Refactorización	Mejora estructural que conserva el comportamiento esperado.
Punto de entrada	Función desde la que comienza la ejecución coordinada del programa.

Referencias del capítulo

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2nd ed.). The MIT Press.

CLRS Kit Español. (2026). Código fuente y documentación de funciones, parámetros, llamadas y retornos [Repositorio de software].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

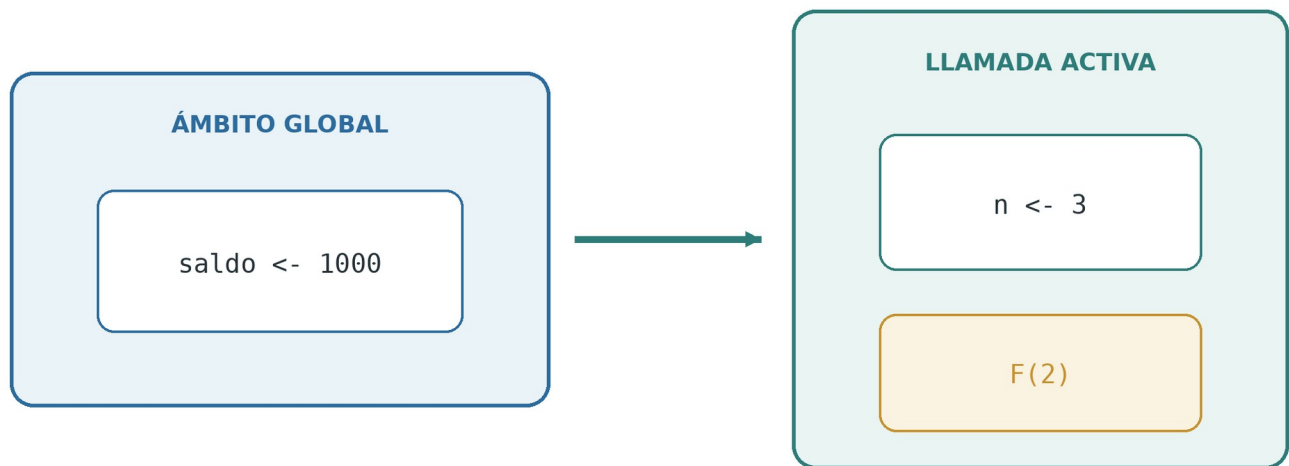
Continuidad: las funciones ya permiten dividir y coordinar responsabilidades. El capítulo 8 profundizará en qué variables puede ver cada función, cómo se conserva el estado durante una llamada y cómo una función puede invocarse a sí misma.

CAPÍTULO

8

Ámbito, llamadas y recursión inicial

Qué información puede ver cada función y cómo se conservan sus llamadas



Ámbito, llamadas y recursión inicial

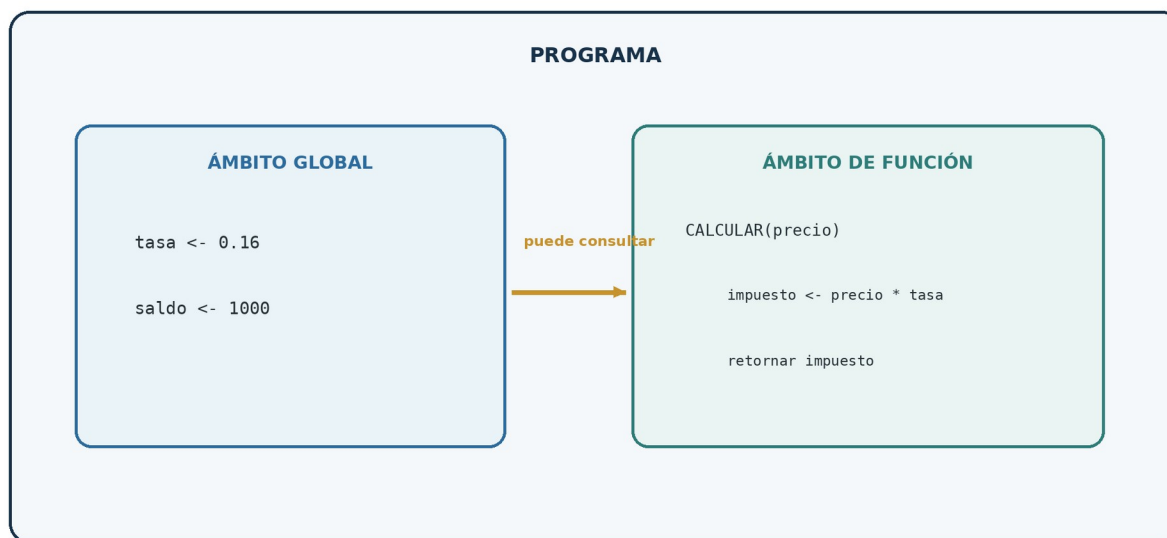
¿Qué información puede ver cada función y qué ocurre cuando una función vuelve a llamarse?

En el capítulo anterior, las funciones permitieron dividir un programa en responsabilidades. Sin embargo, esa división abre nuevas preguntas. Dos funciones pueden utilizar el nombre `saldo` sin referirse al mismo valor. Una función puede consultar un dato exterior, pero crear otra variable con el mismo nombre. Además, una llamada puede quedar suspendida mientras otra llamada se ejecuta.

Comprender estas situaciones exige dos modelos complementarios: el ámbito, que determina dónde puede encontrarse un nombre, y el marco de llamada, que conserva los parámetros y variables locales de una ejecución concreta. Sobre estos modelos se construye la recursión: una función puede iniciar una nueva llamada de sí misma sin perder la información de las llamadas anteriores.

Promesa de aprendizaje: al terminar podrás distinguir variables globales, parámetros y variables locales; explicar el ocultamiento de nombres; seguir llamadas anidadas; reconocer qué conserva cada llamada; y construir funciones recursivas sencillas con caso base y progreso demostrable.

El ámbito determina dónde puede utilizarse un nombre



El ámbito no indica cuánto dura un valor; indica desde qué parte del programa puede encontrarse su nombre.

Figura 8.1. El ámbito como región de visibilidad de nombres. Elaboración propia.

Alcance del capítulo: la recursión se estudiará como mecanismo de programación. La resolución formal de recurrencias y su análisis matemático pertenecen a una materia posterior de algoritmos.

8.1 Un mismo nombre no siempre representa la misma variable

Cuando una función utiliza un identificador, el programa debe decidir a qué variable se refiere. No basta con conocer el texto del nombre: también importa el lugar donde aparece.

Dos variables llamadas total

```
total <- 100

MOSTRAR_TOTAL()
  total <- 25
  escribir total

PRINCIPAL()
  MOSTRAR_TOTAL()
  escribir total
```

La función muestra 25 porque su asignación crea o reutiliza un nombre local de esa función. Al regresar a `PRINCIPAL`, el nombre global conserva 100. Las dos variables comparten su escritura, pero no su identidad.

Idea clave: el ámbito responde a la pregunta “¿desde qué parte del programa puede localizarse este nombre?”.

8.2 Ámbito global

Una variable creada fuera del cuerpo de una función pertenece al ámbito global. Su nombre puede ser consultado desde el nivel superior y, cuando no existe un nombre local que lo oculte, desde las funciones.

Consulta de una variable global

```
tasa <- 0.16

CALCULAR_IMPUESTO(precio)
  retornar precio * tasa

PRINCIPAL()
  impuesto <- CALCULAR_IMPUESTO(500)
  escribir impuesto
```

`CALCULAR\_IMPUESTO` recibe `precio` como parámetro y consulta `tasa` desde el ámbito exterior. El ejemplo funciona, pero la dependencia no aparece en la interfaz de la función.

Advertencia: una función que depende de muchas variables globales puede ser difícil de reutilizar y probar. Cuando un dato forma parte real de la tarea, suele ser más claro recibirlo mediante un parámetro.

8.3 Ámbito de función

Cada función crea su propio ámbito para parámetros y variables asignadas en su cuerpo. Esos nombres se utilizan durante la llamada y no quedan disponibles como variables locales de otras funciones.

Variable local

```
CALCULAR_AREA(radio)
  area <- PI() * radio^2
  retornar area

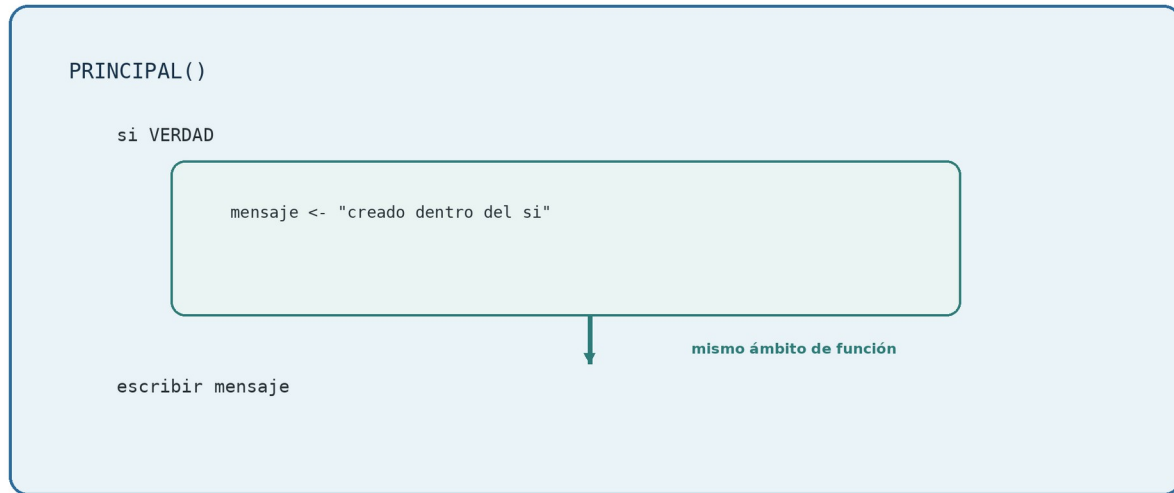
PRINCIPAL()
  resultado <- CALCULAR_AREA(5)
  escribir resultado
  // escribir area <- area no pertenece a PRINCIPAL
```

El parámetro `radio` y la variable `area` pertenecen a la función `CALCULAR\_AREA`. `PRINCIPAL` recibe el valor retornado, pero no obtiene acceso directo a esos nombres.

Nombre	Lugar de creación	Visibilidad principal
`tasa`	Nivel superior	Ámbito global.
`radio`	Parámetro	Cuerpo de `CALCULAR_AREA`.
`area`	Asignación dentro de la función	Cuerpo de `CALCULAR_AREA`.
`resultado`	Asignación en `PRINCIPAL`	Ámbito de `PRINCIPAL`.

8.4 Los bloques de control no crean otro ámbito de variables

En CLRS, los bloques de control no crean un ámbito de variables adicional



La indentación crea un bloque de ejecución, pero no una nueva zona de variables.

Figura 8.2. Bloque de ejecución y ámbito de variables. Elaboración propia.

En CLRS, la indentación delimita qué instrucciones pertenecen a `si`, `mientras` o `para`, pero esos bloques no crean un ámbito adicional de variables. Una asignación dentro del bloque pertenece al ámbito global o de función que lo contiene.

Nombre creado dentro de un bloque

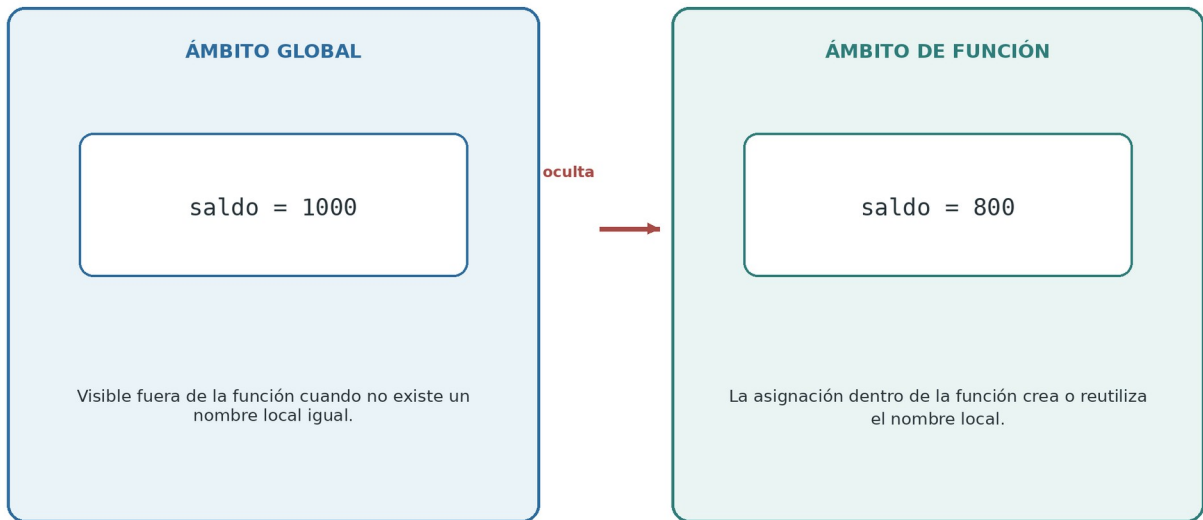
```

PRINCIPAL()
  si VERDAD
    mensaje <- "Operación permitida"
  escribir mensaje
  
```

Importante: esta regla es propia de CLRS Kit y no debe generalizarse a todos los lenguajes. En C#, JavaScript con `let` o Java, determinados bloques sí pueden introducir ámbitos distintos.

8.5 Ocultamiento de nombres

Un nombre local puede ocultar temporalmente a otro nombre exterior



Al terminar la llamada, el saldo global sigue siendo 1000; el saldo local pertenecía a esa función.

Figura 8.3. Ocultamiento de una variable global por una local. Elaboración propia.

Cuando una función posee un parámetro o una variable local con el mismo nombre que una variable exterior, el nombre local tiene prioridad dentro de esa función. Este fenómeno se conoce como ocultamiento.

El nombre local tiene prioridad

```
saldo <- 1000

REGISTRAR_GASTO(gasto)
  saldo <- 800
  saldo <- saldo - gasto
  escribir "Saldo local: " + saldo

PRINCIPAL()
  REGISTRAR_GASTO(100)
  escribir "Saldo global: " + saldo
```

La primera salida muestra 700; la segunda muestra 1000. La asignación dentro de la función no actualiza el nombre global: crea o reutiliza el nombre local `saldo`.

Criterio de diseño: evita reutilizar nombres globales como variables locales cuando pueda producir una interpretación equivocada. El ocultamiento puede ser válido, pero debe ser intencional.

8.6 Pasar el estado mediante parámetros y retornos

Si una función debe transformar un valor exterior, una forma clara consiste en recibir el valor actual y retornar el nuevo. La función no intenta modificar directamente el estado global.

Transformación explícita del saldo

```

RESTAR_GASTO(saldoActual, gasto)
  retornar saldoActual - gasto

PRINCIPAL()
  saldo <- 1000
  saldo <- RESTAR_GASTO(saldo, 100)
  escribir saldo

```

La asignación en `PRINCIPAL` hace visible el cambio de estado: entrega 1000, recibe 900 y actualiza su propia variable. La función permanece independiente de un nombre global específico.

Idea clave: los parámetros comunican información hacia la función; el retorno comunica un resultado hacia el punto de llamada.

8.7 Cada llamada crea una ejecución distinta

Una función puede invocarse varias veces con argumentos diferentes. Cada llamada necesita conservar sus propios parámetros y variables locales hasta terminar.

Dos llamadas independientes

```

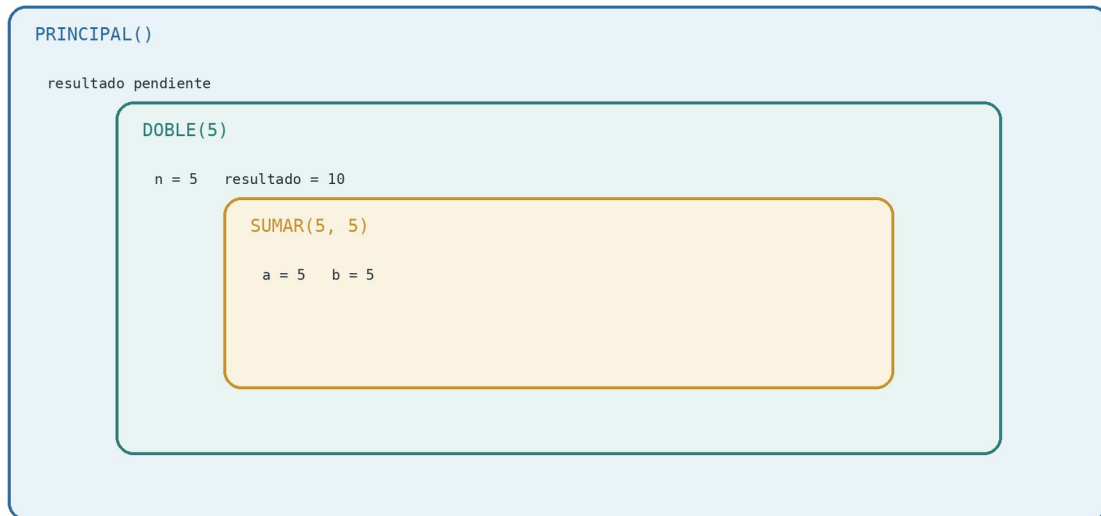
DOBLE(n)
  resultado <- n * 2
  retornar resultado

PRINCIPAL()
  a <- DOBLE(5)
  b <- DOBLE(8)

```

En la primera llamada, `n` vale 5 y `resultado` vale 10. En la segunda, `n` vale 8 y `resultado` vale 16. No son cambios sucesivos sobre una única variable compartida: son ejecuciones diferentes de la misma definición.

Cada llamada conserva sus propios parámetros y variables locales



La llamada más reciente termina primero; después se reanuda la llamada que la esperaba.

Figura 8.4. Marcos de llamadas anidadas. Elaboración propia.

8.8 El marco de llamada

Para razonar sobre una llamada, utilizaremos un modelo mental llamado marco de llamada. Cada marco conserva al menos:

- el nombre de la función que se está ejecutando;
- los valores recibidos por sus parámetros;
- las variables locales creadas durante esa llamada;
- el punto al que debe regresar cuando termine;
- las operaciones pendientes que utilizarán el valor retornado.

Nota: el marco de llamada es un modelo pedagógico del estado de ejecución. Los detalles internos del runtime pueden ser más complejos, pero este modelo permite seguir correctamente funciones anidadas y recursivas.

8.9 Llamadas anidadas y orden de retorno

Una función llama a otra

```
SUMAR(a, b)
  retornar a + b

DOBLE(n)
  retornar SUMAR(n, n)

PRINCIPAL()
  resultado <- DOBLE(5)
  escribir resultado
```

Paso	Marco activo	Acción
1	`PRINCIPAL`	Invoca `DOBLE(5)`.
2	`DOBLE`	El parámetro `n` recibe 5 e invoca `SUMAR(5, 5)`.
3	`SUMAR`	Retorna 10.
4	`DOBLE`	Recibe 10 y lo retorna.
5	`PRINCIPAL`	Asigna 10 a `resultado`.

La llamada más reciente termina primero. Esta disciplina se conoce como último en entrar, primero en salir. No es necesario memorizar el término para programar, pero sí comprender el orden.

8.10 Referencias antes de la primera asignación

CLRS declara las variables de manera implícita mediante su primera asignación textual y organiza los nombres por ámbito de función. El analizador puede reconocer el nombre aunque una referencia aparezca antes de esa asignación. Sin embargo, la existencia del nombre no garantiza que ya tenga un valor útil durante la ejecución.

Orden confuso que debe evitarse

```
PRINCIPAL()
  escribir total
  total <- 50
```

Aunque el nombre pertenezca al ámbito de PRINCIPAL, se intenta utilizar antes de establecer su valor esperado.

Práctica recomendada: inicializa una variable antes de leerla o usarla en una expresión. La elevación del nombre es un detalle semántico, no una razón para escribir programas en un orden confuso.

8.11 La instrucción `leer` y las variables existentes

La instrucción `leer` asigna un valor recibido, pero no introduce por sí sola una nueva declaración implícita. Por ello, el destino debe contar con una asignación declarativa en el mismo ámbito o existir en uno exterior.

Preparar el destino antes de leer

```
PRINCIPAL()
  edad <- 0
  escribir "Ingresa tu edad"
  leer edad
```

La asignación inicial establece el nombre `edad` en el ámbito de `PRINCIPAL`. Después, `leer` sustituye su valor.

Conexión: esta regla explica por qué en ejemplos anteriores se inicializaban variables con `0`, `""` o `FALSO` antes de solicitar datos.

8.12 Cuando una función se llama a sí misma

Una función es recursiva cuando una de sus rutas de ejecución contiene una llamada a la misma función. La nueva llamada no reemplaza a la anterior: crea otro marco con sus propios parámetros.

Cuenta regresiva recursiva

```
CUENTA_REGRESIVA(n)
  si n = 0
    escribir "Fin"
    retornar 0

  escribir n
  retornar CUENTA_REGRESIVA(n - 1)

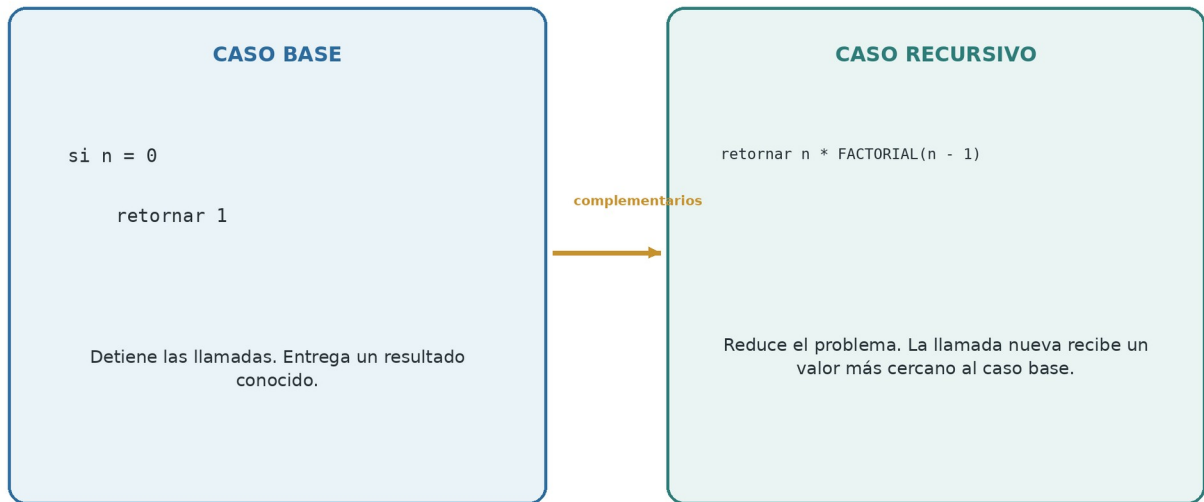
PRINCIPAL()
  CUENTA_REGRESIVA(3)
```

La primera llamada recibe 3. Antes de terminar, invoca otra con 2; después otra con 1; y finalmente una con 0. La llamada con 0 detiene el proceso.

Idea clave: la recursión no consiste en repetir exactamente la misma llamada. Cada llamada debe representar una versión más pequeña o más cercana a la terminación.

8.13 Caso base y caso recursivo

Toda recursión necesita un caso base y un avance hacia él



Sin caso base o sin progreso, la función continúa llamándose hasta fallar.

Figura 8.5. Dos componentes inseparables de la recursión. Elaboración propia.

El caso base resuelve directamente una situación mínima y evita nuevas llamadas. El caso recursivo expresa la solución en función de una versión reducida del mismo problema.

Componente	Pregunta de revisión	Ejemplo en factorial
Caso base	¿Cuándo puede responderse sin otra llamada?	<code>`n = 0`</code> , retorna 1.
Reducción	¿Cómo se aproxima la entrada al caso base?	Usa <code>`n - 1`</code> .
Combinación	¿Qué se hace con el resultado recibido?	Multiplica por <code>`n`</code> .

8.14 Factorial como primera función recursiva

El factorial de un entero no negativo puede definirse mediante una relación consigo mismo. Para ``n > 0``, el factorial de ``n`` es ``n`` multiplicado por el factorial de ``n - 1``. El factorial de cero vale 1.

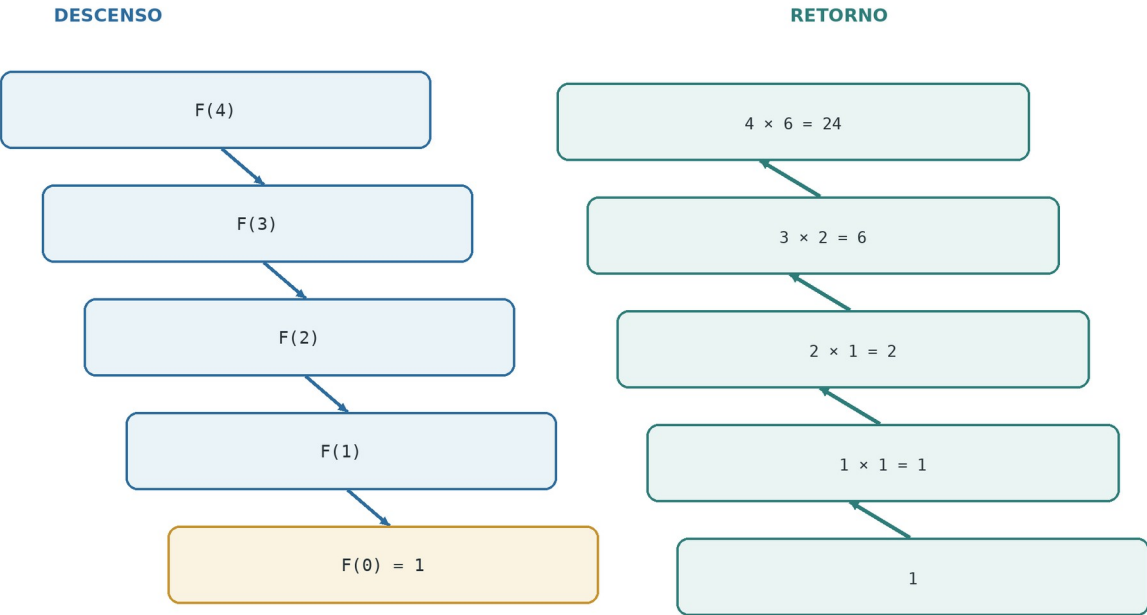
Factorial recursivo

```
FACTORIAL(n)
  si n = 0
    retornar 1

  retornar n * FACTORIAL(n - 1)

PRINCIPAL()
  escribir FACTORIAL(4)
```

Factorial de 4: primero se apilan llamadas y después se resuelven retornos



Cada retorno completa una multiplicación que había quedado pendiente en una llamada anterior.

Figura 8.6. Descenso y retorno de FACTORIAL(4). Elaboración propia.

Las multiplicaciones no se completan inmediatamente. Cada llamada espera el valor producido por la siguiente. Después del caso base, los marcos se resuelven en orden inverso.

8.15 Traza de `FACTORIAL(4)`

Llamada	Decisión	Expresión pendiente	Valor retornado
`FACTORIAL(4)`	No es base.	`4 × FACTORIAL(3)`	24
`FACTORIAL(3)`	No es base.	`3 × FACTORIAL(2)`	6
`FACTORIAL(2)`	No es base.	`2 × FACTORIAL(1)`	2
`FACTORIAL(1)`	No es base.	`1 × FACTORIAL(0)`	1

Llamada	Decisión	Expresión pendiente	Valor retornado
`FACTORIAL(0)`	Caso base.	Ninguna.	1

Pausa y piensa: antes de ejecutar `FACTORIAL(5)`, escribe la secuencia de llamadas y después la secuencia de valores retornados.

8.16 Errores de terminación recursiva

Sin caso base

```
REPETIR(n)
  escribir n
  retornar REPETIR(n - 1)
```

Sin progreso hacia el caso base

```
FACTORIAL(n)
  si n = 0
    retornar 1

  retornar n * FACTORIAL(n)
```

En el primer ejemplo ninguna entrada detiene las llamadas. En el segundo existe un caso base, pero el argumento no cambia; si `n` comienza distinto de cero, nunca se aproxima a él.

Regla de seguridad: antes de ejecutar una función recursiva, identifica explícitamente el caso base y demuestra con palabras cómo cada llamada se acerca a ese caso.

8.17 Entradas fuera del dominio

La definición anterior de factorial supone enteros no negativos. Una entrada negativa produce una secuencia que se aleja de cero: -1, -2, -3 y así sucesivamente. La función debe validar su precondition.

Factorial con validación

```
FACTORIAL(n)
  si no ES_NUM(n) o n < 0
    LANZAR_ERROR("El factorial requiere un número no negativo")

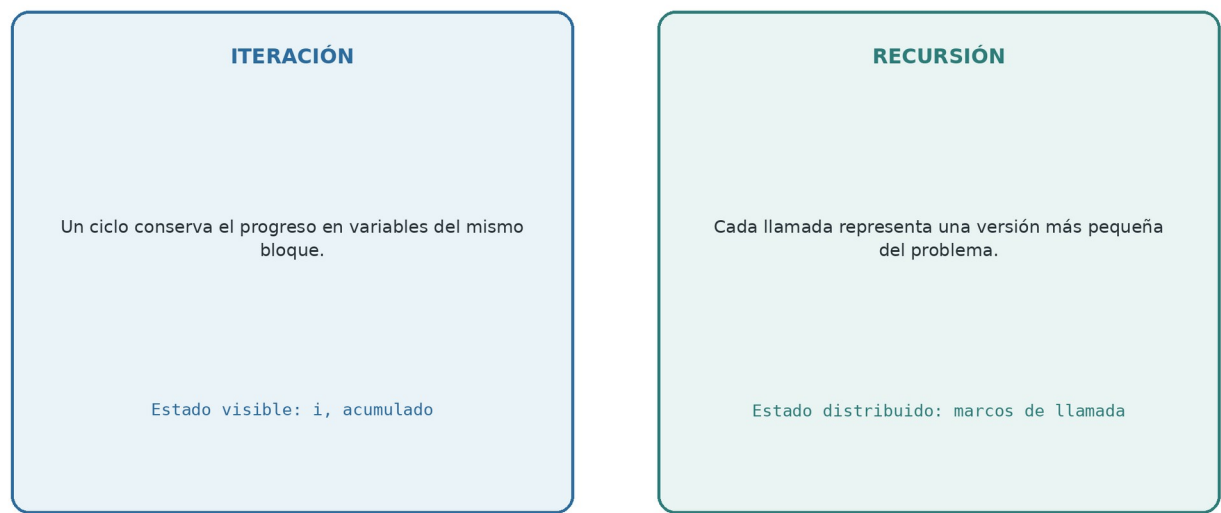
  si n = 0
    retornar 1

  retornar n * FACTORIAL(n - 1)
```

Importante: una función puede tener un caso base correcto y aun así fallar para entradas que no pertenecen al problema definido.

8.18 Recursión e iteración

Iteración y recursión pueden expresar el mismo proceso de maneras distintas



La recursión no es automáticamente mejor ni más rápida; se elige cuando refleja con claridad la estructura del problema.

Figura 8.7. Dos maneras de conservar el progreso. Elaboración propia.

Factorial iterativo
<pre>FACTORIAL_ITERATIVO(n) resultado <- 1 para i <- 1 hasta n resultado <- resultado * i retornar resultado</pre>

Las dos versiones pueden producir el mismo resultado. La iterativa conserva el progreso en `i` y `resultado`. La recursiva distribuye el progreso entre llamadas, donde cada marco conserva su valor de `n` y una multiplicación pendiente.

Criterio	Iteración	Recursión
Estructura visible	Ciclo.	Llamada a la misma función.
Terminación	Condición y actualización.	Caso base y reducción.
Estado	Variables del ciclo.	Marcos de llamada.
Riesgo típico	Ciclo infinito.	Llamadas sin fin o profundidad excesiva.

Criterio	Iteración	Recursión
Elección	Procesos repetitivos directos.	Problemas definidos naturalmente por subproblemas semejantes.

Criterio práctico: no elijas recursión solo porque el lenguaje la permite. Elige la forma que haga más clara la estructura del problema y sus condiciones de terminación.

8.19 Recursión y costo

Cada llamada recursiva realiza trabajo y crea un nuevo marco. En `FACTORIAL(n)`, la entrada disminuye una unidad hasta cero, por lo que la cantidad de llamadas crece de forma lineal con `n`. De manera intuitiva, su tiempo es $O(n)$ y la profundidad de llamadas también es $O(n)$.

CLRS Kit puede reconocer determinadas recurrencias y mostrar una clasificación cuando la reducción es demostrable. Este capítulo utiliza el resultado como apoyo visual, no como sustituto de una traza manual.

Límite conceptual: saber que una función es recursiva no basta para determinar su costo. Importan la cantidad de llamadas generadas y cómo disminuye el tamaño de la entrada.

8.20 Laboratorio CLRS: total acumulado recursivo

Construiremos una función que sume todos los enteros desde `n` hasta 0. El problema se reduce restando uno y el caso base devuelve cero.

Suma recursiva

```
SUMAR_HASTA(n)
    si no ES_NUM(n) o n < 0
        LANZAR_ERROR("Se requiere un número no negativo")

    si n = 0
        retornar 0

    retornar n + SUMAR_HASTA(n - 1)

PRINCIPAL()
    escribir SUMAR_HASTA(5)
```

Llamada	Expresión pendiente
<code>SUMAR_HASTA(5)</code>	<code>5 + SUMAR_HASTA(4)</code>
<code>SUMAR_HASTA(4)</code>	<code>4 + SUMAR_HASTA(3)</code>
<code>SUMAR_HASTA(3)</code>	<code>3 + SUMAR_HASTA(2)</code>
<code>SUMAR_HASTA(2)</code>	<code>2 + SUMAR_HASTA(1)</code>
<code>SUMAR_HASTA(1)</code>	<code>1 + SUMAR_HASTA(0)</code>

Llamada	Expresión pendiente
`SUMAR_HASTA(0)`	`0`

Experimento: cambia el caso base para retornar 1. Ejecuta de nuevo y explica por qué todos los resultados aumentan una unidad.

8.21 Caso longitudinal: funciones sin estado global oculto

El registro de gastos puede organizarse evitando que las funciones dependan de una variable global `saldo`. Cada función recibe lo que necesita y retorna un resultado explícito.

Registro con transformación explícita del estado

```
ES_GASTO_VALIDO(saldoActual, gasto)
    retornar ES_NUM(gasto) y gasto > 0 y gasto <= saldoActual

APLICAR_GASTO(saldoActual, gasto)
    retornar saldoActual - gasto

MOSTRAR_ESTADO(gasto, saldoActual)
    escribir "Gasto: " + gasto
    escribir "Saldo: " + saldoActual

PRINCIPAL()
    saldo <- 1000
    gasto <- 200

    si ES_GASTO_VALIDO(saldo, gasto)
        saldo <- APLICAR_GASTO(saldo, gasto)
        MOSTRAR_ESTADO(gasto, saldo)
```

El estado cambia en `PRINCIPAL`, donde la asignación es visible. Las funciones no crean otra variable `saldo` que pueda confundirse con la principal.

Reflexión: compara esta versión con una función que leyera y modificara una variable global. ¿Cuál permite probar `APLICAR\_GASTO` con mayor facilidad?

8.22 Reconstruye el modelo mental

1. El ámbito determina dónde puede encontrarse un identificador.
2. Las variables del nivel superior pertenecen al ámbito global.
3. Los parámetros y asignaciones dentro de una función pertenecen a su ámbito local.
4. Los bloques `si`, `mientras` y `para` no crean un ámbito de variables adicional en CLRS.
5. Un nombre local puede ocultar a otro exterior con la misma escritura.
6. Cada llamada conserva sus propios parámetros, variables y punto de retorno.

7. Las llamadas anidadas terminan desde la más reciente hacia la más antigua.
8. La recursión crea nuevas llamadas de la misma función.
9. Una función recursiva necesita un caso base y una reducción hacia él.
10. Las entradas deben pertenecer al dominio donde la reducción alcanza el caso base.
11. Iteración y recursión son herramientas distintas; ninguna es superior en todos los casos.

Comprobación activa: explica por qué dos llamadas simultáneamente pendientes de `FACTORIAL`` pueden tener parámetros `n`` distintos sin interferirse.

Ejercicios

Resuelve primero las trazas y predicciones sin ejecutar.

1. Clasifica cinco nombres de un programa como globales, parámetros o locales. **[Reconocer]**
2. Explica por qué una variable local no puede utilizarse directamente desde `PRINCIPAL``. **[Explicar]**
3. Predice la salida de un programa con una variable global y otra local llamadas `total``. **[Predecir]**
4. Reescribe una función que depende de `tasa`` global para recibirla como parámetro. **[Mejorar]**
5. Construye un ejemplo donde una variable creada dentro de `si`` se utilice después del bloque. **[Aplicar]**
6. Explica por qué ese comportamiento no debe suponerse universal en otros lenguajes. **[Analizar]**
7. Dibuja los marcos creados por `DOBLE(5)`` cuando `DOBLE`` llama a `SUMAR``. **[Representar]**
8. Ordena los retornos de tres funciones anidadas. **[Analizar]**
9. Corrige un programa que usa una variable antes de inicializarla. **[Mejorar]**
10. Prepara correctamente una variable antes de usarla como destino de `leer``. **[Aplicar]**
11. Escribe una cuenta regresiva recursiva desde 5. **[Construir]**
12. Identifica el caso base y la reducción de `SUMAR_HASTA``. **[Reconocer]**
13. Traza `FACTORIAL(3)`` indicando llamadas y retornos. **[Analizar]**
14. Modifica factorial para rechazar entradas negativas. **[Construir]**
15. Detecta por qué `F(n) -> F(n + 1)`` no termina con caso base `n = 0`` para entradas positivas. **[Analizar]**
16. Escribe una función recursiva que muestre los números desde 1 hasta `n``. **[Construir]**
17. Compara una solución iterativa y recursiva para sumar de 1 a `n``. **[Evaluar]**
18. Explica qué estado conserva un ciclo y qué estado conservan las llamadas. **[Explicar]**
19. Diseña casos de prueba para una función recursiva con entradas 0, 1, 5 y -1. **[Probar]**
20. Refactoriza el registro de gastos para evitar una dependencia global oculta. **[Construir]**
21. Genera el diagrama de flujo de una función recursiva y localiza la llamada a sí misma. **[Representar]**
22. Activa el visualizador de costo para factorial y compara su resultado con tu traza. **[Explorar]**

Mini proyecto: potencia mediante llamadas recursivas

Situación: construye `POTENCIA(base, exponente)` para exponentes enteros no negativos, utilizando la relación entre una potencia y otra con exponente menor.

- Define el caso base `exponente = 0`.
- Reduce el exponente en una unidad en cada llamada.
- Valida que el exponente sea numérico y no negativo.
- Prueba los casos `2^0`, `2^1`, `2^5`, `0^3` y una entrada negativa.
- Dibuja los marcos de `POTENCIA(2, 4)`.
- Escribe también una versión iterativa y compáralas.
- Explica por qué la profundidad de llamadas crece con el exponente.

Criterio de éxito: la función debe terminar para todas las entradas aceptadas, producir los valores esperados y permitir explicar cada llamada pendiente.

Autoevaluación

Afirmación	Sí	Todavía no
Distingo ámbito global y ámbito de función.	☐	☐
Comprendo que los bloques de control no crean otro ámbito de variables en CLRS.	☐	☐
Puedo explicar el ocultamiento de nombres.	☐	☐
Puedo evitar dependencias globales mediante parámetros y retornos.	☐	☐
Comprendo que cada llamada conserva su propio estado.	☐	☐
Puedo seguir llamadas anidadas y sus retornos.	☐	☐
Inicializo variables antes de utilizarlas.	☐	☐
Puedo identificar caso base y caso recursivo.	☐	☐
Puedo demostrar que una entrada progresa hacia el caso base.	☐	☐
Puedo trazar una función recursiva sencilla.	☐	☐
Puedo comparar iteración y recursión sin asumir que una siempre es mejor.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas explicar el ámbito de cada nombre de un programa y seguir una función recursiva desde la primera llamada hasta el último retorno.

Glosario

Término	Definición de trabajo
Ámbito	Región del programa donde puede encontrarse un nombre.
Ámbito global	Ámbito de los nombres creados fuera de las funciones.
Ámbito de función	Ámbito de parámetros y variables locales de una función.
Variable local	Variable perteneciente al ámbito de una función.
Ocultamiento	Prioridad de un nombre local sobre otro exterior con la misma escritura.
Dependencia global	Uso de un nombre exterior que no aparece en la interfaz de la función.
Marco de llamada	Estado propio de una ejecución concreta de una función.
Llamada anidada	Llamada realizada mientras otra llamada permanece pendiente.
Recurción	Uso de una función que se llama a sí misma directa o indirectamente.
Caso base	Situación resuelta sin iniciar otra llamada recursiva.
Caso recursivo	Situación que reduce el problema y utiliza otra llamada.
Profundidad	Cantidad de llamadas pendientes simultáneamente.
Desenrollado	Proceso de resolver y retirar llamadas desde la más reciente.
Precondición	Condición que una entrada debe cumplir para que la función sea válida.

Referencias del capítulo

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2nd ed.). The MIT Press.

CLRS Kit Español. (2026). Documentación y código fuente del analizador semántico, funciones y llamadas [Repositorio de software].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

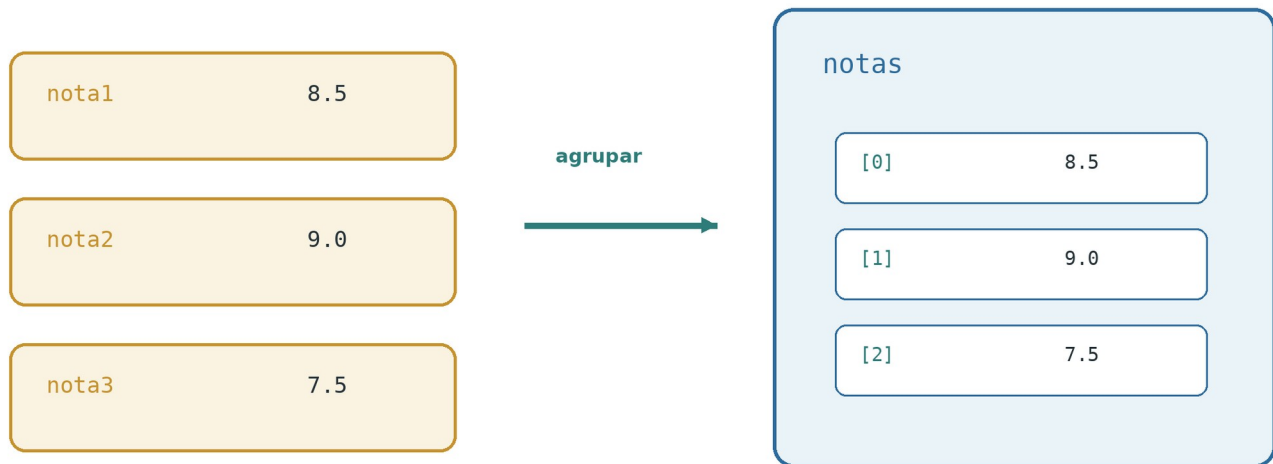
Continuidad: con las funciones y su estado ya comprendidos, el capítulo 9 ampliará el modelo desde valores individuales hacia arreglos: colecciones cuyos elementos pueden recorrerse, transformarse y enviarse a funciones.

CAPÍTULO

9

De variables aisladas a arreglos

Índices, recorridos, operaciones y estructuras multidimensionales



De variables aisladas a arreglos

¿Cómo se aplica un mismo proceso a una cantidad creciente de valores?

Tres calificaciones pueden representarse con `nota1`, `nota2` y `nota3`. El programa todavía parece manejable. Pero si después se solicitan veinte, cien o una cantidad desconocida, la estrategia de crear un nombre diferente para cada valor deja de escalar. También obliga a repetir las mismas operaciones una y otra vez.

Un arreglo reúne varios elementos bajo un solo identificador y permite seleccionar cada uno mediante una posición. Esta idea transforma el problema: en lugar de diseñar una instrucción para cada dato, se diseña un proceso que recorre posiciones.

Promesa de aprendizaje: al terminar podrás crear y recorrer arreglos, relacionar longitud e índices, sumar, contar, buscar y modificar elementos, utilizar funciones de biblioteca, representar tablas bidimensionales y enviar arreglos como argumentos.

Un arreglo relaciona un nombre con una secuencia de posiciones

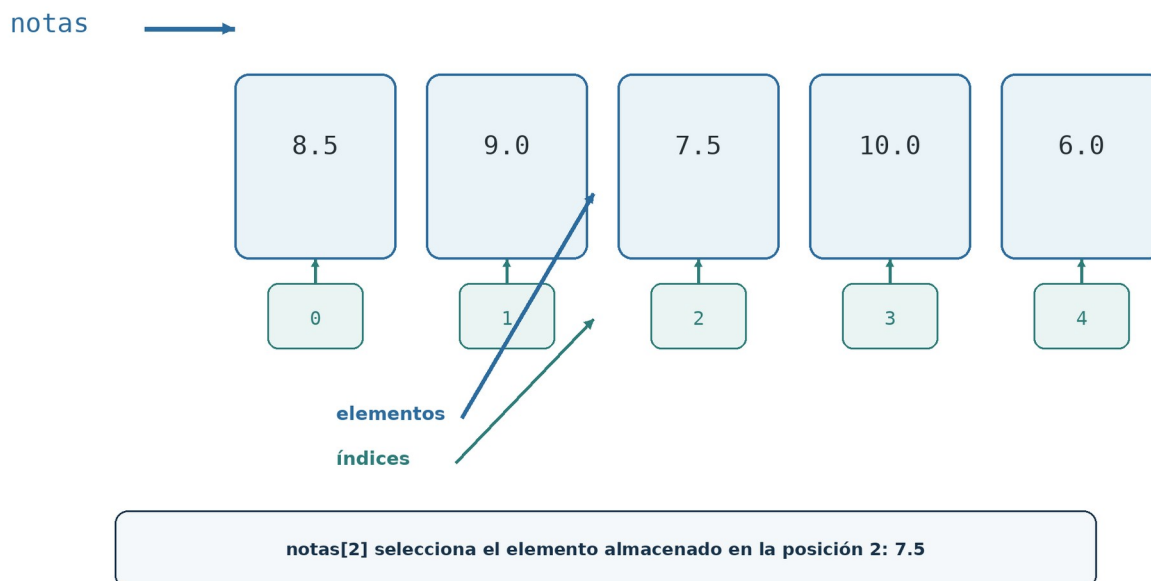


Figura 9.1. Nombre, elementos e índices de un arreglo. Elaboración propia.

Alcance del capítulo: se estudiarán operaciones fundamentales sobre colecciones. El procesamiento especializado de texto y algoritmos adicionales sobre colecciones se desarrollarán en el capítulo 10.

9.1 Cuando las variables aisladas ya no alcanzan

Una solución que no escala

```
nota1 <- 8.5
nota2 <- 9.0
nota3 <- 7.5

promedio <- (nota1 + nota2 + nota3) / 3
```

La solución no es incorrecta. El problema aparece cuando cambia la cantidad de datos. Agregar una cuarta calificación exige crear otro nombre y modificar la fórmula. La estructura del programa depende de una cantidad fija escrita de antemano.

Los mismos datos como colección

```
notas[0] <- 8.5
notas[1] <- 9.0
notas[2] <- 7.5
```

Ahora los valores comparten el identificador `notas`. Cada uno se distingue mediante un índice entre corchetes. La cantidad puede crecer sin inventar nuevos nombres para el mismo tipo de información.

Idea clave: un arreglo no elimina los valores individuales. Los organiza como elementos de una colección que puede procesarse mediante posiciones.

9.2 Arreglo, elemento e índice

Un arreglo es una colección ordenada de elementos accesibles mediante índices. Ordenada significa que cada elemento ocupa una posición identificable; no significa que los valores estén ordenados de menor a mayor.

Término	Ejemplo	Significado
Arreglo	`notas`	Colección completa.
Elemento	`notas[2]`	Valor localizado en una posición.
Índice	`2`	Expresión que selecciona la posición.
Asignación	`notas[2] <- 7.5`	Coloca o reemplaza un valor.
Acceso	`escribir notas[2]`	Obtiene el valor de una posición.

En CLRS los índices se escriben entre corchetes y pueden ser expresiones. `notas[i]` selecciona una posición cuyo índice se obtiene del valor actual de `i`.

Advertencia: no confundas el índice con el elemento. En `notas[2]`, el índice es 2 y el elemento podría ser 7.5, 10 o cualquier otro valor almacenado.

9.3 Crear un arreglo mediante asignaciones

CLRS no exige declarar previamente el tamaño de un arreglo. El tamaño y el número de dimensiones se determinan conforme se accede a posiciones mediante asignaciones.

Creación gradual

```
edades[0] <- 19
edades[1] <- 21
edades[2] <- 18
```

La primera asignación introduce el identificador `edades` como arreglo y establece un elemento en la posición 0. Las asignaciones posteriores incorporan otras posiciones.

En CLRS, nuevas posiciones determinan dinámicamente el tamaño observado

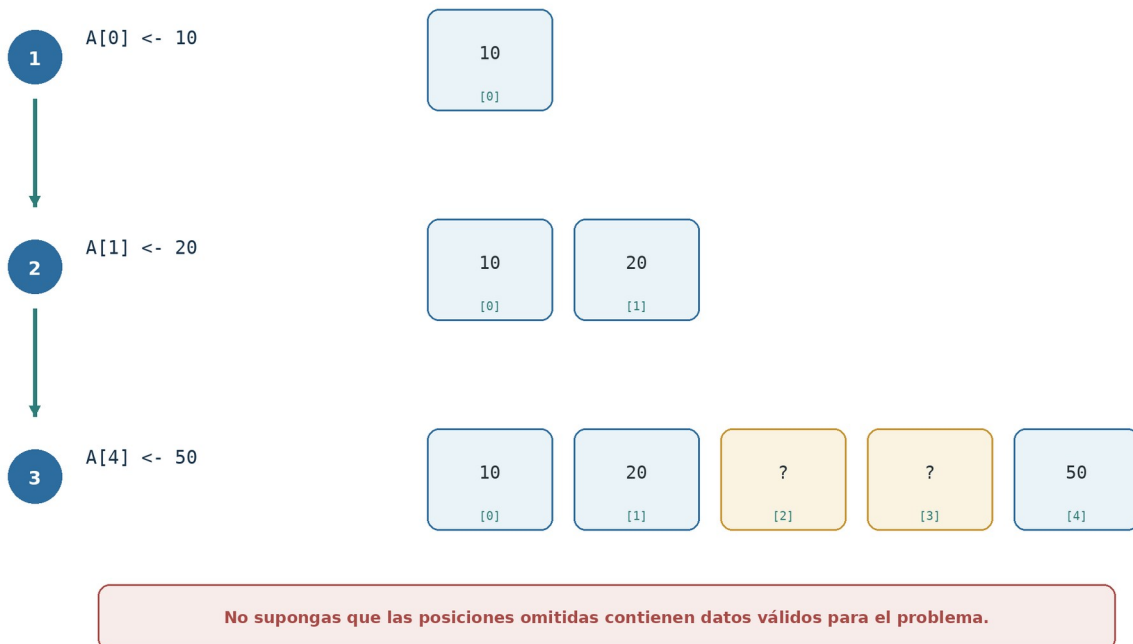


Figura 9.2. Crecimiento dinámico observado mediante nuevas posiciones. Elaboración propia.

Importante: la flexibilidad dinámica no elimina la necesidad de diseñar posiciones coherentes. Saltar índices puede dejar huecos que no representan datos válidos para el problema.

9.4 Leer y actualizar elementos

Acceso y actualización

```
precios[0] <- 15
precios[1] <- 22

escribir precios[0]

precios[0] <- precios[0] + 5
escribir precios[0]
```

La primera escritura muestra 15. La actualización obtiene el valor actual de `precios[0]`, suma 5 y vuelve a asignar el resultado en la misma posición. La segunda escritura muestra 20.

Modelo mental: actualizar un elemento transforma solo la posición indicada; el resto del arreglo conserva su estado.

Estado	`precios[0]`	`precios[1]`
Después de crear	15	22
Después de actualizar	20	22

9.5 Longitud y límites válidos

La función `LONG(arreglo)` devuelve la longitud observada de la colección. Cuando los índices comienzan en cero, una colección de longitud `n` utiliza normalmente las posiciones desde 0 hasta `n - 1`.

La longitud cuenta elementos; el último índice es longitud menos uno



```
LONG(datos) = 5
```

```
Primer índice = 0
```

```
Último índice = LONG(datos) - 1 = 4
```

`datos[5]`

queda fuera del conjunto de posiciones
existentes

Contar elementos y nombrar posiciones son operaciones relacionadas, pero no idénticas.

Figura 9.3. Relación entre longitud e índices válidos. Elaboración propia.

Consultar longitud y último elemento

```

cantidad <- LONG(datos)
ultimoIndice <- cantidad - 1
ultimoValor <- datos[ultimoIndice]

```

Error frecuente: usar `LONG(datos)` como último índice. Si la longitud es 5, el último índice normal es 4. Esta diferencia produce los errores de una posición estudiados en los ciclos.

9.6 Recorrer todos los elementos

Un recorrido utiliza una variable de iteración como índice. El ciclo comienza en 0 y termina en `LONG(arreglo) - 1`.

Recorrido completo

```

para i <- 0 hasta LONG(datos) - 1
  escribir datos[i]

```

En cada iteración cambia `i`; por ello cambia también la posición seleccionada por `datos[i]`. La misma instrucción se aplica a todos los elementos existentes.

Recorrer significa visitar cada posición según una regla

i	datos[i]	operación	total después
0	12	0 + 12	12
1	7	12 + 7	19
2	15	19 + 15	34
3	6	34 + 6	40

El índice cambia; el acumulador conserva el resultado parcial.

Figura 9.4. Traza de un recorrido con acumulación. Elaboración propia.

Pausa y piensa: si `LONG(datos)` es 4, escribe los valores que toma `i` y las expresiones concretas que reemplazan a `datos[i]` en cada iteración.

9.7 Recorridos parciales y descendentes

No siempre es necesario recorrer toda la colección. Los límites del ciclo pueden describir un segmento o un orden diferente.

Desde la segunda posición

```
para i <- 1 hasta LONG(datos) - 1
  escribir datos[i]
```

De la última posición a la primera

```
para i <- LONG(datos) - 1 bajando 0
  escribir datos[i]
```

El primer ciclo omite la posición 0. El segundo conserva los mismos elementos, pero cambia el orden de visita. Diseñar un recorrido exige responder qué posiciones deben visitarse y en qué secuencia.

Advertencia: un recorrido parcial puede ser correcto o puede omitir datos por error. Los límites deben derivarse del problema, no elegirse por costumbre.

9.8 Sumar y obtener un promedio

La suma de una colección utiliza un acumulador. El promedio divide el total entre la cantidad de elementos.

Suma y promedio manuales

```
notas[0] <- 8
notas[1] <- 9
notas[2] <- 7

total <- 0
para i <- 0 hasta LONG(notas) - 1
  total <- total + notas[i]

promedio <- total / LONG(notas)
escribir promedio
```

El acumulador comienza en cero porque cero es el valor neutro de la suma. Después de cada iteración representa la suma parcial de los elementos ya visitados.

Caso límite: antes de dividir, considera qué ocurriría con un arreglo sin elementos. Una división entre `LONG(notas)` solo es válida cuando la longitud es mayor que cero.

9.9 Contar elementos que cumplen una condición

Contar calificaciones aprobatorias

```
aprobadas <- 0

para i <- 0 hasta LONG(notas) - 1
  si notas[i] >= 6
    aprobadas <- aprobadas + 1

escribir aprobadas
```

El contador no suma los valores del arreglo. Suma una unidad cuando el elemento satisface la condición. Por eso responde cuántos elementos cumplen una regla, no cuánto valen en conjunto.

Contraste: un acumulador incorpora el valor del elemento; un contador incorpora normalmente una unidad por coincidencia.

9.10 Encontrar máximos y mínimos

Para conservar el mayor valor observado se necesita una referencia inicial válida. Una estrategia general consiste en utilizar el primer elemento y recorrer desde la segunda posición.

Máximo de un arreglo no vacío

```
mayor <- datos[0]

para i <- 1 hasta LONG(datos) - 1
  si datos[i] > mayor
    mayor <- datos[i]

escribir mayor
```

Inicializar `mayor <- 0` sería incorrecto si todos los valores fueran negativos. El primer elemento pertenece al mismo dominio de los datos y evita esa suposición.

Precondición: este patrón requiere al menos un elemento. La función o el programa debe comprobar `LONG(datos) > 0` antes de acceder a `datos[0]`.

9.11 Buscar un valor

La búsqueda lineal visita posiciones hasta encontrar una coincidencia o terminar la colección. Una bandera puede recordar si el valor apareció y una variable puede conservar su posición.

Búsqueda lineal completa

```

objetivo <- 15
encontrado <- FALSO
posicion <- -1

para i <- 0 hasta LONG(datos) - 1
  si datos[i] = objetivo y no encontrado
    encontrado <- VERDAD
    posicion <- i

escribir encontrado
escribir posicion

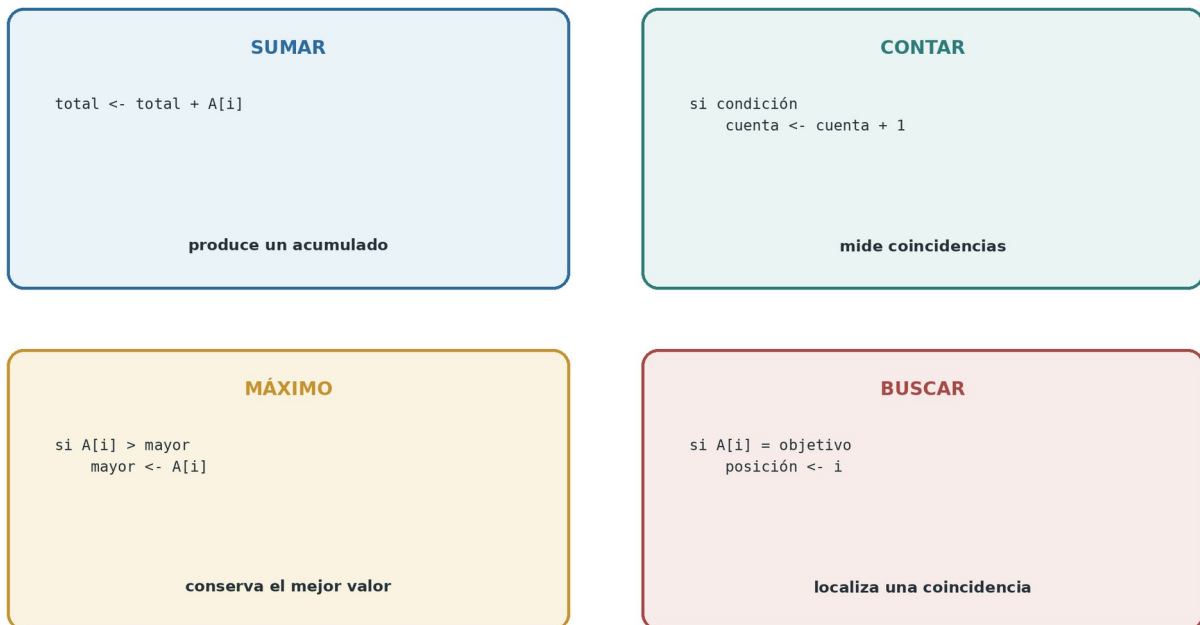
```

La posición -1 funciona como señal de ausencia porque no es un índice normal del arreglo. La condición `no encontrado` conserva la primera coincidencia si el valor aparece varias veces.

Nota: la biblioteca también ofrece `INDICE(arreglo, valor)` y `CONT(arreglo, valor)`. Conviene comprender primero el recorrido que esas operaciones resumen.

9.12 Cuatro patrones sobre un mismo recorrido

Un recorrido puede aplicar distintos patrones sobre los mismos elementos



La estructura del ciclo se repite; cambia el estado que se actualiza y la pregunta que se responde.

Figura 9.5. Patrones de suma, conteo, máximo y búsqueda. Elaboración propia.

Los patrones comparten el esqueleto del recorrido, pero responden preguntas diferentes. La elección del estado inicial y la regla de actualización determina el significado del resultado.

Pregunta	Estado inicial	Actualización típica
¿Cuál es la suma?	`total <- 0`	`total <- total + A[i]`
¿Cuántos cumplen?	`cuenta <- 0`	`cuenta <- cuenta + 1` si se cumple.
¿Cuál es el mayor?	`mayor <- A[0]`	Reemplazar si aparece uno mayor.
¿Dónde está?	`posicion <- -1`	Guardar el índice de la coincidencia.

9.13 Sustituir elementos y producir una copia

Sustituir valores negativos

```
para i <- 0 hasta LONG(datos) - 1
  si datos[i] < 0
    datos[i] <- 0
```

Este recorrido modifica el arreglo original posición por posición. Cuando se necesita conservar la colección original, puede crearse una copia antes de transformar.

Trabajar sobre una copia

```
copia <- COPIA(datos)
para i <- 0 hasta LONG(copia) - 1
  si copia[i] < 0
    copia[i] <- 0
```

Idea clave: pregunta siempre si la operación debe transformar la colección existente o producir otra versión. La decisión afecta el estado que observarán las instrucciones posteriores.

9.14 Operaciones de biblioteca sobre arreglos

CLRS Kit incluye funciones para operaciones frecuentes. Estas funciones reducen código repetitivo, pero su uso debe partir de una intención clara.

Función	Propósito	Resultado
`AGREGA(A, valor)`	Añadir al final.	Arreglo.
`INSER(A, indice, valor)`	Insertar en una posición.	Arreglo.
`ELIM(A, indice)`	Eliminar una posición.	Arreglo.
`CONT(A, valor)`	Comprobar existencia.	Lógico.
`INDICE(A, valor)`	Obtener una posición.	Número.
`ORDENA(A)`	Ordenar elementos.	Arreglo.

Función	Propósito	Resultado
`INVER(A)`	Invertir el orden.	Arreglo.
`COPIA(A)`	Crear una copia.	Arreglo.
`SUM(A)`	Sumar elementos.	Número.
`PROM(A)`	Obtener promedio.	Número.

Asignar el resultado de una operación

```
datos <- AGREGA(datos, 40)
datos <- INSER(datos, 1, 15)
datos <- ELIM(datos, 0)
copia <- COPIA(datos)
```

Práctica segura: cuando una función de biblioteca devuelve un arreglo, asigna explícitamente su resultado. Así el cambio de estado queda visible en el programa.

9.15 Agregar, insertar y eliminar no significan lo mismo

Operación	Ejemplo conceptual	Efecto
Agregar	`[10, 20] -> [10, 20, 30]`	Coloca al final.
Insertar en 1	`[10, 20] -> [10, 15, 20]`	Desplaza elementos posteriores.
Eliminar 1	`[10, 15, 20] -> [10, 20]`	Retira la posición y reajusta la secuencia.
Reemplazar 1	`[10, 20] -> [10, 99]`	Conserva cantidad y cambia un elemento.

La distinción importa porque insertar o eliminar cambia la relación entre índices y elementos posteriores. Una posición que antes contenía cierto valor puede referirse a otro después de la operación.

Advertencia: si recorres y eliminas elementos simultáneamente, los índices pueden desplazarse. En este nivel, separa primero la identificación de posiciones y después la modificación, o recorre con una estrategia cuidadosamente justificada.

9.16 Asignar arreglos completos

CLRS permite asignar un arreglo completo a una variable o asignar a un arreglo el contenido correspondiente de otra variable. La documentación de la extensión describe estas asignaciones como copias de contenido.

Copias mediante asignación

```
original[0] <- 10
original[1] <- 20

copia <- original
copia[0] <- 99
```

El modelo documentado implica que `copia` recibe el contenido del arreglo, en vez de convertirse simplemente en otro nombre para la misma colección. Aun así, `COPIA` expresa con mayor claridad la intención cuando la duplicación es parte importante de la solución.

Idea editorial: prefiere una operación cuyo nombre comunique la intención cuando esa claridad ayude al lector o a quien mantendrá el programa.

9.17 Arreglos como argumentos y parámetros

Para enviar un arreglo a una función se utiliza su identificador como argumento. En la definición, el parámetro incluye pares de corchetes vacíos que indican el número de dimensiones esperado.

La interfaz indica cuántas dimensiones espera el parámetro



El identificador del arreglo se entrega como argumento; el parámetro usa [] para expresar su forma esperada.

Figura 9.6. Arreglo como argumento de una función. Elaboración propia.

Función que recibe un arreglo

```

PROMEDIO(valores[])
  si LONG(valores) = 0
    LANZAR_ERROR("Se requiere al menos un valor")

  total <- 0
  para i <- 0 hasta LONG(valores) - 1
    total <- total + valores[i]

  retornar total / LONG(valores)

```

Importante: `valores[]` aparece solo en la lista de parámetros para expresar una dimensión. La llamada utiliza `PROMEDIO(notas)`, no `PROMEDIO(notas[])`.

9.18 Diseñar funciones sobre colecciones

Una función sobre arreglos debe declarar claramente qué colección necesita, qué precondiciones supone y qué resultado produce.

Función	Precondición	Retorno
`SUMAR(A[])`	Elementos numéricos.	Suma.
`MAXIMO(A[])`	Arreglo no vacío y numérico.	Mayor elemento.
`CONTAR_APROBADOS(A[])`	Calificaciones comparables.	Cantidad.
`BUSCAR(A[], objetivo)`	Objetivo comparable.	Índice o -1.
`REEMPLAZAR_NEGATIVOS(A[])`	Elementos numéricos.	Arreglo transformado.

Las funciones de consulta, como sumar o buscar, pueden retornar un número sin modificar el arreglo. Las funciones de transformación pueden trabajar sobre una copia y retornar la nueva colección para hacer explícito el resultado.

Criterio de diseño: un nombre como `PROCESAR(A[])` no revela si la función suma, ordena, modifica o muestra. Nombra la operación concreta.

9.19 Arreglos multidimensionales

Una dimensión adicional permite representar información organizada por dos coordenadas, como filas y columnas. Cada par de índices selecciona una celda.

Crear una matriz de calificaciones

```

notas[0][0] <- 8
notas[0][1] <- 7
notas[0][2] <- 9

notas[1][0] <- 6
notas[1][1] <- 10
notas[1][2] <- 8

```

Un arreglo bidimensional localiza valores mediante fila y columna

	columna 0	columna 1	columna 2
fila 0	8 [0][0]	7 [0][1]	9 [0][2]
fila 1	6 [1][0]	10 [1][1]	8 [1][2]
fila 2	9 [2][0]	8 [2][1]	7 [2][2]

notas[1][2] selecciona la fila 1 y la columna 2: 8

Figura 9.7. Filas, columnas y acceso bidimensional. Elaboración propia.

En `notas[fila][columna]`, el primer índice selecciona una fila y el segundo una posición dentro de esa fila. Esta convención debe mantenerse de forma consistente en todo el programa.

9.20 Recorrer filas y columnas

Un arreglo bidimensional requiere normalmente dos ciclos anidados: el ciclo exterior avanza por filas y el interior por columnas.

Recorrido de una matriz de 3 por 3

```

para fila <- 0 hasta 2
  para columna <- 0 hasta 2
    escribir notas[fila][columna]

```

Por cada valor de `fila`, el ciclo interior recorre todas las columnas. El elemento seleccionado cambia mediante la combinación de ambos índices.

Fila	Columnas visitadas	Accesos
0	0, 1, 2	`notas[0][0]`, `notas[0][1]`, `notas[0][2]`
1	0, 1, 2	`notas[1][0]`, `notas[1][1]`, `notas[1][2]`
2	0, 1, 2	`notas[2][0]`, `notas[2][1]`, `notas[2][2]`

Costo intuitivo: una matriz con `n` filas y `m` columnas requiere `n \* m` visitas para un recorrido completo. El visualizador puede representar este trabajo como $O(nm)$.

9.21 Enviar una matriz a una función

Parámetro de dos dimensiones

```

MOSTRAR_MATRIZ(M[[]], filas, columnas)
  para i <- 0 hasta filas - 1
    para j <- 0 hasta columnas - 1
      escribir M[i][j]

PRINCIPAL()
  datos[0][0] <- 10
  datos[0][1] <- 20
  datos[1][0] <- 30
  datos[1][1] <- 40

  MOSTRAR_MATRIZ(datos, 2, 2)

```

Los dos pares de corchetes de `M[[]]` indican que la función espera un arreglo bidimensional. Los parámetros `filas` y `columnas` proporcionan los límites utilizados por los recorridos.

Importante: el número de dimensiones forma parte de la interfaz. Una función diseñada para `A[]` no expresa la misma estructura que una diseñada para `M[[]]`.

9.22 Errores frecuentes con arreglos

Error	Consecuencia	Revisión
Usar longitud como índice	Acceso fuera del último elemento.	Terminar en `LONG(A) - 1`.
Comenzar en 1 por costumbre	Se omite la posición 0.	Verificar índice inicial.
Inicializar máximo con 0	Falla con datos negativos.	Usar el primer elemento.
Dividir entre longitud cero	Operación inválida.	Comprobar arreglo no vacío.
Confundir índice y valor	Se compara o muestra el dato equivocado.	Nombrar ambas ideas.
Modificar mientras se desplazan índices	Se omiten o repiten elementos.	Diseñar estrategia de modificación.
Invertir fila y columna	Se accede a otra celda.	Mantener convención.

Error	Consecuencia	Revisión
Omitir `[ ]` en parámetro	La interfaz no expresa colección.	Declarar dimensiones.
Usar `[ ]` en la llamada	Sintaxis incorrecta.	Pasar solo el identificador.

Método de depuración: anota en cada iteración el índice, el elemento seleccionado y el estado actualizado. La mayoría de los errores de recorrido se vuelven visibles en una tabla pequeña.

9.23 Laboratorio CLRS: registrar varios gastos

El caso longitudinal ya puede conservar cada gasto, no solo el total acumulado. El arreglo permite revisar los movimientos después de capturarlos.

Registro y resumen de gastos

```

CALCULAR_TOTAL(gastos[])
  total <- 0
  para i <- 0 hasta LONG(gastos) - 1
    total <- total + gastos[i]
  retornar total

MOSTRAR_GASTOS(gastos[])
  para i <- 0 hasta LONG(gastos) - 1
    escribir "Gasto " + i + ": " + gastos[i]

PRINCIPAL()
  gastos[0] <- 120
  gastos[1] <- 85.5
  gastos[2] <- 300

  MOSTRAR_GASTOS(gastos)
  escribir "Total: " + CALCULAR_TOTAL(gastos)
  escribir "Promedio: " + PROM(gastos)

```

`CALCULAR\_TOTAL` consulta todos los elementos y retorna una cantidad. `MOSTRAR\_GASTOS` produce salida. `PRINCIPAL()` crea la colección y coordina ambas responsabilidades.

Experimento: agrega un cuarto gasto mediante `AGREGA`, elimina el segundo mediante `ELIM` y vuelve a calcular el total. Predice primero cómo cambiarán los índices.

9.24 Diagramas, costo y colecciones

El diagrama de flujo permite observar el ciclo que recorre el arreglo, pero una tabla de traza sigue siendo la mejor herramienta para relacionar índices, elementos y estado parcial. El visualizador de costo ayuda a reconocer cuántas veces se ejecuta el cuerpo del ciclo.

- Un acceso directo como `A[3]` representa trabajo constante para el modelo elemental.
- Un recorrido completo de `n` elementos produce crecimiento lineal $O(n)$.
- Dos recorridos consecutivos siguen siendo lineales respecto de `n`.

- Dos recorridos anidados sobre una matriz de n por m producen $O(nm)$.
- Una función de biblioteca puede ocultar un recorrido; su costo declarado debe considerarse al analizar el programa.

Advertencia: una función corta en el código no implica trabajo constante. `ORDENA(A)` ocupa una sola línea, pero su operación depende de la cantidad de elementos.

9.25 Reconstruye el modelo mental

1. Un arreglo reúne elementos bajo un solo identificador.
2. Cada índice selecciona una posición; el elemento es el valor almacenado allí.
3. CLRS determina dinámicamente el tamaño y las dimensiones conforme se accede a posiciones.
4. La longitud cuenta elementos y el último índice normal es longitud menos uno.
5. Un recorrido aplica el mismo proceso a varias posiciones.
6. Sumar, contar, buscar y hallar máximos comparten el ciclo, pero actualizan estados distintos.
7. Modificar un elemento no es lo mismo que insertar o eliminar una posición.
8. Las funciones de biblioteca resumen operaciones frecuentes y devuelven resultados que conviene asignar explícitamente.
9. Una función recibe un arreglo mediante un parámetro con corchetes vacíos.
10. Una dimensión adicional permite representar filas y columnas.
11. Los ciclos anidados recorren combinaciones de índices.
12. Las precondiciones de longitud y contenido deben verificarse antes de ciertas operaciones.

Comprobación activa: explica la diferencia entre `LONG(A)`, `A[LONG(A) - 1]` y `A[LONG(A)]`.

Ejercicios

Resuelve primero las predicciones y trazas sin ejecutar el código.

1. Convierte cinco variables de calificaciones en un arreglo. **[Aplicar]**
2. Identifica arreglo, índice y elemento en `precios[i + 1]`. **[Reconocer]**
3. Predice el estado después de actualizar `A[1] <- A[1] * 2`. **[Predecir]**
4. Escribe un recorrido que muestre todos los elementos. **[Aplicar]**
5. Escribe un recorrido descendente. **[Aplicar]**
6. Explica por qué el último índice es `LONG(A) - 1`. **[Explicar]**
7. Traza la suma de `[4, 7, 2, 9]`. **[Analizar]**
8. Construye una función `SUMAR(A[])`. **[Construir]**
9. Construye una función `PROMEDIO(A[])` que rechace un arreglo vacío. **[Construir]**
10. Cuenta cuántos valores son negativos. **[Aplicar]**
11. Encuentra el menor valor sin inicializarlo con cero. **[Construir]**
12. Busca la primera posición de un objetivo y retorna -1 si no aparece. **[Construir]**
13. Modifica una copia para reemplazar valores negativos por cero. **[Mejorar]**

14. Compara reemplazar, insertar y agregar. **[Analizar]**
15. Predice los índices después de eliminar la posición 1. **[Predecir]**
16. Usa ``CONT`` e ``INDICE`` para comprobar y localizar un valor. **[Aplicar]**
17. Crea una copia, invierte su orden y conserva el original. **[Construir]**
18. Explica cuándo usarías ``ORDENA`` y cuándo no modificarías el orden. **[Evaluar]**
19. Construye una matriz de 2 filas y 3 columnas. **[Aplicar]**
20. Traza el orden de visitas de dos ciclos anidados. **[Analizar]**
21. Escribe ``SUMAR_MATRIZ(M[[]], filas, columnas)``. **[Construir]**
22. Detecta un acceso fuera de límites en un programa dado. **[Depurar]**
23. Diseña casos de prueba para máximo: positivos, negativos, un elemento y vacío. **[Probar]**
24. Analiza el costo de un recorrido y de dos recorridos anidados. **[Analizar]**

Mini proyecto: analizador de calificaciones

Situación: construye un programa que almacene calificaciones, muestre cada una y calcule total, promedio, máximo, mínimo y cantidad de aprobadas.

- Crea las calificaciones en un arreglo.
- Divide el programa en funciones con parámetros `[]``.
- Valida que las operaciones de máximo y mínimo reciban al menos un elemento.
- Incluye una búsqueda de una calificación indicada.
- Genera una copia ordenada sin perder el orden original.
- Escribe casos de prueba con valores repetidos y negativos inválidos.
- Explica el costo de cada recorrido principal.

Criterio de éxito: el programa debe permitir comprender la solución leyendo ``PRINCIPAL()`` y cada función debe responder una pregunta concreta sobre la colección.

Autoevaluación

Afirmación	Sí	Todavía no
Distingo arreglo, elemento e índice.	☐	☐
Puedo crear y actualizar posiciones.	☐	☐
Relaciono longitud con límites válidos.	☐	☐
Puedo recorrer todos los elementos.	☐	☐
Puedo construir suma, conteo, máximo y búsqueda.	☐	☐

Afirmación	Sí	Todavía no
Comprendo la diferencia entre reemplazar, insertar y eliminar.	☐	☐
Puedo usar operaciones de biblioteca de forma explícita.	☐	☐
Puedo enviar un arreglo a una función mediante <code>A[]</code> .	☐	☐
Puedo representar y recorrer una matriz.	☐	☐
Puedo detectar errores de una posición y arreglos vacíos.	☐	☐
Puedo explicar el crecimiento de recorridos simples y anidados.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas diseñar una función que recorra un arreglo, justificar sus límites, trazar sus estados y explicar qué resultado produce.

Glosario

Término	Definición de trabajo
Arreglo	Colección ordenada de elementos accesibles mediante índices.
Elemento	Valor almacenado en una posición.
Índice	Expresión que selecciona una posición.
Longitud	Cantidad observada de elementos de la colección.
Recorrido	Proceso de visitar posiciones según un orden.
Acumulador	Variable que conserva un resultado parcial.
Contador	Variable que registra cuántas veces ocurre una condición.
Búsqueda lineal	Recorrido secuencial para localizar una coincidencia.
Copia	Colección independiente con contenido duplicado.
Inserción	Incorporación de un elemento en una posición, desplazando otros.
Eliminación	Retiro de una posición de la secuencia.
Matriz	Arreglo bidimensional organizado por filas y columnas.
Dimensión	Cantidad de índices necesarios para seleccionar un elemento.
Parámetro de arreglo	Parámetro que declara dimensiones mediante corchetes vacíos.
Límite	Primera o última posición incluida en un recorrido.
Hueco	Posición omitida entre índices utilizados que puede carecer de un dato válido.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación y código fuente de arreglos, accesos, asignaciones y parámetros [Repositorio de software].

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Gaddis, T. (2018). Starting out with programming logic and design (5th ed.). Pearson.

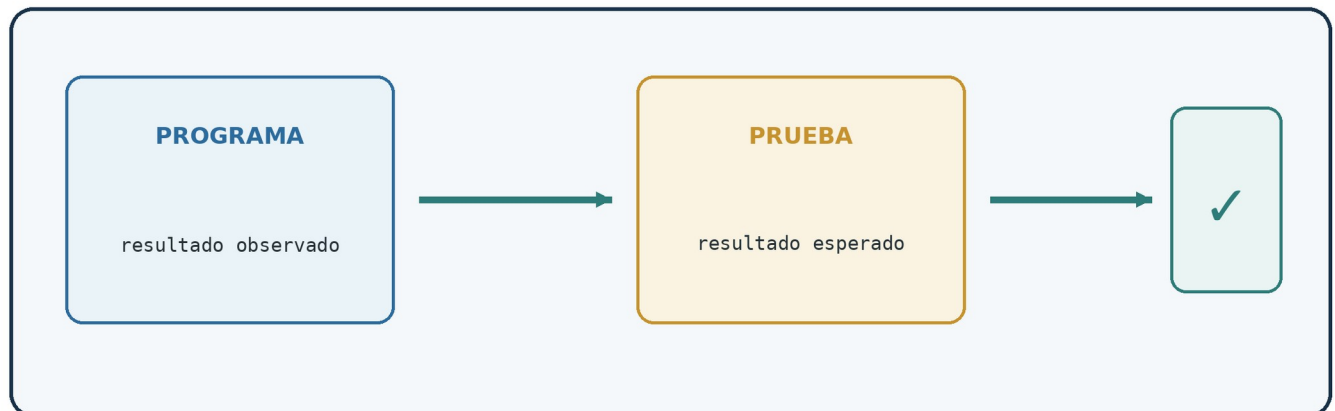
Continuidad: los arreglos permiten procesar conjuntos de valores. El capítulo 10 aplicará esta misma idea a las cadenas: secuencias de caracteres que pueden normalizarse, dividirse, buscarse y combinarse con otras colecciones.

CAPÍTULO

11

Saber si un programa realmente funciona

Especificar, probar, diagnosticar y mejorar soluciones



Ejecutar sin errores no demuestra que el problema esté resuelto.

Saber si un programa realmente funciona

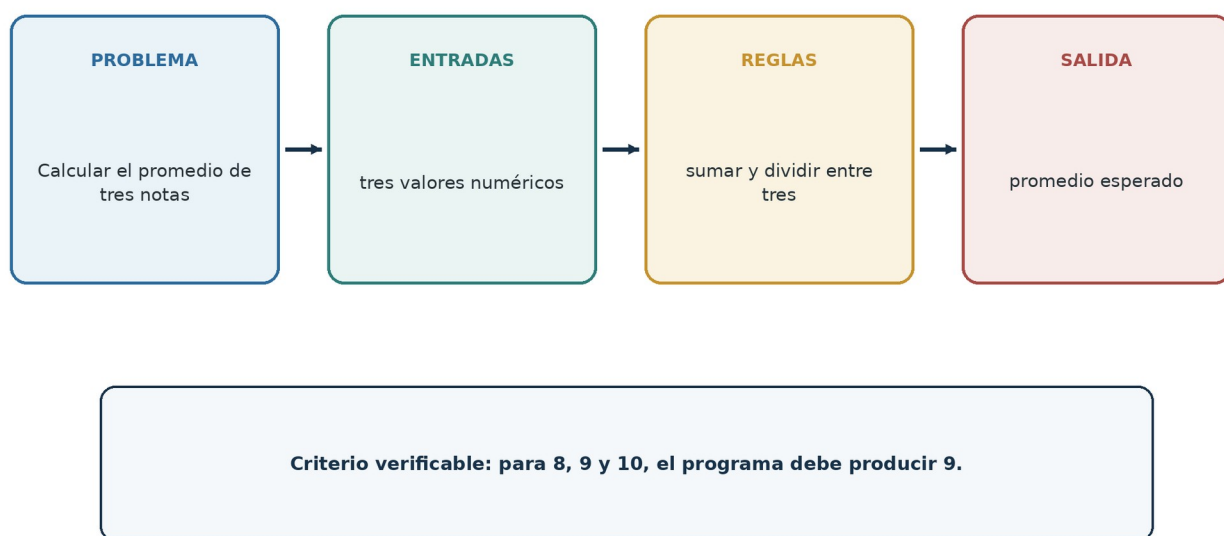
¿Cómo se pasa de “parece funcionar” a una comprobación sistemática?

Un programa puede ejecutarse sin mostrar mensajes de error y, aun así, calcular mal un promedio, aceptar datos imposibles o ignorar un caso límite. También puede fallar de manera evidente, pero dejar un mensaje que no explica por sí solo la causa. La corrección no se obtiene observando una sola ejecución favorable: se construye relacionando una especificación, casos de prueba, resultados esperados y evidencia obtenida durante la ejecución.

En este capítulo el código deja de verse únicamente como una solución que debe escribirse. Se convierte en un objeto que debe poder explicarse, someterse a pruebas, diagnosticar y mejorar. Las herramientas de CLRS Kit ayudan a localizar errores y visualizar recorridos, pero la interpretación sigue dependiendo de comprender qué debía hacer el programa.

Promesa de aprendizaje: al terminar podrás formular una especificación verificable, diseñar casos normales, límite e inválidos, distinguir errores sintácticos, semánticos, de ejecución y lógicos, seguir un proceso de depuración, usar diagnósticos y diagramas como evidencia, y revisar la calidad de un programa.

Una especificación convierte una intención en criterios comprobables



Sin un resultado esperado, una ejecución solo muestra lo que ocurrió, no si ocurrió lo correcto.

Figura 11.1. De la intención al criterio verificable. Elaboración propia.

11.1 Ejecutarse no es lo mismo que funcionar correctamente

Decir que un programa “funciona” puede significar varias cosas diferentes: que inicia, que no se detiene con un error, que produce una salida o que realmente satisface las reglas del problema. Solo la última interpretación permite hablar de corrección respecto de una especificación.

Afirmación	Lo que demuestra	Lo que todavía no demuestra
El archivo se ejecuta	La sintaxis y la estructura permiten iniciar.	Que la lógica sea correcta.
No aparece un error	No ocurrió una falla detectada en esa ejecución.	Que todos los caminos sean seguros.
Produce un número	Existe una salida observable.	Que sea el número esperado.
Coincide en un ejemplo	Ese caso concreto fue resuelto.	Que funcione para límites y datos inválidos.
Supera casos diseñados desde la especificación	Existe evidencia organizada.	Una garantía absoluta para toda entrada posible.

Idea clave: la prueba no demuestra que un programa sea perfecto; aporta evidencia de que se comporta como se espera en situaciones elegidas deliberadamente.

11.2 Especificar antes de probar

Una prueba necesita una expectativa. Para obtenerla, el problema debe describirse con suficiente precisión. Una especificación básica identifica entradas, salidas, reglas, restricciones y casos especiales.

Elemento	Pregunta	Ejemplo: promedio de tres notas
Entradas	¿Qué recibe el programa?	Tres valores numéricos.
Salida	¿Qué debe producir?	La media aritmética.
Regla	¿Cómo se obtiene?	Suma de las notas dividida entre tres.
Restricción	¿Qué valores son aceptables?	Notas entre 0 y 10.
Caso especial	¿Qué ocurre con un dato inválido?	Se rechaza y se informa la causa.

La especificación no necesita comenzar como un documento extenso. Sin embargo, debe permitir que otra persona determine si una ejecución es correcta sin adivinar la intención del autor.

Pausa y piensa: “calcular un descuento” todavía es ambiguo. ¿El porcentaje se expresa como 15 o como 0.15? ¿Puede ser mayor que 100 %? ¿El precio puede ser negativo? Convierte esas dudas en reglas explícitas.

11.3 Precondiciones y resultados esperados

Una precondición describe lo que debe cumplirse antes de ejecutar una operación. Un resultado esperado describe lo que debe observarse después cuando las precondiciones se cumplen.

Contrato de una función

```
CALCULAR_DESCUENTO(precio, porcentaje)
  retornar precio * porcentaje
```

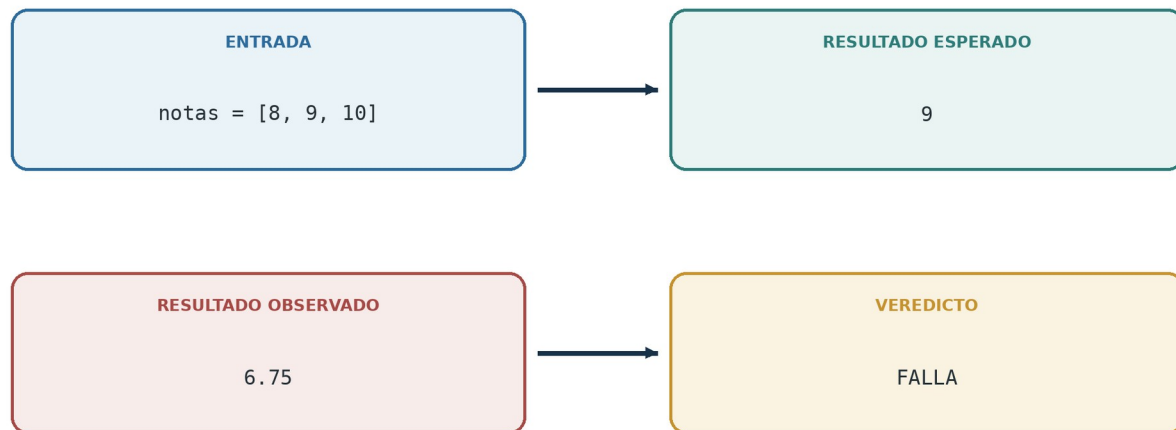
Precondiciones posibles: $\text{precio} \geq 0$ y $0 \leq \text{porcentaje} \leq 1$. Resultado esperado: monto no negativo y no mayor que el precio.

Las precondiciones ayudan a separar dos preguntas: ¿la función calcula bien con datos válidos? y ¿el programa controla correctamente los datos inválidos? Ambas son importantes, pero no son la misma prueba.

Importante: un resultado inesperado puede deberse a una lógica incorrecta o a que la prueba proporcionó datos fuera del dominio sin definir qué debía ocurrir.

11.4 Anatomía de un caso de prueba

Un caso de prueba relaciona una situación con una expectativa



La diferencia entre 9 y 6.75 orienta la investigación: el divisor usado no coincide con la especificación.

Una prueba útil registra evidencia, no solo la frase “no funciona”.

Figura 11.2. Componentes de un caso de prueba. Elaboración propia.

Un caso de prueba debe poder repetirse. Por ello conviene registrar al menos la entrada, el resultado esperado, el resultado observado y el veredicto. También puede incluir una breve razón que explique qué aspecto verifica.

ID	Propósito	Entrada	Esperado	Observado	Resultado
P-01	Caso normal	8, 9, 10	9	9	Pasa
P-02	Límite inferior	0, 0, 0	0	0	Pasa
P-03	Valor inválido	-1, 8, 9	Rechazo	-1.33	Falla

Idea clave: un caso de prueba no es solo una entrada. Es una afirmación acerca de lo que debe ocurrir con esa entrada.

11.5 Casos normales, límite e inválidos

Categoría	Propósito	Ejemplo
Normal	Representar un uso frecuente dentro del dominio.	Notas 7, 8 y 9.
Límite	Examinar extremos y cambios de comportamiento.	0, 10 y valores cercanos.
Inválido	Comprobar rechazo o manejo definido.	Cadena, nota negativa o mayor que 10.
Especial	Revisar una regla poco frecuente pero permitida.	Todas las notas iguales.
Regresión	Confirmar que una corrección anterior no se perdió.	Caso que antes reveló un defecto.

Los casos normales muestran que el recorrido común funciona. Los límites y los datos inválidos suelen descubrir defectos que permanecen ocultos en el centro del rango.

Los límites concentran errores que los valores centrales no revelan



Rango permitido: $0 \leq \text{edad} \leq 120$ **Casos sugeridos:** -1, 0, 1, 119, 120 y 121

Figura 11.3. Selección de valores alrededor de un límite. Elaboración propia.

11.6 Probar alrededor de los límites

Cuando una regla cambia en un punto, conviene probar el valor inmediatamente anterior, el valor límite y el inmediatamente posterior. Esta estrategia revela comparaciones incorrectas como usar `<` cuando se necesitaba `<=`.

Clasificación con un posible error de frontera

```

si edad > 18
    escribir "Acceso permitido"
sino
    escribir "Acceso restringido"
Si la regla permite acceso desde los 18 años, el operador correcto debe incluir el límite.

```

Edad	Esperado según la regla	Resultado del código	Conclusión
17	Restringido	Restringido	Correcto
18	Permitido	Restringido	Falla de frontera
19	Permitido	Permitido	Correcto

Pausa y piensa: para un arreglo de longitud 5, selecciona pruebas para el primer índice, el último índice válido y el primer índice inválido.

11.7 Probar caminos de decisión

Una estructura condicional crea caminos alternativos. Probar únicamente un camino deja los demás sin observar. La cobertura de ramas consiste, en este nivel, en diseñar al menos un caso que haga verdadera y otro que haga falsa cada decisión relevante.

Clasificación de saldo

```

si saldo < 0
    escribir "Deuda"
sino si saldo = 0
    escribir "Sin saldo"
sino
    escribir "Disponible"

```

Caso	Saldo	Rama esperada
R-01	-1	Deuda
R-02	0	Sin saldo
R-03	1	Disponible

No basta con contar líneas ejecutadas. Lo importante es que cada comportamiento definido por la especificación haya sido observado al menos una vez.

Advertencia: cubrir todas las ramas no garantiza que todas las expresiones, combinaciones y estados hayan sido comprobados. Es un criterio útil, no una prueba total.

11.8 Probar ciclos y colecciones

Los ciclos requieren casos que revelen su inicio, progreso y terminación. Para un recorrido de arreglo, una colección vacía, una de un elemento y otra con varios elementos producen evidencias distintas.

Situación	Qué permite observar
Arreglo vacío	Si el ciclo evita accesos inexistentes y si el resultado inicial tiene sentido.
Un elemento	Si los límites incluyen exactamente la única posición.
Varios elementos	Si la actualización se repite y acumula correctamente.
Elemento buscado al inicio	Si la búsqueda reconoce la primera posición.
Elemento buscado al final	Si el último índice se procesa.
Elemento ausente	Si la terminación y el resultado de ausencia son correctos.

Máximo con una inicialización defectuosa

```
MAXIMO(valores[])
  mayor <- 0
  para i <- 0 hasta LONG(valores) - 1
    si valores[i] > mayor
      mayor <- valores[i]
  retornar mayor
```

Con [-8, -3, -10], la función retorna 0, aunque 0 no pertenece al arreglo. El caso con todos los valores negativos revela el defecto.

Una corrección posible consiste en inicializar `mayor` con el primer elemento y recorrer desde el índice 1, después de validar que el arreglo no esté vacío.

11.9 Cuatro familias de errores

No todos los errores aparecen en el mismo momento



Un mensaje de diagnóstico puede señalar la ubicación, pero comprender la causa requiere relacionarlo con la intención del programador

Figura 11.4. Momento de aparición de distintos errores. Elaboración propia.

Clasificar el error orienta la investigación. Los errores de sintaxis y muchos errores semánticos pueden detectarse antes de ejecutar. Los errores de ejecución aparecen durante un recorrido concreto. Los errores lógicos requieren comparar el comportamiento con la especificación.

Tipo	Ejemplo conceptual	Pregunta útil
Sintaxis	Falta un paréntesis o la indentación no forma el bloque esperado.	¿Qué estructura esperaba el analizador?
Semántico	Se invoca una función inexistente o se usa una referencia no válida.	¿Qué nombre o relación no puede resolverse?
Ejecución	Una función de biblioteca o <code>`LANZAR_ERROR`</code> detiene el programa.	¿Qué estado y entrada condujeron a la falla?
Lógico	El promedio divide entre cuatro cuando existen tres notas.	¿En qué difiere el observado del esperado?

11.10 Leer diagnósticos sin obedecerlos mecánicamente

CLRS Kit presenta diagnósticos estáticos en línea, mensajes en español y entradas en el panel Problemas. Cuando la ubicación está disponible, el mensaje señala línea y columna. Esta información reduce la región que debe inspeccionarse, pero no reemplaza la interpretación.

Ejemplo de diagnóstico

× Error de sintaxis

Se esperaba «)», pero se encontró «escribir».
Línea 2, columna 5

Estado: No ejecutado

1. Identifica la familia del error.
2. Lee qué se esperaba y qué se encontró.
3. Revisa la ubicación indicada y la instrucción anterior.
4. Relaciona el mensaje con la estructura completa, no solo con el carácter marcado.
5. Corrige una causa probable y vuelve a ejecutar.

Advertencia: el lugar señalado puede ser donde el analizador descubre la inconsistencia, no necesariamente donde comenzó. Un paréntesis faltante en una línea anterior puede hacer que el error aparezca después.

Nota técnica: los detalles internos se ocultan en la salida normal. El modo `CLRS\_DEBUG=1` está destinado al desarrollo de la extensión, no a la resolución ordinaria de ejercicios.

11.11 Diagnósticos estáticos y estado del editor

Los diagnósticos estáticos se actualizan mientras se edita. Esto permite detectar referencias, problemas estructurales o errores léxicos antes de intentar ejecutar. Sin embargo, un editor sin advertencias no implica que el resultado sea correcto.

Señal	Interpretación adecuada
Subrayado y entrada en Problemas	Existe un problema detectado por el análisis disponible.
Sin diagnósticos	No se detectó un problema estático conocido.
Error al ejecutar	El recorrido concreto produjo una falla.
Salida inesperada sin error	Probable defecto lógico o especificación incompleta.

La ausencia de diagnósticos debe dar paso a pruebas, no a una confianza automática. El analizador no intenta inferir todos los tipos ni todos los valores que existirán durante la ejecución.

Idea clave: las herramientas responden preguntas diferentes: el diagnóstico pregunta si detecta una inconsistencia; la prueba pregunta si el comportamiento satisface una expectativa.

11.12 Depurar mediante hipótesis

Depurar es un ciclo de investigación, no una secuencia de cambios al azar

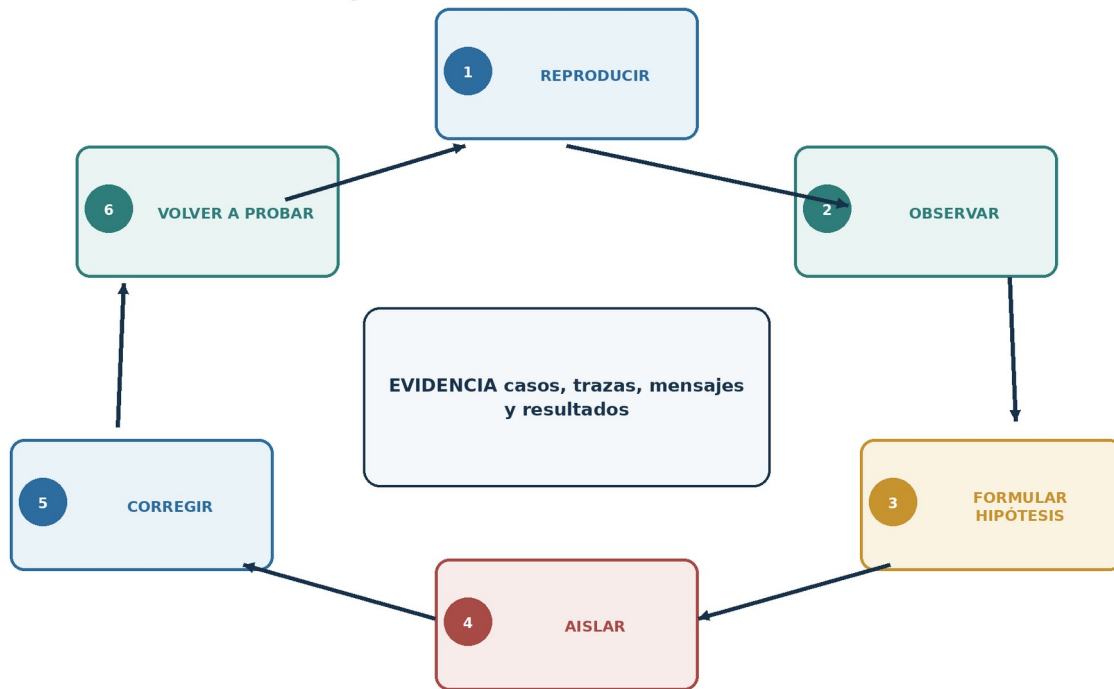


Figura 11.5. Ciclo de depuración basado en evidencia. Elaboración propia.

Cambiar varias líneas al azar puede ocultar la causa original e introducir defectos nuevos. Una depuración disciplinada comienza con un caso reproducible y una diferencia concreta entre lo esperado y lo observado.

Paso	Acción
Reproducir	Conservar una entrada que provoque siempre el problema.
Observar	Registrar salida, estado, mensaje y punto del recorrido.
Hipótesis	Proponer una causa que explique la evidencia.
Aislar	Reducir el programa, la entrada o la función implicada.
Corregir	Aplicar el cambio mínimo coherente con la causa.
Volver a probar	Ejecutar el caso original y los casos relacionados.

11.13 Usar trazas para localizar el primer estado incorrecto

Una salida final equivocada suele originarse varias instrucciones antes. La traza permite comparar estados esperados y observados hasta localizar la primera divergencia.

Promedio ponderado con una regla equivocada

```
total <- 0
total <- total + nota1 * 0.10
total <- total + nota2 * 0.25
total <- total + nota3 * 0.40
total <- total + nota4 * 0.15
escribir total
```

Los pesos suman 0.90. La tercera ponderación debía ser 0.50.

Paso	Valor esperado de total	Valor observado	¿Coinciden?
Inicial	0	0	Sí
Nota 1	0.8	0.8	Sí
Nota 2	3.05	3.05	Sí
Nota 3	7.55	6.65	No
Nota 4	8.75	7.85	No

La primera diferencia aparece al procesar la tercera nota. Esta observación reduce la investigación a una regla concreta en lugar de revisar todo el programa.

Método práctico: detente en la primera divergencia. Los errores posteriores pueden ser consecuencias del mismo defecto.

11.14 Aislar funciones antes de revisar el programa completo

Las funciones permiten probar responsabilidades de manera independiente. Si el total de una compra es incorrecto, puede comprobarse por separado el subtotal, el descuento, el impuesto y la combinación final.

Pruebas directas

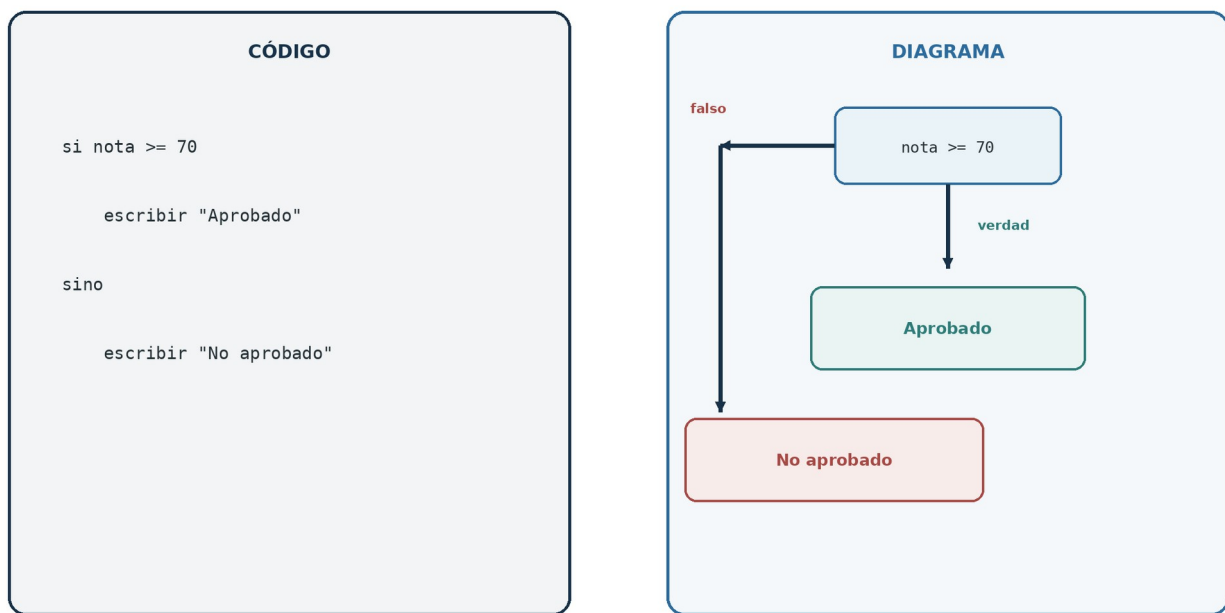
```
PRINCIPAL()
  escribir CALCULAR_SUBTOTAL(100, 3)           // esperado: 300
  escribir CALCULAR_DESCUENTO(300, 0.10)       // esperado: 30
  escribir CALCULAR_IMPUESTO(270, 0.16)        // esperado: 43.2
  escribir CALCULAR_TOTAL(300, 30, 43.2)       // esperado: 313.2
```

Cuando las funciones intermedias producen lo esperado, la causa puede encontrarse en la coordinación, en el orden de los argumentos o en el uso de los retornos. Cuando una función aislada falla, el área de búsqueda se reduce.

Idea clave: la descomposición funcional no solo organiza el código; también crea unidades que pueden especificarse y probarse por separado.

11.15 El diagrama de flujo como instrumento de revisión

Código y diagrama muestran el mismo recorrido desde perspectivas distintas



El diagrama ayuda a revisar caminos; el código conserva las expresiones exactas y los detalles de implementación.

Figura 11.6. Correspondencia entre código y diagrama de flujo. Elaboración propia.

El comando para mostrar el diagrama de flujo genera una representación visual del programa y de sus funciones. La visualización ayuda a revisar decisiones, ciclos y llamadas que pueden resultar difíciles de seguir en un bloque extenso.

- Comprueba que cada condición tenga los caminos previstos.
- Localiza ciclos cuyo regreso no conduce a una actualización visible.
- Revisa si una función queda aislada porque nunca se invoca.
- Compara el orden general del diagrama con la especificación.
- Exporta el SVG cuando necesites documentar la revisión.

Límite de la figura: el diagrama muestra la estructura del recorrido, pero no demuestra que una fórmula, un límite o una condición sean correctos. Debe leerse junto con casos de prueba y trazas.

11.16 Revisar caminos imposibles o faltantes

Una cadena `sino si` se evalúa en orden. Una condición demasiado amplia puede absorber casos que deberían llegar a una rama posterior.

Orden defectuoso de condiciones

```
si calificacion >= 60
  escribir "Aprobado"
sino si calificacion >= 90
  escribir "Excelente"
sino
  escribir "No aprobado"
```

La rama "Excelente" nunca se alcanza: todo valor mayor o igual que 90 ya satisface la primera condición.

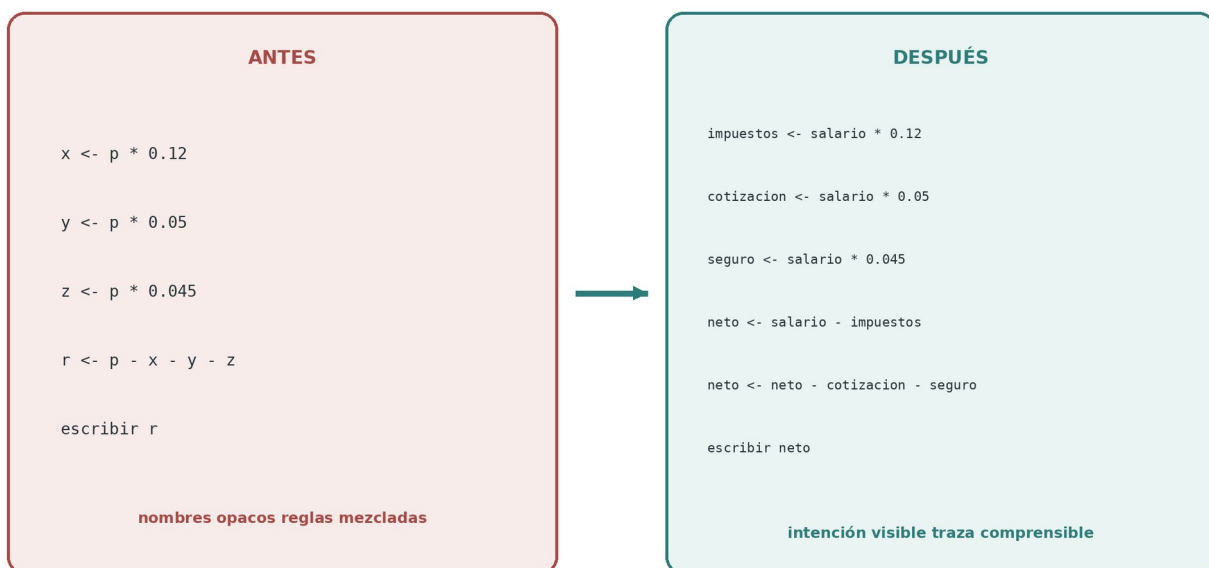
El diagrama permite observar el orden, pero una tabla de casos muestra la consecuencia: para 95, el resultado esperado "Excelente" se convierte en "Aprobado". La corrección consiste en ordenar primero la condición más específica.

Pausa y piensa: diseña tres calificaciones que recorran todas las ramas después de corregir el orden.

11.17 Calidad: hacer visible la intención

La calidad del código no sustituye las pruebas. Sin embargo, un programa legible permite construir mejores hipótesis, seguir estados y reconocer reglas. La claridad reduce el costo humano de comprobar y modificar.

Mejorar calidad reduce la dificultad de comprobar y modificar



La calidad no garantiza corrección, pero hace más probable descubrir y corregir errores.

Figura 11.7. Refactorización orientada a la comprensión. Elaboración propia.

Práctica	Beneficio para la comprobación
Nombres significativos	Permiten relacionar variables y funciones con la especificación.
Funciones pequeñas y cohesivas	Facilitan pruebas aisladas y localización de causas.
Evitar duplicación	Impide que una regla tenga versiones inconsistentes.
Condiciones simples	Hace visibles los caminos y los límites.
Comentarios útiles	Explican decisiones o restricciones no evidentes.
Eliminar código innecesario	Reduce estados y caminos que deben analizarse.

11.18 Comentarios que ayudan y comentarios que estorban

Un comentario útil explica una decisión, una precondition, una limitación o una razón que no puede expresarse claramente con nombres y estructura. Un comentario que repite la instrucción añade ruido y puede quedar desactualizado.

Comentario	Evaluación
<code>`// suma uno a i` junto a `i <- i + 1`</code>	Repite lo visible; aporta poco.
<code>`// -1 termina la captura y no se registra como gasto`</code>	Explica el significado del centinela.
<code>`// usar 0.50 porque la tercera nota vale 50 %`</code>	Documenta una regla del problema.
<code>`// corregido`</code>	No explica qué cambió ni por qué.

Criterio: prefiere que el código diga qué hace y reserva el comentario para explicar por qué se tomó una decisión.

11.19 Simplificar condiciones sin cambiar su significado

Las condiciones duplicadas o contradictorias dificultan la cobertura de ramas. Antes de simplificar, utiliza una tabla de casos para asegurar que el comportamiento permanezca igual.

Condición redundante
<pre>si edad >= 18 y edad > 15 escribir "Mayor de edad"</pre> Si edad >= 18, entonces edad > 15 ya se cumple. La segunda comparación no añade un caso nuevo.

Versión más clara

```
si edad >= 18
    escribir "Mayor de edad"
```

Después de refactorizar, deben volver a ejecutarse los casos de frontera 17, 18 y 19. Una mejora estructural también puede introducir errores y necesita verificación.

11.20 Volver a probar después de una corrección

Corregir el caso que reveló un defecto es necesario, pero insuficiente. El cambio puede afectar comportamientos que antes eran correctos. Las pruebas de regresión repiten casos anteriores para comprobar que la solución no retrocedió.

Después de corregir...	Vuelve a probar...
Un límite de edad	Valores por debajo, en y por encima del límite.
La inicialización del máximo	Arreglos positivos, negativos, de un elemento y vacíos.
El orden de una cadena `sino si`	Un caso para cada rama.
Una fórmula de descuento	Porcentaje cero, normal y máximo.
Una función compartida	Todos los programas o funciones que la invocan.

Idea clave: cada defecto descubierto debe convertirse en un caso que pueda repetirse. Así, el error deja de ser solo un incidente y se transforma en conocimiento comprobable.

11.21 Laboratorio CLRS: comprobar el registro de gastos

El registro de gastos ya contiene validación, ciclos, funciones, arreglos y texto. Ahora se convertirá en un objeto de prueba.

Regla	Caso	Resultado esperado
Un gasto debe ser positivo	-20	Rechazo con mensaje.
Un gasto no debe superar el saldo	saldo 100, gasto 120	Rechazo; saldo permanece 100.
El centinela no se registra	-1	Termina sin modificar arreglos.
Un gasto válido reduce el saldo	saldo 100, gasto 40	Saldo 60 y movimiento agregado.
El total coincide con los gastos	20, 30, 10	Total 60.
La categoría se normaliza	" alimentos "	Categoría comparable con "ALIMENTOS".

Función sometida a prueba

```
ES_GASTO_VALIDO(gasto, saldo)
    retornar ES_NUM(gasto) y gasto > 0 y gasto <= saldo
```

Prueba primero la función de validación con valores concretos. Después prueba la coordinación: confirma que el programa no modifique el saldo ni agregue elementos cuando la validación sea falsa.

Laboratorio: construye una tabla con entrada, saldo inicial, longitud inicial del arreglo, resultado esperado, saldo observado y longitud observada. Ejecuta al menos seis casos.

11.22 Investigación guiada de un defecto lógico

Programa con una falla

```
PROMEDIO(notas[])
    total <- 0
    para i <- 0 hasta LONG(notas) - 1
        total <- total + notas[i]
    retornar total / 4

PRINCIPAL()
    notas[0] <- 8
    notas[1] <- 9
    notas[2] <- 10
    escribir PROMEDIO(notas)
```

6. Especifica el resultado esperado.
7. Ejecuta y registra el resultado observado.
8. Traza `total` durante el ciclo.
9. Localiza la primera operación que no coincide con la regla.
10. Formula la corrección mínima.
11. Agrega casos con uno, tres y cinco elementos.
12. Refactoriza para dividir entre `LONG(notas)`.

Conclusión esperada: el ciclo acumula correctamente; el defecto se encuentra en una suposición fija sobre la longitud. La colección debe determinar el divisor.

11.23 Lista de revisión antes de considerar terminado un programa

Área	Preguntas
Especificación	¿Entradas, salidas, reglas y restricciones están claras?
Casos	¿Existen pruebas normales, límite, inválidas y especiales?

Área	Preguntas
Decisiones	¿Cada rama relevante se ejecutó?
Ciclos	¿Se probaron cero, una y varias iteraciones cuando aplica?
Funciones	¿Se probaron de forma aislada y con sus precondiciones?
Diagnósticos	¿El panel Problemas está limpio y los mensajes fueron comprendidos?
Resultados	¿Se compararon esperado y observado?
Regresión	¿Se repitieron los casos anteriores después de cambios?
Calidad	¿Los nombres, funciones y condiciones expresan intención?
Documentación	¿El diagrama y los comentarios explican decisiones reales?

11.24 Reconstruye el modelo mental

Comprobar un programa comienza antes de ejecutarlo. La especificación define lo esperado; los casos de prueba seleccionan situaciones; la ejecución produce evidencia; el diagnóstico y la traza ayudan a localizar causas; la corrección se confirma mediante nuevas pruebas y regresión; y la calidad del código reduce la dificultad de todo el proceso.

13. Ejecutarse sin errores no demuestra corrección.
14. Una prueba necesita una expectativa derivada de la especificación.
15. Los casos normales, límite e inválidos revelan comportamientos distintos.
16. Las decisiones requieren casos para sus ramas.
17. Los ciclos requieren pruebas de inicio, progreso y terminación.
18. Los errores sintácticos, semánticos, de ejecución y lógicos aparecen de formas diferentes.
19. Un diagnóstico orienta, pero no explica automáticamente la causa.
20. La depuración debe apoyarse en evidencia e hipótesis.
21. Las trazas localizan la primera divergencia del estado.
22. Los diagramas ayudan a revisar recorridos, no fórmulas.
23. La calidad facilita comprobar, modificar y explicar.
24. Después de corregir, deben repetirse casos de regresión.

Comprobación activa: explica cómo investigarías un programa que no muestra errores, pero produce 6.75 cuando esperabas 9.

Ejercicios

Realiza primero los ejercicios de especificación y predicción sin ejecutar.

1. Especifica entradas, salida, reglas y restricciones de un conversor Celsius-Fahrenheit. [\[Especificar\]](#)
2. Diseña tres casos normales y cuatro casos límite para una nota permitida entre 0 y 10. [\[Diseñar\]](#)
3. Completa una tabla de prueba para `CALCULAR_AREA_CIRCULO(radio)`. [\[Aplicar\]](#)

4. Explica por qué una entrada aislada no constituye un caso de prueba completo. [Analizar]
5. Diseña casos que cubran las ramas de una clasificación negativa, cero y positiva. [Diseñar]
6. Selecciona pruebas para un arreglo vacío, de un elemento y de cinco elementos. [Diseñar]
7. Identifica un error de frontera causado por usar `<` en lugar de `<=`. [Analizar]
8. Clasifica cuatro ejemplos como sintácticos, semánticos, de ejecución o lógicos. [Reconocer]
9. Corrige un programa con un paréntesis faltante y explica por qué el diagnóstico aparece en la línea siguiente. [Diagnosticar]
10. Construye una traza para una suma acumulada que omita el último elemento. [Trazar]
11. Prueba `MAXIMO` con todos los valores negativos y explica la falla de inicialización. [Analizar]
12. Aísla y prueba las funciones de una calculadora de compra. [Probar]
13. Diseña una prueba de regresión para un defecto de descuento ya corregido. [Diseñar]
14. Genera un diagrama de flujo e identifica una rama que nunca se alcanza. [Representar]
15. Refactoriza variables `x`, `y`, `z` y `r` para hacer visible una fórmula de salario. [Mejorar]
16. Elimina una condición redundante y demuestra con casos que el comportamiento no cambia. [Mejorar]
17. Escribe un comentario útil para un centinela y otro que deba eliminarse. [Evaluar]
18. Construye una lista de revisión para una búsqueda lineal. [Integrar]
19. Investiga un promedio que divide siempre entre cuatro, aunque el arreglo tenga otra longitud. [Depurar]
20. Documenta un defecto real encontrado en tus programas: entrada, esperado, observado, causa y corrección. [Integrar]

Mini proyecto: expediente de comprobación

Situación: elige uno de los programas desarrollados en capítulos anteriores y construye un expediente que permita a otra persona comprobarlo sin preguntarte qué debería ocurrir.

El expediente debe incluir:

- Descripción precisa del problema.
- Entradas, salidas, reglas, restricciones y casos especiales.
- Tabla con al menos diez casos de prueba.
- Casos normales, límite, inválidos y de cobertura de ramas.
- Resultados observados y veredictos.
- Una traza de un caso significativo.
- Captura o transcripción de un diagnóstico corregido.
- Diagrama de flujo exportado.
- Una refactorización de calidad y sus pruebas de regresión.
- Conclusión sobre limitaciones que permanecen.

Criterio de éxito: otra persona debe poder repetir las pruebas, reconocer qué comportamiento se esperaba y comprender la evidencia utilizada para aceptar o rechazar el programa.

Autoevaluación

Responde sin consultar el capítulo. Después utiliza un programa real para comprobar tus respuestas.

Afirmación	Sí	Todavía no
Puedo distinguir ejecución de corrección.	☐	☐
Puedo escribir una especificación verificable.	☐	☐
Diseño casos normales, límite e inválidos.	☐	☐
Puedo cubrir ramas y casos de ciclos.	☐	☐
Distingo cuatro familias de errores.	☐	☐
Puedo interpretar un diagnóstico sin corregir al azar.	☐	☐
Uso trazas para localizar la primera divergencia.	☐	☐
Puedo probar funciones de forma aislada.	☐	☐
Uso el diagrama como instrumento de revisión.	☐	☐
Vuelvo a probar después de corregir.	☐	☐
Puedo mejorar nombres, condiciones y funciones.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas especificar un programa, diseñar pruebas reproducibles, localizar un defecto mediante evidencia y demostrar que la corrección no dañó casos anteriores.

Glosario

Término	Definición de trabajo
Especificación	Descripción verificable de entradas, salidas, reglas y restricciones.
Precondición	Condición que debe cumplirse antes de usar una operación.
Resultado esperado	Comportamiento que debe observarse para una entrada dada.
Caso de prueba	Entrada, expectativa, resultado observado y veredicto reproducibles.
Caso límite	Prueba situada en un extremo o cambio de comportamiento.
Cobertura de ramas	Evidencia de que los caminos alternativos fueron ejecutados.
Defecto	Causa en el programa que produce un comportamiento incorrecto.
Falla	Manifestación observable de un defecto durante una ejecución.

Término	Definición de trabajo
Diagnóstico	Mensaje producido por una herramienta para señalar un problema detectado.
Depuración	Proceso de reproducir, localizar y corregir la causa de una falla.
Hipótesis	Explicación provisional que debe contrastarse con evidencia.
Regresión	Pérdida de un comportamiento que antes era correcto.
Refactorización	Mejora estructural que pretende conservar el comportamiento.
Traza	Registro ordenado de estados y pasos de ejecución.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación técnica de diagnósticos, ejecución y diagramas de flujo [Repositorio de software].

McConnell, S. (2004). Code complete: A practical handbook of software construction (2nd ed.). Microsoft Press.

Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). John Wiley & Sons.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

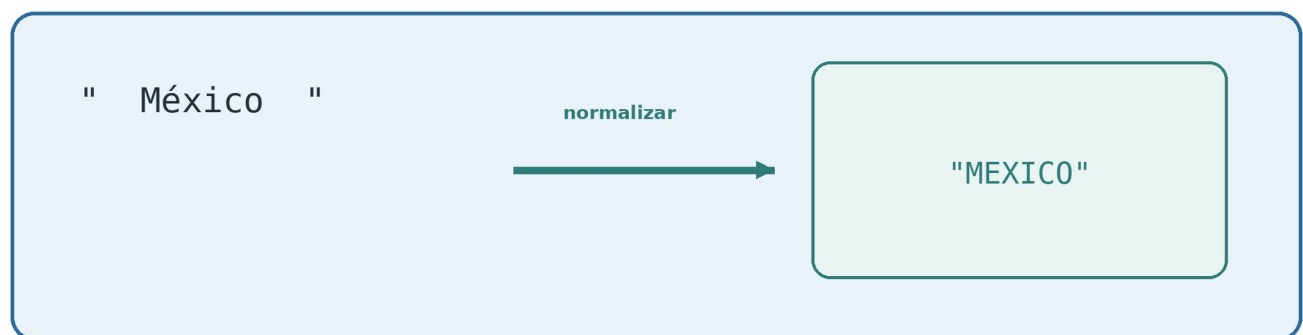
Continuidad: el programa ya puede especificarse, probarse y mejorarse. El capítulo 12 integrará la biblioteca estándar, la persistencia básica y un proyecto completo que reúna todo el recorrido del libro.

CAPÍTULO

10

Tratar el texto como información procesable

Caracteres, normalización, búsqueda, división y análisis



Tratar el texto como información procesable

¿Cómo deja una cadena de ser un bloque indivisible y se convierte en una secuencia que puede analizarse?

Para una persona, “ México ”, “MÉXICO” y “méxico” pueden expresar el mismo nombre. Para un programa, las diferencias de mayúsculas, espacios y caracteres forman parte de la representación. Si se comparan sin preparación, pueden considerarse valores distintos.

Procesar texto exige observar dos niveles a la vez. Una cadena tiene un significado completo —un nombre, una frase o un registro—, pero también está formada por caracteres y fragmentos que pueden recorrerse, buscarse, separarse y reorganizarse.

Promesa de aprendizaje: al terminar podrás tratar cadenas como secuencias, acceder a caracteres, normalizar texto, extraer subcadenas, buscar y reemplazar fragmentos, dividir y unir cadenas, y construir algoritmos elementales de análisis textual.

Procesar texto suele requerir una cadena de transformaciones

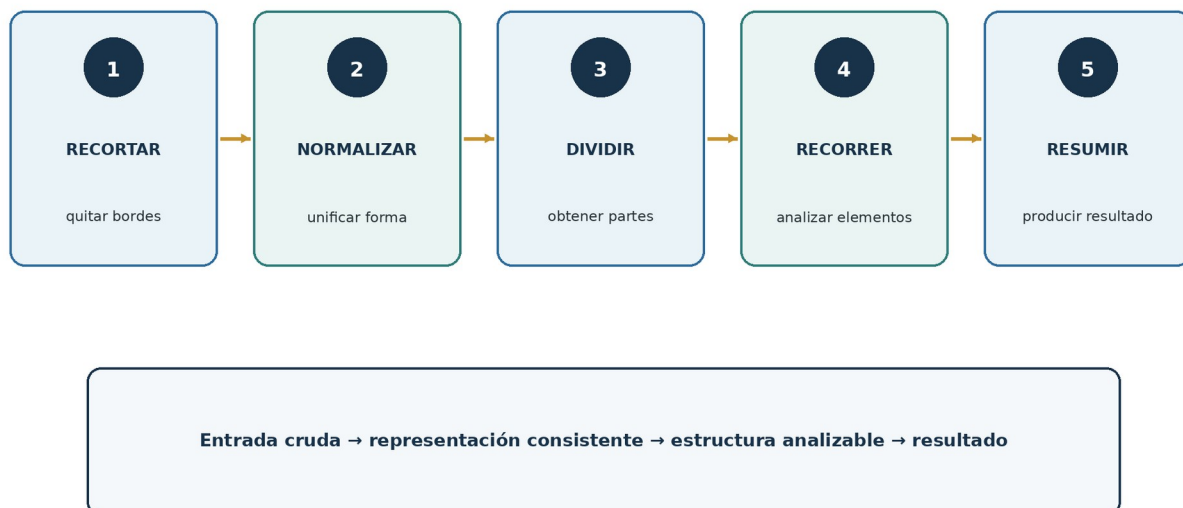


Figura 10.1. Cadena general de procesamiento de texto. Elaboración propia.

Conexión: el capítulo anterior introdujo arreglos. Ahora veremos que muchas operaciones textuales producen arreglos o necesitan recorrerlos.

10.1 Una cadena es un valor completo y una secuencia

Una cadena puede utilizarse como un valor completo: asignarse, compararse, escribirse o enviarse a una función. También puede observarse como una secuencia ordenada de caracteres. Esta segunda perspectiva permite analizar su estructura interna.

Una cadena puede observarse como una secuencia de caracteres

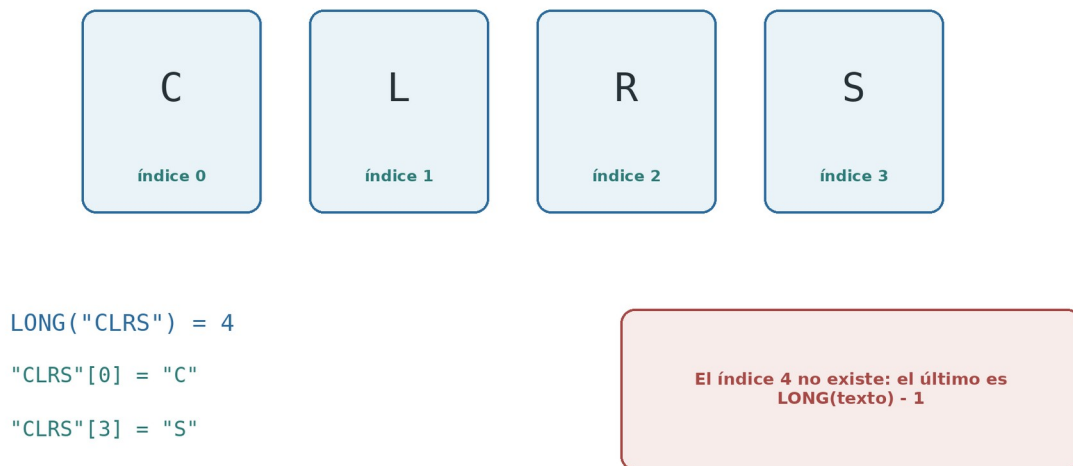


Figura 10.2. Cadena como secuencia indexada. Elaboración propia.

Acceder a caracteres

```
texto <- "CLRS"
escribir LONG(texto)
escribir texto[0]
escribir texto[LONG(texto) - 1]
```

El primer carácter ocupa el índice 0. El último ocupa $LONG(\text{texto}) - 1$. La misma relación estudiada para arreglos se reutiliza aquí.

Advertencia: una cadena vacía tiene longitud 0; por tanto, no posee un primer carácter. Antes de acceder a `texto[0]`, comprueba que exista al menos una posición.

10.2 Longitud, índices y límites

La longitud responde cuántos caracteres contiene la cadena. No responde cuál es el último índice. Si la longitud es 5, las posiciones válidas son 0, 1, 2, 3 y 4.

Expresión	Resultado	Interpretación
<code>`LONG("casa")`</code>	4	Cantidad de caracteres.
<code>`"casa"[0]`</code>	<code>`"c"`</code>	Primer carácter.
<code>`"casa"[3]`</code>	<code>`"a"`</code>	Último carácter.
<code>`"casa"[4]`</code>	Inválido	Está fuera de los límites.

Pausa y piensa: para la cadena ``"algoritmo"``, escribe la longitud, el último índice y los caracteres almacenados en las posiciones 0, 4 y 8.

10.3 Recorrer caracteres

Un recorrido visita los caracteres en orden y permite aplicar una misma pregunta o transformación a cada posición.

Mostrar cada carácter

```
texto <- "CLRS"

para i <- 0 hasta LONG(texto) - 1
  escribir texto[i]
```

La variable ``i`` representa la posición actual. El acceso ``texto[i]`` recupera el carácter asociado con esa posición.

Contar vocales

```
CONTAR_VOCALES(texto)
  cantidad <- 0
  texto <- MAYUS(texto)

  para i <- 0 hasta LONG(texto) - 1
    caracter <- texto[i]

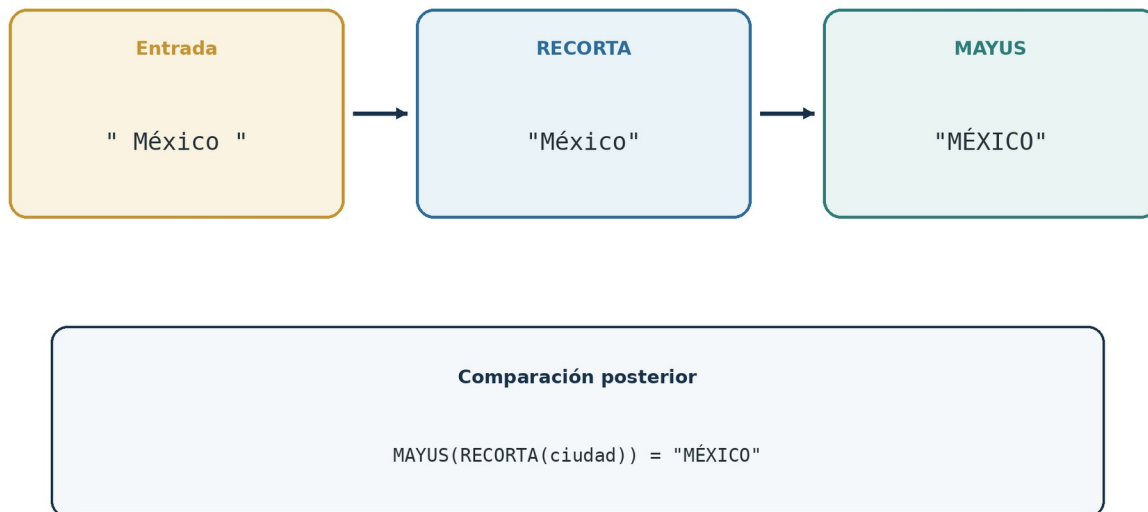
    si caracter = "A" o caracter = "E" o caracter = "I" o caracter = "O" o caracter =
"U"
      cantidad <- cantidad + 1

  retornar cantidad
```

El texto se normaliza primero a mayúsculas. Así se evita duplicar cada comparación para versiones mayúsculas y minúsculas.

10.4 Normalizar antes de comparar

Normalizar significa convertir distintas representaciones aceptables en una forma común. La normalización no corrige cualquier entrada; reduce únicamente diferencias que el problema considera irrelevantes.

Normalizar reduce diferencias que no son relevantes para el problema

La normalización elegida depende del significado: no siempre deben eliminarse acentos, símbolos o espacios internos.

Figura 10.3. Normalización de espacios y mayúsculas. Elaboración propia.

Comparación normalizada

```

ciudad <- " México "
ciudad <- RECORTA(ciudad)
ciudad <- MAYUS(ciudad)

si ciudad = "MÉXICO"
    escribir "Coincidencia"

```

Operación	Propósito	Precaución
`RECORTA`	Eliminar espacios al inicio y al final.	No elimina necesariamente espacios internos.
`MAYUS`	Unificar letras en mayúsculas.	Puede alterar la presentación original.
`MINUS`	Unificar letras en minúsculas.	Tiene el mismo propósito comparativo.

Importante: conserva el texto original cuando su forma sea relevante para mostrarlo después. Puedes crear una variable normalizada separada.

10.5 Comparación exacta y comparación preparada

Una comparación exacta pregunta si dos representaciones coinciden carácter por carácter. Una comparación preparada transforma primero ambos valores según las reglas del problema.

Dos estrategias

```
original <- " Ana "
escribir original = "ana"

normalizado <- MINUS(RECORTA(original))
escribir normalizado = "ana"
```

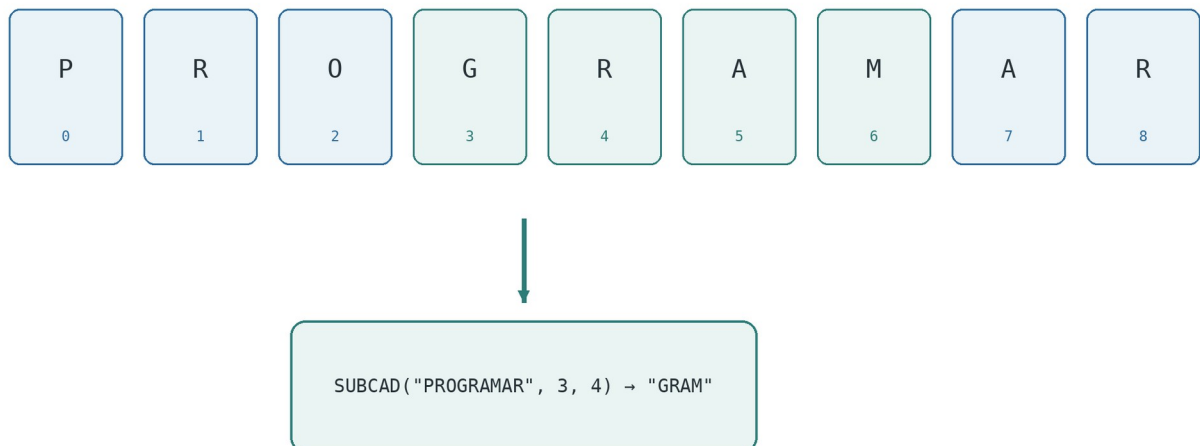
La primera comparación puede producir falso porque conserva espacios y mayúscula inicial. La segunda expresa una regla más cercana a “ignorar espacios exteriores y diferencia entre mayúsculas y minúsculas”.

Contraejemplo: normalizar números de identificación, contraseñas o códigos sensibles a mayúsculas puede destruir información válida. La normalización depende del dominio.

10.6 Extraer subcadenas

Una subcadena es una región continua de otra cadena. Puede representar un prefijo, un código interno, una fecha, una extensión o cualquier fragmento delimitado por posiciones.

Una subcadena selecciona una región continua del texto



Antes de extraer, identifica el índice inicial y la cantidad o límite que utiliza la función.

Figura 10.4. Selección de una subcadena. Elaboración propia.

Extraer una región

```
palabra <- "PROGRAMAR"
fragmento <- SUBCAD(palabra, 3, 4)
escribir fragmento
```

Antes de utilizar una función de subcadena, verifica cómo interpreta sus parámetros: posición inicial, longitud o posición final. El capítulo usa la convención documentada por la biblioteca de CLRS Kit.

Advertencia: una posición inicial negativa o un fragmento que excede la longitud disponible debe tratarse como un caso inválido o limitarse de manera explícita.

10.7 Prefijos, sufijos y formatos

Los prefijos aparecen al comienzo y los sufijos al final. Muchas validaciones sencillas pueden expresarse mediante estas regiones.

Obtener una extensión

```
archivo <- "reporte.txt"
extension <- SUBCAD(archivo, LONG(archivo) - 3, 4)

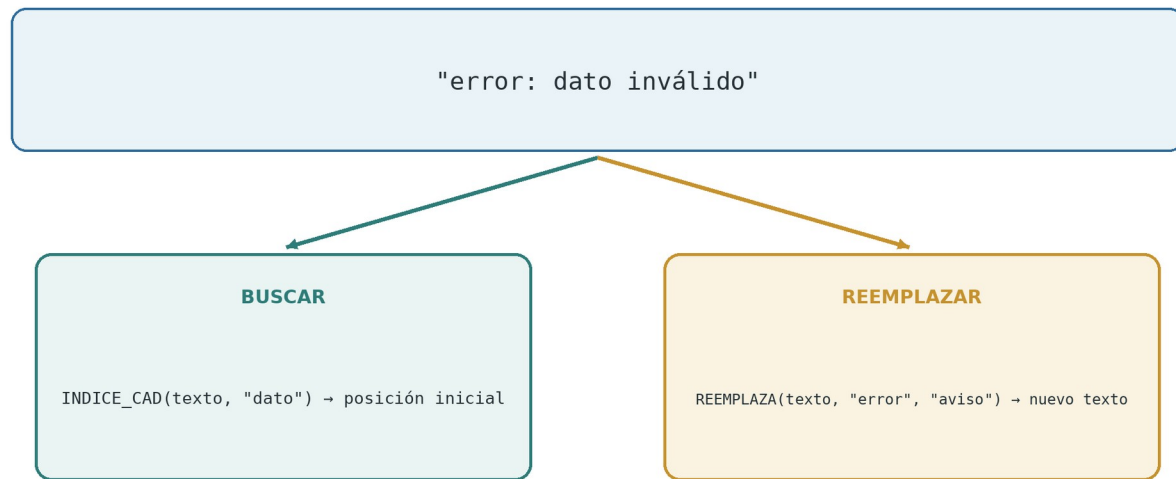
si MINUS(extension) = ".txt"
  escribir "Archivo de texto"
```

Esta solución funciona únicamente si el formato garantiza una extensión de cuatro caracteres. Para nombres generales conviene localizar primero el último punto o emplear una función específica de búsqueda.

Caso límite: ``archivo`` no contiene punto; ``datos.tar.gz`` contiene más de uno. El algoritmo debe decidir qué significado se asigna a cada caso.

10.8 Buscar texto dentro de texto

Buscar una subcadena permite saber si aparece un fragmento y, cuando la función lo permite, en qué posición comienza.

Buscar localiza; reemplazar produce una nueva representación

La cadena original puede conservarse; asigna el resultado cuando quieras actualizar el estado.

Figura 10.5. Buscar y reemplazar fragmentos. Elaboración propia.

Buscar una palabra

```

mensaje <- "error: dato inválido"
posicion <- INDICE_CAD(mensaje, "dato")

si posicion >= 0
  escribir "La palabra comienza en: " + posicion
  
```

Cuando no existe coincidencia, las funciones de búsqueda suelen devolver un valor especial. Confirma el valor utilizado por la biblioteca antes de construir la condición.

Idea clave: buscar responde una pregunta sobre la cadena; no modifica su contenido.

10.9 Reemplazar fragmentos

Reemplazar toma una cadena, localiza un patrón y produce una representación con otro fragmento en su lugar.

Reemplazo y asignación

```

mensaje <- "error: dato inválido"
corregido <- REEMPLAZA(mensaje, "error", "aviso")

escribir mensaje
escribir corregido
  
```

La cadena original puede permanecer sin cambios. Para actualizar el estado de `mensaje`, asigna el resultado a la misma variable: `mensaje <- REEMPLAZA(...)`.

Advertencia: reemplazar todas las coincidencias puede modificar fragmentos dentro de palabras más largas. Define con precisión qué patrón buscas.

10.10 Dividir una cadena en partes

Dividir transforma una cadena estructurada en un arreglo de campos. El separador indica dónde termina una parte y comienza la siguiente.

Dividir transforma texto en arreglo; unir realiza el recorrido inverso

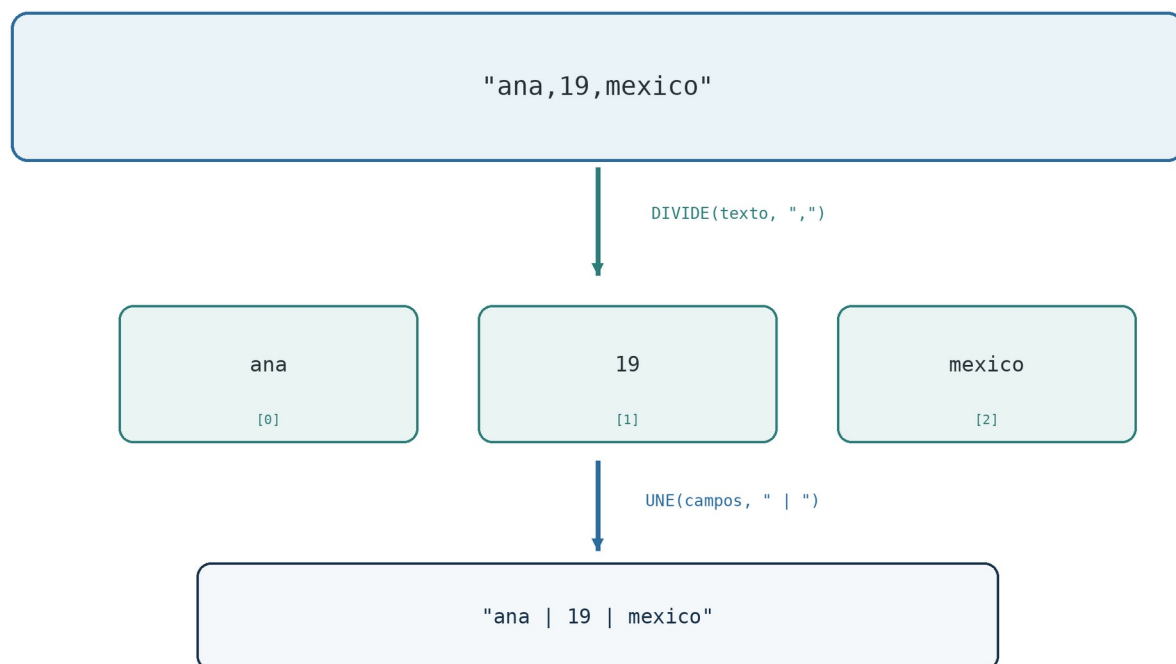


Figura 10.6. División y unión de cadenas. Elaboración propia.

Separar campos

```
linea <- "ana,19,mexico"
campos <- DIVIDE(linea, ",")

escribir campos[0]
escribir campos[1]
escribir campos[2]
```

Después de dividir, el programa debe validar cuántos campos obtuvo. Una línea incompleta o un separador dentro de un valor puede romper la interpretación esperada.

Importante: dividir no comprende por sí mismo el significado de cada campo. Solo produce partes según un separador. El programa debe interpretarlas y convertirlas cuando sea necesario.

10.11 Unir elementos en una cadena

Unir realiza la transformación inversa: toma un arreglo de cadenas y coloca un separador entre sus elementos.

Construir una línea

```
campos[0] <- "ana"
campos[1] <- "19"
campos[2] <- "mexico"

línea <- UNE(campos, ",")
escribir línea
```

La unión es útil para presentar listas, construir registros simples o preparar texto para almacenamiento. No agrega automáticamente comillas, escapes ni reglas de formatos complejos.

10.12 Texto y conversión de tipos

Los datos obtenidos al dividir una cadena siguen siendo texto. Si un campo representa un número, debe comprobarse y convertirse antes de utilizarse en cálculos.

Interpretar un campo numérico

```
línea <- "producto,3,49.90"
campos <- DIVIDE(línea, ",")

si ES_NUM(campos[1]) y ES_NUM(campos[2])
  cantidad <- A_NUM(campos[1])
  precio <- A_NUM(campos[2])
  total <- cantidad * precio
  escribir total
sino
  escribir "Registro inválido"
```

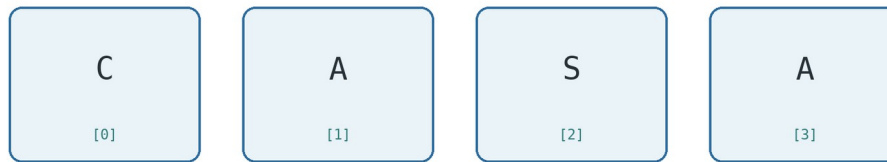
Idea clave: la estructura textual y el tipo conceptual no son lo mismo. ``3`` representa caracteres hasta que se convierte en un valor numérico.

10.13 Contar frecuencias

Un análisis de frecuencia cuenta cuántas veces aparece cada carácter o palabra. El problema combina recorrido, comparación, contadores y colecciones.

Contar frecuencias combina texto, arreglos y recorridos

```
texto = "CASA"
```



carácter	frecuencia
A	2
C	1
S	1

Cada carácter se convierte en una categoría; el conteo actualiza el estado asociado con esa categoría.

Figura 10.7. Frecuencias de caracteres. Elaboración propia.

Frecuencia de una letra

```
CONTAR_CARACTER(texto, objetivo)
  cantidad <- 0
  texto <- MAYUS(texto)
  objetivo <- MAYUS(objetivo)

  para i <- 0 hasta LONG(texto) - 1
    si texto[i] = objetivo
      cantidad <- cantidad + 1

  retornar cantidad
```

Contar todos los caracteres distintos requiere además una estructura para recordar cuáles ya fueron procesados y su frecuencia. Ese problema se retomará en ejercicios y en cursos posteriores con estructuras especializadas.

10.14 Buscar palabras y contar apariciones

Para analizar palabras, primero se normaliza y divide el texto. Después se recorre el arreglo resultante.

Contar una palabra

```

CONTAR_PALABRA(texto, objetivo)
  texto <- MINUS(RECORTA(texto))
  objetivo <- MINUS(RECORTA(objetivo))
  palabras <- DIVIDE(texto, " ")
  cantidad <- 0

  para i <- 0 hasta LONG(palabras) - 1
    si palabras[i] = objetivo
      cantidad <- cantidad + 1

  retornar cantidad

```

Esta versión es pedagógica y supone palabras separadas por un único espacio. Signos de puntuación, saltos de línea y espacios repetidos requieren una normalización más completa.

Contraejemplo: en ``"casa, casa"`, dividir solo por espacios produce ``"casa,"` y ``"casa"`. La coma impide que ambas partes sean iguales.

10.15 Eliminar duplicados de una lista textual

Eliminar duplicados exige conservar el primer ejemplar de cada valor y omitir los siguientes. La comparación debe realizarse sobre una representación normalizada si el problema considera equivalentes distintas formas.

Idea de eliminación de duplicados

```

SIN_DUPLICADOS(valores[])
  unicos[0] <- valores[0]

  para i <- 1 hasta LONG(valores) - 1
    candidato <- MINUS(RECORTA(valores[i]))
    encontrado <- FALSO

    para j <- 0 hasta LONG(unicos) - 1
      si MINUS(RECORTA(unicos[j])) = candidato
        encontrado <- VERDAD

    si no encontrado
      AGREGA(unicos, valores[i])

  retornar unicos

```

La versión supone al menos un elemento. Una implementación completa debe tratar el arreglo vacío.

La solución utiliza ciclos anidados: por cada candidato revisa los valores ya conservados. Su costo puede crecer cuadráticamente.

10.16 Invertir texto

Invertir una cadena permite practicar recorridos descendentes y construcción progresiva de texto.

Construir una cadena invertida

```

INVERTIR_TEXTO(texto)
  invertido <- ""

  para i <- LONG(texto) - 1 bajando 0
    invertido <- invertido + texto[i]

  retornar invertido

```

Para ``CLRS``, el recorrido visita S, R, L y C. Cada carácter se concatena al resultado parcial.

Pausa y piensa: completa una tabla de estado con las columnas ``i``, ``texto[i]`` e ``invertido`` para la entrada ``SOL``.

10.17 Palíndromos

Un palíndromo se lee igual de izquierda a derecha y de derecha a izquierda. La comparación debe definir primero qué diferencias se ignorarán.

Comprobación mediante inversión

```

ES_PALINDROMO(texto)
  normalizado <- MINUS(RECORTA(texto))
  retornar normalizado = INVERTIR_TEXTO(normalizado)

```

Esta versión no elimina espacios internos ni signos. Por ello reconoce ``radar``, pero no necesariamente ``Anita lava la tina``. Una versión más amplia debe filtrar caracteres antes de comparar.

Importante: la definición operacional forma parte del algoritmo. “Ignorar espacios y puntuación” debe expresarse mediante instrucciones concretas.

10.18 Ordenar texto y colecciones de palabras

Ordenar palabras puede facilitar búsquedas, presentación y detección de duplicados. La biblioteca puede ordenar arreglos, pero el resultado depende de la forma exacta de las cadenas.

Ordenar palabras normalizadas

```

texto <- "Pera manzana uva"
palabras <- DIVIDE(MINUS(texto), " ")
ORDENA(palabras)

escribir UNE(palabras, " ")

```

Normalizar antes de ordenar evita que las mayúsculas introduzcan diferencias inesperadas. Sin embargo, las reglas lingüísticas completas pueden depender del idioma y no reducirse al orden interno de caracteres.

10.19 Laboratorio CLRS: analizador elemental de texto

Construiremos un programa que reciba una frase y produzca longitud, cantidad de palabras, vocales y una versión normalizada.

Analizador de texto

```

CONTAR_VOCALES(texto)
  cantidad <- 0
  texto <- MAYUS(texto)

  para i <- 0 hasta LONG(texto) - 1
    c <- texto[i]
    si c = "A" o c = "E" o c = "I" o c = "O" o c = "U"
      cantidad <- cantidad + 1

  retornar cantidad

PRINCIPAL()
  escribir "Introduce una frase"
  leer frase

  normalizada <- RECORTA(frase)
  palabras <- DIVIDE(normalizada, " ")

  escribir "Caracteres: " + LONG(normalizada)
  escribir "Palabras: " + LONG(palabras)
  escribir "Vocales: " + CONTAR_VOCALES(normalizada)
  escribir "Mayúsculas: " + MAYUS(normalizada)

```

El conteo de palabras funciona correctamente solo bajo la suposición de un espacio entre palabras. Prueba entradas con espacios repetidos y registra el comportamiento antes de proponer una mejora.

Laboratorio: ejecuta el programa con ``"hola mundo"`, `` hola mundo ``, `"hola mundo"`` y una cadena vacía. Construye una tabla de resultados observados.

10.20 Caso longitudinal: categorías y conceptos de gastos

El registro de gastos puede almacenar conceptos y categorías como cadenas. Para generar resúmenes confiables, conviene normalizarlas antes de comparar.

Normalizar una categoría

```

NORMALIZAR_CATEGORIA(categoria)
    retornar MAYUS(RECORTA(categoria))

PRINCIPAL()
    categorias[0] <- " alimentos "
    categorias[1] <- "ALIMENTOS"
    categorias[2] <- "Transporte"

    para i <- 0 hasta LONG(categorias) - 1
        categorias[i] <- NORMALIZAR_CATEGORIA(categorias[i])

    escribir UNE(categorias, ", ")

```

Después de normalizar, las dos primeras categorías pueden tratarse como equivalentes. El siguiente paso sería contar frecuencias o acumular montos por categoría.

Decisión de diseño: guarda por separado la forma original cuando necesites presentarla exactamente como fue escrita por la persona.

10.21 Errores frecuentes al procesar texto

Error	Consecuencia	Revisión
Acceder sin comprobar longitud	Índice inválido en cadenas vacías.	Validar <code>LONG(texto) > 0</code> .
Comparar sin normalizar	Equivalencias humanas aparecen como distintas.	Definir reglas de normalización.
Normalizar de más	Se pierde información significativa.	Conservar original y justificar cambios.
Suponer un separador perfecto	Campos incompletos o desalineados.	Validar cantidad y contenido.
Usar texto numérico en cálculos	Conversión implícita o resultado inesperado.	Comprobar y convertir.
Ignorar puntuación	Conteos de palabras incorrectos.	Limpiar o tokenizar según reglas.
No asignar un reemplazo	El estado original permanece igual.	Guardar el valor producido.
Confundir índice con longitud	Acceso fuera del final.	Usar <code>LONG(texto) - 1</code> .

10.22 Crecimiento del trabajo en algoritmos de texto

Recorrer una cadena de longitud n una vez produce trabajo lineal, $O(n)$. Buscar cada carácter dentro de otra colección o comparar cada palabra con todas las anteriores puede generar ciclos anidados y aproximarse a $O(n^2)$.

Operación conceptual	Patrón	Costo intuitivo
Obtener longitud disponible por biblioteca	Consulta directa	$O(1)$ o dependiente de implementación.
Contar vocales	Un recorrido	$O(n)$.
Invertir texto	Un recorrido	$O(n)$.
Comparar cada elemento con los anteriores	Ciclos anidados	$O(n^2)$.
Dividir texto	Recorrer y crear partes	Aproximadamente $O(n)$.

Límite del análisis: estas clasificaciones expresan crecimiento general. El comportamiento concreto de las funciones de biblioteca depende de su implementación.

10.23 Reconstruye el modelo mental

El texto puede estudiarse como valor completo, secuencia de caracteres o conjunto de fragmentos. Las funciones de cadena permiten cambiar de una representación a otra: normalizar, extraer, buscar, reemplazar, dividir y unir. Los algoritmos combinan estas transformaciones con recorridos, decisiones, arreglos y funciones.

1. Una cadena tiene longitud e índices válidos desde 0 hasta longitud menos uno.
2. Recorrer permite examinar cada carácter.
3. Normalizar reduce diferencias irrelevantes definidas por el problema.
4. Las subcadenas representan regiones continuas.
5. Buscar localiza; reemplazar produce una nueva representación.
6. Dividir produce un arreglo de campos.
7. Unir produce una cadena a partir de una colección.
8. Los campos numéricos siguen siendo texto hasta comprobarse y convertirse.
9. Contar frecuencias combina recorridos, comparaciones y acumuladores.
10. Los casos límite incluyen cadenas vacías, separadores repetidos y puntuación.

Comprobación activa: explica cómo transformarías `" Ana,19,Mexico "` en tres campos limpios y cómo convertirías la edad en número.

Ejercicios

1. Para `"PSEUDOCODIGO"`, determina longitud, último índice y caracteres en 0, 5 y 11. **[Reconocer]**
2. Escribe un recorrido que muestre cada carácter junto con su índice. **[Aplicar]**
3. Construye una función que cuente espacios. **[Construir]**
4. Normaliza un nombre eliminando espacios exteriores y convirtiéndolo a mayúsculas. **[Aplicar]**
5. Explica por qué no conviene normalizar una contraseña. **[Analizar]**
6. Extrae los cuatro últimos caracteres de un nombre de archivo. **[Aplicar]**
7. Diseña casos de prueba para la extracción de extensiones. **[Probar]**

8. Busca una palabra y muestra su posición o un mensaje de ausencia. **[Construir]**
9. Reemplaza guiones por espacios y conserva el texto original. **[Aplicar]**
10. Divide ``rojo,verde,azul`` y recorre el arreglo resultante. **[Aplicar]**
11. Une un arreglo de nombres con ``; ``. **[Aplicar]**
12. Interpreta ``producto,4,25.5`` y calcula el total. **[Construir]**
13. Cuenta cuántas veces aparece una letra sin distinguir mayúsculas. **[Construir]**
14. Cuenta una palabra en una frase y documenta las limitaciones por puntuación. **[Analizar]**
15. Invierte una cadena y completa su traza. **[Aplicar]**
16. Amplía `ES\_PALINDROMO` para ignorar espacios. **[Construir]**
17. Elimina duplicados de una lista de categorías normalizadas. **[Construir]**
18. Ordena una lista de palabras después de normalizarla. **[Aplicar]**
19. Compara el costo de contar vocales y eliminar duplicados con ciclos anidados. **[Analizar]**
20. Genera un resumen de frecuencias de categorías de gasto. **[Integrar]**

Mini proyecto: analizador de palabras

Situación: construye un programa que reciba un texto y muestre una versión normalizada, cantidad de caracteres, palabras, vocales, palabra buscada, texto invertido y lista de palabras ordenadas.

Requisitos:

- Conservar el texto original.
- Crear funciones separadas para normalización, conteo de vocales e inversión.
- Dividir el texto en palabras y validar el arreglo obtenido.
- Permitir buscar una palabra sin distinguir mayúsculas.
- Ordenar las palabras normalizadas.
- Diseñar casos normales, vacíos, con espacios repetidos y puntuación.
- Generar el diagrama de flujo y explicar los recorridos principales.

Criterio de éxito: el programa debe explicar mediante sus funciones qué transformación se aplica en cada etapa y debe reconocer explícitamente sus limitaciones.

Autoevaluación

Responde sin consultar el capítulo.

Afirmación	Sí	Todavía no
Puedo relacionar longitud con índices válidos.	☐	☐
Puedo recorrer una cadena carácter por carácter.	☐	☐

Afirmación	Sí	Todavía no
Sé cuándo normalizar y cuándo conservar el original.	☐	☐
Puedo extraer una subcadena sin exceder límites.	☐	☐
Distingo buscar de reemplazar.	☐	☐
Puedo dividir una cadena y validar sus campos.	☐	☐
Puedo unir un arreglo de cadenas.	☐	☐
Comprendo que los campos numéricos siguen siendo texto.	☐	☐
Puedo construir conteos e inversiones.	☐	☐
Puedo identificar casos límite y costos básicos.	☐	☐

Criterio de dominio: estás listo para continuar cuando puedas transformar una entrada textual cruda en una estructura validada, recorrerla y producir un resultado explicable mediante funciones.

Glosario

Término	Definición de trabajo
Cadena	Secuencia ordenada de caracteres tratada también como un valor completo.
Carácter	Unidad individual localizada mediante un índice.
Normalización	Transformación hacia una representación común según reglas del problema.
Subcadena	Región continua extraída de una cadena.
Prefijo	Fragmento situado al inicio.
Sufijo	Fragmento situado al final.
Patrón	Texto que se desea localizar o reemplazar.
Separador	Fragmento que delimita partes al dividir.
Campo	Parte interpretada de una cadena estructurada.
Tokenización	Separación del texto en unidades analizables.
Frecuencia	Cantidad de apariciones de un valor.
Palíndromo	Texto que coincide con su versión invertida bajo reglas definidas.

Referencias del capítulo

CLRS Kit Español. (2026). Documentación y código fuente de las funciones estándar para cadenas y arreglos [Repositorio de software].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Gaddis, T. (2022). Starting out with programming logic and design (6th ed.). Pearson.

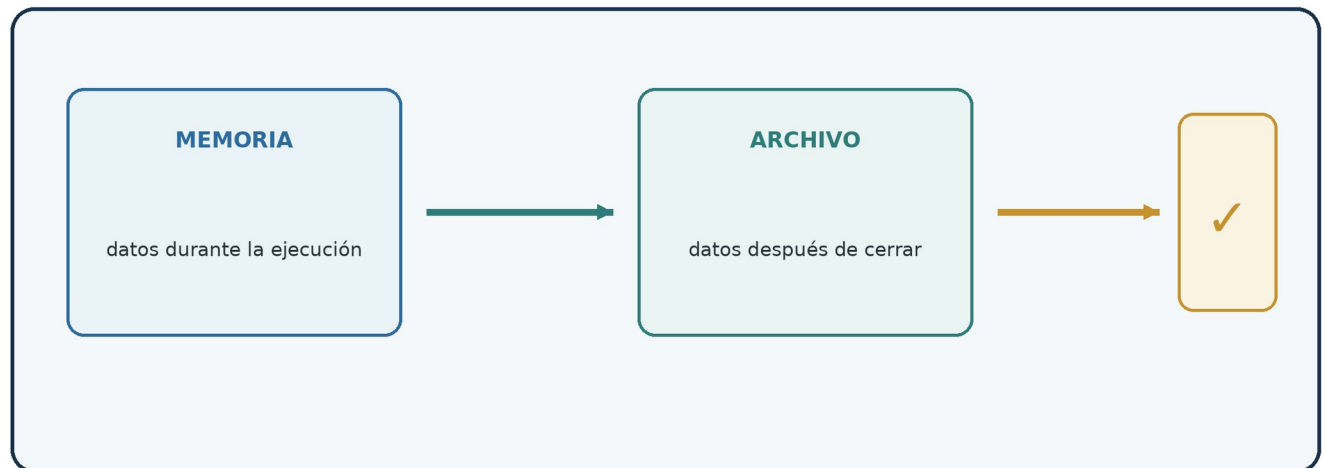
Continuidad: los arreglos y las cadenas ya permiten representar información compleja. El capítulo 11 se centrará en especificar, probar, diagnosticar y mejorar programas completos.

CAPÍTULO

12

Integrar datos, archivos y decisiones de diseño

Biblioteca estándar, persistencia y proyecto integrador



El programa completo une estructuras, biblioteca, pruebas y decisiones justificadas.

Integrar datos, archivos y decisiones de diseño

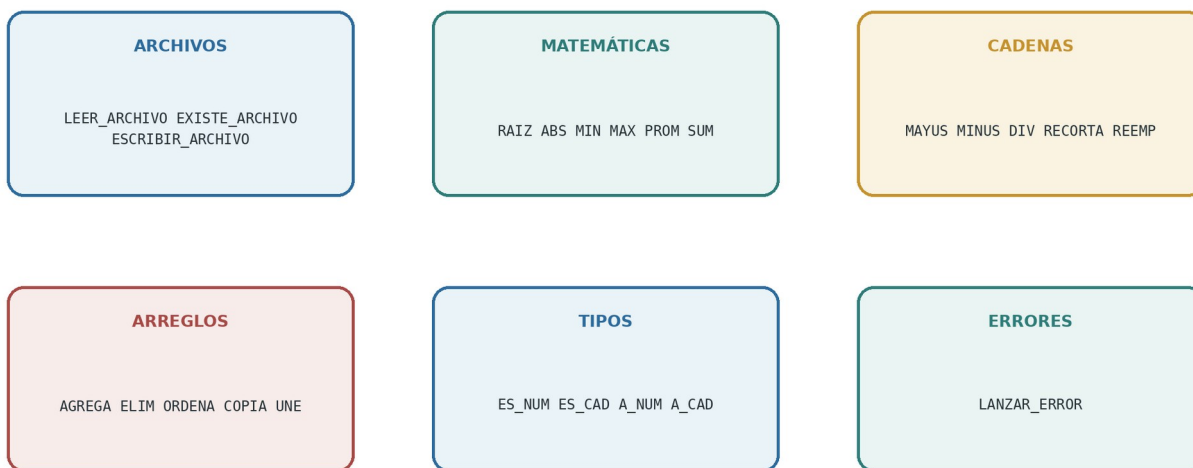
¿Cómo se convierte un conjunto de técnicas en un programa completo que conserva información?

Hasta ahora, cada ejecución comenzó con un estado nuevo. Las variables y los arreglos permitieron organizar información mientras el programa estaba activo, pero al cerrarlo los datos desaparecieron. Un registro de gastos que olvida todos los movimientos deja de ser útil, aunque sus cálculos sean correctos.

El cierre de la obra integra tres capacidades. Primero, seleccionar funciones de la biblioteca estándar según el problema. Segundo, conservar información mediante archivos de texto. Tercero, justificar cómo se relacionan entrada, validación, funciones, colecciones, pruebas, diagramas y costo dentro de un proyecto completo.

Promesa de aprendizaje: al terminar podrás elegir funciones de biblioteca por su contrato, crear, escribir, leer, medir y eliminar archivos, diseñar un formato textual básico, separar procesamiento y almacenamiento, reconstruir datos persistentes y completar un proyecto integrador documentado y probado.

La biblioteca estándar agrupa capacidades que ya tienen nombre y contrato



Elegir una función exige conocer qué recibe, qué devuelve, qué modifica y cuánto trabajo realiza.

Figura 12.1. Familias principales de la biblioteca estándar de CLRS. Elaboración propia.

Alcance: la persistencia se estudiará con archivos de texto y formatos sencillos. Bases de datos, serialización estructurada, concurrencia y seguridad de almacenamiento pertenecen a cursos posteriores.

12.1 La biblioteca como conjunto de decisiones ya implementadas

Una biblioteca estándar reúne operaciones comunes para que cada programa no tenga que reconstruirlas. Usar ``RAIZ``, ``DIV``, ``ORDENA`` o ``LEER_ARCHIVO`` delega detalles de implementación, pero no elimina la necesidad de comprender el problema.

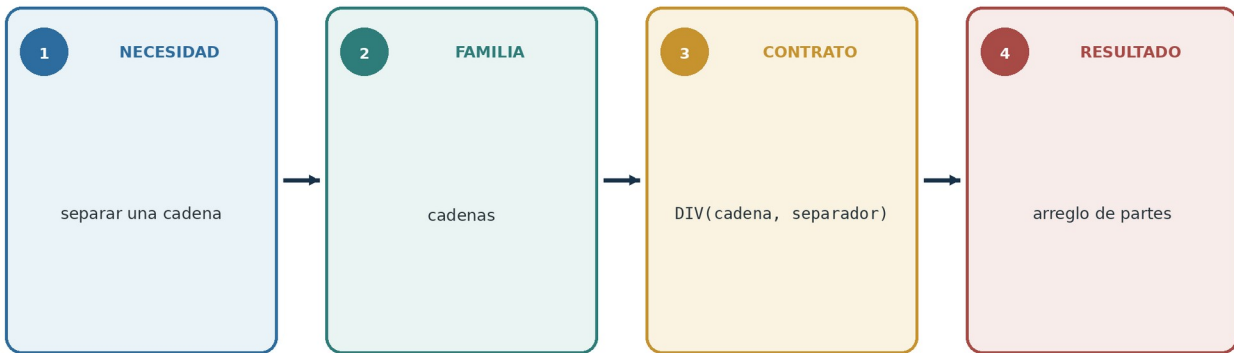
La pregunta correcta no es “¿qué función puedo usar?”, sino “¿qué transformación necesito y qué contrato ofrece la función?”.

Necesidad	Función posible	Resultado
Obtener la raíz cuadrada	<code>`RAIZ(x)`</code>	Número.
Normalizar una categoría	<code>`MAYUS(RECORTA(texto))`</code>	Cadena.
Separar campos	<code>`DIV(texto, separador)`</code>	Arreglo.
Comprobar un tipo	<code>`ES_NUM(valor)`</code>	Lógico.
Agregar un elemento	<code>`AGREGA(arreglo, valor)`</code>	Arreglo modificado.
Leer un archivo	<code>`LEER_ARCHIVO(ruta)`</code>	Cadena con su contenido.

Idea clave: la biblioteca reduce código repetitivo; la responsabilidad de elegir, validar y combinar sus operaciones sigue perteneciendo a quien diseña el programa.

12.2 Elegir por contrato, no por semejanza del nombre

Seleccionar una función comienza por la necesidad, no por memorizar nombres



Después se revisan casos límite: cadena vacía, separador ausente, campos incompletos y tipos incompatibles.

Figura 12.2. Proceso para seleccionar una función de biblioteca. Elaboración propia.

El contrato de una función incluye al menos sus entradas, su resultado observable y sus condiciones de uso. Dos funciones pueden parecer cercanas y resolver problemas distintos.

Función	Pregunta que responde	Detalle relevante
`ES_NUM(valor)`	¿El valor ya es numérico?	No convierte.
`ES_CAD_NUM(cadena)`	¿La cadena representa un número?	Examina texto.
`A_NUM(valor)`	¿Cuál es su conversión numérica?	Puede producir un valor no útil si la entrada no es válida.
`INDICE(arreglo, valor)`	¿En qué posición aparece?	Retorna una posición, no un lógico.
`CONT(arreglo, valor)`	¿Está contenido?	Retorna `VERDAD` o `FALSO`.

Pausa y piensa: para validar una entrada textual antes de sumarla, ¿usarías únicamente `A\_NUM` o combinarías una comprobación con la conversión? Justifica el orden.

12.3 Las familias de funciones trabajan juntas

Un problema real rara vez se resuelve con una sola llamada. La preparación de una categoría puede combinar funciones de cadenas; el análisis de cantidades puede unir división, conversión y operaciones de arreglos.

Preparar una categoría

```
NORMALIZAR_CATEGORIA(categoria)
  categoria <- RECORTA(categoria)
  categoria <- MAYUS(categoria)
  retornar categoria
```

Separar y convertir un registro

```
campos <- DIV("COMIDA|150.50", "|")
categoria <- campos[0]
monto <- A_NUM(campos[1])
```

La composición hace visible una cadena de transformaciones. Cada paso puede comprobarse por separado y el resultado intermedio conserva un nombre significativo.

Advertencia: encadenar muchas llamadas en una sola expresión puede ocultar qué transformación falló. Usa variables intermedias cuando aporten observabilidad.

12.4 El costo no desaparece dentro de la biblioteca

Una llamada breve puede representar trabajo proporcional al tamaño de los datos

EXISTE_ARCHIVO(ruta)	Comprobar metadatos	$O(1)$
LEER_ARCHIVO(ruta)	Leer n caracteres	$O(n)$
ESCRIBIR_ARCHIVO(ruta, texto)	Agregar m caracteres	$O(m)$
DIV(texto, separador)	Recorrer la cadena	$O(n)$
ORDENA(arreglo)	Ordenar n elementos	$O(n \log n)$

La biblioteca oculta detalles de implementación, no elimina el costo de procesar datos.

Figura 12.3. Trabajo oculto detrás de llamadas breves. Elaboración propia.

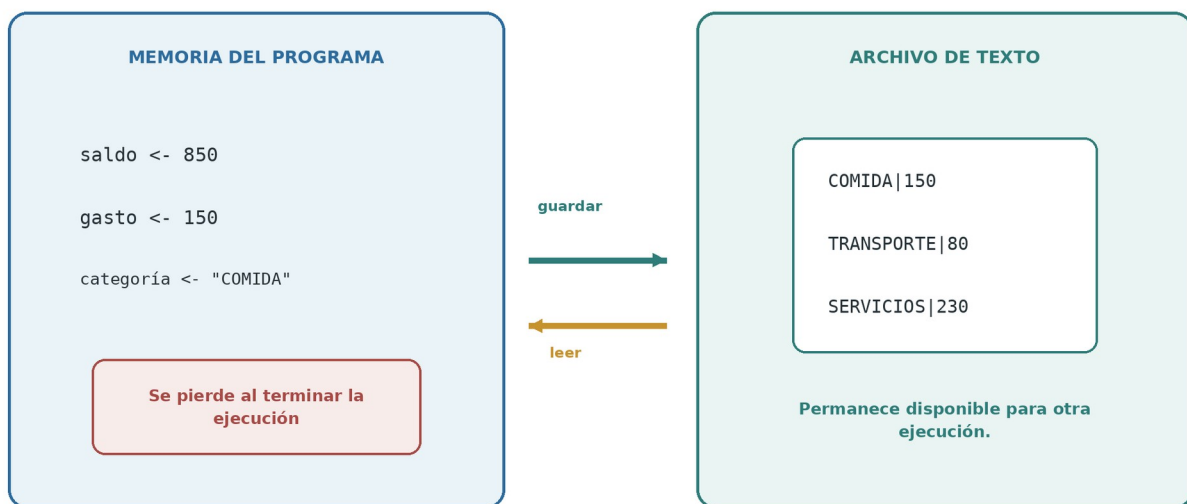
La notación de una llamada puede ocupar una sola línea mientras la operación recorre una cadena, copia un arreglo o reordena elementos. El visualizador de costo permite mantener esta relación entre sintaxis breve y trabajo realizado.

Patrón	Ejemplo	Interpretación
Constante	<code>`EXISTE_ARCHIVO(ruta)`</code>	No depende directamente de la longitud del contenido.
Lineal	<code>`LEER_ARCHIVO(ruta)`</code>	Procesa una cantidad proporcional al tamaño leído.
Lineal	<code>`DIV(texto, " ")`</code>	Examina la cadena para localizar separadores.
$n \log n$	<code>`ORDENA(valores)`</code>	Ordena una colección.

Importante: en esta obra el costo se utiliza para explicar crecimiento, no para reemplazar mediciones ni demostrar formalmente la implementación.

12.5 De memoria temporal a información persistente

Persistir significa conservar información más allá de una ejecución



El archivo no reemplaza las variables: permite reconstruir datos cuando el programa vuelve a comenzar.

Figura 12.4. Relación entre memoria de ejecución y archivo persistente. Elaboración propia.

La memoria del programa mantiene el estado necesario para la ejecución actual. Un archivo conserva una representación de ciertos datos para que otra ejecución pueda recuperarlos.

Persistir no significa guardar todas las variables. Significa decidir qué información merece sobrevivir, con qué formato se escribirá y cómo podrá reconstruirse después.

Pregunta	Decisión de diseño
¿Qué debe conservarse?	Los movimientos, no mensajes temporales ni contadores de interfaz.
¿Cuándo se guarda?	Después de aceptar y validar cada registro.
¿Cómo se representa?	Una línea por movimiento.
¿Cómo se recupera?	Leer, dividir líneas, dividir campos y convertir cantidades.
¿Qué sucede si falta el archivo?	Crearlo o iniciar un historial vacío.

Idea clave: un archivo contiene una representación de los datos, no los conceptos del programa. Recuperar exige interpretar esa representación.

12.6 Rutas y nombres de archivo

Las funciones de archivo reciben una ruta. Para los primeros programas conviene utilizar un nombre relativo sencillo, como `"gastos.txt"`, que se resolverá desde el directorio de ejecución.

Ruta básica

```
ruta <- "gastos.txt"
escribir "Archivo de trabajo: " + ruta
```

Las rutas absolutas dependen del sistema operativo y del equipo. Una ruta relativa facilita trasladar el proyecto, aunque exige comprender desde qué carpeta se ejecuta.

Advertencia: no construyas rutas mezclando separadores específicos del sistema sin necesidad. En un proyecto introductorio, conserva los archivos dentro de la carpeta de trabajo.

12.7 Comprobar y crear sin destruir información

`EXISTE_ARCHIVO(ruta)` permite decidir si es necesario preparar un archivo. `CREAR_ARCHIVO(ruta)` crea un archivo vacío; si se utiliza sobre uno existente, su contenido puede reemplazarse por vacío. Por eso no debe ejecutarse indiscriminadamente en cada inicio.

Preparación segura

```
PREPARAR_ARCHIVO(ruta)
  si no EXISTE_ARCHIVO(ruta)
    CREAR_ARCHIVO(ruta)
```

Práctica peligrosa

```
PRINCIPAL( )
  ruta <- "gastos.txt"
  CREAR_ARCHIVO(ruta)
  escribir "Archivo preparado"
```

Si el archivo ya contenía movimientos, esta decisión puede borrar el historial.

Idea clave: antes de crear, sobrescribir o eliminar, el programa debe decidir qué debe ocurrir con la información existente.

12.8 Escribir agrega contenido

`ESCRIBIR\_ARCHIVO(ruta, contenido)` agrega el contenido al final. Esta semántica es apropiada para un historial en el que cada operación produce una nueva línea.

Agregar un movimiento

```
ruta <- "gastos.txt"
registro <- "COMIDA|150.50\n"
ESCRIBIR_ARCHIVO(ruta, registro)
```

El salto de línea separa registros consecutivos. Sin él, dos escrituras podrían quedar unidas y perder la frontera entre movimientos.

Advertencia: agregar no corrige registros previos. Para modificar una línea específica sería necesario leer, transformar todo el contenido y volver a escribir con una estrategia diferente.

12.9 Leer y distinguir ausencia de contenido

`LEER\_ARCHIVO(ruta)` devuelve el contenido completo como cadena. Cuando el archivo no existe, la implementación de CLRS devuelve una cadena vacía. Esto evita una falla inmediata, pero obliga a diferenciar entre “archivo ausente” y “archivo existente sin contenido” cuando esa diferencia sea importante.

Lectura defensiva

```
contenido <- LEER_ARCHIVO("gastos.txt")

si ES_VAC(RECORTA(contenido))
  escribir "No hay movimientos registrados"
sino
  escribir contenido
```

Para conocer la existencia de manera explícita se utiliza `EXISTE\_ARCHIVO`. Para conocer el tamaño se utiliza `PESO\_ARCHIVO`, que retorna cero si el archivo no existe.

Situación	`EXISTE_ARCHIVO`	`LEER_ARCHIVO`	`PESO_ARCHIVO`
No existe	`FALSO`	''''	0
Existe y está vacío	`VERDAD`	''''	0
Existe con datos	`VERDAD`	Contenido	Número mayor que 0

Importante: una misma observación, como contenido vacío, puede corresponder a estados distintos. Elige la función que permita responder la pregunta real.

12.10 Diseñar el ciclo de vida del archivo

El ciclo de vida de un archivo debe diseñarse de forma explícita

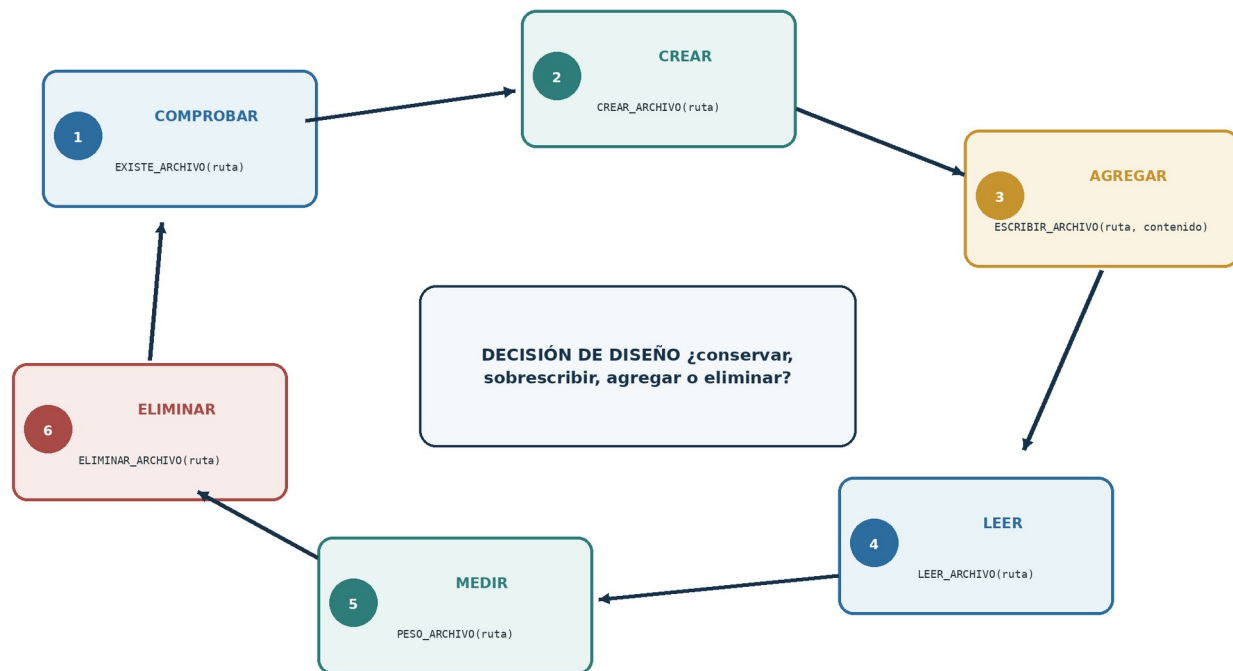


Figura 12.5. Operaciones del ciclo de vida de un archivo. Elaboración propia.

No todos los programas necesitan todas las operaciones. Un historial puede comprobar, crear, agregar y leer. Una herramienta de mantenimiento podría además medir o eliminar.

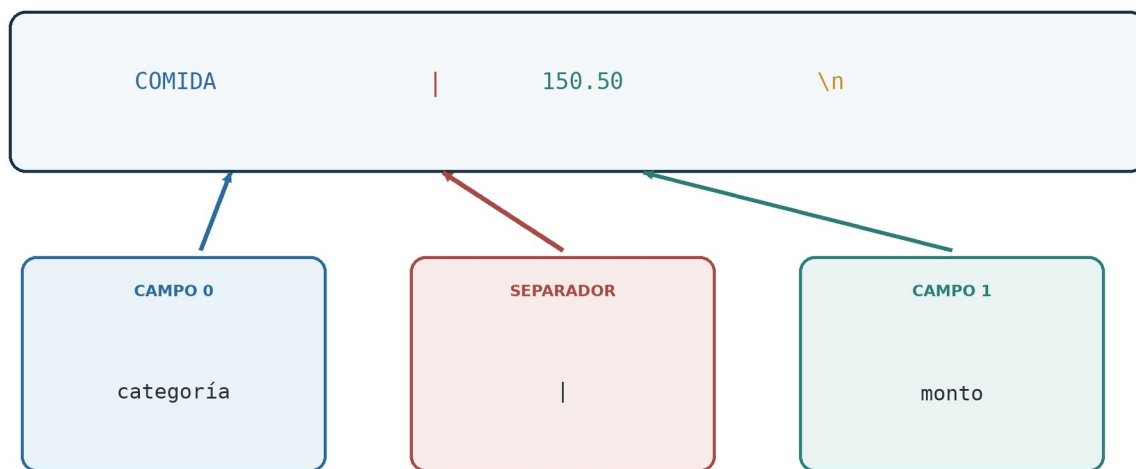
Operación	Cuándo utilizarla	Riesgo
Comprobar	Antes de depender de un archivo.	Confundir ausencia con vacío.
Crear	Al inicializar un recurso nuevo.	Vaciar uno existente.
Agregar	Al registrar nueva información.	Duplicar datos.
Leer	Al reconstruir el estado.	Interpretar un formato dañado.

Operación	Cuándo utilizarla	Riesgo
Medir	Al informar tamaño o detectar vacío.	No confundir bytes con cantidad de registros.
Eliminar	Al reiniciar deliberadamente.	Pérdida irreversible del historial.

Pausa y piensa: ¿un botón “reiniciar historial” debería llamar directamente a `ELIMINAR\_ARCHIVO` o pedir una confirmación y volver a crear el archivo? Explica el flujo.

12.11 Un formato de registros separado por campos

Un formato de texto define cómo escribir y reconstruir cada registro



Limitación: si la categoría contiene “|” o un salto de línea, el formato necesita reglas de escape adicionales.

Figura 12.6. Formato básico de una línea de gasto. Elaboración propia.

Un formato elemental puede asignar una línea a cada registro y utilizar `|` como separador entre categoría y monto.

Formar una línea

```
FORMATEAR_GASTO(categoria, monto)
  categoria <- MAYUS(RECORTA(categoria))
  retornar categoria + "|" + A_CAD(monto) + "\n"
```

El formato es legible y puede procesarse con `DIV`. También tiene límites: no define cómo escapar el separador, los saltos de línea ni campos faltantes. Estas limitaciones deben declararse.

Simplificación pedagógica: el formato sirve para comprender persistencia y reconstrucción. No pretende sustituir CSV, JSON, bases de datos ni formatos con reglas completas de escape.

12.12 Reconstruir registros desde texto

La lectura produce una sola cadena. Para recuperar registros se realiza una transformación por niveles:

1. Recortar saltos sobrantes.
2. Dividir el contenido por líneas.
3. Recorrer cada línea.
4. Dividir cada línea por el separador de campos.
5. Comprobar la cantidad y el tipo de los campos.
6. Convertir el monto a número.

Calcular el total guardado

```
CALCULAR_TOTAL_HISTORIAL(contenido)
  contenido <- RECORTA(contenido)

  si ES_VAC(contenido)
    retornar 0

  lineas <- DIV(contenido, "\n")
  total <- 0

  para i <- 0 hasta LONG(lineas) - 1
    campos <- DIV(lineas[i], "|")

    si LONG(campos) >= 2 y ES_CAD_NUM(campos[1])
      total <- total + A_NUM(campos[1])

  retornar total
```

La función ignora líneas que no contienen al menos dos campos numéricamente interpretables. Esa es una política de tolerancia. Otra solución podría lanzar un error y detenerse. La elección depende del propósito del programa.

Decisión de diseño: tolerar datos dañados permite continuar, pero puede ocultar pérdida de información. Rechazarlos hace visible el problema, pero impide obtener un resultado parcial.

12.13 Separar procesamiento y almacenamiento

Una función de cálculo no debería depender necesariamente de una ruta. Recibir el contenido como cadena permite probarla sin crear archivos.

Separación de responsabilidades

```

LEER_HISTORIAL(ruta)
  retornar LEER_ARCHIVO(ruta)

CALCULAR_TOTAL_HISTORIAL(contenido)
  // procesa la cadena recibida
  retornar total

PRINCIPAL()
  contenido <- LEER_HISTORIAL("gastos.txt")
  total <- CALCULAR_TOTAL_HISTORIAL(contenido)
  escribir "Total: " + total

```

Esta organización permite comprobar `CALCULAR\_TOTAL\_HISTORIAL` con cadenas construidas directamente en pruebas. La interacción con el sistema de archivos queda concentrada.

Idea clave: la persistencia es una responsabilidad; interpretar y calcular sobre los datos es otra. Separarlas mejora las pruebas y permite cambiar el almacenamiento con menor impacto.

12.14 Manejar errores y datos incompatibles

Las operaciones de archivo pueden fallar por permisos, rutas inválidas u otras condiciones externas. CLRS Kit presenta los errores de ejecución con mensajes en español. El programa también debe detectar inconsistencias propias del formato.

Validar una línea

```

ES_LINEA_GASTO_VALIDA(línea)
  campos <- DIV(línea, "|")

  si LONG(campos) != 2
    retornar FALSO

  retornar no ES_VAC(RECORTA(campos[0])) y ES_CAD_NUM(campos[1])

```

`LANZAR\_ERROR` puede utilizarse cuando continuar produciría resultados engañosos.

Rechazar un formato dañado

```

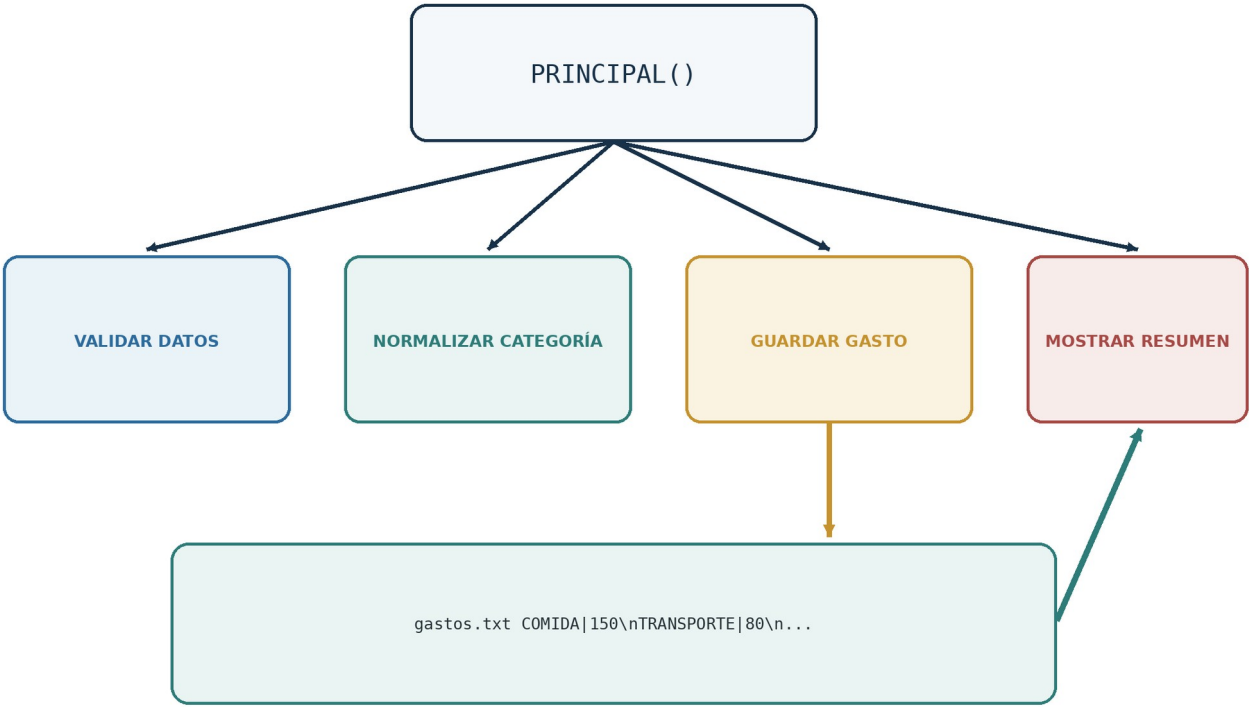
si no ES_LINEA_GASTO_VALIDA(línea)
  LANZAR_ERROR("El historial contiene un registro inválido")

```

Advertencia: no captures o ignores un error solo para que el programa “siga funcionando”. Decide qué información puede recuperarse con seguridad.

12.15 Proyecto integrador: gestor persistente de gastos

El proyecto integrador conecta entrada, reglas, memoria, archivo y presentación



Cada flecha representa una transferencia de datos o una decisión que debe poder explicarse y probarse.

Figura 12.7. Arquitectura del proyecto integrador. Elaboración propia.

El proyecto final amplía el caso longitudinal. Permitirá registrar movimientos, conservarlos en un archivo y recuperar un resumen en ejecuciones posteriores.

Componente	Responsabilidad
`ES_CATEGORIA_VALIDA`	Comprobar que la categoría sea texto no vacío.
`ES_MONTO_VALIDO`	Aceptar números positivos o cadenas numéricas positivas.
`A_MONTO`	Convertir la entrada validada a número.
`NORMALIZAR_CATEGORIA`	Recortar y convertir a mayúsculas.
`PREPARAR_ARCHIVO`	Crear solo cuando no existe.
`GUARDAR_GASTO`	Formar una línea y agregarla.
`CALCULAR_TOTAL_HISTORIAL`	Reconstruir montos y acumularlos.
`MOSTRAR_HISTORIAL`	Presentar contenido y total.
`PRINCIPAL`	Coordinar el menú y las llamadas.

12.16 Implementación guiada del proyecto

Gestor persistente de gastos

```

ES_CATEGORIA_VALIDA(valor)
    retornar ES_CAD(valor) y no ES_VAC(RECORTA(valor))

ES_MONTO_VALIDO(valor)
    si ES_NUM(valor)
        retornar valor > 0

    si ES_CAD(valor)
        retornar ES_CAD_NUM(valor) y A_NUM(valor) > 0

    retornar FALSO

A_MONTO(valor)
    si ES_NUM(valor)
        retornar valor

    retornar A_NUM(valor)

NORMALIZAR_CATEGORIA(categoria)
    retornar MAYUS(RECORTA(categoria))

PREPARAR_ARCHIVO(ruta)
    si no EXISTE_ARCHIVO(ruta)
        CREAR_ARCHIVO(ruta)

GUARDAR_GASTO(ruta, categoria, monto)
    categoria <- NORMALIZAR_CATEGORIA(categoria)
    registro <- categoria + "|" + A_CAD(monto) + "\n"
    ESCRIBIR_ARCHIVO(ruta, registro)

CALCULAR_TOTAL_HISTORIAL(contenido)
    contenido <- RECORTA(contenido)

    si ES_VAC(contenido)
        retornar 0

    lineas <- DIV(contenido, "\n")
    total <- 0

    para i <- 0 hasta LONG(lineas) - 1
        campos <- DIV(lineas[i], "|")

        si LONG(campos) >= 2 y ES_CAD_NUM(campos[1])
            total <- total + A_NUM(campos[1])

    retornar total

MOSTRAR_HISTORIAL(ruta)
    contenido <- LEER_ARCHIVO(ruta)

    si ES_VAC(RECORTA(contenido))
        escribir "No hay movimientos registrados"
        retornar 0

    escribir "--- HISTORIAL ---"
    escribir contenido
    retornar CALCULAR_TOTAL_HISTORIAL(contenido)

PRINCIPAL()
    ruta <- "gastos.txt"

```

```

PREPARAR_ARCHIVO(ruta)
opcion <- 0

mientras opcion != 3
  escribir "1. Registrar gasto"
  escribir "2. Mostrar historial"
  escribir "3. Salir"
  leer opcion

  si opcion = 1
    categoria <- ""
    monto <- ""

    escribir "Categoría:"
    leer categoria
    escribir "Monto:"
    leer monto

    si ES_CATEGORIA_VALIDA(categoria) y ES_MONTO_VALIDO(monto)
      monto <- A_MONTO(monto)
      GUARDAR_GASTO(ruta, categoria, monto)
      escribir "Gasto guardado"
    sino
      escribir "Datos inválidos"
  sino si opcion = 2
    total <- MOSTRAR_HISTORIAL(ruta)
    escribir "Total acumulado: " + total
  sino si opcion != 3
    escribir "Opción inválida"

```

Observación: el programa conserva los registros en texto y vuelve a calcular el total al leerlos. No mantiene todavía identificadores, fechas, edición ni eliminación selectiva. Esas ausencias son limitaciones declaradas, no errores ocultos.

12.17 Seguir una sesión completa

Paso	Acción	Estado o archivo
1	El programa inicia.	Comprueba `gastos.txt`.
2	El archivo no existe.	Lo crea vacío.
3	Se registra `Comida`, 150.	Normaliza a `COMIDA`.
4	Guarda el movimiento.	Archivo: `COMIDA 150\n`.
5	Se registra `Transporte`, 80.	Agrega otra línea.
6	Se solicita historial.	Lee todo el contenido.
7	Divide líneas y campos.	Convierte 150 y 80.
8	Acumula los montos.	Total = 230.

La traza permite comprobar que la persistencia no consiste únicamente en escribir. El recorrido completo incluye validación, representación, almacenamiento, recuperación, interpretación y cálculo.

Pausa y piensa: ¿qué resultado obtendría la función si el archivo contiene una línea vacía al final? Explica el efecto de `RECORTA` antes de `DIV`.

12.18 Casos de prueba del proyecto

Caso	Estado inicial	Acción	Resultado esperado
Archivo ausente	No existe	Iniciar	Se crea vacío.
Historial vacío	Existe y mide 0	Mostrar	Mensaje sin movimientos; total 0.
Gasto válido	Vacío	Registrar COMIDA, 150	Se agrega una línea.
Monto textual válido	Vacío	Registrar "80.5"	Se convierte y guarda.
Monto negativo	Cualquier estado	Registrar -10	Se rechaza.
Categoría vacía	Cualquier estado	Registrar espacios	Se rechaza.
Dos registros	Dos líneas válidas	Mostrar	Total correcto.
Línea dañada	Campo numérico inválido	Calcular	Se ignora según la política actual.
Reinicio	Archivo con datos	Volver a ejecutar	Los datos permanecen.

Prueba de regresión esencial: después de cualquier cambio, confirma que iniciar el programa no vacíe un archivo existente.

12.19 Revisar decisiones y limitaciones

Un proyecto integrador debe explicar no solo lo que hace, sino por qué se diseñó de esa manera y qué no resuelve.

Decisión	Razón	Limitación
Una línea por gasto	Facilita agregar y leer.	No permite saltos en campos.
Separador ` `	Es visible y fácil de dividir.	Requiere escape si aparece en texto.
Agregar al final	Conserva historial.	Editar exige reescritura.
Total recalculado	Evita guardar datos derivados.	Lee todo el archivo.
Funciones separadas	Facilita pruebas.	Aumenta cantidad de componentes.
Ignorar líneas dañadas	Permite resultado parcial.	Puede ocultar pérdida.

Criterio profesional inicial: una simplificación es aceptable cuando está declarada, sus consecuencias se comprenden y no se presenta como una solución universal.

12.20 Entregables del proyecto final

El proyecto puede presentarse como evidencia acumulada de todo el aprendizaje del libro.

1. Archivo `.clrs`` ejecutable.
2. Descripción del problema y del usuario previsto.
3. Especificación de entradas, salidas, reglas y restricciones.
4. Mapa de funciones y responsabilidades.
5. Descripción del formato persistente.
6. Tabla de casos de prueba y resultados observados.
7. Diagrama de flujo exportado.
8. Explicación del costo de los recorridos principales.
9. Lista de errores encontrados y correcciones.
10. Limitaciones y posibles extensiones.

Criterio	Evidencia de dominio
Corrección	Los casos normales y límite producen resultados esperados.
Persistencia	Los datos sobreviven a una nueva ejecución.
Validación	Las entradas inválidas no contaminan el archivo.
Organización	Las funciones tienen responsabilidades reconocibles.
Representación	El formato puede escribirse y reconstruirse.
Pruebas	La evidencia permite repetir la comprobación.
Costo	Se distinguen recorridos lineales y operaciones constantes.
Comunicación	Las decisiones y limitaciones están justificadas.

12.21 Reconstruye el recorrido completo del libro

El programa final puede comprenderse como una cadena de decisiones acumuladas:

1. Un problema se convierte en entradas, reglas y resultados.
2. Los valores forman un estado que cambia mediante instrucciones.
3. Las condiciones eligen caminos.
4. Los ciclos repiten procesos con progreso y terminación.
5. Las funciones dividen responsabilidades y transfieren datos.
6. Los arreglos y cadenas permiten procesar colecciones.
7. Las pruebas comparan resultados observados y esperados.
8. La biblioteca aporta operaciones con contratos conocidos.
9. Los archivos conservan representaciones entre ejecuciones.
10. El proyecto integra código, pruebas, diagramas, costo y explicación.

Idea final: programar no consiste en acumular sintaxis. Consiste en representar un problema, organizar transformaciones, observar evidencia y justificar decisiones hasta obtener una solución comprensible.

Ejercicios

Realiza los ejercicios sobre copias de archivos de práctica. No uses información importante.

1. Clasifica diez funciones de la biblioteca por familia y tipo de resultado. **[Reconocer]**
2. Compara ``ES_NUM``, ``ES_CAD_NUM`` y ``A_NUM`` con tres entradas distintas. **[Analizar]**
3. Selecciona funciones para normalizar una categoría con espacios y minúsculas. **[Aplicar]**
4. Predice el costo de leer, dividir y recorrer un archivo de n caracteres. **[Analizar]**
5. Crea un archivo solo cuando no exista. **[Construir]**
6. Demuestra qué ocurre si ``CREAR_ARCHIVO`` se ejecuta sobre un historial existente. **[Experimentar]**
7. Escribe tres líneas y comprueba el peso del archivo. **[Aplicar]**
8. Diseña un formato ``nombre|calificación`` y declara sus limitaciones. **[Diseñar]**
9. Reconstruye un arreglo de líneas mediante ``DIV``. **[Aplicar]**
10. Valida que cada línea tenga exactamente dos campos. **[Construir]**
11. Calcula el promedio de calificaciones guardadas en texto. **[Integrar]**
12. Decide si una línea dañada debe ignorarse o lanzar un error. Justifica. **[Evaluar]**
13. Separa una función que lee el archivo de otra que procesa su contenido. **[Mejorar]**
14. Prueba una función de procesamiento sin utilizar archivos reales. **[Probar]**
15. Añade una opción para mostrar la cantidad de registros válidos. **[Construir]**
16. Añade una confirmación antes de eliminar el archivo. **[Construir]**
17. Diseña una prueba que demuestre persistencia entre ejecuciones. **[Probar]**
18. Genera el diagrama del proyecto e identifica las llamadas de archivo. **[Representar]**
19. Explica por qué el peso del archivo no equivale a la cantidad de registros. **[Analizar]**
20. Propón una extensión del formato para incluir fecha y describe el impacto. **[Diseñar]**

Variaciones del proyecto integrador

Proyecto	Registro persistente	Procesamiento
Sistema de calificaciones	<code>`alumno nota`</code>	Promedio, máximo y aprobación.
Inventario	<code>`producto cantidad precio`</code>	Existencias y valor total.
Analizador de texto	Texto completo	Frecuencias y palabras comunes.
Registro de hábitos	<code>`fecha hábito estado`</code>	Cumplimiento por período.
Estadísticas personales	Una cantidad por línea	Media, mediana y varianza.

Restricción: elige un proyecto cuya complejidad permita explicar cada decisión. Agregar características sin poder probarlas o justificarlas reduce la calidad del resultado.

Autoevaluación

Responde sin consultar el capítulo. Después demuestra cada respuesta mediante un programa pequeño.

Afirmación	Sí	Todavía no
Selecciono funciones de biblioteca por su contrato.	☐	☐
Distingo comprobar, convertir y procesar.	☐	☐
Comprendo qué operaciones tienen costo lineal.	☐	☐
Puedo crear un archivo sin destruir uno existente.	☐	☐
Puedo agregar y leer contenido.	☐	☐
Distingo ausencia de archivo y archivo vacío.	☐	☐
Puedo diseñar un formato textual básico.	☐	☐
Puedo reconstruir registros desde una cadena.	☐	☐
Separo almacenamiento y procesamiento.	☐	☐
Puedo probar persistencia entre ejecuciones.	☐	☐
Justifico decisiones y limitaciones del proyecto.	☐	☐

Criterio de dominio: has completado la ruta cuando puedes construir un programa persistente, explicar su formato, probarlo desde archivos vacíos y con datos, y justificar la relación entre biblioteca, funciones, costo y calidad.

Glosario

Término	Definición de trabajo
Biblioteca estándar	Conjunto de funciones disponibles para operaciones comunes.
Contrato	Entradas, resultado, efectos y condiciones de uso de una función.
Persistencia	Conservación de información más allá de una ejecución.
Archivo de texto	Secuencia persistente de caracteres.

Término	Definición de trabajo
Ruta	Ubicación utilizada para identificar un archivo.
Formato	Reglas para representar datos mediante texto.
Registro	Grupo de campos que representa una entidad o movimiento.
Separador	Texto utilizado para distinguir campos o registros.
Serialización básica	Conversión de datos del programa a una representación almacenable.
Reconstrucción	Interpretación de la representación para recuperar valores.
Contenido derivado	Dato calculable a partir de otros, como un total.
Tolerancia	Política que permite continuar ante ciertos datos defectuosos.
Sobrescritura	Reemplazo del contenido anterior.
Agregación al final	Incorporación de contenido después de lo existente.
Costo oculto	Trabajo realizado internamente por una llamada breve.
Proyecto integrador	Solución que combina y justifica los aprendizajes principales.

Referencias del capítulo

CLRS Kit Español. (2026). Catálogo semántico de la biblioteca estándar y documentación de funciones de archivos [Repositorio de software].

CLRS Kit Español. (2026). Implementación de la biblioteca estándar: lectura, escritura, creación, comprobación, medición y eliminación de archivos [Código fuente].

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2nd ed.). The MIT Press.

Cierre de la obra: el pseudocódigo dejó de ser una descripción informal y se convirtió en un medio para ejecutar, observar, probar, visualizar y conservar soluciones. El siguiente paso no es memorizar otro lenguaje: es transferir estos modelos mentales a nuevos lenguajes y problemas.